

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 1

Pengantar Time Series Data

Abstrak

Pertemuan ini membahas akuisisi data time series sensor industri: teorema sampling Nyquist-Shannon, bahaya

Sub - CPMK

Sub-CPMK 1.1 – Mampu mengidentifikasi komponen dan kebutuhan teknis sistem otomasi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-1.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan pertama ini adalah pintu gerbang ke seluruh mata kuliah Optimalisasi & Otomasi. Sebelum dapat memprediksi kerusakan mesin, mengoptimalkan jadwal perawatan, atau mengotomasi keputusan operasional, mahasiswa harus terlebih dahulu memahami karakter data yang dihasilkan oleh sensor industri: data berurutan dalam waktu, atau time series. Setiap pengukuran getaran, suhu, arus, maupun aliran yang diambil dari mesin memiliki cap waktu, sehingga ketergantungan antar waktu inilah yang membedakannya dari data tabular biasa. Gambaran umum alur pembahasan pertemuan ini, dari akuisisi sensor mentah hingga keputusan teknis, ditunjukkan pada Gambar 1.



Gambar 1. Alur pertemuan 1: dari akuisisi sensor mentah menjadi keputusan condition monitoring.

Modul ini membahas teori sampling (Teorema Nyquist-Shannon), frekuensi Nyquist, dan bahaya aliasing ketika sampling rate kurang memadai, lalu memperkenalkan metrik dasar getaran (RMS, peak, crest factor, SNR) serta klasifikasi severity ISO 10816 sebagai fondasi

condition monitoring. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan forum diskusi studi kasus.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 1.1:** Mampu mengidentifikasi komponen dan kebutuhan teknis sistem otomasi, yaitu menentukan sampling rate, jenis sensor, dan strategi penyimpanan data yang dibutuhkan untuk akuisisi deret waktu, dengan mempertimbangkan trade-off antara kualitas data dan biaya (CPMK 1).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan sifat temporal dependency pada time series; menguraikan komponen tren, musiman, siklik, dan noise; menghitung lima statistik dasar sinyal; memilih sampling rate yang benar berdasarkan Teorema Nyquist; serta mengimplementasikan semuanya dengan NumPy/Pandas.

KONSEP DASAR TIME SERIES

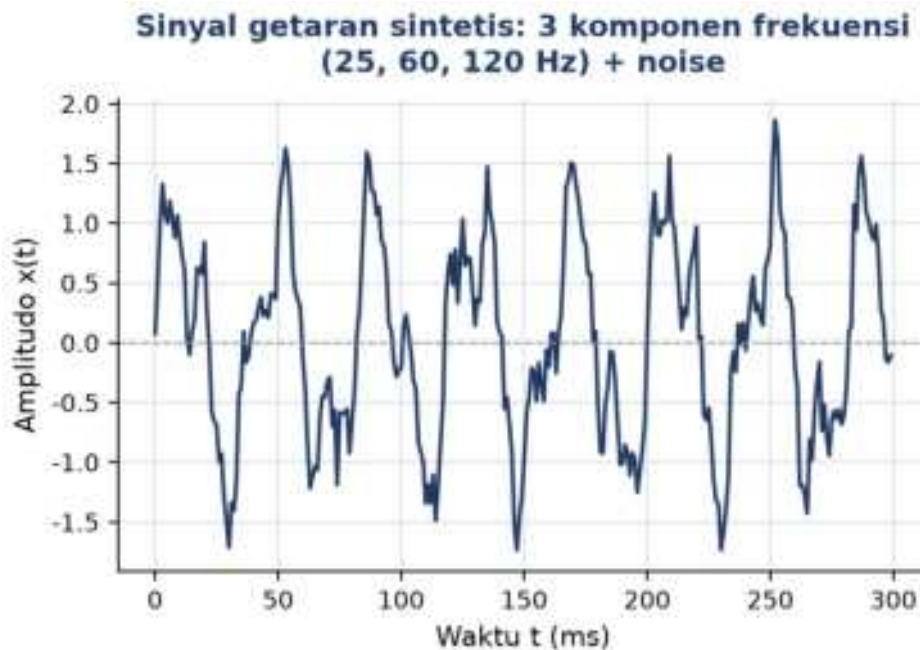
Time series adalah urutan observasi terhadap indeks waktu $t = 1, 2, \dots, N$, dinotasikan $\{x_t\}$. Berbeda dengan data biasa (misalnya tabel pelanggan), urutan dalam time series adalah informasi: mengacak urutannya menghancurkan makna. Setiap titik x_t berhubungan dengan x_{t-1} , x_{t-2} , dan seterusnya; sifat inilah yang disebut temporal dependency.

$$x_t \approx f(x_{t-1}, x_{t-2}, \dots)$$

Sensor sebagai Sumber Data

Mesin industri modern memiliki ratusan sensor: akselerometer (getaran), termokopel (suhu), strain gauge (regangan), sensor arus motor, dan sensor tekanan oli. Semuanya menghasilkan time series. Frekuensi sampling menentukan resolusi data: getaran high-speed bisa memerlukan 10 kHz, suhu cukup 1 Hz, dan kondisi proses cukup satu sampel per menit. Periode sampling dirumuskan $\Delta t = 1/f_s$. Gambar 2 menunjukkan contoh sinyal getaran sintesis hasil superposisi tiga komponen frekuensi ditambah noise, yaitu pola yang tipikal terekam akselerometer pada motor industri.

Pemilihan jenis sensor juga mempertimbangkan aspek ekonomi: akselerometer piezoelectric presisi tinggi untuk vibration monitoring bisa berharga jutaan rupiah per unit ditambah biaya instalasi dan kalibrasi berkala, sedangkan termokopel tipe K jauh lebih murah dan tahan lingkungan panas ekstrem. Pada sistem dengan puluhan hingga ratusan titik ukur, keputusan menempatkan sensor mahal hanya pada komponen kritis seperti bearing utama dan gearbox, sementara komponen sekunder cukup dipantau tidak langsung, adalah trade-off ekonomi khas yang akan terus muncul sepanjang mata kuliah ini.



Gambar 2. Sinyal getaran sintetis: superposisi tiga komponen frekuensi (25, 60, 120 Hz) ditambah noise Gaussian.

Pola Periodik

Banyak sinyal mesin bersifat periodik, seperti getaran berulang setiap rotasi poros, suhu naik-turun mengikuti shift kerja, beban bergeser sesuai musim produksi. Memahami periode dan amplitudo sinyal periodik $x(t) = A \cdot \sin(2\pi ft + \phi)$ adalah dasar untuk semua analisis lanjutan: FFT, deteksi anomali, dan prediksi. Pada Gambar 2 komponen 25 Hz terlihat sebagai pola dominan yang berulang setiap 40 ms, dengan dua komponen frekuensi lebih tinggi (60 dan 120 Hz) menumpangi bentuknya.

Noise dan Sinyal Tercampur

Pengukuran sensor selalu mengandung noise: getaran liar, gangguan elektromagnetik, dan drift sensor. Kemampuan memisahkan sinyal sejati dari noise adalah keterampilan inti seorang engineer data. Rasio sinyal terhadap noise, $SNR = 10 \cdot \log_{10}(P_{\text{sinyal}}/P_{\text{noise}})$, menentukan apakah pola dapat dideteksi atau tertutup derau.

Prinsip utama: Time series bukan sekadar deretan angka; ia adalah jejak digital dari fenomena fisik yang sedang berjalan. Setiap angka adalah potret kondisi mesin pada satu waktu; dengan menyusunnya berurutan diperoleh cerita tentang perilaku mesin, dan dari cerita itulah keputusan teknis diambil: kapan service, kapan mengganti spare part, kapan melakukan optimasi.

KOMPONEN SINYAL TIME SERIES

Setiap deret waktu dari sensor industri pada dasarnya adalah jumlah beberapa komponen tersembunyi. Memahami komponen-komponen ini penting agar data tidak salah dibaca: lonjakan getaran tiba-tiba bisa berarti kerusakan, atau hanya pola harian yang muncul setiap pergantian shift pagi. Model aditifnya, divisualisasikan pada Gambar 3, adalah:

$$y_t = T_t + S_t + C_t + \varepsilon_t \quad (\text{observasi} = \text{tren} + \text{musiman} + \text{siklik} + \text{noise})$$

- **Tren (Tt):** pergerakan jangka panjang yang tidak berulang (naik, turun, atau datar). Pada bearing, getaran rata-rata yang naik selama berbulan-bulan menandakan aus progresif. Tren bisa linear, eksponensial (kerusakan cepat), atau logaritmik (degradasi melambat). Panel kedua Gambar 3 menunjukkan tren linear naik pada deret harian sintetis.
- **Musiman (St):** pola berulang berperiode tetap yang terkait kalender/jadwal: konsumsi listrik tinggi pukul 08.00–17.00 (harian), produksi turun tiap akhir pekan (mingguan), maintenance terjadwal awal bulan (bulanan). Berlaku $S_{t+s} = S_t$ dengan periode s diketahui dari konteks operasional; lihat osilasi berperiode 7 hari pada panel ketiga Gambar 3.
- **Siklik (Ct):** pola berulang tanpa periode tetap, seperti fluktuasi ekonomi dan siklus permintaan industri. Tidak terikat kalender sehingga lebih sulit ditangkap; sering digabung dengan tren saat dekomposisi.
- **Residual/Noise (εt):** sisa acak setelah tren, musiman, dan siklik dipisahkan; idealnya white noise $\varepsilon_t \sim N(0, \sigma^2)$: mean nol, varians konstan, tanpa autokorelasi. Anomali nyata muncul sebagai residual besar yang menyimpang dari distribusi normalnya (lihat panel terbawah Gambar 3).

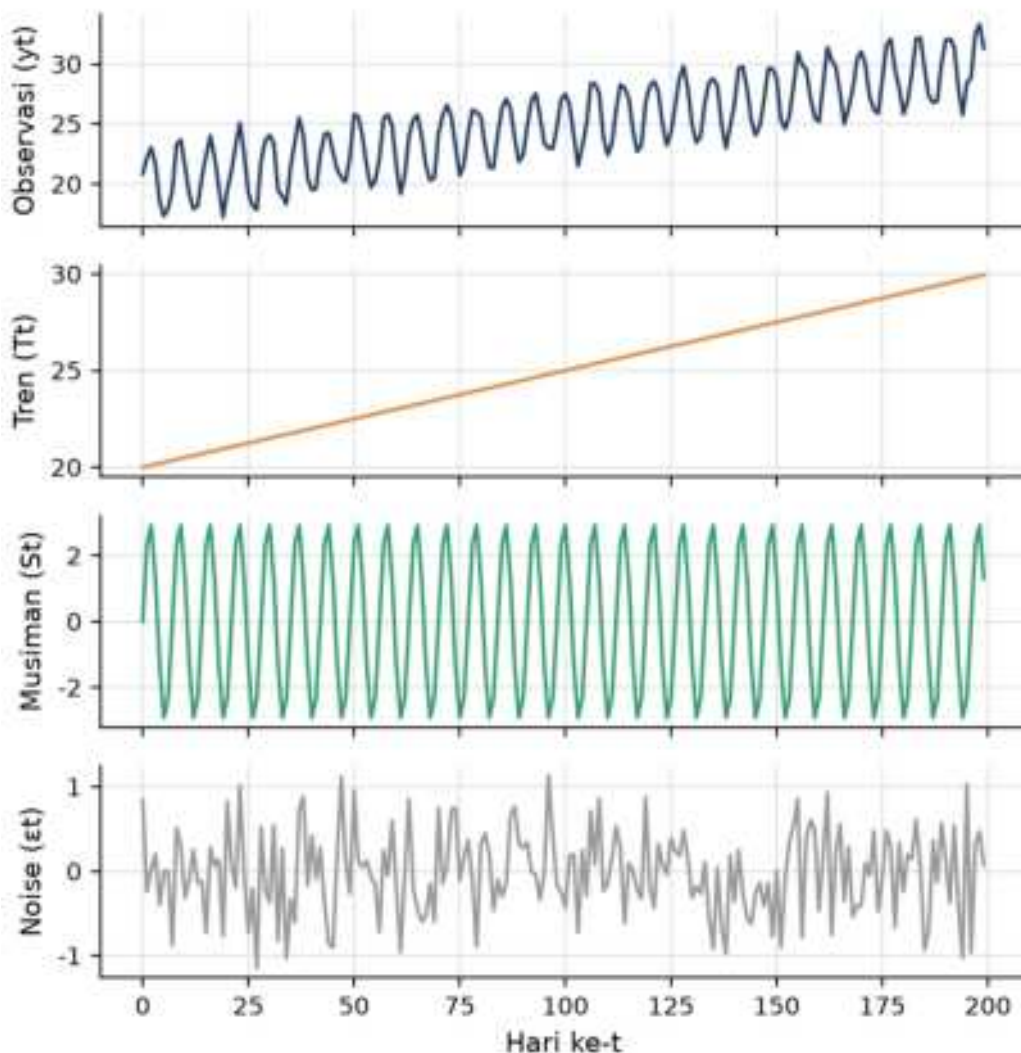
Tabel 1 merangkum keempat komponen tersebut sekaligus, lengkap dengan periode khasnya, contoh kemunculan di industri, dan metode deteksi yang lazim dipakai untuk masing-masing.

Kesalahan yang sering terjadi di lapangan adalah menganggap kenaikan sebuah statistik sebagai anomali tunggal, padahal sesungguhnya gabungan dua komponen berbeda yang kebetulan naik bersamaan. Misalnya, RMS getaran yang meningkat menjelang akhir shift malam bisa jadi bukan indikasi kerusakan, melainkan superposisi tren aus bearing dengan pola musiman beban produksi yang memang lebih tinggi pada jam tersebut. Tanpa dekomposisi, engineer berisiko menjadwalkan maintenance yang tidak perlu, atau mengabaikan tren aus yang sesungguhnya sedang berlangsung.

Tabel 1. Empat komponen sinyal time series beserta periode, contoh industri, dan cara deteksinya.

Komponen	Periode	Contoh Industri	Cara Deteksi
Tren	Tidak ada (monoton)	Aus bearing, drift sensor, korosi pipa	Regresi linear, moving average
Musiman	Tetap (24 j, 7 h, 30 h, 365 h)	Shift kerja, weekend, musim, jadwal	FFT, dekomposisi STL, plot ACF
Siklik	Tidak tetap (> 1 tahun)	Ekonomi, permintaan pasar	Analisis spektral, model ARIMA
Noise	Acak	Kuantisasi, EMI, fluktuasi proses	Analisis residual, uji normalitas

**Dekomposisi aditif $y_t = T_t + S_t + \varepsilon_t$
pada deret harian sintetis**



Gambar 3. Dekomposisi aditif deret harian sintetis menjadi observasi, tren, musiman, dan noise.

Diagnostik visual cepat: Plot deret terhadap waktu, lalu perhatikan: (1) kemiringan jangka panjang = tren; (2) pola berulang berperiode jelas = musiman; (3) fluktuasi besar tanpa

periode = siklik atau anomali; (4) jitter halus konstan = noise. Pemeriksaan visual ini harus menjadi kebiasaan setiap menerima dataset baru.

Contoh penerapan: Pada studi kasus vibration monitoring motor conveyor pabrik semen, plot RMS harian menunjukkan kenaikan tren linear sekitar 15% selama tiga bulan, indikasi aus bearing yang progresif, namun tren ini semula tertutupi oleh osilasi musiman harian akibat pergantian shift kerja pagi-siang-malam. Baru setelah data di-smoothing dengan moving average 24 jam (menghilangkan komponen musiman), tren aus tersebut terlihat jelas dan cukup dini untuk dijadwalkan penggantian bearing sebelum kegagalan, bukan sesudahnya.

STATISTIK DASAR TIME SERIES

Sebagian besar diagnosa kondisi mesin masih dapat dilakukan dengan statistik deskriptif sederhana, jauh sebelum model machine learning diperlukan. Lima metrik fundamental berikut adalah dasar seluruh ekstraksi fitur domain waktu yang akan dibahas kembali pada Pertemuan 6. Gambar 4 mengilustrasikan bagaimana sebuah impulsive event (mis. spalling bearing) menaikkan Peak dan Crest Factor secara tajam sementara RMS relatif tidak berubah; inilah alasan Crest Factor dipakai khusus untuk deteksi dini kerusakan bearing.

- **Mean (μ):** rata-rata aritmatik $\mu = (1/N) \cdot \sum x_i$. Untuk sinyal AC seperti getaran (berosilasi di sekitar nol) mean mendekati nol, sehingga pergeseran mean menandakan DC offset atau drift sensor yang perlu dikalibrasi. Untuk sinyal DC seperti suhu, mean mencerminkan level operasional.
- **RMS:** akar dari rata-rata kuadrat, $RMS = \sqrt{(1/N) \cdot \sum x_i^2}$, yaitu ukuran energi efektif sinyal yang tidak nol meski sinyal berosilasi. Untuk monitoring getaran, RMS adalah metrik utama: ISO 10816 mendefinisikan kelas kondisi mesin berdasarkan RMS getaran (garis putus-putus hijau pada Gambar 4).
- **Standar Deviasi (σ):** ukuran sebaran data terhadap mean; untuk sinyal ber-mean nol $\sigma \approx RMS$. Nilai σ tinggi pada konteks yang biasanya stabil = variabilitas tidak normal, indikator awal kegagalan. Aturan praktis: residual melebihi 3σ dianggap outlier.
- **Peak & Crest Factor:** Peak = nilai absolut maksimum dalam jendela waktu (titik oranye pada Gambar 4); Crest Factor $CF = Peak/RMS$ sensitif terhadap impulsive event seperti benturan atau spalling bearing. Sinusoidal murni memiliki $CF = \sqrt{2} \approx 1,41$; $CF > 4$ menandakan lonjakan abnormal meskipun RMS keseluruhan masih dalam batas.

Contoh angka: Sebuah motor 15 kW yang sehat menghasilkan sinyal getaran dengan $RMS \approx 0,45$ mm/s dan $Peak \approx 0,9$ mm/s, sehingga $Crest\ Factor \approx 2,0$, khas untuk sinyal harmonik normal tanpa impulsive fault. Ketika bearing mulai spalling, $Peak$ melonjak menjadi $\approx 3,2$ mm/s sementara RMS hanya naik ke $\approx 0,52$ mm/s; $Crest\ Factor$ pun melonjak ke $\approx 6,2$, jauh di atas ambang batas 4. Inilah sebabnya $Crest\ Factor$ jauh lebih sensitif sebagai indikator dini kerusakan bearing dibanding RMS saja, yang justru menjadi metrik utama menurut ISO 10816.

Kelima statistik di atas dirangkum pada Tabel 2, dipetakan terhadap apa yang masing-masing sensitif deteksi, aplikasi tipikalnya, dan jenis kerusakan mesin yang paling relevan.

Dalam praktiknya kelima statistik ini jarang dipakai sendiri-sendiri; dashboard condition monitoring modern menghitung semuanya sekaligus pada setiap window data yang masuk, lalu menampilkannya sebagai trend chart berdampingan. Mean dan RMS memantau kondisi umum, sementara $Crest\ Factor$ menjadi early warning khusus untuk fault impulsive seperti spalling yang belum tentu terlihat dari RMS saja. Standar deviasi membantu membedakan mesin yang stabil dari mesin yang mulai tidak menentu, bahkan sebelum nilai rata-ratanya bergeser signifikan.

Tabel 2. Lima statistik dasar time series, sensitivitasnya, aplikasi, dan tipe kerusakan yang terdeteksi.

Statistik	Sensitif terhadap	Aplikasi	Tipe Kerusakan
Mean	DC offset, drift sensor	Kalibrasi, baseline	Drift sensor
RMS	Energi keseluruhan sinyal	Kelas getaran ISO 10816	Unbalance, misalignment
Std (σ)	Variabilitas, sebaran	Deteksi anomali (aturan 3σ)	Kondisi tidak stabil
Peak	Lonjakan tertinggi	Stres mekanik maksimum	Shock, impact event
Crest Factor	Impulsiveness sinyal	Diagnosa bearing	Spalling bearing, gigi gear

Untuk melihat bagaimana kelima statistik ini bereaksi terhadap kejadian nyata di lapangan, perhatikan simulasi pada Gambar 4: satu impulsive event tunggal yang mewakili awal spalling pada elemen bearing disisipkan ke tengah sinyal getaran yang sebelumnya stabil. Perhatikan bagaimana $Peak$ melonjak drastis pada saat kejadian tersebut, sementara RMS keseluruhan sinyal nyaris tidak berubah karena durasi impulsive event yang sangat singkat dibanding total durasi jendela pengukuran. Fenomena inilah yang membuat RMS saja sering kali terlambat mendeteksi kerusakan tahap awal, sedangkan $Crest\ Factor$ yang membandingkan $Peak$ terhadap RMS jauh lebih sensitif terhadap sinyal impulsive semacam ini.



Gambar 4. Peak, RMS, dan Crest Factor pada sinyal getaran dengan satu impulsive event (simulasi spalling bearing).

Window sliding untuk monitoring real-time: Hitung RMS, σ , dan CF pada window bergerak (misal 1 detik = 1024 sampel pada 1 kHz), lalu plot deret RMS bergulir terhadap waktu untuk melihat kapan kondisi mesin mulai memburuk. Window terlalu kecil = noisy; terlalu besar = lambat merespons. Praktik umum: 1 detik untuk vibration high-speed, 10 menit untuk suhu proses.

SAMPLING, FREKUENSI & TEOREMA NYQUIST

Time series dari sensor digital adalah diskretisasi sinyal kontinu di dunia fisik. Seberapa cepat sinyal harus "dipotret"? Jawabannya diberikan Teorema Nyquist-Shannon: untuk merekonstruksi sinyal berfrekuensi maksimum f_{maks} tanpa distorsi, sampling rate harus minimal dua kalinya. Memilih sampling rate yang salah berarti data sudah cacat sejak akuisisi, dan tidak ada algoritma pada pertemuan berikutnya yang mampu memperbaikinya. Gambar 5 mendemonstrasikan konsekuensinya secara visual.

$$f_s \geq 2 \cdot f_{maks}$$

- **Sampling rate (f_s):** banyaknya sampel per detik (Hz). Contoh: sensor getaran 10 kHz mengambil 10.000 sampel per detik; periode sampling $\Delta t = 1/f_s$. Sampling rate menentukan resolusi waktu dan frekuensi maksimum yang dapat dideteksi.
- **Frekuensi Nyquist (f_N):** $f_N = f_s/2$, yaitu frekuensi tertinggi yang masih dapat direpresentasikan tanpa kehilangan informasi. Sampling 1 kHz hanya dapat merekonstruksi sinyal sampai 500 Hz.

- **Aliasing:** distorsi ketika komponen di atas f_N tetap masuk ke ADC: pada Gambar 5, sinyal asli 50 Hz yang di-sampling pada 60 Hz (under-Nyquist) tampak sebagai alias 10 Hz: garis oranye yang menghubungkan titik-titik sampel mengikuti gelombang palsu berfrekuensi jauh lebih rendah dari sinyal aslinya, mengikuti $f_{alias} = |f - k \cdot f_s|$. Pencegahan: pasang anti-aliasing filter (low-pass analog) sebelum ADC dengan cut-off $< f_N$.
- **Oversampling:** sampling jauh di atas Nyquist ($5\text{--}10 \times f_{maks}$). Manfaat: filter anti-aliasing lebih mudah dirancang, noise kuantisasi tersebar lalu dapat difilter, dan tersedia ruang untuk decimation. Biayanya: data lebih besar, storage dan processing lebih berat; di sinilah pertimbangan ekonomi Sub-CPMK 1.1 berperan.

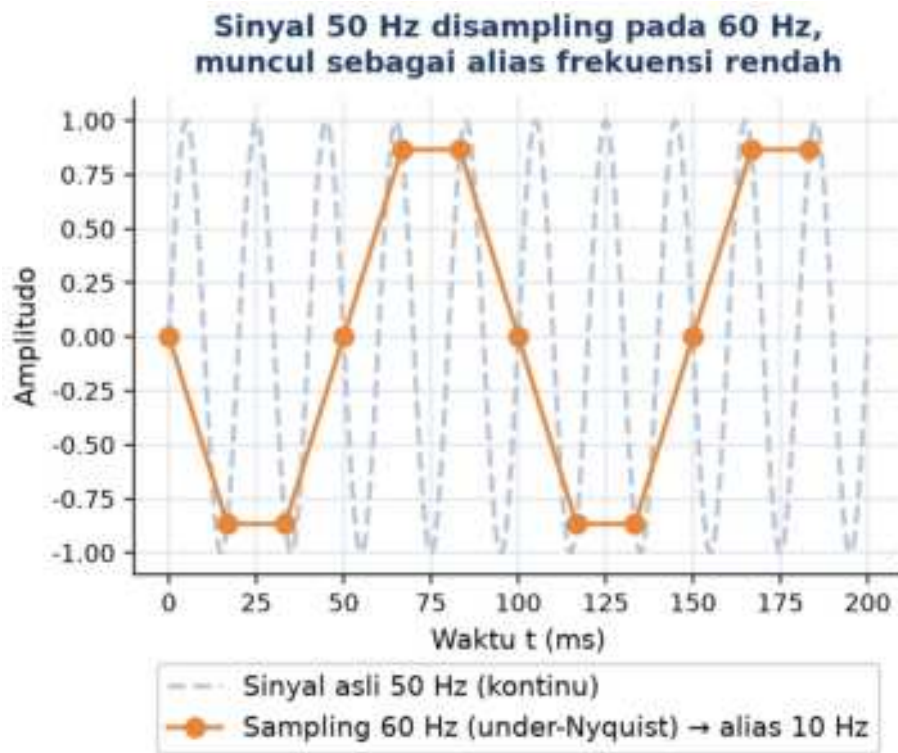
Studi kasus: Sebuah pabrik pernah men-sampling getaran motor pada 2 kHz karena mengasumsikan frekuensi tertinggi yang relevan hanya 1 kHz; belakangan diketahui bearing fault BPFO mesin tersebut sebenarnya berada di 1,4 kHz, DI ATAS frekuensi Nyquist 1 kHz. Akibatnya sinyal fault teralias dan tidak pernah terdeteksi selama delapan bulan hingga terjadi kegagalan katastropik. Setelah sampling rate dinaikkan ke 10 kHz (margin sekitar $7 \times$ di atas BPFO), fault serupa pada mesin sejenis berhasil terdeteksi tiga minggu sebelum kegagalan pada siklus monitoring berikutnya.

Tabel 3 membandingkan rentang frekuensi sinyal dan sampling rate tipikal untuk lima kelompok aplikasi sensor industri yang paling umum dijumpai di lapangan.

Tabel 3. Rentang frekuensi sinyal dan sampling rate tipikal per aplikasi sensor industri.

Aplikasi Sensor	Frekuensi Sinyal	Sampling Rate Tipikal	Catatan
Suhu / tekanan	< 1 Hz	1–10 Hz	Lambat berubah, tanpa oversampling
Aliran (flow)	1–100 Hz	100 Hz – 1 kHz	Tergantung dinamika fluida
Getaran umum mesin	10 Hz – 5 kHz	10 kHz	Standar ISO 10816
Bearing high-speed	1–20 kHz	40–50 kHz	Untuk envelope analysis
Acoustic emission	50 kHz – 1 MHz	1–5 MHz	Mahal; khusus crack detection

Kolom terakhir Tabel 3 memberi catatan praktis, namun keputusan akhir tetap bergantung konteks operasional: pabrik dengan anggaran terbatas mungkin memilih titik terendah dari rentang yang direkomendasikan untuk menghemat biaya storage dan bandwidth jaringan, sementara aplikasi safety-critical seperti bearing high-speed pada turbin justru disarankan memilih titik tertinggi demi margin keamanan ekstra. Perbandingan biaya-manfaat semacam ini adalah inti dari Sub-CPMK 1.1 pertemuan ini: sampling rate bukan sekadar parameter teknis, melainkan keputusan bisnis yang memengaruhi total cost of ownership sistem monitoring.



Gambar 5. Sinyal 50 Hz yang di-sampling pada 60 Hz (under-Nyquist) tampak sebagai alias berfrekuensi 10 Hz.

Cara cepat memilih sampling rate: (1) Identifikasi frekuensi tertinggi yang relevan untuk diagnosa, biasanya characteristic frequency komponen mesin (BPFI/BPFO bearing, gear mesh); (2) kalikan minimal 5 sebagai margin keamanan; (3) pasang anti-aliasing filter analog; (4) verifikasi dengan FFT pada data contoh, pastikan tidak ada peak mencurigakan di dekat f_N .

ANALOGI KEHIDUPAN SEHARI-HARI

Enam analogi berikut (Gambar 6) menghubungkan konsep sampling & statistik sinyal dengan pengalaman sehari-hari mahasiswa, mempermudah intuisi sebelum masuk ke rumus formal.

Analogi-analogi ini sengaja dipilih dari aktivitas yang hampir setiap mahasiswa alami sehari-hari, seperti memotret dengan HP atau mendengarkan musik streaming, karena konsep sampling rate dan aliasing sering terasa abstrak ketika pertama kali dipelajari lewat rumus semata. Dengan mengaitkan $f_N = f_s/2$ pada burst mode kamera, atau RMS pada normalisasi volume Spotify, mahasiswa dapat membangun intuisi fisik terlebih dahulu sebelum kembali ke notasi matematis yang lebih formal.



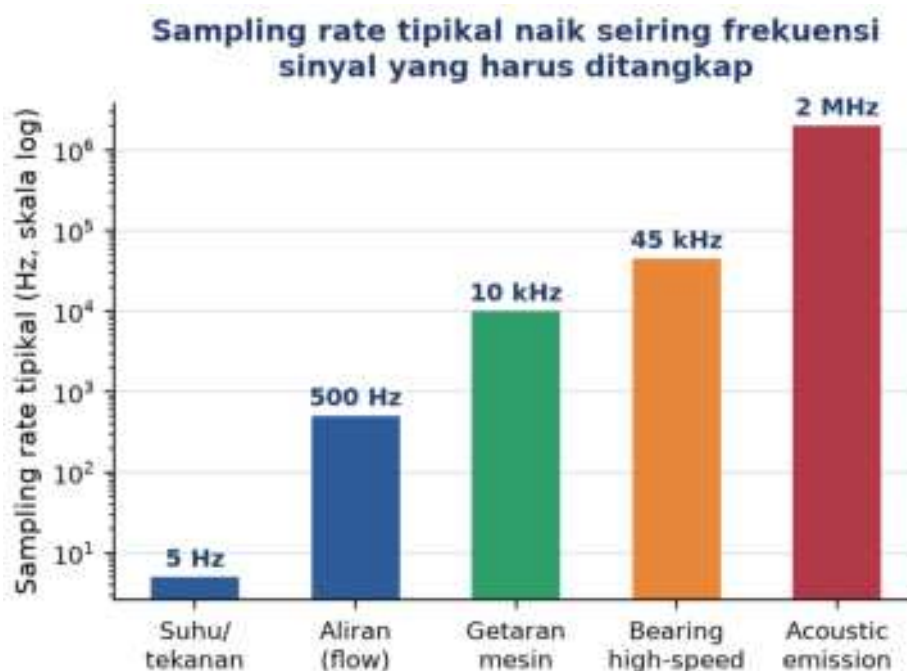
Gambar 6. Enam analogi kehidupan sehari-hari untuk konsep sampling, aliasing, dan statistik sinyal.

- **Burst mode kamera HP & sampling rate:** foto burst 10 fps = sampling 10 sampel/detik. Memotret burung yang mengepak 8 kali per detik butuh burst di atas 16 fps (Nyquist); pada 5 fps kepak tampak bergerak lambat atau terbalik; itulah aliasing (lihat Gambar 6, kartu pertama).
- **Lalu lintas tol & pola musiman:** sensor gerbang tol mencatat mobil per menit selama setahun: terlihat pola harian (puncak commuter pagi-sore), mingguan (Senin–Jumat vs akhir pekan), tahunan (lebaran, libur sekolah), plus tren pertumbuhan, yaitu semua komponen time series dalam satu dataset.
- **Spotify loudness & RMS:** Spotify menormalkan volume antar lagu memakai ukuran energi RMS menuju target -14 LUFS. Logika yang sama dipakai vibration monitoring: bukan nilai puncak sesaat, melainkan energi efektif.
- **Vinyl vs MP3 & kuantisasi:** CD memakai 44,1 kHz, sesuai Nyquist untuk batas pendengaran manusia ± 20 kHz dikali dua plus margin; perbedaan yang dirasakan audiophile adalah efek kuantisasi digital.
- **EKG detak jantung & crest factor:** EKG adalah time series tegangan jantung; detak normal berpuncak P-QRS-T tegas dengan rata-rata rendah = crest factor tinggi. Aritmia mengubah CF dramatis, persis seperti bearing yang mengalami spalling (bandingkan dengan Gambar 4).

- **Cuaca BMKG & multi-resolusi:** BMKG mengukur fenomena pada resolusi berbeda: petir (milidetik) butuh sensor cepat, pergeseran musim cukup data mingguan; setiap fenomena punya characteristic frequency dan sampling rate-nya sendiri.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

Sampling rate tipikal naik seiring frekuensi sinyal yang harus ditangkap tiap aplikasi sensor industri (Gambar 7, skala logaritmik), mulai dari orde Hz untuk suhu/tekanan hingga orde MHz untuk acoustic emission.



Gambar 7. Sampling rate tipikal per aplikasi sensor industri (skala logaritmik).

- **Vibration monitoring motor:** akselerometer pada housing motor, sampling 10 kHz (menangkap sampai 5 kHz), cukup untuk imbalance (1× rpm), misalignment (2× rpm), dan bearing fault. Statistik per detik: RMS, peak, CF; tren RMS naik 7 hari berturut-turut memicu inspeksi.
- **Kontrol suhu proses:** termokopel oven mencatat suhu 1 Hz (suhu lambat berubah): terlihat osilasi kecil kontroler PID, pola lingkungan harian, dan event ramp-up/down. Rolling mean & std 1 menit menjadi baseline; deviasi memicu alarm.
- **Power quality energi:** sensor arus-tegangan motor 3 fasa, 25,6 kHz: fundamental 50 Hz plus harmonik. $THD = \sqrt{(\sum V_n^2)/V_1}$ di atas 5% berarti kualitas daya buruk sehingga motor cepat panas dan efisiensi turun.

- **SCADA & IoT multi-sensor:** puluhan PLC dan ratusan sensor dengan sampling rate berbeda; bandwidth terbatas → sampel tinggi di edge, hitung statistik (RMS, std), kirim hanya fitur ke cloud (edge → feature → cloud).

Kesalahan praktis yang sering terjadi: (1) sampling rate terlalu rendah → aliasing dan peak FFT palsu; (2) tanpa anti-aliasing filter → noise frekuensi tinggi masuk pita rendah; (3) time stamp tidak akurat → korelasi antar sensor keliru; (4) mengabaikan pola musiman → variasi normal dianggap anomali, alarm fatigue; (5) menyimpan raw data tanpa strategi agregasi → storage penuh tanpa insight.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada data sintesis yang meniru sensor industri. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib, scipy, statsmodels; notebook p1_pengantar_time_series.ipynb. Gambar 8 menunjukkan cuplikan Cell 3 yang menghitung lima statistik dasar.

Cell 1: Generate & Visualisasi Deret Waktu

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
np.random.seed(42)

# Simulasi vibration sensor pada motor - sampling 1 kHz, durasi 2 detik
fs = 1000                # Hz, sampling rate
T = 2.0                  # detik
N = int(fs * T)          # jumlah sampel
t = np.linspace(0, T, N, endpoint=False)

# Sinyal getaran: 3 komponen frekuensi + noise gaussian
f1, f2, f3 = 25, 60, 120 # Hz
A1, A2, A3 = 1.0, 0.5, 0.25
x = (A1*np.sin(2*np.pi*f1*t) + A2*np.sin(2*np.pi*f2*t)
     + A3*np.sin(2*np.pi*f3*t) + np.random.normal(0, 0.15, N))
plt.plot(t[200:], x[200:]); plt.xlabel('t (s)'); plt.ylabel('amplitudo');
plt.show()
```

Cell 2: Efek Sampling Rate & Aliasing

```
f_signal = 50            # Hz, sinyal asli
T = 0.1                 # detik (5 periode)
t_cont = np.linspace(0, T, 10000)
sig_cont = np.sin(2*np.pi*f_signal*t_cont)

# Tiga skenario: under-sampled, tepat Nyquist, oversampled
for fs_i, lbl in [(60, 'Under (60 Hz) -> ALIASING'),
                  (100, 'Tepat Nyquist (100 Hz)'),
                  (500, 'Oversampled (500 Hz)')]:
    n_i = int(fs_i * T)
    t_i = np.linspace(0, T, n_i, endpoint=False)
    plt.plot(t_i, np.sin(2*np.pi*f_signal*t_i), 'o-', label=lbl)
plt.plot(t_cont, sig_cont, 'k--', alpha=.4, label='kontinu'); plt.legend();
plt.show()
```


Cell 3: Lima Statistik Dasar

```
mean_val = np.mean(x)
rms_val = np.sqrt(np.mean(x**2))
std_val = np.std(x)
peak_val = np.max(np.abs(x))
cf_val = peak_val / rms_val
print(f"Mean : {mean_val:8.4f} (AC signal -> mendekati 0)")
print(f"RMS : {rms_val:8.4f} (energi efektif; basis ISO 10816)")
print(f"Std : {std_val:8.4f} (mean 0 -> mendekati RMS)")
print(f"Peak : {peak_val:8.4f}")
print(f"CF : {cf_val:8.4f} (sinus murni 1.41; >4 = impulsif)")
```



Gambar 8. Cuplikan Cell 3: perhitungan lima statistik dasar sinyal dengan NumPy.

Cell 4: Dekomposisi Visual Komponen

```
# Deret harian 1 tahun: tren + musiman mingguan + noise
N = 365; t = np.arange(N)
trend = 0.05 * t
seasonal = 3.0 * np.sin(2*np.pi*t/7)
noise = np.random.normal(0, 0.5, N)
y = 20 + trend + seasonal + noise
dates = pd.date_range('2025-01-01', periods=N, freq='D')
series = pd.Series(y, index=dates, name='sensor_harian')
fig, ax = plt.subplots(4, 1, sharex=True, figsize=(9, 7))
for a, (nm, v) in zip(ax, [('observasi', y), ('tren', 20+trend),
                          ('musiman', seasonal), ('noise', noise)]):
    a.plot(dates, v); a.set_ylabel(nm)
plt.tight_layout(); plt.show()
```

TUGAS PERTEMUAN 1

Tugas dikerjakan melalui halaman LMS interaktif (tab Tugas) dan divalidasi otomatis oleh server. Struktur: 10 soal pilihan ganda (@1 poin), 10 soal komputasi easy-medium (@2 poin), dan 5 soal komputasi hard (@4 poin), dengan total 50 poin. Soal komputasi

dikerjakan di Jupyter Notebook lalu kode ditempel ke LMS untuk dijalankan dan dinilai. Distribusi poin ditunjukkan pada Gambar 9. Teks soal di bawah ini disalin persis dari halaman LMS (tab Tugas), termasuk seluruh opsi jawaban dan hint komputasi.



Gambar 9. Distribusi 50 poin Tugas Pertemuan 1 berdasarkan tipe soal.

A. Pilihan Ganda (10 × 1 poin)

- **MC01:** Karakteristik utama yang membedakan time series dari data tabular biasa adalah...
 - (A) Time series selalu memiliki lebih banyak baris data
 - (B) Urutan observasi terhadap waktu adalah informasi yang harus dipertahankan
 - (C) Time series wajib mengandung kolom kategori
 - (D) Time series hanya berasal dari sensor industri
- **MC02:** Sebuah sensor mengambil data dengan sampling rate $f_s = 1000$ Hz. Periode samplingnya (Δt) adalah...
 - (A) 1 detik per sampel
 - (B) 1 ms per sampel ($\Delta t = 1/f_s = 0.001$ s)
 - (C) 1 μ s per sampel
 - (D) 1 menit per 1000 sampel
- **MC03:** Menurut Teorema Nyquist-Shannon, untuk merekonstruksi sebuah sinyal dengan frekuensi maksimum f_{maks} tanpa distorsi, sampling rate harus minimal...
 - (A) $f_s = f_{maks}$
 - (B) $f_s \geq 2 \cdot f_{maks}$
 - (C) $f_s = 0.5 \cdot f_{maks}$
 - (D) $f_s = (f_{maks})^2$

- **MC04:** Sebuah sinyal getaran 80 Hz disampling pada sampling rate 100 Hz. Yang akan terjadi adalah...
 - (A) Sinyal terekam dengan sempurna karena $f_s > f_{\text{signal}}$
 - (B) Aliasing: sinyal tampak sebagai frekuensi palsu (di bawah Nyquist)
 - (C) Amplitudo terlihat 2× lebih besar dari aslinya
 - (D) Phase shift 90° tanpa perubahan amplitudo
- **MC05:** Frekuensi Nyquist (f_N) untuk sampling rate 10 kHz adalah...
 - (A) 10 kHz
 - (B) 20 kHz
 - (C) 5 kHz ($f_N = f_s / 2$)
 - (D) 1 kHz
- **MC06:** Komponen time series yang memiliki periode TETAP dan terkait kalender/jadwal (misalnya pola harian shift kerja, mingguan weekend) disebut...
 - (A) Tren (Trend)
 - (B) Musiman (Seasonal)
 - (C) Siklik (Cyclic)
 - (D) Residual / Noise
- **MC07:** Untuk sinyal AC seperti getaran (yang berosilasi sekitar nol), nilai yang lebih bermakna sebagai ukuran "energi efektif" sinyal adalah...
 - (A) Mean (μ): karena sinyal AC mean-nya ≈ 0 , ini tidak informatif
 - (B) RMS (Root Mean Square): menangkap energi rata-rata
 - (C) Median: robust tapi tidak terkait energi
 - (D) Mode: tidak meaningful untuk sinyal kontinu
- **MC08:** Crest Factor ($CF = \text{peak}/\text{RMS}$) yang TINGGI (>4) pada sinyal vibration bearing menandakan...
 - (A) Bearing dalam kondisi normal: CF tinggi adalah baseline sehat
 - (B) Sensor butuh kalibrasi karena drift
 - (C) Adanya impulsive event seperti spalling pada raceway bearing
 - (D) Sinyal terlalu lemah untuk dianalisis
- **MC09:** Menurut ISO 10816 untuk machine class I (motor <15 kW), RMS getaran < 0.71 mm/s diklasifikasikan sebagai...
 - (A) Unsatisfactory: perlu monitoring intensif
 - (B) Unacceptable: stop produksi segera
 - (C) Baik (Good): kondisi mesin sehat
 - (D) Class boundary: tidak bisa dikategorikan

- **MC10:** Untuk monitoring vibration bearing high-speed yang dapat mencapai frekuensi sinyal 5 kHz, sampling rate yang direkomendasikan praktek industri (dengan margin keamanan) adalah...
 - (A) 1 kHz: cukup untuk overview
 - (B) 5 kHz: tepat di Nyquist boundary
 - (C) 25–50 kHz: oversampling 5–10× untuk margin & anti-aliasing filter design
 - (D) 1 MHz: lebih tinggi selalu lebih baik

B. Komputasi Easy–Medium (10 × 2 poin)

- **C1:** Hitung mean (rata-rata) dari deret x dataset referensi. Untuk sinyal AC murni mean ≈ 0 . Cetak nilai dengan `print(np.mean(x))`.
 - 💡 **HINT:** Mean dihitung dengan `np.mean()` atas deret x. Untuk sinyal AC murni nilainya mendekati nol.
- **C2:** Hitung RMS (Root Mean Square) dari x: $RMS = \sqrt{(1/N) \cdot \sum x^2}$. Untuk sinyal getaran ini, RMS adalah ukuran energi efektif. Cetak nilai RMS.
 - 💡 **HINT:** RMS = akar dari rata-rata kuadrat sinyal; gunakan `np.sqrt` dan `np.mean` pada `x**2`.
- **C3:** Hitung standar deviasi (σ) dari x menggunakan `np.std(x, ddof=1)` (sample std). Untuk sinyal AC dengan mean ≈ 0 , $\sigma \approx RMS$.
 - 💡 **HINT:** Gunakan `np.std(x, ddof=1)` untuk sample standard deviation; untuk mean ≈ 0 hasilnya mendekati RMS.
- **C4:** Hitung Peak (nilai absolut maksimum) dari x: $peak = \max(|x|)$. Peak menangkap lonjakan tertinggi sinyal.
 - 💡 **HINT:** Peak = nilai absolut maksimum; kombinasikan `np.abs` dengan `np.max` atas x.
- **C5:** Hitung Crest Factor ($CF = peak / RMS$) dari x. Sinyal sinusoidal murni: $CF = \sqrt{2} \approx 1.41$. Sinyal multi-frekuensi normal: $CF \approx 2-3$.
 - 💡 **HINT:** Crest Factor = Peak ÷ RMS; bagi hasil peak (soal C4) dengan hasil RMS (soal C2).
- **C6:** Hitung frekuensi Nyquist untuk sampling rate $f_s = 10000$ Hz. Cetak nilai dalam Hz: $f_N = f_s / 2$.
 - 💡 **HINT:** Frekuensi Nyquist = $f_s \div 2$. Substitusikan sampling rate dari soal.
- **C7:** Hitung periode sampling (Δt) dalam milisekon untuk sampling rate $f_s = 1000$ Hz. $\Delta t = 1/f_s$ dalam detik, kalikan 1000 untuk dapat ms.
 - 💡 **HINT:** $\Delta t = 1 \div f_s$ (detik); kalikan 1000 untuk milisekon. Masukkan f_s dari soal.
- **C8:** Hitung jumlah sampel total yang dihasilkan oleh sensor dengan sampling rate $f_s = 2000$ Hz selama durasi 5 detik. Rumus: $N = f_s \times T$.

- 💡 **HINT:** $N = f_s \times T$; kalikan sampling rate dengan durasi dari soal, lalu konversi ke integer.
- **C9:** Hitung RMS dari sebuah sinusoidal MURNI dengan amplitudo $A = 2.0$ (tanpa noise). Rumus teoritis: $RMS = A/\sqrt{2}$. Verifikasi dengan generate sinyal: $x = 2.0 \cdot \sin(2\pi \cdot 10 \cdot t)$ selama 1 detik dengan $f_s = 1000$.
 - 💡 **HINT:** RMS sinusoidal murni $= A \div \sqrt{2}$. Generate sinyal dengan `np.linspace` dan `np.sin`, lalu verifikasi dengan formula RMS numerik.
- **C10:** Hitung Signal-to-Noise Ratio (SNR) dalam dB dari sinyal dengan power signal $P_s = 100$ dan power noise $P_n = 1$. Rumus: $SNR_{dB} = 10 \cdot \log_{10}(P_s/P_n)$.
 - 💡 **HINT:** $SNR_{dB} = 10 \times \log_{10}(P_s \div P_n)$; pakai `np.log10` dengan power signal dan noise dari soal.

C. Komputasi Hard (5 × 4 poin)

- **C11:** Implementasikan perhitungan RMS teoritis dari sinyal multi-frekuensi berdasarkan amplitudonya (rumus Parseval). Untuk dataset referensi (3 sinusoid $A_1=1.0$, $A_2=0.5$, $A_3=0.25$ + noise $\sigma=0.15$), gunakan formula: $RMS_{total} = \sqrt{(\sum(A_i^2/2) + \sigma^2_{noise})}$. Bandingkan dengan RMS empiris (numerik) dari deret x . Cetak nilai RMS teoritis.
 - 💡 **HINT:** Tiap sinusoid menyumbang power $A_i^2/2$; jumlahkan semua kontribusi ditambah variansi noise (σ^2), lalu akarkan untuk RMS total. Bandingkan dengan RMS numerik `np.sqrt(np.mean(x**2))`.
- **C12:** Implementasikan demonstrasi aliasing dari awal. Generate sinyal 50 Hz murni dengan resolusi tinggi (5000 sampel/detik selama 0.1 detik) → kemudian sampel-ulang pada frekuensi under-Nyquist 60 Hz. Hitung frekuensi alias yang muncul menggunakan rumus: $f_{alias} = |f_{signal} - k \cdot f_s|$ untuk k integer terdekat. Untuk $f=50$ Hz, $f_s=60$ Hz, alias muncul di...
 - 💡 **HINT:** Generate sinyal kontinu beresolusi tinggi, lalu downsample ke f_s under-Nyquist. Frekuensi alias $= |f_{signal} - k \cdot f_s|$ dengan k integer terdekat (gunakan `round`); verifikasi posisi peak dengan FFT pada hasil downsample.
- **C13:** Implementasikan Rolling RMS dari awal (tanpa `pandas`) pada dataset referensi x . Window = 256 sampel. Untuk tiap posisi i : hitung RMS dari window $x[i:i+256]$. Cetak nilai maksimum dari rolling RMS (deteksi window dengan energi tertinggi).
 - 💡 **HINT:** Geser window selebar 256 sampel di sepanjang x (loop atau list comprehension), hitung RMS tiap window dengan `np.sqrt(np.mean)`, lalu ambil nilai maksimumnya. Aplikasi: deteksi window dengan event abnormal di vibration monitoring.

- **C14:** Implementasikan identifikasi periode sinyal periodik menggunakan autokorelasi (ACF) dari awal. Generate sinyal test: `np.random.seed(7); N=1000; sig = np.sin(2π·t/50) + 0.2·noise` dengan periode TRUE = 50. Hitung ACF normalized, cari indeks lag pertama setelah lag 5 dimana ACF mencapai puncak lokal; ini estimasi periode. Cetak nilai estimasi periode.
 - 💡 **HINT:** Center sinyal (kurangi mean), hitung autokorelasi dengan `np.correlate` mode 'full' lalu ambil sisi positif dan normalisasi terhadap lag 0. Cari puncak lokal pertama setelah lag 5 dengan membandingkan tiap `acf[i]` terhadap tetangganya; indeks itu adalah estimasi periode.
- **C15:** Hitung Total Harmonic Distortion (THD) dari dataset referensi. THD adalah rasio RMS semua harmonik (A2, A3) terhadap RMS fundamental (A1), dalam persen. Rumus: $THD\% = \sqrt{(\sum A_h^2/2)} / (A_1/\sqrt{2}) \times 100\%$ untuk $h \geq 2$. Cetak nilai THD dalam persen.
 - 💡 **HINT:** RMS fundamental = $A_1/\sqrt{2}$; RMS harmonik = $\sqrt{(\sum A_h^2/2)}$ untuk $h \geq 2$. $THD\% = (RMS \text{ harmonik} \div RMS \text{ fundamental}) \times 100$. Aplikasi: THD penting di power quality monitoring motor.

FORUM DISKUSI: STUDI KASUS

Rendi, junior engineer di pabrik komponen otomotif, ditugaskan menyiapkan sistem vibration monitoring pertama untuk 8 motor induksi 11 kW pada line CNC machining. Manajemen ingin mulai sederhana: baseline statistik time series plus alert, tanpa machine learning dulu. Karakteristik frekuensi yang harus terdeteksi: unbalance $1 \times rpm = 24 \text{ Hz}$, misalignment $2 \times rpm = 48 \text{ Hz}$, bearing fault $BPFI \approx 121 \text{ Hz}$ / $BPFO \approx 78 \text{ Hz}$, gear mesh sampai $\pm 1,5 \text{ kHz}$. DAQ mendukung maksimal 50 kHz (16-bit), tetapi penyimpanan terbatas: 15 hari raw di edge, setelah itu hanya agregat statistik harian.

- **Diskusi 1:** Berapa sampling rate yang direkomendasikan, dan mengapa? Pertimbangkan frekuensi maksimum yang harus terdeteksi, margin oversampling untuk desain anti-aliasing filter, serta trade-off penyimpanan dan bandwidth.
- **Diskusi 2:** Statistik time series mana yang wajib dihitung dan disimpan, pada window berapa lama? Timbang cakupan diagnostik (banyak metrik) versus kesederhanaan pemantauan harian.
- **Diskusi 3:** Rancang sistem alert rule-based berbasis ISO 10816 (threshold RMS untuk Class I: $< 0,71$ baik; $0,71\text{--}1,8$ satisfactory; $1,8\text{--}4,5$ unsatisfactory; $> 4,5$ unacceptable), crest factor sebagai indikator spalling, dan konsep persistence untuk mencegah false alarm dari spike sesaat.

Jawaban ditulis pada tab Forum LMS (minimal 30 kata per pertanyaan) dan akan tampil sebagai diskusi kelas.

DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. *IEEE Transactions on Knowledge and Data Engineering*, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. <https://doi.org/10.1609/aaai.v37i9.26317>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 2

Filter & Smoothing Time Series Data

Abstrak

Pertemuan ini membahas teknik filtering dan smoothing untuk data time series sensor industri yang selalu tercampur

Sub - CPMK

Sub-CPMK 1.2 – Mampu menganalisis kebutuhan sistem untuk efisiensi dan keberlanjutan

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-2.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan kedua ini melanjutkan pemahaman dasar time series dari Pertemuan 1 menuju keterampilan praktis pertama: membersihkan sinyal sensor dari noise sebelum dianalisis lebih lanjut. Setiap pengukuran sensor industri (getaran, suhu, arus, tekanan) selalu tercampur noise dan spike sporadis. Modul ini membahas empat teknik filter utama yaitu Moving Average, Exponential Moving Average (EMA), Savitzky-Golay, dan median filter, beserta trade-off masing-masing dan bagaimana menyusun pipeline denoising yang tidak merusak kandungan sinyal asli. Gambar 1 menunjukkan posisi Pertemuan 2 dalam keseluruhan alur mata kuliah.



Gambar 1. Posisi Pertemuan 2 (Filter & Smoothing) dalam alur mata kuliah Optimalisasi & Otomasi.

Materi mencakup teori sumber-sumber noise pada sensor industri, empat teknik filter time-domain beserta parameter dan trade-off-nya, pemilihan filter berdasarkan tipe noise, serta pipeline rekomendasi untuk vibration monitoring. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan latihan komputasi.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 1.2:** Mampu menganalisis kebutuhan sistem untuk efisiensi dan keberlanjutan, yaitu memilih teknik filter yang tepat berdasarkan trade-off komputasi, lag, dan preservasi sinyal untuk sistem monitoring yang berkelanjutan (CPMK 1).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan sumber-sumber noise pada sensor industri; menerapkan Moving Average dan EMA beserta pemilihan parameternya; menerapkan Savitzky-Golay dan median filter untuk preservasi

puncak dan penghapusan spike; serta menyusun pipeline filter yang tepat untuk vibration monitoring.

NOISE DALAM TIME SERIES

Setiap pengukuran sensor mengandung noise: model sinyal teramati adalah $x_{\text{obs}}(t) = s(t) + n(t)$, yaitu sinyal asli $s(t)$ ditambah noise $n(t)$. Sumber noise pada sensor industri beragam: thermal noise (Johnson noise) dari komponen elektronik, quantization noise akibat resolusi ADC terbatas, electromagnetic interference (EMI) dari motor dan inverter di sekitar sensor, mechanical noise dari getaran struktur, dan environmental drift akibat perubahan suhu lingkungan.

$$SNR = P_{\text{sinyal}} / P_{\text{noise}}$$

Trade-off smoothing: Tanpa filter, noise menimbulkan konsekuensi nyata: RMS bias ke atas (nilai RMS jadi lebih besar dari nilai sebenarnya), kesalahan deteksi peak, spektrum FFT menjadi berisik, dan autokorelasi menurun. Namun smoothing yang berlebihan sama berbahayanya dengan tidak sama sekali: kurang smoothing menyisakan noise yang mengaburkan analisis, sementara smoothing berlebihan justru menghapus kandungan sinyal yang bermakna (mis. impulsive event tanda kerusakan dini bearing). Kunci keseimbangan ini terletak pada pemilihan window size, $w_{\text{optimal}} = f(f_{\text{sinyal}}, f_{\text{noise}})$.

- **Causal (trailing):** filter hanya memakai data masa lalu (window trailing), cocok untuk real-time monitoring karena bisa dihitung saat data baru masuk, tetapi punya delay/lag terhadap sinyal asli.
- **Non-causal (centered):** filter memakai data masa lalu dan masa depan sekaligus (window centered), menghasilkan zero phase shift (tanpa lag) tetapi hanya bisa dihitung offline karena butuh data masa depan yang belum tersedia saat real-time.

MOVING AVERAGE

Moving Average (MA) adalah filter smoothing paling fundamental: merata-ratakan k titik data di sekitar setiap sampel. Ada dua varian utama tergantung posisi window terhadap titik saat ini.

$$\text{Centered: } y'_t = (1/k) \cdot \sum x_{t+i}, i=-k/2..k/2 \quad \text{Trailing: } y'_t = (1/k) \cdot \sum x_{t-i}, i=0..k-1$$

- **MA Centered:** rata-rata simetris kiri-kanan titik t ; zero phase shift tetapi butuh data masa depan sehingga hanya cocok analisis offline.

- **MA Trailing:** rata-rata hanya dari titik t ke belakang; causal dan real-time-ready, tetapi menimbulkan lag $\approx (k-1)/2$ sampel terhadap sinyal asli.
- **Pemilihan window:** window k dipilih mengikuti rasio sampling rate terhadap frekuensi cutoff yang diinginkan, $k \approx f_s/f_{\text{cutoff}}$. Window terlalu kecil menyisakan noise; window terlalu besar menumpulkan puncak dan menambah lag.

Kelemahan MA: bobot uniform pada seluruh titik window membuat frequency response-nya berbentuk sinc dengan sidelobe yang bocor (tidak benar-benar memotong frekuensi tinggi secara bersih), muncul boundary effect (NaN atau distorsi di tepi data), dan pada varian trailing menimbulkan lag. Implementasi praktis: ``np.convolve(x, np.ones(k)/k, mode='valid')`` atau ``pd.Series(x).rolling(k).mean()``. Trik praktis: pakai trailing MA $k=10-20$ sebagai baseline bergerak, lalu tandai deviasi lebih dari 3σ sebagai kandidat anomali untuk diinvestigasi.

Tabel 1 merangkum pemilihan window k terhadap sampling rate dan frekuensi cutoff efektif yang dihasilkan, beserta lag yang ditimbulkan pada varian trailing.

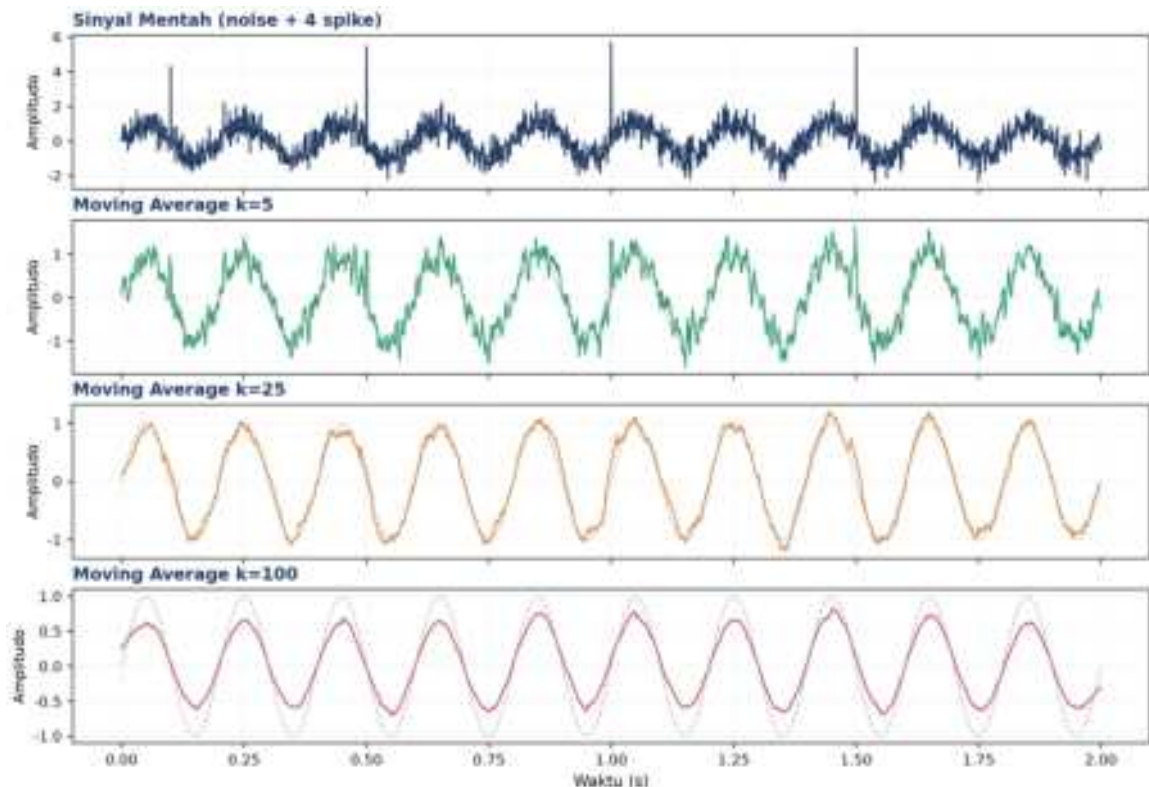
Tabel 1. Pemilihan window Moving Average terhadap sampling rate dan frekuensi cutoff efektif.

Sampling Rate	Window k	fcutoff Efektif	Lag (ms)
1000 Hz	5	~200 Hz	2 ms
1000 Hz	25	~40 Hz	12 ms
100 Hz	10	~10 Hz	50 ms
10 Hz	10	~1 Hz	500 ms
1 Hz	10	~0.1 Hz	5000 ms

Contoh Konkret: Pemilihan Window pada Deteksi Bearing Fault. Sebagai ilustrasi konkret pemilihan window, perhatikan accelerometer pada bearing motor 1500 rpm ($f_s=1000$ Hz) yang harus mendeteksi indikasi awal spalling pada frekuensi karakteristik bearing (BPFO) sekitar 120 Hz. Bila window k dipilih terlalu besar ($k=100$, fcutoff efektif ~10 Hz menurut Tabel 1), komponen 120 Hz justru ikut teredam habis sehingga sinyal kerusakan hilang dari hasil filter, kesalahan fatal untuk aplikasi predictive maintenance. Sebaliknya window $k=5$ (fcutoff efektif ~200 Hz) mempertahankan komponen 120 Hz tetapi hanya sedikit meredam noise broadband di sekitarnya. Pelajaran praktisnya: window Moving Average wajib dipilih berdasarkan frekuensi terendah yang ingin dipertahankan dalam analisis lanjutan, bukan sekadar mengejar tampilan sinyal yang tampak mulus secara visual.

Penanganan Boundary Effect: Boundary effect juga perlu ditangani secara eksplisit saat implementasi produksi. Pada mode ``valid`` (NumPy), output kehilangan $(k-1)$ titik di kedua ujung sinyal karena window tidak memiliki data tetangga yang cukup untuk dirata-ratakan;

pada mode `same`, titik-titik tepi diisi dengan hasil zero-padding yang dapat mendistorsi nilai dekat awal maupun akhir rekaman, sehingga menyesatkan bila kebetulan kejadian penting (mis. start-up mesin) berada tepat di tepi buffer data. Praktik yang disarankan untuk sistem streaming: rekam buffer tambahan sepanjang minimal $k/2$ sampel di kedua sisi window analisis sebelum menerapkan filter, lalu buang buffer tersebut setelah proses filtering selesai, sehingga seluruh titik yang dilaporkan ke sistem monitoring benar-benar bebas dari distorsi tepi.



Gambar 2. Sinyal noisy (noise + 4 spike) dan hasil Moving Average pada tiga level window $k=5$, 25, dan 100.

Gambar 2 memperlihatkan efek window size secara langsung: $k=5$ hanya sedikit meredam noise dan spike masih tampak jelas; $k=25$ menghasilkan kompromi yang baik antara smoothing dan preservasi bentuk gelombang; sedangkan $k=100$ sudah terlalu agresif, mengaburkan amplitudo asli sinyal sinusoidal 5 Hz.

EXPONENTIAL MOVING AVERAGE (EMA)

EMA menghitung rata-rata bergerak secara rekursif, memberi bobot lebih besar pada observasi terbaru dan bobot yang meluruh secara eksponensial untuk observasi lama.

$$EMA_t = \alpha \cdot x_t + (1-\alpha) \cdot EMA_{t-1}, \quad \alpha \in (0, 1]$$

Bobot pengamatan ke- i dari belakang adalah $w_i = \alpha \cdot (1-\alpha)^i$, sehingga effective window (jumlah sampel yang secara efektif berkontribusi) kira-kira $N \approx 2/\alpha - 1$. Pemilihan α : α kecil (mis. 0,05) menghasilkan smoothing agresif dengan lag besar; α besar (mis. 0,5) menghasilkan filter yang responsif tetapi kurang meredam noise. Aturan praktis untuk menyamakan EMA dengan MA window- N adalah $\alpha \approx 2/(N+1)$.

- **Inisialisasi:** $EMA_0 = x_0$ (paling umum), $EMA_0 = \text{mean}(x[:k])$ untuk mengurangi bias awal, atau memakai periode warm-up sebelum EMA dipakai untuk keputusan.
- **Implementasi pandas:** `pd.Series(x).ewm(alpha= $\alpha$ , adjust=False).mean()`, perhatikan parameter `adjust=False` wajib dipakai untuk mendapatkan bentuk rekursif standar; `adjust=True` (default pandas) akan menghitung ulang bobot dari seluruh history, bukan formula rekursif yang dimaksud.
- **EMA vs MA:** EMA punya lag lebih kecil dan tidak perlu menyimpan seluruh window (hanya nilai EMA sebelumnya), sementara MA punya frequency response yang lebih bersih (less sidelobe leakage). EWMA control chart ($|x - EMA| > 3 \cdot \sqrt{EMA_var}$) adalah teknik standar Statistical Process Control (SPC)/Six Sigma untuk deteksi pergeseran baseline.

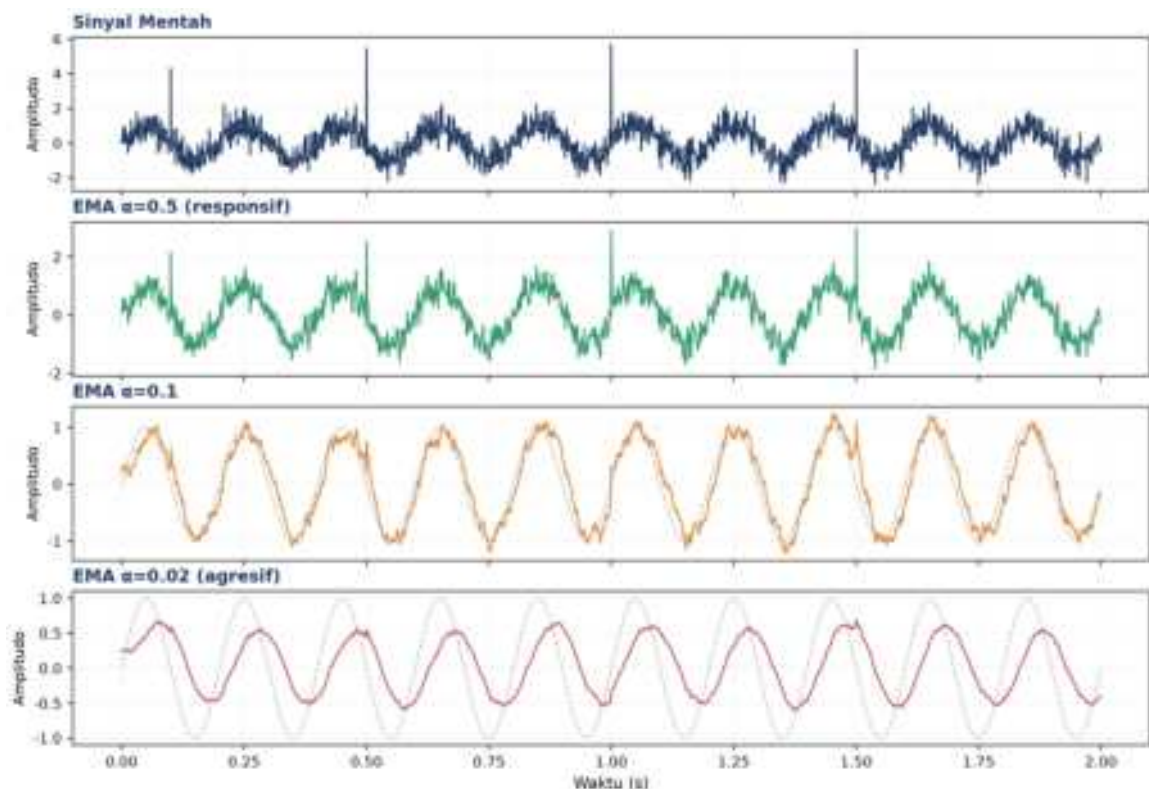
Tabel 2 merangkum karakter EMA pada beberapa nilai α beserta effective window dan use case yang sesuai.

Tabel 2. Pemilihan α EMA terhadap effective window dan karakter hasil filter.

α	Effective Window N	Karakter	Use Case
0,5	3	Sangat responsif, noise ikut lolos	Deteksi event cepat
0,3	5-6	Responsif, smoothing sedang	Alert real-time
0,1	19	Seimbang	Baseline umum
0,05	39	Smoothing kuat	Tren jangka menengah
0,01	199	Hampir flat	Baseline drift jangka panjang

Efisiensi Memori pada Sistem Monitoring Skala Besar: EMA banyak dipakai sebagai baseline adaptif pada sistem SCADA karena sifat rekursifnya sangat hemat memori: hanya satu nilai (EMA sebelumnya) yang perlu disimpan per channel sensor, berbeda dengan Moving Average yang butuh menyimpan seluruh k sampel terakhir dalam buffer. Untuk sistem monitoring dengan ratusan channel sensor berjalan pada embedded controller dengan RAM terbatas, perbedaan kebutuhan memori ini signifikan: MA $k=100$ pada 200 channel membutuhkan buffer 20.000 titik data, sedangkan EMA hanya membutuhkan 200 nilai skalar untuk mencapai smoothing setara.

Adaptive α : Pemilihan α juga dapat dilakukan secara adaptif (bukan konstan) berdasarkan kondisi operasi mesin: α diperbesar sementara saat start-up atau perubahan beban mendadak (agar EMA cepat mengejar perubahan level baseline yang legitimate), lalu dikembalikan ke nilai kecil semula saat kondisi steady-state tercapai. Teknik ini, dikenal sebagai variable-rate EMA atau adaptive exponential smoothing, mencegah trade-off klasik antara responsivitas dan stabilitas: pada α tetap kecil, EMA lambat mengikuti perubahan beban legitimate (false alarm delay); pada α tetap besar, EMA terlalu sensitif terhadap noise sesaat (false alarm rate tinggi).



Gambar 3. Sinyal noisy dan hasil EMA pada tiga nilai α : 0,5 (responsif), 0,1, dan 0,02 (agresif).

Perbandingan tiga level α : Terlihat pada Gambar 3 bahwa $\alpha=0,5$ nyaris tidak meredam noise (bahkan spike ikut lolos sebagian), $\alpha=0,1$ memberi smoothing yang seimbang, dan $\alpha=0,02$ menghasilkan kurva sangat halus namun dengan lag fase yang jelas terhadap sinyal asli garis putus-putus abu-abu.

SAVITZKY-GOLAY & MEDIAN FILTER

Savitzky-Golay (SG) bekerja dengan mem-fit polinomial orde p pada window $(2k+1)$ titik memakai least squares, lalu output-nya adalah nilai polinomial tersebut di titik pusat window; pendekatan ini menjaga bentuk puncak dan lembah jauh lebih baik dibanding MA. Median filter mengganti tiap titik dengan median dari window di sekitarnya, sangat efektif

untuk impulse noise/outlier karena sifatnya nonlinear (tidak ada sidelobe leakage seperti filter linear).

$$\text{Median: } y[t] = \text{median}(x[t-k : t+k+1])$$

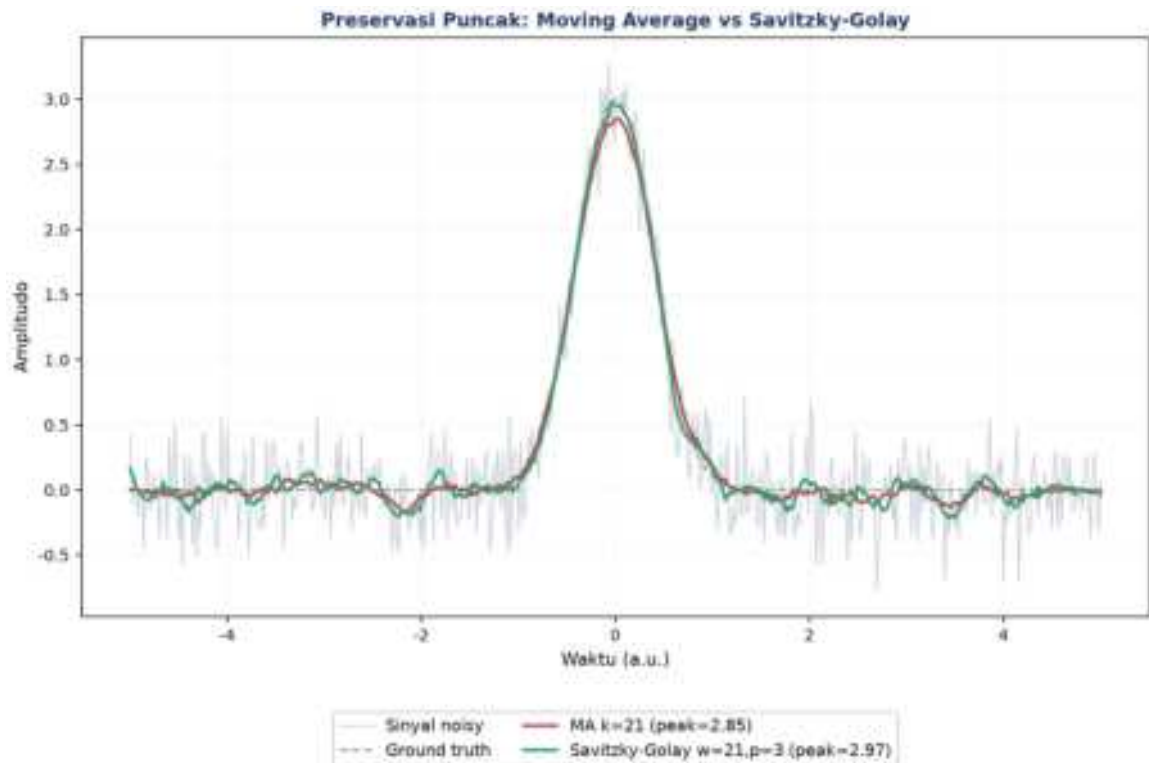
- **Parameter SG:** window w harus ganjil (rentang umum 5-101), orde polinomial p umumnya 2 (jaga smoothness), 3 (jaga curvature), atau 4-5 (jaga inflection point); syarat $p < w$. Titik awal yang aman: $w=11$, $p=2$.
- **Kapan pakai apa:** MA cocok untuk noise Gaussian dan kesederhanaan implementasi; EMA cocok untuk kebutuhan real-time adaptif; SG cocok saat preservasi puncak krusial (spektroskopi, analisis getaran); Median cocok khusus untuk outlier/spike/salt-and-pepper noise.
- **Chaining filter:** median dahulu untuk menghapus spike, baru Savitzky-Golay untuk smoothing yang tetap menjaga bentuk puncak: kombinasi ini menghasilkan sinyal bersih tanpa kehilangan informasi penting.

Tabel 3 membandingkan keempat filter dari sisi tipe noise yang paling cocok ditangani, kemampuan preservasi puncak, dan cocok-tidaknya untuk real-time.

Tabel 3. Perbandingan empat teknik filter terhadap tipe noise, preservasi puncak, dan kecocokan real-time.

Filter	Tipe Noise Terbaik	Preserve Peak?	Real-time?	Komputasi
Moving Average	Gaussian, broadband	Tidak	Ya (trailing)	$O(N)$
EMA	Gaussian, drift lambat	Tidak	Ya	$O(N)$
Savitzky-Golay	Gaussian, perlu jaga puncak	Ya	Tidak (perlu window penuh)	$O(N \cdot w \cdot p)$
Median	Impulsive/spike/outlier	Sebagian	Ya	$O(N \cdot w \cdot \log w)$

Constraint Komputasi pada Embedded Controller: Pemilihan filter yang tepat berdasarkan Tabel 3 juga harus mempertimbangkan constraint komputasi pada embedded controller: Savitzky-Golay dengan kompleksitas $O(N \cdot w \cdot p)$ menjadi kandidat termahal terutama saat window w besar dan orde polinomial p tinggi, sehingga pada sistem edge computing dengan sumber daya terbatas (mis. microcontroller ARM Cortex-M4 tanpa FPU), penerapan SG real-time pada banyak channel sensor sekaligus dapat menjadi bottleneck komputasi. Solusi praktis: precompute koefisien Savitzky-Golay untuk kombinasi (w, p) yang sudah ditentukan sebagai lookup table konstan, sehingga operasi filtering di runtime tereduksi menjadi dot product sederhana antara window data dengan koefisien yang sudah dihitung sebelumnya, alih-alih menyelesaikan sistem persamaan least squares setiap kali filter dipanggil pada tiap sampel baru.

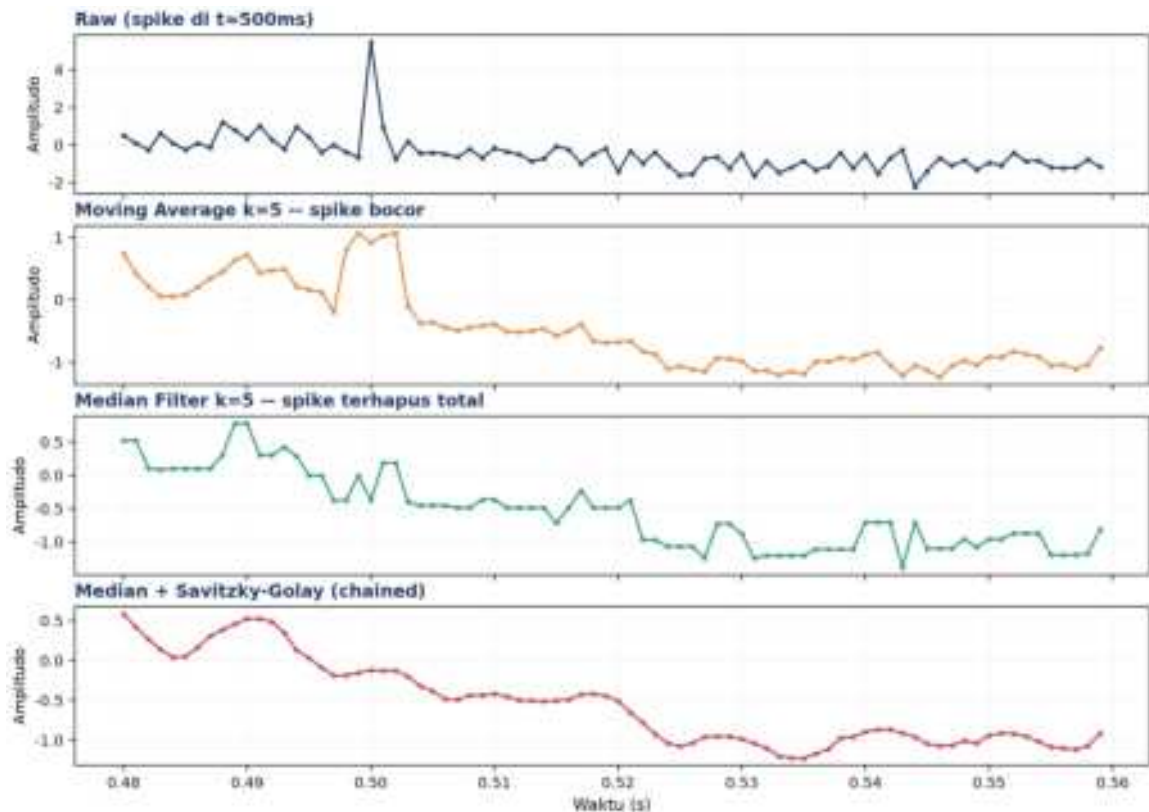


Gambar 4. Preservasi puncak pada sinyal Gaussian ber-noise: Moving Average $k=21$ vs Savitzky-Golay $w=21, p=3$.

Gambar 4 menunjukkan MA $k=21$ memangkas puncak sinyal dari nilai asli 3,0 menjadi hanya 2,85, sementara Savitzky-Golay $w=21, p=3$ mempertahankan puncak lebih dekat ke nilai asli (2,97), yaitu perbedaan krusial saat puncak tersebut merepresentasikan impulsive event nyata seperti dampak awal kerusakan bearing, bukan sekadar noise.

Studi Kasus: Spike EMI pada Monitoring Bearing. Studi kasus nyata: pada monitoring bearing dengan sampling rate 1000 Hz, spike EMI berdurasi singkat (1-3 sampel, setara 1-3 milidetik) kerap muncul akibat starting arus motor tetangga pada instalasi kelistrikan yang sama. Bila spike ini tidak dihapus sebelum perhitungan RMS, nilai RMS getaran bisa melonjak signifikan sesaat dan memicu false alarm pada sistem monitoring kondisi, padahal kondisi mesin sesungguhnya normal. Median filter dengan window kecil ($w=3-5$) sangat efektif menghapus spike berdurasi pendek ini tanpa mendistorsi bentuk gelombang getaran periodik yang justru ingin dipertahankan untuk analisis lanjutan.

Peringatan Window Terlalu Besar: Sebaliknya, hati-hati menerapkan median filter dengan window terlalu besar ($w>15$) pada sinyal getaran yang mengandung impact event nyata (bukan spike noise), seperti benturan awal spalling bearing: window besar berisiko menghapus impact event legitimate tersebut karena dianggap outlier statistik, padahal justru itulah informasi diagnostik paling berharga yang sedang dicari oleh sistem predictive maintenance.

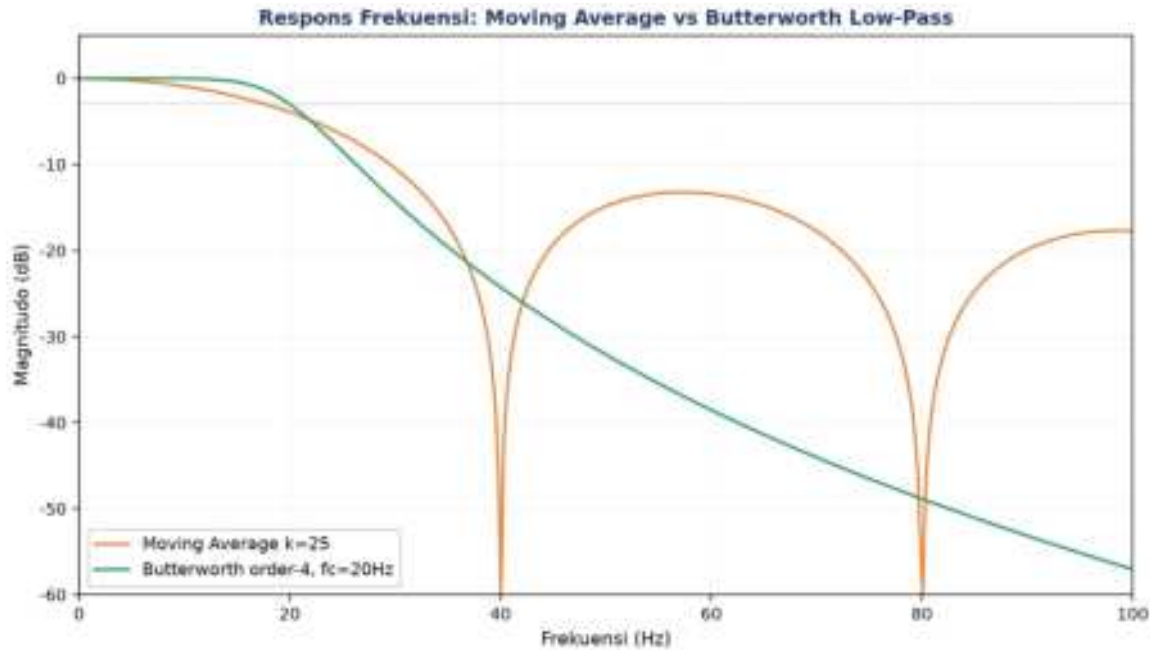


Gambar 5. Penghapusan spike: sinyal mentah, Moving Average $k=5$ (spike bocor), Median Filter $k=5$ (spike terhapus total), dan hasil chaining Median+Savitzky-Golay.

Linear vs nonlinear filtering: Perbandingan pada Gambar 5 memperjelas perbedaan fundamental filter linear vs nonlinear: MA $k=5$ hanya meredam sebagian amplitudo spike (spike bocor ke tetangganya), sementara median filter $k=5$ menghapus spike sepenuhnya dari sinyal. Panel terakhir menunjukkan hasil chaining median lalu Savitzky-Golay: sinyal bersih dari spike sekaligus halus tanpa distorsi bentuk gelombang.

Untuk memahami mengapa MA meninggalkan sidelobe (osilasi residual pada frekuensi tertentu) sementara filter Butterworth memberi roll-off yang lebih bersih, Gambar 6 membandingkan respons frekuensi keduanya secara eksplisit.

Implikasi pada Desain Anti-Aliasing: Perbedaan ini bukan sekadar teoretis: pada sistem anti-aliasing analog sebelum ADC, filter Butterworth (atau varian Chebyshev/Bessel) selalu dipilih dibanding pendekatan moving-average karena roll-off yang monoton menjamin komponen di atas Nyquist benar-benar teredam, sedangkan sidelobe MA berpotensi meloloskan sebagian energi frekuensi tinggi yang kemudian ter-alias kembali ke pita frekuensi rendah setelah sampling, mencemari hasil analisis FFT dengan komponen palsu yang tidak pernah benar-benar ada pada sinyal asli.



Gambar 6. Respons frekuensi Moving Average $k=25$ (sinc-like, sidelobe bocor) dibandingkan Butterworth low-pass order-4 $f_c=20\text{Hz}$ (roll-off bersih, monoton turun).

Moving Average menunjukkan pola sidelobe klasik filter sinc: magnitudo turun ke titik nol (null) di frekuensi tertentu (40 dan 80 Hz untuk $k=25$ pada $f_s=1000$ Hz) namun kembali naik di antaranya, artinya sebagian noise pada rentang frekuensi tersebut justru LOLOS tidak teredam. Butterworth order-4 menurun secara monoton dan konsisten setelah cutoff, sehingga jauh lebih andal dipakai sebagai anti-aliasing filter analog sebelum ADC.

ANALOGI KEHIDUPAN SEHARI-HARI

Enam analogi berikut menghubungkan konsep filter & smoothing dengan pengalaman sehari-hari mahasiswa, mempermudah intuisi sebelum masuk ke rumus formal.

- **Noise-Cancelling Headphone:** sinyal antiphase dijumlahkan ke sinyal noise ambien sehingga saling meniadakan (music = signal – noise); analog dengan filter yang mengurangi komponen tak diinginkan dari sinyal campuran.
- **Kacamata Renang Anti-Embun:** lapisan anti-kabut pada kacamata renang bertindak seperti low-pass filter bertekstur, yaitu trade-off antara kejernihan pandang (passband) dan proteksi dari embun (attenuation).
- **Autofokus Kamera HP:** perpindahan fokus kamera HP antar objek berlangsung mulus, bukan instan; ini persis perilaku EMA: $\text{focus}_t = \alpha \cdot \text{target} + (1-\alpha) \cdot \text{focus}_{t-1}$.
- **Shutter Speed Eksposur:** durasi shutter terbuka meng-integrasi cahaya sepanjang periode tersebut, konsep yang identik dengan window Moving Average yang mengintegrasi/merata-ratakan sinyal sepanjang periode window.

- **Photoshop Gaussian vs Median Blur:** Gaussian blur meratakan piksel secara linear (mirip MA), sedangkan median blur mengganti tiap piksel dengan median tetangganya, sangat efektif menghapus noise 'salt-and-pepper' (bintik hitam-putih acak) tanpa mengaburkan tepi gambar.
- **Audacity Denoise Audio:** fitur Noise Reduction pada software audio memakai kombinasi spectral subtraction, median filter, dan smoothing mirip Savitzky-Golay, yaitu pipeline yang persis sama dengan preprocessing sinyal vibrasi di industri.

APLIKASI FILTER DI MESIN INDUSTRI

Pipeline pada Gambar 7 merangkum urutan filter yang direkomendasikan dalam praktik vibration monitoring: median filter dahulu untuk menghapus spike, lalu Savitzky-Golay untuk smoothing yang menjaga puncak, EMA untuk baseline yang halus, baru kemudian analisis (RMS/FFT/deteksi anomali).



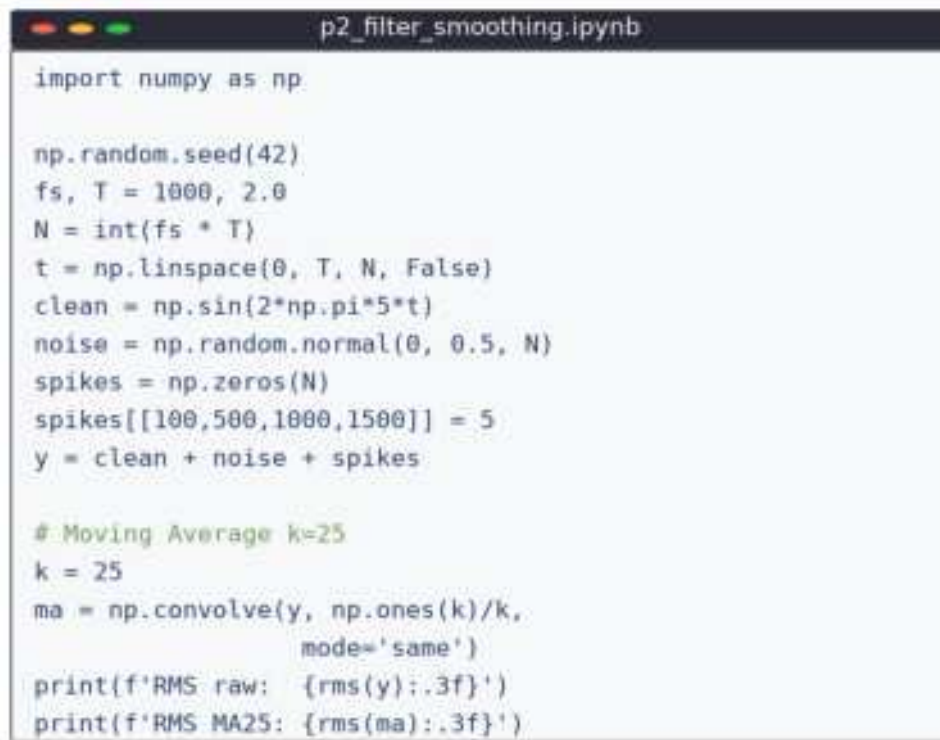
Gambar 7. Pipeline filter rekomendasi untuk vibration monitoring: Raw Signal → Median Filter → Savitzky-Golay → EMA → Analisis.

- **Vibration RMS Baseline:** EMA dengan $\alpha \approx 0,05$ dipakai sebagai baseline bergerak RMS getaran ($EMA_t = 0,05 \cdot x_t + 0,95 \cdot EMA_{t-1}$); alert dipicu bila RMS melebihi $baseline + 3\sigma$, jauh lebih stabil dibanding threshold statis.
- **Spectral Peak Preservation:** Savitzky-Golay $w=11, p=3$ dipakai untuk identifikasi peak spektral bearing fault (BPFI/BPFO/BSF/FTF, lihat Pertemuan 7); MA akan menumpulkan peak spektral sehingga severity kerusakan bisa terestimasi lebih rendah dari kondisi sebenarnya.
- **Spike Removal:** median filter $w=5$ diterapkan sebelum perhitungan RMS untuk menghapus spike EMI/mechanical shock sesaat yang tidak merepresentasikan kondisi mesin sesungguhnya.
- **Anti-Aliasing Pre-Filter:** filter low-pass analog dengan cutoff = $f_s/2$ (Nyquist) wajib dipasang di hardware sebelum ADC. Untuk $f_s=10$ kHz, cutoff harus di sekitar 4 kHz agar komponen di atas Nyquist tidak menyebabkan aliasing.

Peringatan: Kesalahan praktis yang sering terjadi: (1) menerapkan smoothing time-domain sebelum FFT sehingga mendistorsi spektrum; (2) memakai MA panjang untuk deteksi shock/impact padahal tujuannya justru kontradiktif dengan sifat MA yang menumpulkan puncak; (3) lupa memasang anti-aliasing filter analog; (4) parameter α/k tidak disesuaikan dengan sampling rate sistem.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada data sintesis yang meniru sensor industri. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, matplotlib; notebook p2_filter_smoothing.ipynb. Gambar 8 menunjukkan cuplikan Cell 1 yang membangkitkan sinyal sintesis dan menerapkan Moving Average.



```
import numpy as np

np.random.seed(42)
fs, T = 1000, 2.0
N = int(fs * T)
t = np.linspace(0, T, N, False)
clean = np.sin(2*np.pi*5*t)
noise = np.random.normal(0, 0.5, N)
spikes = np.zeros(N)
spikes[[100,500,1000,1500]] = 5
y = clean + noise + spikes

# Moving Average k=25
k = 25
ma = np.convolve(y, np.ones(k)/k,
                  mode='same')
print(f'RMS raw: {rms(y):.3f}')
print(f'RMS MA25: {rms(ma):.3f}')
```

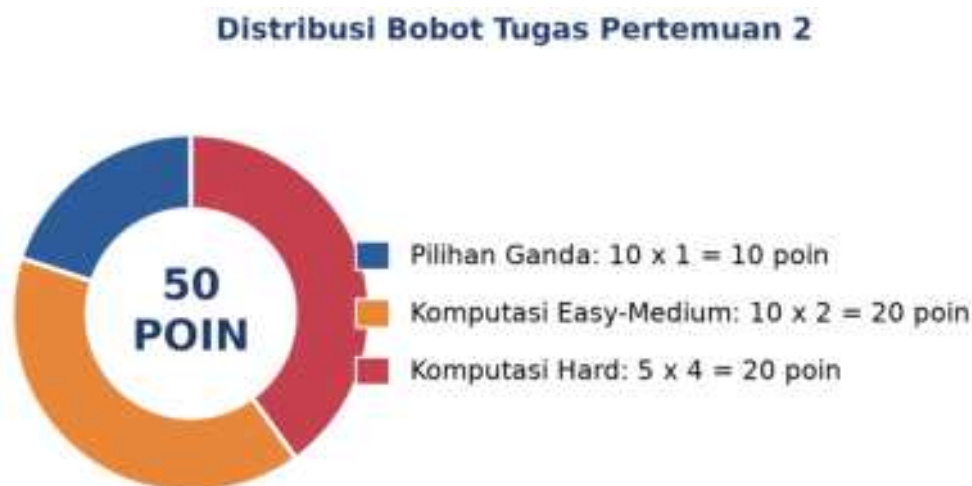
Gambar 8. Cuplikan Cell 1: pembangkitan sinyal sintesis (noise + 4 spike) dan Moving Average $k=25$ dengan NumPy.

- **Cell 1:** generate sinyal noisy ($fs=1000$ Hz, 5 Hz sine + noise $\sigma=0,5$ + 4 spike di indeks [100,500,1000,1500]) + MA $k=5/25/100$, plot 4-panel, cetak RMS raw/clean/MA25.
- **Cell 2:** EMA manual (rekursif) vs `pandas.ewm()`, bandingkan $\alpha=[0,5; 0,1; 0,02]$ dengan overlay sinyal bersih sebagai ground truth.
- **Cell 3:** Savitzky-Golay vs Moving Average pada sinyal Gaussian peak ($N=500$), cetak persentase preservasi puncak (SG $\sim 95\%$, MA $< 70\%$).

- **Cell 4:** Median filter untuk penghapusan spike: tabel nilai pada 4 lokasi spike (Raw / MA k=5 / Median k=5 / Median+SG chained).

TUGAS PERTEMUAN 2

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.



Gambar 9. Distribusi 50 poin Tugas Pertemuan 2 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Tujuan utama dari filter low-pass pada data sensor adalah...
 - A. Memperbesar amplitudo sinyal asli
 - B. Mengurangi noise frekuensi tinggi sambil mempertahankan komponen frekuensi rendah
 - C. Mengubah deret waktu menjadi domain frekuensi
 - D. Menghitung autokorelasi sinyal
2. Pernyataan yang benar mengenai Trailing Moving Average (causal) adalah...
 - A. Tidak memerlukan data masa lalu
 - B. Cocok untuk real-time monitoring tapi memiliki lag $\approx (k-1)/2$ sampel
 - C. Selalu menghasilkan zero phase shift
 - D. Memerlukan polynomial fitting
3. Pada formula $EMAt = \alpha \cdot xt + (1-\alpha) \cdot EMAt-1$, jika $\alpha = 0,5$ maka...
 - A. Output didominasi sepenuhnya oleh history (lag besar)
 - B. Bobot setara antara observasi terbaru dan history, sangat responsif tapi noisy

- C. EMA tidak akan pernah mendekati sinyal asli
- D. α tidak boleh bernilai 0,5 secara matematis

4. Keunggulan utama Savitzky-Golay (SG) dibandingkan Moving Average untuk analisis getaran adalah...

- A. SG selalu lebih cepat secara komputasi
- B. SG menjaga amplitudo puncak/lembah jauh lebih baik (preserve features)
- C. SG tidak memerlukan parameter window
- D. SG bisa digunakan tanpa perlu polynomial order

5. Untuk menghapus impulse spike (lonjakan tunggal seperti EMI dari relay) dari sinyal sensor, filter yang paling efektif adalah...

- A. Moving Average dengan window besar
- B. EMA dengan α besar (responsif)
- C. Median filter, karena sifat nonlinear-nya menelan spike sepenuhnya
- D. Savitzky-Golay dengan polynomial order tinggi

6. Dampak utama dari memilih window MA yang terlalu besar pada deteksi anomali sinyal getaran adalah...

- A. Tidak ada dampak, sinyal tetap utuh
- B. Transient/lonjakan singkat ikut tersmoothing sehingga anomali terlewatkan, lag deteksi besar
- C. Konsumsi memori turun drastis
- D. Output menjadi sama dengan EMA dengan $\alpha=1$

7. Hubungan antara α EMA dan effective window N equivalent adalah...

- A. $N = \alpha \times 100$
- B. $\alpha = 2/(N+1)$, atau ekuivalen $N = 2/\alpha - 1$
- C. $N = \log(\alpha)$
- D. Tidak ada hubungan formal

8. Yang bukan merupakan sumber noise pada sensor industri adalah...

- A. Thermal noise pada resistor
- B. Quantization noise dari ADC resolusi terbatas
- C. EMI dari motor/inverter di lingkungan
- D. Bias DC dari kalibrasi (ini systematic offset, bukan noise)

9. Pipeline pre-processing standar untuk vibration data dengan noise Gaussian + spike adalah...

- A. FFT terlebih dahulu, lalu MA, lalu SG
- B. Median (hapus spike), lalu Savitzky-Golay (preserve peak smoothing), lalu analisis
- C. EMA dengan α besar, lalu MA window besar, lalu median

- D. Tidak ada urutan baku, tergantung intuisi

10. Mengapa tidak disarankan menerapkan smoothing time-domain (MA/EMA) sebelum FFT pada analisis getaran?

- A. FFT tidak akan berjalan pada sinyal yang di-filter
- B. Filter mendistorsi spektrum, peak amplitudo dan posisi frekuensi tidak akurat sehingga severity assessment salah
- C. Smoothing meningkatkan noise di domain frekuensi
- D. Operasi Python tidak mendukung pipeline tersebut

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Dataset referensi (dipakai C1-C10, dibuat sekali di Jupyter):

```
import numpy as np
np.random.seed(42)
fs = 1000; T_dur = 2.0; N = int(fs * T_dur)
t = np.linspace(0, T_dur, N, endpoint=False)
clean = np.sin(2*np.pi*5*t)           # 5 Hz pure baseline
noise = np.random.normal(0, 0.5, N)   # heavy gaussian noise
spikes = np.zeros(N); spikes[[100,500,1000,1500]] = 5
y = clean + noise + spikes             # deret yang akan difilter
```

C1. Hitung RMS dari sinyal y mentah (raw, sebelum filter). Bandingkan dengan RMS clean signal ($\approx 0,707$); selisihnya menunjukkan kontribusi noise + spike.

- 💡 **HINT:** RMS = akar dari rata-rata kuadrat sinyal; gunakan `np.sqrt` dan `np.mean` pada `y**2`.

C2. Terapkan Moving Average window $k=10$ memakai `np.convolve(y, np.ones(10)/10, mode='same')`. Cetak mean dari hasil filter.

- 💡 **HINT:** Pakai `np.convolve` dengan kernel `np.ones(k)/k` dan `mode='same'`, lalu ambil rata-rata hasilnya dengan `np.mean`.

C3. Terapkan Moving Average window $k=50$. Cetak standar deviasi (`ddof=1`) dari hasil filter; std akan jauh lebih kecil dari raw karena noise tersmoothing.

- 💡 **HINT:** Terapkan moving average dengan `np.convolve` (kernel `np.ones(k)/k`), lalu hitung std sampel dengan `np.std(..., ddof=1)`.

C4. Terapkan EMA dengan $\alpha=0,1$ memakai `pandas`: `ema = pd.Series(y).ewm(alpha=0.1, adjust=False).mean().values`. Cetak nilai EMA pada indeks terakhir.

- 💡 **HINT:** Gunakan `pd.Series(y).ewm(alpha=..., adjust=False).mean()`, lalu ambil nilai pada indeks terakhir.

C5. Terapkan EMA dengan $\alpha=0,5$ (sangat responsif). Cetak nilai EMA pada indeks 1500 (segera setelah spike ke-4). EMA dengan α besar akan menyerap spike dengan kuat.

- 💡 **HINT:** Hitung EMA responsif dengan `ewm(alpha=..., adjust=False).mean()`, lalu ambil nilai pada indeks yang diminta soal.

C6. Terapkan Savitzky-Golay (window=11, polyorder=3) dari `scipy.signal`. Cetak nilai filtered pada indeks 1000 (lokasi spike).

- 💡 **HINT:** Gunakan `scipy.signal.savgol_filter` dengan window dan polyorder dari soal, lalu ambil nilai pada indeks yang diminta.

C7. Terapkan Median filter dengan `kernel_size=5` dari `scipy.signal`. Cetak nilai pada indeks 100 (lokasi spike pertama). Median filter akan sepenuhnya menghapus spike.

- 💡 **HINT:** Gunakan `scipy.signal.medfilt` dengan `kernel_size` dari soal, lalu ambil nilai pada indeks spike yang diminta.

C8. Hitung effective window N equivalent untuk EMA dengan $\alpha=0,1$. Rumus: $N = 2/\alpha - 1$. Cetak nilai N.

- 💡 **HINT:** Effective window $N = 2/\alpha - 1$. Substitusikan nilai α dari soal.

C9. Hitung RMS dari residual (selisih antara y dengan clean): $\text{residual} = y - \text{clean}$. Cetak RMS residual, yang mengukur kontaminasi noise + spike total.

- 💡 **HINT:** Hitung $\text{residual} = y - \text{clean}$, lalu RMS-nya dengan `np.sqrt(np.mean(residual**2))`.

C10. Terapkan MA $k=25$ pada y lalu hitung RMS hasilnya. Bandingkan dengan RMS clean (0,707); MA window 25 cukup baik untuk recovery RMS asli.

- 💡 **HINT:** Terapkan moving average $k=25$ dengan `np.convolve`, lalu hitung RMS hasilnya dengan `np.sqrt(np.mean)`.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal (bukan memanggil fungsi library siap pakai untuk bagian intinya), 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan Moving Average dari awal tanpa `np.convolve` atau `pandas`. Untuk window $k=15$ dan dataset referensi y, gunakan loop: untuk tiap titik i, hitung mean dari $y[\max(0, i-7) : \min(N, i+8)]$ (centered, dengan boundary handling). Cetak mean dari MA hasil.

- 💡 **HINT:** Loop tiap titik i, ambil window centered $y[\max(0, i-7) : \min(N, i+8)]$ dan hitung mean-nya untuk mengisi `out[i]`; akhirnya ambil rata-rata seluruh out.

C12 (Hard). Implementasikan EMA dari awal tanpa `pandas`. Untuk dataset y dengan $\alpha=0,05$ dan inisialisasi $\text{EMA}[0] = \text{mean}(y[:10])$ (bukan default $y[0]$). Cetak nilai EMA pada indeks 100.

- 💡 **HINT:** Inisialisasi $\text{EMA}[0] = \text{mean}$ dari 10 titik pertama, lalu iterasi rumus rekursif $\text{EMA}_i = \alpha \cdot y_i + (1-\alpha) \cdot \text{EMA}_{i-1}$; ambil nilai pada indeks yang diminta.

C13 (Hard). Implementasikan Savitzky-Golay dari awal tanpa `scipy.signal.savgol_filter`. Untuk `window=11`, `polyorder=3`, dan dataset `y`: untuk tiap pusat window, fit polinomial orde 3 dengan `np.polyfit(idx, y[i-5:i+6], 3)`, lalu evaluasi di pusat (`idx=0`). Cetak nilai SG manual pada indeks 500.

- 💡 **HINT:** Untuk tiap pusat window, fit polinomial orde 3 dengan `np.polyfit` pada 11 titik sekitar, lalu evaluasi di pusat (`idx=0`) memakai `np.polyval`; ambil nilai pada indeks yang diminta dan validasi terhadap `savgol_filter`.

C14 (Hard). Implementasikan anomaly detection berbasis EMA + aturan 3σ . Generate sinyal baru: `y_anom = clean + noise` (tanpa spike), lalu inject 3 anomali besar: `y_anom[[250,750,1250]] += 4.0`. Hitung $EMA(\alpha=0,05)$ lalu `residual = y_anom - EMA`. Hitung `n_detect = jumlah |residual| > 3*std(residual)`. Cetak jumlah anomali terdeteksi.

- 💡 **HINT:** Bangun sinyal beranomali sesuai soal, hitung EMA-nya, lalu `residual = sinyal - EMA`; hitung berapa titik dengan `|residual|` melebihi $3 \times \text{std residual}$.

C15 (Hard). Implementasikan filter pipeline lengkap pada `y`: (1) median filter `k=5` untuk hapus spike, (2) Savitzky-Golay `w=11, p=3` untuk smoothing yang preserve peak, (3) EMA $\alpha=0,1$ untuk baseline yang halus. Hitung RMS dari output akhir pipeline. Hasil seharusnya mendekati RMS clean ($\approx 0,707$).

- 💡 **HINT:** Rangkai tiga tahap berurutan: `medfilt` → `savgol_filter` → `ewm().mean()` sesuai parameter soal, lalu hitung RMS output akhir dengan `np.sqrt(np.mean())`.

DAFTAR PUSTAKA

- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. IEEE Transactions on Knowledge and Data Engineering, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Khan, S., & Lee, D.-H. (2017). An adaptive dynamically weighted median filter for impulse noise removal. EURASIP Journal on Advances in Signal Processing, 2017, 67. <https://doi.org/10.1186/s13634-017-0502-z>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. Eng, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>

Schmid, M., Rath, D., & Diebold, U. (2022). Why and how Savitzky-Golay filters should be replaced. *ACS Measurement Science Au*, 2(2), 185–196.

<https://doi.org/10.1021/acsmeasuresciau.1c00054>

Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 3

Visualisasi & Akuisisi Pipeline Time

Abstrak

Pertemuan ini membahas pipeline akuisisi data time series dari sensor fisik hingga dashboard produksi: konfigurasi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 1.3 – Mampu mempertimbangkan aspek ekonomi dalam desain sistem otomasi

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-3.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Setelah sinyal bisa diakuisisi dan dibersihkan (Pertemuan 1-2), sistem monitoring nyata membutuhkan rantai akuisisi data yang andal dari sensor fisik hingga dashboard produksi. Pertemuan ketiga ini membahas konfigurasi sensor dan ADC (dynamic range, LSB, bandwidth), arsitektur penyimpanan tiered, streaming data dari puluhan motor sekaligus, hingga visualisasi time series yang efektif untuk dashboard produksi. Gambar 1 menunjukkan lima tahap pipeline akuisisi data dari sensor hingga analitik.



Gambar 1. Pipeline akuisisi data lima tahap: Sensor, Signal Conditioning, ADC, Edge Device & Bus, Storage/Cloud, Analytics.

Kesalahan analisis pada pertemuan-pertemuan berikutnya (ekstraksi fitur, pemodelan) sering kali berakar jauh di hulu pada tahap akuisisi ini, misalnya peak FFT yang salah karena ADC yang saturated. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan forum diskusi studi kasus.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 1.3:** Mampu mempertimbangkan aspek ekonomi dalam desain sistem otomasi, yaitu memilih resolusi ADC, sampling rate, dan arsitektur penyimpanan yang selalu merupakan trade-off antara kualitas data dan biaya (CPMK 1).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan komponen sensor chain dari sensor hingga penyimpanan; menghitung dynamic range dan resolusi ADC; memilih sampling rate dan arsitektur tiered storage yang tepat; serta memilih jenis visualisasi time series yang sesuai untuk berbagai kasus monitoring industri.

SENSOR & ADC: KONFIGURASI YANG TEPAT

Sinyal analog dari sensor harus melalui rantai signal conditioning (amplifikasi, filtering anti-aliasing, isolation) sebelum masuk ADC. Kualitas hasil digitalisasi ditentukan oleh resolusi ADC (jumlah bit) dan sampling rate.

$$LSB = V_{range} / 2^N \quad SNR_{kuantisasi} = 6,02 \cdot N + 1,76 \text{ dB}$$

- **Akselerometer Vibration:** IEPE/ICP (100 mV/g, rentang 0,5 Hz-10 kHz) untuk pengukuran presisi tinggi, atau MEMS (DC-1 kHz) untuk aplikasi umum; bandwidth minimal 5x kecepatan rotasi dan 2x frekuensi bearing fault yang ingin dideteksi; metode pemasangan stud lebih baik dari magnetic, magnetic lebih baik dari adhesive.
- **ADC Resolution Trade-off:** 12-bit menghasilkan 4096 level (SNR sekitar 74 dB) untuk monitoring umum; 16-bit menghasilkan 65536 level (SNR sekitar 98 dB) sebagai standar vibration; 24-bit Sigma-Delta menghasilkan 16 juta level (SNR sekitar 144 dB) untuk aplikasi precision/audio.
- **Sampling Rate Selection:** praktik industri memakai oversampling 5-10 kali frekuensi maksimum yang relevan. Contoh: motor 1500 RPM menghasilkan frekuensi dasar 25 Hz, harmonik sampai 1 kHz, dan bearing fault sampai 5 kHz, sehingga sampling rate minimum berada di rentang 25-50 kHz.

Tabel 1 merangkum lima aplikasi sensor umum beserta sampling rate khas, resolusi ADC, dan storage yang dihasilkan per jam untuk 1 channel.

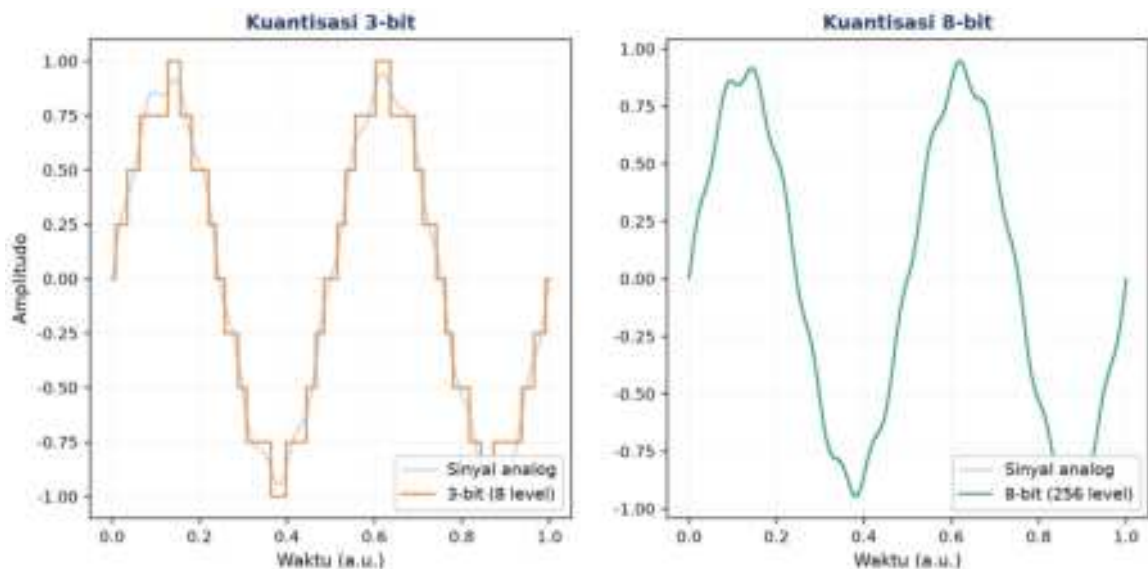
Tabel 1. Sampling rate, resolusi ADC, dan storage per jam untuk lima aplikasi sensor industri.

Aplikasi	Sensor	fs Khas	ADC bit	Storage/jam (1 ch)
Temperatur Proses	RTD/TC	1 Hz	16-bit	~7 KB
Vibration Bearing	IEPE	10 kHz	16-bit	~70 MB
Vibration High-Speed	IEPE	50 kHz	24-bit	~520 MB
Power Quality	Current/Voltage	20 kHz	16-bit	~140 MB
Acoustic Emission	Piezo wide-band	1 MHz	14-bit	~6 GB

Peringatan: Kesalahan konfigurasi ADC yang umum terjadi: (1) rentang tegangan terlalu lebar menyebabkan hilangnya sekitar 7 bit resolusi efektif untuk sinyal kecil; (2) rentang terlalu sempit menyebabkan clipping; (3) lupa memasang anti-aliasing filter analog sebelum ADC; (4) crosstalk antar-channel pada ADC multiplexed.

Gambar 2 memperlihatkan efek kuantisasi ADC pada dua resolusi berbeda: bit-depth rendah (3-bit) menghasilkan staircase kasar yang jauh dari sinyal analog aslinya,

sementara 8-bit sudah cukup halus mengikuti bentuk gelombang meskipun masih ada sedikit distorsi.



Gambar 2. Efek kuantisasi ADC pada resolusi 3-bit (8 level) dibandingkan 8-bit (256 level).

SAMPLING RATE DAN TEOREMA NYQUIST

Memilih sampling rate yang salah menyebabkan sinyal berfrekuensi tinggi tampak sebagai alias berfrekuensi rendah yang palsu. Gambar 3 mendemonstrasikan konsekuensi ini secara visual: sinyal 3 Hz yang disampling pada 12 Hz (di atas syarat Nyquist) berhasil direkonstruksi dengan benar, sedangkan sampling pada 4 Hz (di bawah syarat Nyquist) menghasilkan alias berfrekuensi 1 Hz yang sama sekali tidak merepresentasikan sinyal aslinya.

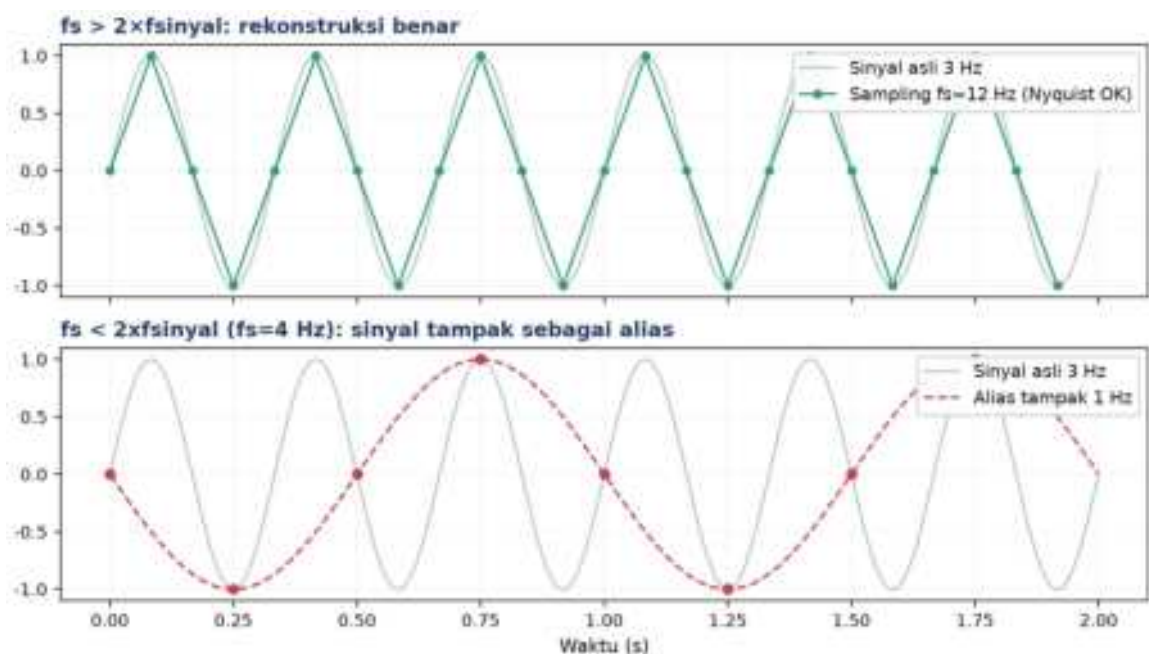
$$f_s \geq 2 \cdot f_{maks} \quad f_{alias} = |f_{sinyal} - k \cdot f_s|$$

Konsekuensi Aliasing pada Diagnosis Getaran: Konsekuensi aliasing pada analisis getaran bisa berakibat fatal secara diagnostik: bila sistem monitoring bearing men-sampling pada $f_s=1000$ Hz padahal frekuensi kerusakan sebenarnya (BPFO) berada di 1200 Hz (melebihi Nyquist 500 Hz), maka energi kerusakan tersebut akan muncul sebagai alias di sekitar $|1200-1000|=200$ Hz pada spektrum FFT, sebuah frekuensi yang sama sekali tidak berkaitan dengan geometri bearing yang sesungguhnya. Analis yang tidak menyadari fenomena ini berisiko salah mendiagnosis sumber kerusakan, atau lebih buruk, mengabaikan sinyal peringatan dini karena dianggap noise acak pada frekuensi yang tidak relevan.

Solusi standar industri adalah memasang anti-aliasing filter analog (low-pass hardware, bukan digital) SEBELUM tahap ADC, dengan cutoff frequency di bawah $f_s/2$. Filter digital

tidak bisa menggantikan peran ini karena proses sampling sudah terjadi lebih dulu: begitu sinyal ter-alias saat sampling, informasi frekuensi aslinya hilang permanen dan tidak dapat direkonstruksi ulang oleh pemrosesan digital apa pun setelahnya, betapapun canggihnya algoritma yang digunakan.

Contoh Perhitungan Sampling Rate: Sebagai perhitungan praktis: untuk mendeteksi harmonik hingga orde ke-5 dari frekuensi putaran motor 3000 rpm (50 Hz), sistem monitoring memerlukan sampling rate $f_s \geq 2 \times (5 \times 50) = 500$ Hz secara teori, namun praktik industri umumnya menerapkan margin keamanan $2,56 \times$ (mengikuti rekomendasi standar analisis FFT windowed) sehingga f_s yang dipilih menjadi sekitar 1280 Hz, dibulatkan ke nilai standar terdekat pada datasheet DAQ komersial, misalnya 2000 Hz.



Gambar 3. Demonstrasi sampling di atas syarat Nyquist ($f_s=12$ Hz, rekonstruksi benar) dan di bawahnya ($f_s=4$ Hz, sinyal tampak sebagai alias 1 Hz).

Format data yang dipilih juga memengaruhi efisiensi penyimpanan dan kecepatan akses: CSV mudah dibaca manusia tetapi lambat diproses; Parquet (columnar, terkompresi) menjadi standar untuk batch analytics karena bisa 10-20 kali lebih kecil dari CSV; HDF5 populer untuk data ilmiah; TDMS adalah format proprietary National Instruments yang menjadi standar industri vibrasi; sedangkan protokol InfluxDB line protocol dipakai untuk streaming langsung ke time-series database.

STORAGE & STREAMING TIME SERIES

Volume data akuisisi tumbuh cepat: sensor vibrasi 10 kHz menghasilkan sekitar 70 MB per jam per channel, atau 1,7 GB per hari; untuk 8 motor sekaligus totalnya mencapai 13,6 GB

per hari. Arsitektur tiered storage menjadi solusi standar untuk mengelola volume ini secara ekonomis.

$$\text{Bandwidth (B/s)} = fs \cdot (\text{bits}/8) \cdot \text{channels} \quad \text{Storage} = \text{Bandwidth} \cdot \text{durasi}$$

- **Edge Buffer:** circular buffer pada NVMe SSD industrial (kapasitas 256 GB-1 TB) menyimpan data mentah full-resolution selama beberapa hari sebagai buffer forensik sebelum di-overwrite.
- **Time Series Database:** database time-series khusus seperti InfluxDB, TimescaleDB (ekstensi PostgreSQL), Prometheus, atau QuestDB, dengan downsampling otomatis dan retention policy per tier.
- **Streaming Protocol:** MQTT ringan dengan QoS 0-2 cocok untuk IoT skala besar; Kafka untuk throughput tinggi skala enterprise; OPC UA untuk integrasi sistem industri klasik.
- **Aggregation Strategy:** edge device menghitung statistik ringkas (RMS, peak per detik) dari data mentah 10 kHz, lalu hanya mengirim agregat 1 Hz ke cloud, menyisakan data mentah tetap tersimpan lokal untuk investigasi forensik bila diperlukan.

Tabel 2 merangkum empat tingkat (tier) arsitektur penyimpanan dari data mentah resolusi penuh hingga arsip jangka panjang.

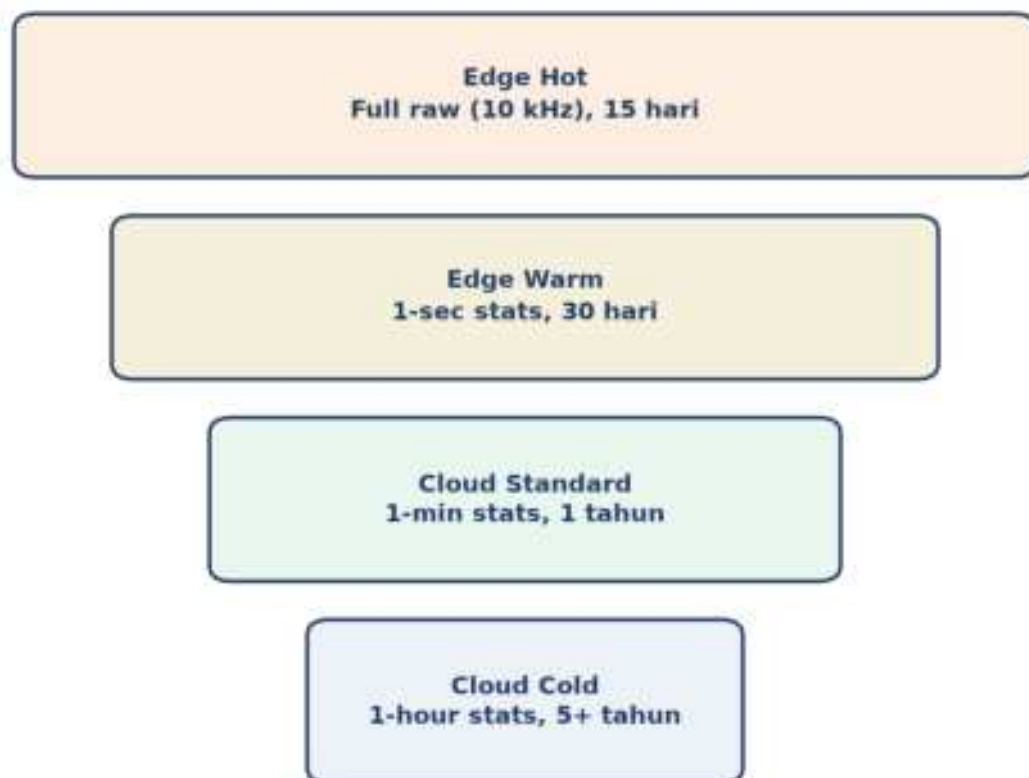
Tabel 2. Arsitektur tiered storage: resolusi data, retention, teknologi penyimpanan, dan penggunaannya.

Tier	Resolusi	Retention	Storage Tech	Use Case
Edge Hot	Full raw (10 kHz)	15 hari	NVMe SSD industrial	Forensic investigation
Edge Warm	1-sec stats	30 hari	Local TSDB	Recent dashboards
Cloud Standard	1-min stats	1 tahun	InfluxDB Cloud	Trending, historical
Cloud Cold	1-hour stats	5+ tahun	S3 + Parquet	Compliance, audit

Perhitungan Kapasitas Edge Buffer: Perhitungan kapasitas edge buffer perlu mempertimbangkan skenario forensik konkret: bila terjadi kegagalan bearing mendadak, tim maintenance idealnya dapat menelusuri kembali data mentah resolusi penuh beberapa hari sebelum kejadian untuk mengidentifikasi tanda-tanda awal yang terlewat oleh sistem alert otomatis. Dengan asumsi 8 motor pada $fs=10$ kHz masing-masing menghasilkan 70 MB/jam, total volume harian mencapai 13,6 GB, sehingga retensi 15 hari pada tier Edge Hot membutuhkan kapasitas SSD sekitar 204 GB, sudah termasuk margin untuk metadata dan indexing.

Pemilihan teknologi penyimpanan pada tiap tier juga mempertimbangkan pola akses data yang berbeda: tier Edge Hot dioptimalkan untuk write throughput tinggi (data terus mengalir masuk secara real-time) dan sesekali random-read saat investigasi forensik, sehingga NVMe SSD dengan endurance tinggi (TBW besar) menjadi pilihan tepat. Sebaliknya, tier Cloud Cold dioptimalkan untuk cost-per-GB serendah mungkin karena data jarang diakses (hanya untuk audit atau compliance), sehingga object storage seperti Amazon S3 Glacier atau setara jauh lebih ekonomis dibanding menyimpan seluruh histori pada database aktif.

Transisi data antar tier (tiering policy) umumnya diotomatisasi melalui job terjadwal yang menghitung agregat statistik (mean, RMS, peak per menit/jam) dari data resolusi penuh sebelum data mentah tersebut dihapus dari tier hot, sehingga informasi tren jangka panjang tetap tersedia meski detail resolusi tinggi sudah tidak lagi disimpan. Kebijakan retention period pada tiap tier idealnya disesuaikan dengan regulasi industri yang berlaku (mis. ISO 55001 untuk asset management), bukan sekadar keputusan teknis semata.



Gambar 4. Diagram tiered storage dari Edge Hot (resolusi penuh, retensi pendek) hingga Cloud Cold (agregat jam-an, retensi bertahun-tahun).

VISUALISASI TIME SERIES INDUSTRI

Empat jenis plot utama dipakai dalam monitoring time series industri: line plot, spectrogram, heatmap, dan plot statistik (boxplot/violin). Pemilihan yang tepat bergantung pada jumlah sensor yang dipantau bersamaan dan sifat sinyal yang ingin ditonjolkan.

- **Line Plot:** cocok untuk 1-beberapa sensor, batasi maksimal sekitar 10 ribu titik ditampilkan (downsampling untuk data lebih besar), beri judul informatif, unit eksplisit pada sumbu, grid transparan (alpha 0,3), dan colormap yang accessible seperti viridis, bukan rainbow.
- **Spectrogram:** menampilkan sumbu waktu vs frekuensi dengan warna merepresentasikan amplitudo, ideal untuk analisis startup/shutdown, bearing degradation, dan gear mesh modulation. Dihitung via `scipy.signal.spectrogram``.
- **Heatmap Multi-Sensor:** baris merepresentasikan sensor, kolom merepresentasikan waktu, warna merepresentasikan nilai metrik; ideal untuk memantau 100 lebih motor sekaligus karena pola anomali langsung terlihat sebagai blok warna berbeda.
- **Boxplot & Violin:** membandingkan distribusi antar shift, bulan, atau unit mesin untuk mendeteksi pergeseran (drift) kondisi operasional.

Tabel 3 merangkum jenis plot terbaik beserta tooling yang umum dipakai untuk beberapa kasus visualisasi khas.

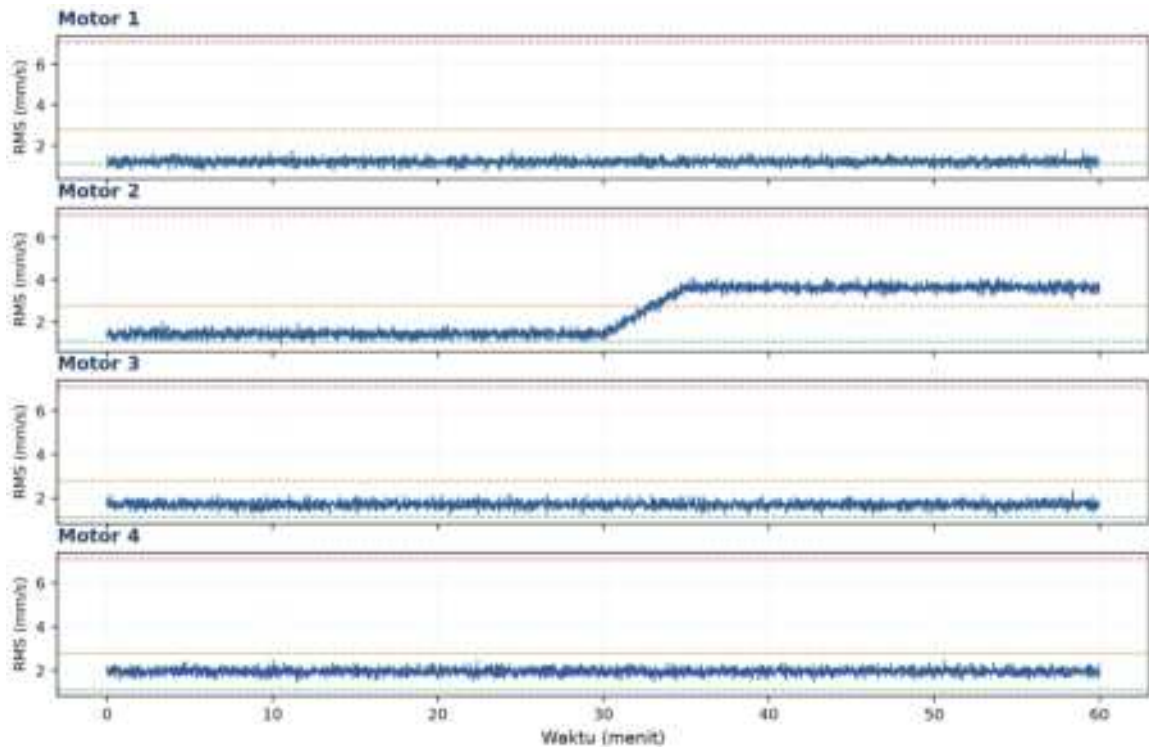
Tabel 3. Pemilihan jenis plot dan tooling untuk berbagai kasus visualisasi time series industri.

Kasus	Plot Terbaik	Tooling
Trend RMS 1 motor, 1 minggu	Line + threshold overlay	matplotlib / plotly
Motor startup analysis	Spectrogram	scipy + imshow
100 motor, daily RMS	Heatmap (annotated)	seaborn / Grafana
Per-shift performance	Boxplot side-by-side	seaborn boxplot
Real-time monitoring wall	Live dashboard	Grafana + InfluxDB

Konvensi Warna Threshold: Pemilihan warna threshold pada line plot multi-motor idealnya mengikuti konvensi universal yang sudah dikenal operator: hijau untuk kondisi baik, oranye untuk peringatan, dan merah untuk kritis, konsisten dengan skema warna traffic light yang dipakai di hampir semua dashboard SCADA industri. Konsistensi warna ini penting agar operator shift baru dapat langsung membaca kondisi mesin tanpa perlu mempelajari legenda khusus tiap sistem, mempercepat respons saat terjadi kondisi abnormal yang membutuhkan tindakan cepat.

Overlay threshold ISO 10816 pada Gambar 5 sekaligus menunjukkan manfaat visualisasi multi-motor dalam satu plot dibanding memantau tiap motor secara terpisah: anomali

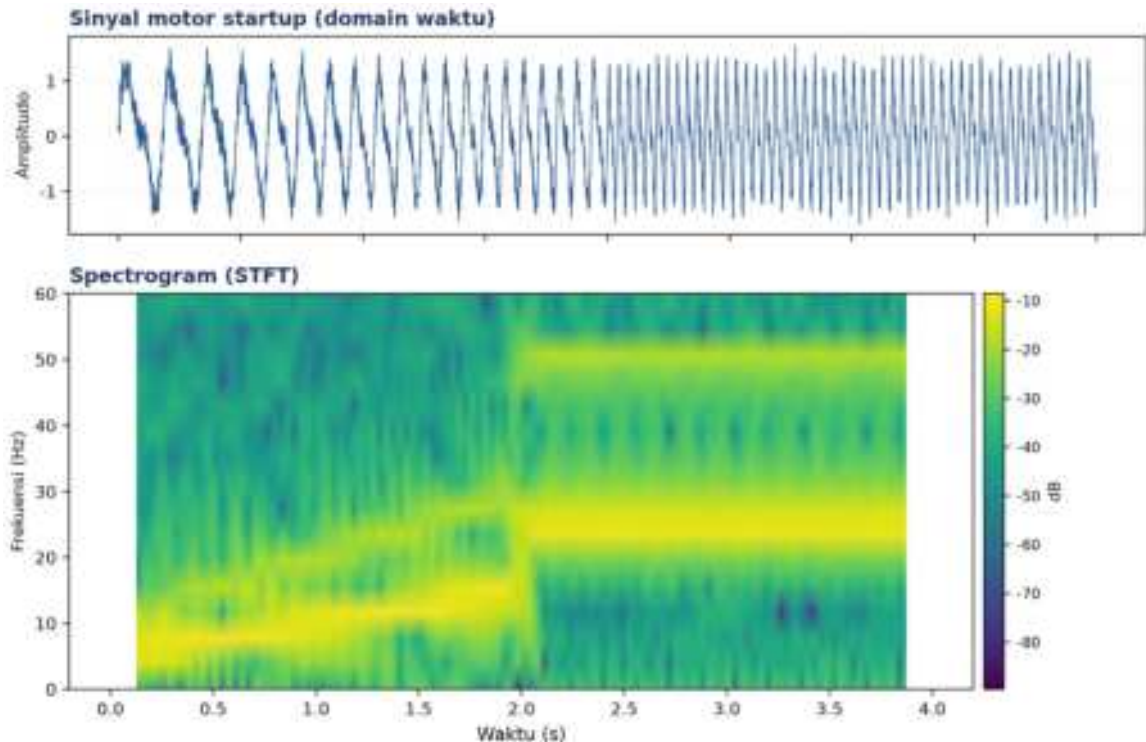
gradual pada Motor 2 (mulai menit ke-30) dapat langsung dibandingkan konteksnya terhadap tiga motor sejenis yang beroperasi normal pada rentang waktu yang sama, sehingga operator dapat membedakan dengan cepat apakah kenaikan RMS disebabkan oleh gangguan spesifik pada satu unit mesin, atau justru gangguan sistemik yang memengaruhi seluruh lini produksi (misalnya fluktuasi tegangan suplai listrik).



Gambar 5. Line plot RMS getaran 4 motor dengan overlay threshold ISO 10816 (garis putus-putus hijau/oranye/merah); Motor 2 menunjukkan anomali gradual mulai menit ke-30.

Peringatan: Anti-pattern visualisasi yang harus dihindari: (1) memplot ribuan titik tanpa downsampling sehingga file menjadi berat dan berjejal; (2) auto-scaling sumbu-y tanpa baseline sehingga fluktuasi kecil tampak dramatis; (3) memakai colormap rainbow untuk data kontinu, yang tidak perceptually uniform; (4) memplot tanpa unit yang jelas pada sumbu.

Spectrogram pada Motor Variable Speed Drive: Spectrogram terutama berguna pada motor dengan Variable Speed Drive (VSD), di mana kecepatan putar (dan seluruh frekuensi karakteristik yang bergantung padanya, seperti frekuensi putaran rotor dan harmonik gear mesh) berubah secara kontinu mengikuti perintah kecepatan dari kontroler. Pada kondisi ini, spektrum FFT tunggal yang dihitung dari keseluruhan durasi rekaman menjadi sulit diinterpretasi karena mencampur seluruh rentang frekuensi operasi sekaligus; spectrogram menyelesaikan masalah ini dengan menampilkan evolusi spektrum terhadap waktu, sehingga jejak tiap komponen frekuensi dapat ditelusuri seiring perubahan kecepatan motor.

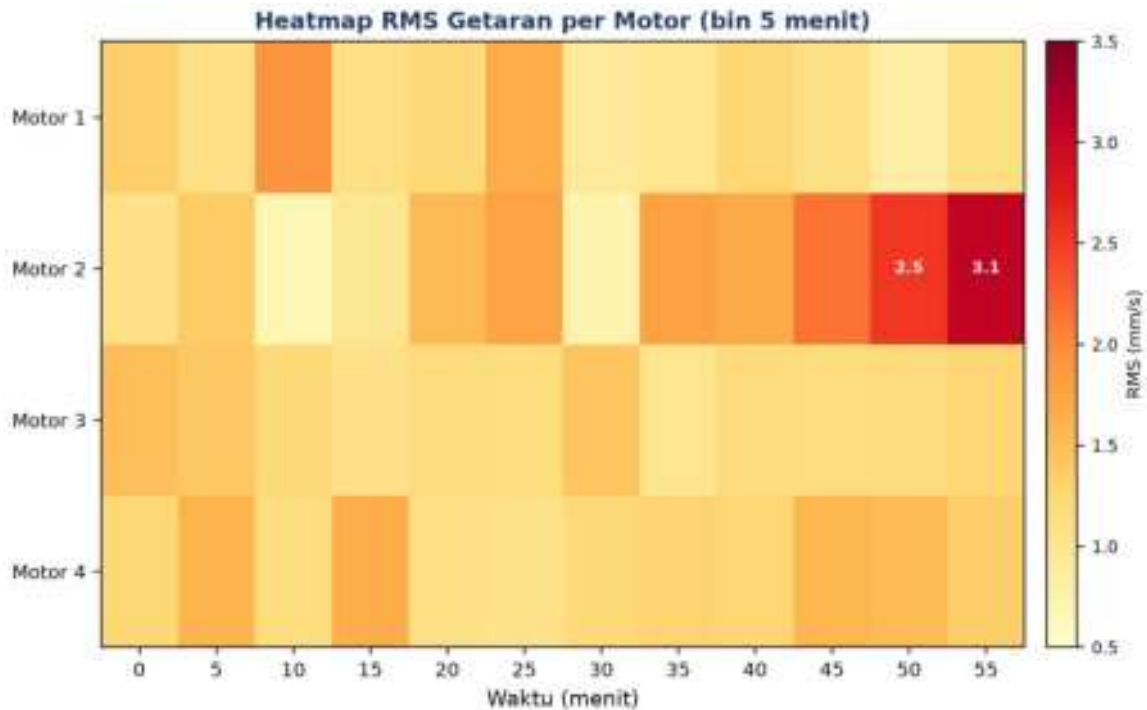


Gambar 6. Spectrogram sinyal motor startup: frekuensi dasar naik dari 5 Hz ke 25 Hz disertai harmonik kedua, terlihat jelas pada representasi waktu-frekuensi dibanding domain waktu saja.

Gambar 6 menegaskan keunggulan spectrogram: pada domain waktu (panel atas) perubahan frekuensi selama startup sulit dibaca langsung, sedangkan spectrogram (panel bawah) menunjukkan jejak frekuensi dasar naik dari 5 Hz hingga stabil di 25 Hz beserta harmonik keduanya secara eksplisit.

Pemilihan Bin Waktu Heatmap: Praktik terbaik desain heatmap untuk monitoring skala besar melibatkan pemilihan bin waktu yang tepat: bin terlalu halus (mis. per-detik) pada horizon waktu panjang (bulanan) menghasilkan ribuan kolom yang tidak lagi terbaca visual, sementara bin terlalu kasar (mis. per-hari) pada horizon pendek (harian) menyembunyikan pola anomali jangka pendek yang justru penting untuk deteksi dini. Aturan praktis: pilih bin sedemikian rupa sehingga total kolom heatmap tetap berada di kisaran 50-200 agar tetap terbaca dalam satu layar tanpa perlu scroll horizontal berlebihan.

Pewarnaan (colormap) heatmap juga krusial untuk interpretasi yang benar: colormap sequential seperti viridis atau plasma cocok untuk data yang secara alami memiliki urutan rendah-ke-tinggi (seperti RMS getaran), sedangkan colormap diverging (mis. RdBu) lebih tepat dipakai saat nilai nol memiliki makna khusus (misalnya deviasi dari baseline). Menambahkan anotasi angka pada sel yang melewati ambang batas, seperti dicontohkan pada Gambar 7, membantu operator langsung memahami skala masalah tanpa harus menerka dari gradasi warna semata, terutama penting untuk laporan yang akan dicetak hitam-putih.



Gambar 7. Heatmap RMS getaran 4 motor pada bin waktu 5 menit; sel bernilai tinggi ($>2,5$ mm/s) diberi anotasi angka pada Motor 2 yang mengalami tren kenaikan.

Pola anomali pada Motor 2 di Gambar 7 langsung terlihat sebagai blok warna merah gelap yang meluas ke arah kanan (waktu berjalan), jauh lebih cepat dibaca dibanding menelusuri 4 line plot terpisah satu per satu.

ENAM ANALOGI AKUISISI UNTUK MEMORI VISUAL

- **Mikrofon Studio Rekaman:** rangkaian mikrofon, preamp, ADC, dan digital audio workstation (DAW) persis mencerminkan sensor chain: sensor, signal conditioning, ADC, dan sistem akuisisi.
- **Kamera DSLR vs HP:** kamera DSLR menangkap RAW 14-bit sementara HP umumnya JPEG 8-bit; perbedaan ini identik dengan konsep dynamic range ADC yang dibahas di awal modul.
- **Streaming Netflix vs Download:** menonton video streaming (buffer kecil, real-time) berbeda dengan mengunduh seluruh file (arsip lengkap); ini analog dengan edge buffer versus cloud archive dalam arsitektur tiered storage.
- **Google Maps Zoom Levels:** level zoom pada peta digital memuat ubin (tile) dengan resolusi berbeda sesuai kebutuhan; konsep yang sama dipakai pada tiered storage dan agregasi data time series.

- **Dashboard Pesawat/Mobil:** dashboard pesawat dan mobil menampilkan tiap sensor dengan visualisasi yang sesuai karakternya (gauge untuk kecepatan, grafik untuk tren), sama seperti pemilihan jenis plot yang tepat untuk tiap jenis sinyal.
- **Termometer vs Thermal Imaging:** termometer memberi satu angka skalar (seperti line plot), sedangkan kamera thermal memberi peta spasial suhu (seperti heatmap); keduanya cocok untuk kebutuhan yang berbeda.

APLIKASI DI INDUSTRI: EMPAT STUDI KASUS

- **Vibration Monitoring Pabrik Semen:** 20 lebih motor mill grinding berkapasitas 1-15 MW dipantau dengan akselerometer IEPE, ADC 16-bit pada $f_s=25$ kHz; edge menghitung RMS dan Crest Factor per detik, dikirim via MQTT ke InfluxDB, ditampilkan di dashboard Grafana dengan alert tier mengikuti ISO 10816 kelas II (notice $>2,8$; warning $>4,5$; critical $>7,1$ mm/s).
- **SCADA Kelapa Sawit:** ratusan sensor RTD/thermocouple/pressure/level dengan PLC Allen-Bradley, sampling $f_s=1$ Hz, protokol Modbus TCP ke SCADA Wonderware, historian SQL Server dengan retensi 5 tahun.
- **Power Quality Gardu Distribusi:** current transformer dan potential transformer pada gardu 20 kV, ADC simultaneous-sampling 16-bit pada 20 kHz, FFT real-time untuk THD dan harmonik hingga orde 50, protokol IEC 61850, metrik dihitung per 10 menit mengikuti standar EN 50160.
- **Patient Monitor ICU:** ECG 500 Hz 12-bit, SpO2 1 Hz, NIBP event-based, suhu 0,1 Hz, membutuhkan sinkronisasi timestamp lintas sampling rate berbeda; penyimpanan mengikuti standar HL7 FHIR.

Pelajaran dari lapangan: Beberapa pelajaran dari implementasi nyata: (1) lupa memasang cable shielding menyebabkan interferensi elektromagnetik dari inverter VSD; (2) timeout edge gateway yang terlalu agresif menyebabkan kehilangan data; (3) tidak ada sinkronisasi timestamp (perlu protokol NTP atau PTP IEEE 1588) menyebabkan korelasi antar sensor keliru; (4) query dashboard yang berat karena tanpa agregat pre-computed.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada data sintesis yang meniru sensor industri. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, matplotlib; notebook p3_visualisasi_akuisisi.ipynb. Gambar 8 menunjukkan cuplikan Cell 2 yang membangun plot multi-sensor dengan overlay threshold ISO 10816.



```
import matplotlib.pyplot as plt

thr = dict(good=1.12, warn=2.8, crit=7.1)
fig, axes = plt.subplots(4, 1,
                        sharex=True)

for i, ax in enumerate(axes):
    ax.plot(t, rms[:, i])
    ax.axhline(thr['good'], ls='--',
               color='green')
    ax.axhline(thr['warn'], ls='--',
               color='orange')
    ax.axhline(thr['crit'], ls='--',
               color='red')
    ax.set_title(f'Motor {i+1}')

plt.savefig('rms_panel.png', dpi=300)
```

Gambar 8. Cuplikan Cell 2: multi-sensor line plot 4-panel dengan overlay threshold ISO 10816.

- **Cell 1:** simulasi 4 motor pabrik, vibration RMS per detik selama 1 jam; motor 2 diberi anomali gradual di menit 30-35.
- **Cell 2:** multi-sensor line plot 4-panel dengan overlay threshold ISO 10816 (thr_good=1,12; thr_warn=2,8; thr_crit=7,1 mm/s).
- **Cell 3:** spectrogram sinyal startup motor (frequency sweep 5 ke 25 Hz plus harmonik di t=2 detik), verifikasi peak frekuensi di t=3,5 detik (perkiraan 22,5 Hz).
- **Cell 4:** heatmap hasil resample 5-menit max, ekspor perbandingan ukuran file CSV vs Parquet.

Keempat cell tersebut, bila dijalankan berurutan dalam satu notebook, menghasilkan alur kerja end-to-end mulai dari data mentah tersimulasi hingga perbandingan efisiensi format penyimpanan, mencerminkan pipeline visualisasi dan akuisisi data yang telah dibahas sepanjang pertemuan ini secara utuh dan siap diadaptasi ke data sensor riil di lapangan.

TUGAS PERTEMUAN 3

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.



Gambar 9. Distribusi 50 poin Tugas Pertemuan 3 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Komponen utama dalam sebuah sensor chain industri dari sensor fisik hingga storage adalah...

- A. Hanya sensor dan komputer, komponen lain tidak diperlukan
- B. Sensor, Signal Conditioning, ADC, Edge Device, Storage/Cloud secara berurutan
- C. Sensor, AI model langsung, output
- D. ADC, Cloud, Sensor (urutan terbalik)

2. Dynamic Range (DR) sebuah ADC N-bit dapat dihitung dengan rumus...

- A. $DR = N \text{ dB}$
- B. $DR = 6,02 \cdot N + 1,76 \text{ dB}$
- C. $DR = 2^N \text{ dB}$
- D. $DR = \log_2(N) \text{ dB}$

3. Untuk monitoring vibration motor 1500 RPM dengan kebutuhan deteksi bearing fault hingga 5 kHz, sampling rate yang tepat adalah...

- A. 1 kHz, sudah cukup
- B. 5 kHz, tepat di Nyquist boundary
- C. 25-50 kHz, oversampling 5-10 kali untuk margin anti-aliasing filter
- D. 1 MHz, selalu ambil yang tertinggi

4. Manfaat utama dari arsitektur tiered storage (hot/warm/cold) untuk data time series adalah...

- A. Selalu mendapat performa query yang sama
- B. Mengoptimalkan biaya storage: data yang sering diakses di SSD cepat, data lama di tier murah dengan downsampling otomatis

- C. Menghilangkan kebutuhan untuk database
 - D. Membuat data tidak bisa di-query lagi
5. Saat sebuah dashboard menampilkan line plot dari 100 motor bersamaan, masalah utamanya adalah...
- A. Tidak ada masalah, line plot selalu pilihan terbaik
 - B. Visual clutter, pola abnormal sulit terlihat; heatmap dengan baris per sensor jauh lebih readable
 - C. Browser tidak bisa render 100 line
 - D. Memerlukan komputer dengan RAM 32 GB
6. Untuk visualisasi perubahan frekuensi dominan seiring waktu (misalnya motor startup), plot yang tepat adalah...
- A. Line plot biasa
 - B. Boxplot per shift
 - C. Spectrogram (waktu vs frekuensi, warna merepresentasikan amplitudo)
 - D. Pie chart
7. Pernyataan yang benar mengenai format data time series...
- A. CSV adalah format terbaik untuk dataset besar (>1 GB)
 - B. Parquet (columnar, compressed) jauh lebih efisien dari CSV untuk batch analytics, biasanya 10-20 kali lebih kecil
 - C. JSON adalah format paling efisien untuk numerical data
 - D. XML wajib untuk semua time series
8. Yang bukan merupakan masalah saat sebuah ADC rentangnya terlalu lebar (mis. $\pm 10V$) untuk sinyal sensor lemah ($\pm 100\text{ mV}$) adalah...
- A. Resolusi efektif berkurang drastis, kehilangan sekitar 7 bit untuk sinyal 1% dari rentang
 - B. SNR rendah karena sebagian besar level ADC tidak digunakan
 - C. Sinyal pasti clipping dan terdistorsi (ini justru terjadi saat rentang terlalu sempit, bukan terlalu lebar)
 - D. Solusi: pakai gain amplifier sebelum ADC untuk mencocokkan rentang
9. Streaming protocol yang tepat untuk monitoring puluhan motor di pabrik dengan latency rendah dan resource gateway terbatas adalah...
- A. HTTP polling tiap detik
 - B. MQTT (lightweight pub/sub, low overhead, ideal untuk IoT)
 - C. FTP file transfer setiap menit
 - D. Email dengan attachment data
10. Anti-pattern yang harus dihindari saat membuat visualisasi time series adalah...

- A. Memberi label sumbu dengan unit yang jelas
- B. Memakai colormap viridis untuk data kontinu
- C. Memplot ribuan titik tanpa downsampling sehingga berat, dan memakai rainbow colormap yang tidak perceptually uniform
- D. Memberi judul plot yang informatif

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

C1. Hitung bandwidth (byte per detik) dari sebuah ADC yang men-sampling pada $f_s=10$ kHz, resolusi 16-bit, 1 channel. Rumus: $BW = f_s \cdot (\text{bits}/8) \cdot \text{channels}$.

- 💡 **HINT:** Bandwidth = sample_rate x (bit_depth / 8) x jumlah channel. Substitusikan nilai dari soal.

C2. Hitung storage requirement dalam MB untuk 1 jam data ADC $f_s=10$ kHz, 16-bit, 1 channel. Konversi: 1 MB = 1024^2 byte.

- 💡 **HINT:** Storage = bandwidth x durasi (detik), lalu bagi 1024^2 untuk konversi ke MB. Masukkan nilai dari soal.

C3. Hitung Dynamic Range (dB) untuk ADC 16-bit. Rumus: $DR = 6,02 \cdot N + 1,76$.

- 💡 **HINT:** Dynamic range (dB) = $6,02 \times N_{\text{bits}} + 1,76$. Masukkan jumlah bit dari soal.

C4. Hitung LSB (Least Significant Bit) dalam microvolt untuk ADC $\pm 10V$ (rentang 20V), 16-bit. Rumus: $LSB = V_{\text{range}} / 2^N$, kalikan $1e6$ untuk dapat microvolt.

- 💡 **HINT:** $LSB = \text{rentang tegangan} / 2^N$; kalikan $1e6$ untuk konversi ke microvolt. Masukkan rentang dan jumlah bit dari soal.

C5. Dari dataset referensi simulasi 4 motor (mean RMS per motor), hitung mean dari motor_1_rms. Cetak `np.mean(df['motor_1_rms'])`.

- 💡 **HINT:** Gunakan `df['motor_1_rms'].mean()` untuk menghitung rata-rata kolom tersebut.

C6. Hitung nilai maksimum dari motor_2_rms yang mengandung event anomali injeksi. Nilai max akan jauh di atas baseline.

- 💡 **HINT:** Gunakan `df['motor_2_rms'].max()` untuk menemukan nilai puncak kolom tersebut.

C7. Hitung jumlah titik di motor_2_rms yang melebihi 2,8 mm/s (threshold ISO 10816 warning). Pakai boolean indexing.

- 💡 **HINT:** Buat mask boolean dengan kondisi `df['motor_2_rms'] > threshold` lalu jumlahkan dengan `.sum()` untuk menghitung jumlah sampel yang memenuhi.

C8. Lakukan resampling 5-menit max aggregation dengan pandas: `df.resample('5min').max()`. Cetak jumlah baris dari hasil resample (60 menit / 5 menit).

- 💡 **HINT:** Gunakan `df.resample('5min').max()` lalu hitung jumlah baris hasilnya dengan `len()`.

C9. Hitung jumlah motor yang max RMS-nya melebihi 2,5 mm/s dari dataset referensi 4 motor.

- 💡 **HINT:** Hitung nilai maksimum tiap kolom dengan `df.max()`, lalu hitung berapa kolom yang melebihi threshold dengan kondisi boolean dan `.sum()`.

C10. Hitung frekuensi peak dari FFT sinyal sinusoidal murni 50 Hz dengan `fs=5000` Hz selama 1 detik. Pakai `np.fft.rfft` dan `np.fft.rfftfreq`.

- 💡 **HINT:** Bangun sinyal, hitung FFT dengan `np.fft.rfft` dan frekuensinya dengan `np.fft.rfftfreq`, lalu ambil frekuensi pada magnitudo terbesar via `np.argmax(np.abs(...))`.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan analisis spectrogram lengkap. Generate sinyal test 30 Hz murni dengan `fs=1000` Hz, durasi 2 detik. Gunakan `scipy.signal.spectrogram` dengan `nperseg=128`, `noverlap=64`. Cari frekuensi rata-rata dengan power tertinggi sepanjang waktu (ambil mean power per frekuensi, cari `argmax`).

- 💡 **HINT:** Gunakan `scipy.signal.spectrogram`, rata-ratakan power sepanjang sumbu waktu (`np.mean(Sxx, axis=1)`), lalu ambil frekuensi dengan power rata-rata terbesar.

C12 (Hard). Demonstrasikan aliasing detection melalui FFT. Generate sinyal 120 Hz murni tapi disampling pada `fs=100` Hz (di bawah Nyquist) selama 1 detik. Hitung FFT, cetak frekuensi peak yang tampak, yaitu frekuensi alias $|f_{\text{sinyal}} - fs|$, bukan 120 Hz asli.

- 💡 **HINT:** Sampel sinyal di bawah Nyquist, lalu cari frekuensi peak via FFT (`rfft/rfftfreq + argmax`); frekuensi alias = $|f - k \cdot fs|$ untuk k integer terdekat.

C13 (Hard). Implementasikan heatmap aggregation pipeline pada dataset referensi 4 motor: (1) resample 5-menit max, (2) buat matrix transposed (baris=motor, kolom=bin waktu), (3) hitung jumlah cell yang melebihi threshold 2,5 mm/s.

- 💡 **HINT:** Resample data ke bin 5 menit dengan `resample('5min').max()`, lalu hitung berapa sel yang melebihi threshold dengan kondisi boolean dan `np.sum`.

C14 (Hard). Hitung rata-rata pairwise correlation antar 4 motor di dataset referensi. Pakai `df.corr()`, ambil hanya bagian off-diagonal upper triangle, hitung mean. Untuk 4 motor independen (kecuali 1 motor bermasalah), korelasi seharusnya sangat kecil.

- 💡 **HINT:** Hitung matriks korelasi dengan `df.corr()`, ambil elemen off-diagonal segitiga atas (`np.triu_indices_from, k=1`), lalu rata-ratakan dengan `np.mean`.

C15 (Hard). Hitung storage requirement edge tier dalam GB untuk arsitektur: 8 motor, $f_s=10$ kHz, 16-bit, retensi 15 hari. Rumus: $\text{storage_GB} = f_s \cdot (\text{bits}/8) \cdot \text{channels} \cdot \text{detik_per_hari} \cdot \text{hari} / 1024^3$.

- 💡 **HINT:** $\text{Storage} = f_s \times (\text{bits}/8) \times \text{channel} \times \text{detik_per_hari} \times \text{jumlah_hari}$, lalu bagi 1024^3 untuk konversi ke GB; substitusikan parameter dari soal.

DAFTAR PUSTAKA

- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. *IEEE Transactions on Knowledge and Data Engineering*, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 4

Preprocessing Dasar: Pembersihan

Abstrak

Pertemuan ini membahas tahap fundamental preprocessing data sensor time series pada konteks predictive

Sub - CPMK

Sub-CPMK 2.1 – Mampu menganalisis dampak sosial dan lingkungan dari sistem otomasi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-4.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan keempat ini membahas tahap fundamental preprocessing data sensor time series pada konteks predictive maintenance. Prinsip "Garbage In, Garbage Out" berlaku mutlak: keputusan preprocessing yang keliru dapat menghapus sinyal peringatan dini kerusakan yang justru ingin dideteksi, sehingga berdampak pada keandalan operasional dan keselamatan kerja secara luas. Gambar 1 menunjukkan pipeline enam tahap pembersihan data yang dibahas pada pertemuan ini.



Gambar 1. Pipeline pembersihan data time series enam tahap: Inspeksi Awal, Missing Values, Outlier Detection, Noise Reduction, Resampling, dan Validasi Akhir.

Preprocessing memakan sekitar 60-80% waktu kerja data scientist dalam praktiknya. Contoh skala data: accelerometer 200 Hz yang direkam selama 8 jam menghasilkan 5,76 juta titik data yang harus diperiksa dan dibersihkan sebelum dianalisis. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan forum diskusi studi kasus.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 2.1:** Mampu menganalisis dampak sosial dan lingkungan dari sistem otomasi, yaitu memahami bagaimana keputusan preprocessing data (pembersihan missing value, outlier, dan noise) memengaruhi keandalan sistem monitoring yang pada akhirnya berdampak pada keselamatan operasional dan lingkungan kerja (CPMK 2).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan pipeline pembersihan data time series enam tahap; mendeteksi dan menangani missing values dengan teknik yang sesuai; mendeteksi outlier dengan Z-score dan IQR tanpa membuang sinyal

kerusakan dini; menerapkan noise reduction yang tepat; serta melakukan resampling dan normalisasi data multi-sensor.

MISSING VALUES: DETEKSI & PENANGANAN

Missing values dideteksi dengan `df.isnull().sum()` untuk menghitung jumlah nilai kosong per kolom, `df.isna().mean()*100` untuk persentasenya, atau `df[df.isnull().any(axis=1)]` untuk menampilkan baris yang mengandung nilai kosong.

- **Drop:** menghapus baris yang mengandung NaN (`df.dropna()`); cocok bila persentase missing kecil (<5%) dan tersebar acak.
- **Forward/Backward Fill:** mengisi dengan nilai observasi terakhir (`df.fillna(method='ffill')`); cocok untuk data yang lambat berubah.
- **Mean/Median Fill:** mengisi dengan rata-rata atau median kolom (`df.fillna(df.mean())`); sederhana tetapi menumpulkan variabilitas data.
- **Interpolasi:** mengisi berdasarkan interpolasi nilai tetangga (`df.interpolate(method='time')`); umumnya menjadi pilihan terbaik untuk time series numerik.

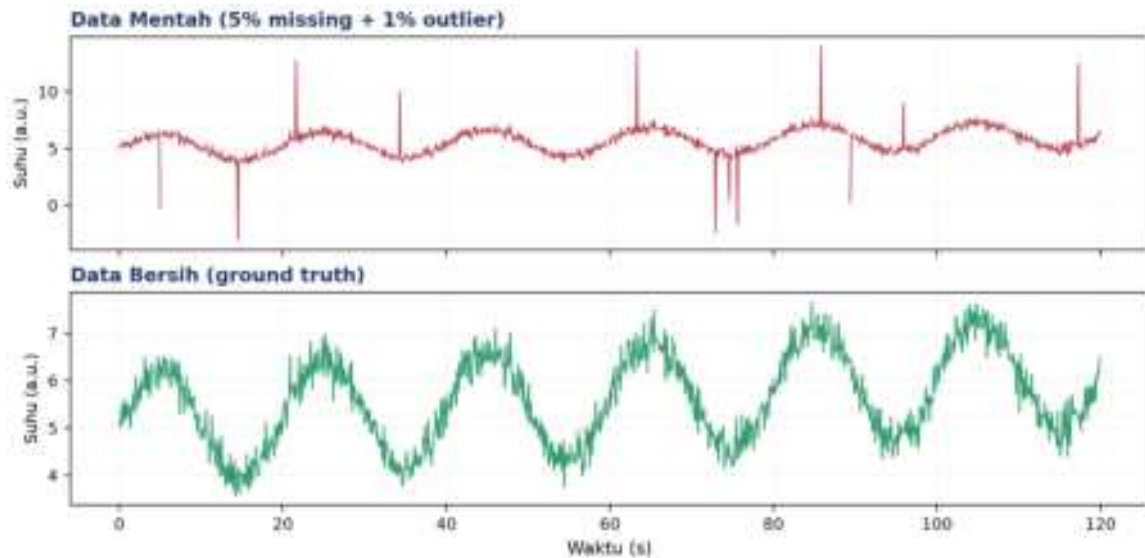
Tabel 1 merangkum lima pola missing values yang umum dijumpai beserta karakteristik dan teknik penanganan yang disarankan.

Tabel 1. Pola missing values pada data sensor dan teknik penanganan yang disarankan.

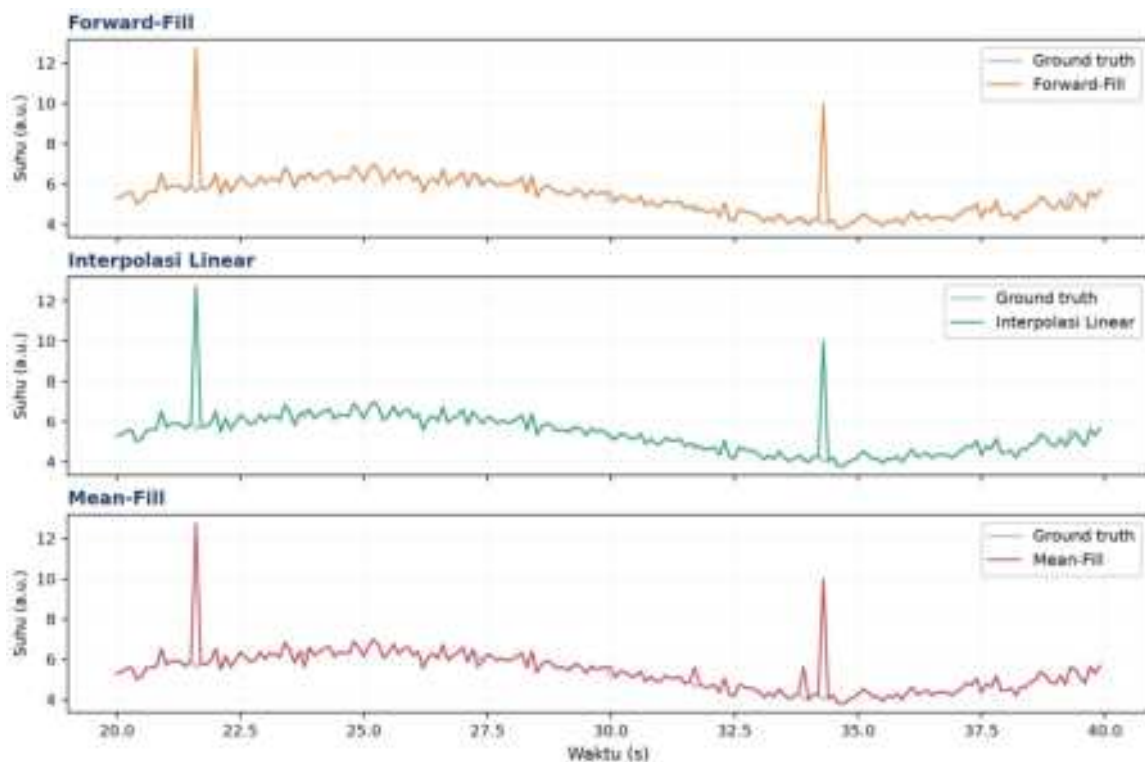
Pola Missing	Karakteristik	Teknik Disarankan
MCAR (Acak Sepenuhnya)	Hilang tersebar tanpa pola	Drop atau Mean Fill
MAR (Acak Bersyarat)	Hilang berkorelasi dengan kolom lain	Interpolasi atau model-based imputation
MNAR (Tidak Acak)	Hilang ketika sensor jenuh/di luar batas	Tandai dengan flag, jangan diisi sembarangan
Gap waktu pendek (<5 sampel)	Komunikasi sesaat terputus	Forward-fill atau interpolasi linear
Gap waktu panjang (>1 menit)	Mesin off/sensor maintenance	Tandai segmen, hindari interpolasi paksa

Mengapa Klasifikasi Pola Missing Penting: Penting dibedakan antara MCAR, MAR, dan MNAR karena kesalahan klasifikasi pola missing values dapat menghasilkan bias sistematis pada analisis lanjutan. Contoh nyata MNAR pada sensor industri: accelerometer yang mencatat nilai kosong justru saat amplitudo getaran melampaui rentang pengukuran (sensor saturasi/clipping), sehingga dalam kasus ini, data yang hilang JUSTRU berkorelasi dengan kondisi paling kritis yang ingin dideteksi. Mengisi nilai kosong tersebut dengan mean atau interpolasi akan secara sistematis menyembunyikan kejadian ekstrem yang

sebenarnya paling penting untuk sistem predictive maintenance, sehingga pendekatan yang benar adalah menandainya dengan flag khusus (mis. ``sensor_saturated=True``) alih-alih mengisi nilainya sama sekali.



Gambar 2. Perbandingan data mentah (5% missing + 1% outlier) dengan data bersih (ground truth).



Gambar 3. Perbandingan tiga teknik imputasi: Forward-Fill, Interpolasi Linear, dan Mean-Fill pada segmen data yang sama.

Gambar 3 memperlihatkan interpolasi linear mengikuti bentuk data ground truth paling dekat, forward-fill menciptakan efek "tangga" yang kurang halus, sementara mean-fill paling menyimpang karena mengabaikan tren lokal sinyal.

Peringatan: Mengisi missing values saat mesin sedang shutdown dengan mean atau forward-fill membuat model seolah-olah melihat mesin terus berjalan normal, padahal sesungguhnya tidak beroperasi, sehingga data menjadi menyesatkan bagi analisis lanjutan.

OUTLIER DETECTION & PENANGANAN

Dua metode deteksi outlier yang paling umum dipakai adalah Z-score dan IQR (Interquartile Range).

Z-score: $z = (x - \mu) / \sigma$, outlier jika $|z| > 3$ **IQR:** outlier jika $x < Q1 - 1,5 \cdot IQR$ atau $x > Q3 + 1,5 \cdot IQR$

- **Removal:** menghapus titik yang terdeteksi sebagai outlier (`df[z<3]`).
- **Capping/Winsorizing:** membatasi nilai ekstrem ke persentil tertentu tanpa menghapus titik data (`np.clip(x, q01, q99)`).
- **Flag & Investigate:** menandai titik sebagai outlier tanpa mengubah nilainya (`df['is_outlier']=(z>3)`), direkomendasikan khusus untuk monitoring kondisi mesin agar sinyal kerusakan tidak hilang.
- **Imputasi:** mengganti nilai outlier dengan median lokal (`x[outlier]=median(x)`).

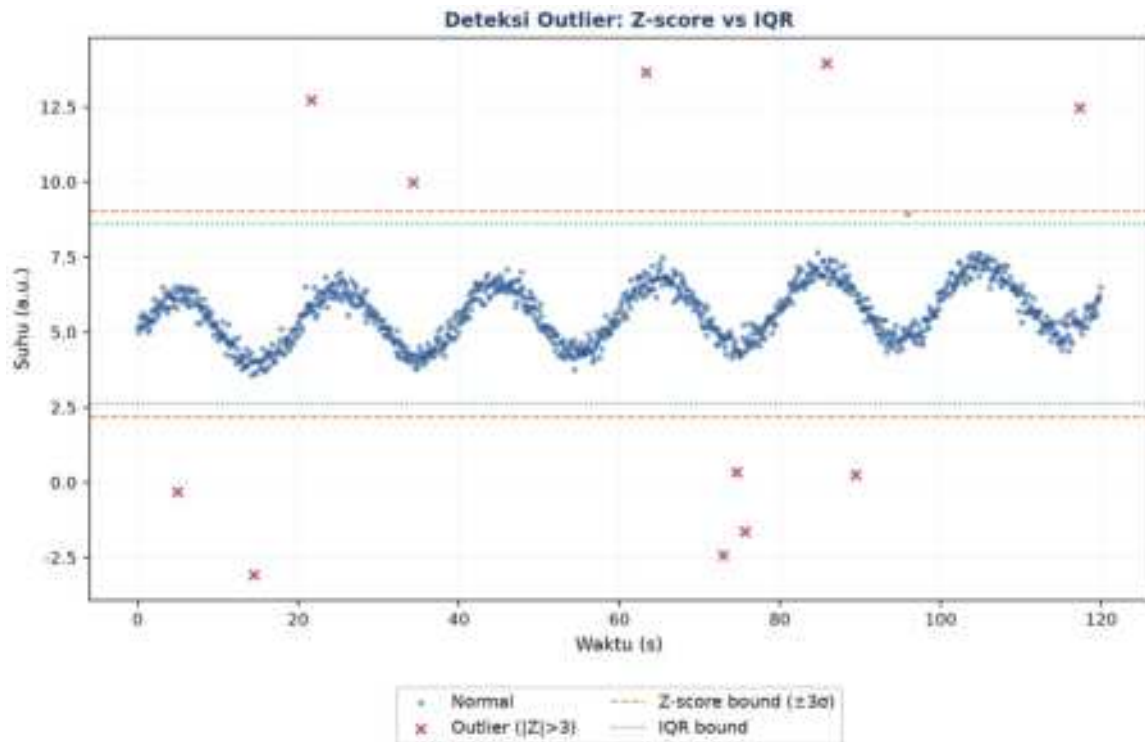
Tabel 2 membandingkan lima metode deteksi outlier dari sisi asumsi distribusi, keunggulan, dan keterbatasannya.

Tabel 2. Perbandingan lima metode deteksi outlier terhadap asumsi distribusi, keunggulan, dan keterbatasan.

Metode	Asumsi Distribusi	Keunggulan	Keterbatasan
Z-Score	Mendekati normal (Gaussian)	Cepat, intuitif, mudah diinterpretasi	Sensitif terhadap outlier itu sendiri
IQR	Bebas asumsi distribusi	Robust, tidak dipengaruhi outlier ekstrem	Lebih konservatif, kadang melewatkan outlier moderat
Modified Z-Score (MAD)	Bebas asumsi distribusi	Robust dengan skala mirip Z-score (threshold 3,5)	Perlu perhitungan tambahan
Rolling Z-Score	Stasioner dalam window	Cocok untuk time series dengan tren/musiman	Pemilihan window size krusial
Isolation Forest	Bebas asumsi (model ML)	Multi-variati, menangkap outlier kompleks	Black-box, butuh tuning hyperparameter

Kapan Isolation Forest Diperlukan: Isolation Forest layak dipertimbangkan ketika outlier bersifat multivariat, yaitu tidak terdeteksi bila tiap sensor dianalisis satu per satu, tetapi menjadi jelas saat kombinasi beberapa sensor diperiksa bersamaan. Contohnya, suhu

bearing 60°C dan RPM 1500 masing-masing normal bila dilihat terpisah, namun kombinasi keduanya (suhu tinggi pada RPM rendah) justru mengindikasikan gesekan berlebih akibat pelumasan yang buruk, yaitu pola yang hanya terlihat pada ruang fitur multi-dimensi, bukan pada distribusi univariat Z-score atau IQR sederhana.



Gambar 4. Deteksi outlier dengan batas Z-score (garis putus-putus oranye) dan IQR (garis titik-titik hijau); titik merah ditandai sebagai outlier.

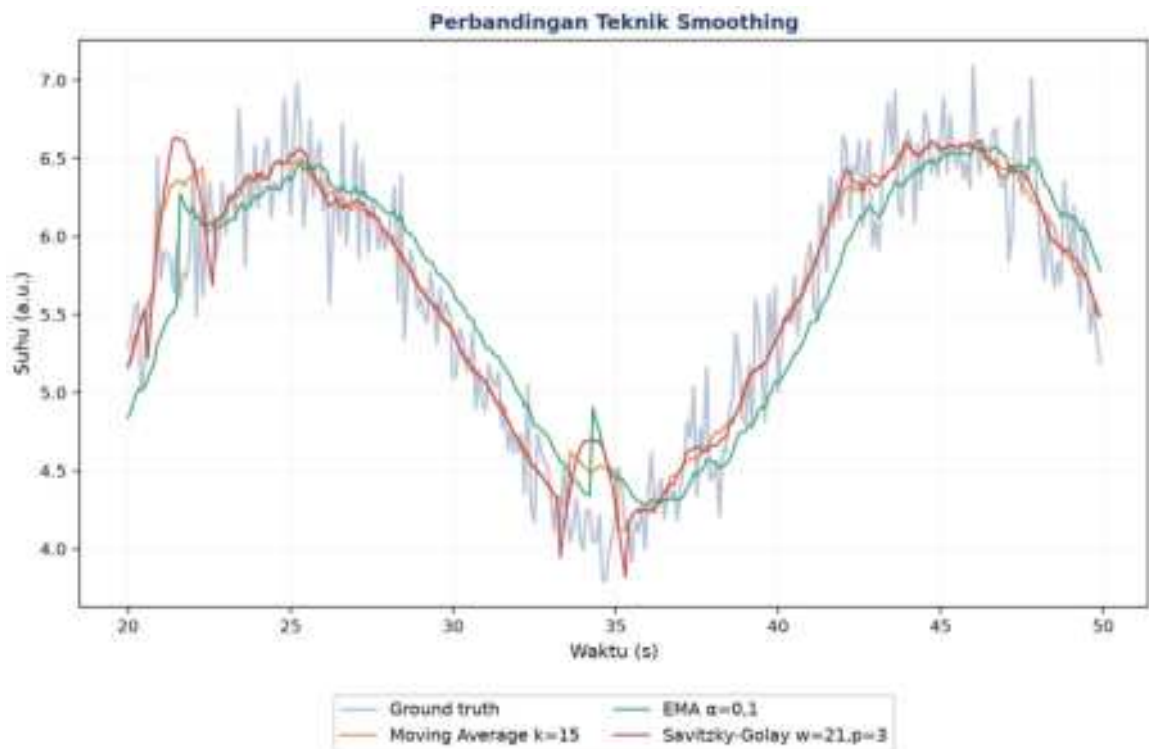
Studi kasus: Sebuah bearing pompa sentrifugal dengan amplitudo getaran rata-rata 2 mm/s tiba-tiba melonjak ke 25 mm/s selama 0,3 detik, menghasilkan Z-score sekitar 8 yang jelas terdeteksi sebagai outlier oleh algoritma standar. Namun lonjakan ini sebenarnya adalah impulse dari bearing yang mulai retak; menghapusnya sebagai outlier berarti membuang sinyal peringatan dini yang justru paling berharga.

NOISE REDUCTION: SMOOTHING & FILTER

Empat teknik smoothing yang umum dipakai: Moving Average (`df.rolling(w).mean()`, window besar berarti lebih halus tetapi delay lebih besar), Exponential Moving Average (`df.ewm(alpha=0.3).mean()`, memberi bobot lebih besar pada data terbaru), Savitzky-Golay Filter (`savgol_filter(x,51,3)`, mempertahankan bentuk puncak lebih baik), dan Low-Pass Filter (`scipy.signal.butter(N, fc)`, Butterworth/FIR untuk filtering yang lebih presisi).

Kapan Memilih Butterworth: Low-Pass Filter Butterworth layak menjadi pilihan default saat presisi kontrol frekuensi cutoff menjadi prioritas, karena parameternya (orde N ,

frekuensi cutoff f_c) langsung dapat diterjemahkan ke spesifikasi respons frekuensi yang diinginkan, berbeda dengan Moving Average yang parameter window-nya (k) hanya berhubungan tidak langsung dengan frekuensi cutoff efektif melalui hubungan $f_{\text{cutoff}} \approx f_s/k$. Pada scipy, ``butter(N, fc, fs=fs)`` mengembalikan koefisien filter yang kemudian diterapkan via ``filtfilt`` (zero-phase, non-causal) untuk analisis offline, atau ``lfilter`` (causal, ada phase lag) untuk aplikasi real-time.



Gambar 5. Perbandingan Moving Average, EMA, dan Savitzky-Golay pada segmen data yang sama.

Cara memilih window size: Untuk mempertahankan komponen 1 Hz pada data yang di-sampling 100 Hz, sebaiknya window moving average tidak melebihi 100 sampel; window yang lebih besar akan meredam komponen frekuensi tersebut secara tidak sengaja.

Peringatan: Smoothing yang terlalu kuat dapat menyamarkan impulse kerusakan bearing (buruk untuk deteksi kerusakan), meskipun bermanfaat untuk melihat tren suhu jangka panjang. Pemilihan intensitas smoothing harus disesuaikan dengan tujuan analisis.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Bersih-bersih Dapur sebelum Memasak:** mempersiapkan bahan sebelum memasak, mencuci dan memotong, adalah analogi langsung dari tahap preprocessing sebelum data siap dianalisis.

- **Sambungan Telepon Terputus Sesaat:** otak manusia otomatis mengisi kata yang hilang saat sambungan telepon terputus sesaat, mirip interpolasi; gap pendek aman ditebak, tetapi gap panjang berbahaya untuk ditebak sembarangan.
- **Filter Foto Smartphone:** filter noise reduction yang terlalu kuat pada foto membuat wajah tampak seperti plastik, sama seperti smoothing berlebihan yang menghilangkan detail bermakna pada sinyal.
- **Termometer yang Kadang Salah Baca:** membedakan termometer yang salah baca (outlier error) dari suhu ekstrem nyata seperti gelombang panas adalah persoalan yang identik dengan membedakan outlier sensor dari kejadian fisik nyata.
- **Membandingkan Nilai Ujian Antar Sekolah:** membandingkan nilai ujian antar sekolah dengan skala penilaian berbeda membutuhkan normalisasi/standardisasi, sama seperti menyelaraskan skala fitur multi-sensor sebelum dianalisis bersama.

RESAMPLING, NORMALISASI & VALIDASI AKHIR

Min-Max: $x_{norm} = (x - x_{min}) / (x_{max} - x_{min})$

Z-score: $x_{std} = (x - \mu) / \sigma$

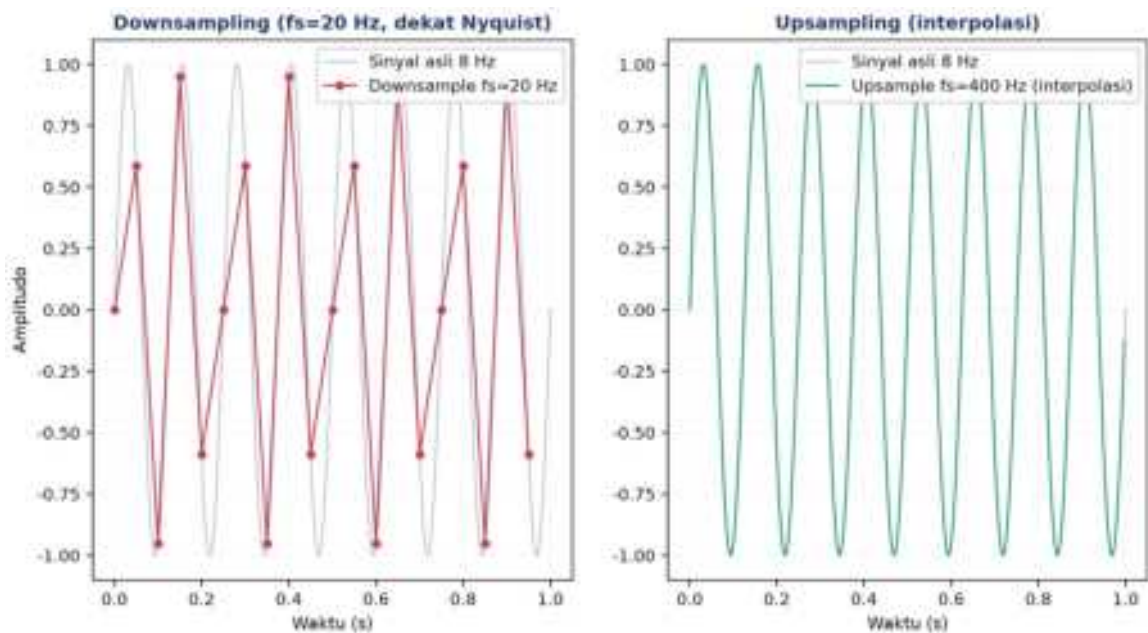
- **Downsampling/Aggregation:** agregasi data ke sampling rate lebih rendah (`df.resample('1S').mean()`); harus hati-hati terhadap potensi aliasing.
- **Upsampling/Interpolation:** interpolasi data ke sampling rate lebih tinggi (`df.resample('10S').interpolate()`).
- **Min-Max Scaling:** menyamakan rentang ke [0,1] (`MinMaxScaler()`); cocok untuk algoritma seperti k-NN dan neural network.
- **Z-Score Standardization:** menyamakan mean=0, std=1 (`StandardScaler()`); cocok untuk PCA dan regresi linear, lebih robust terhadap outlier dibanding Min-Max.

Tabel 3 merangkum konteks aplikasi dan teknik resampling/normalisasi yang sesuai.

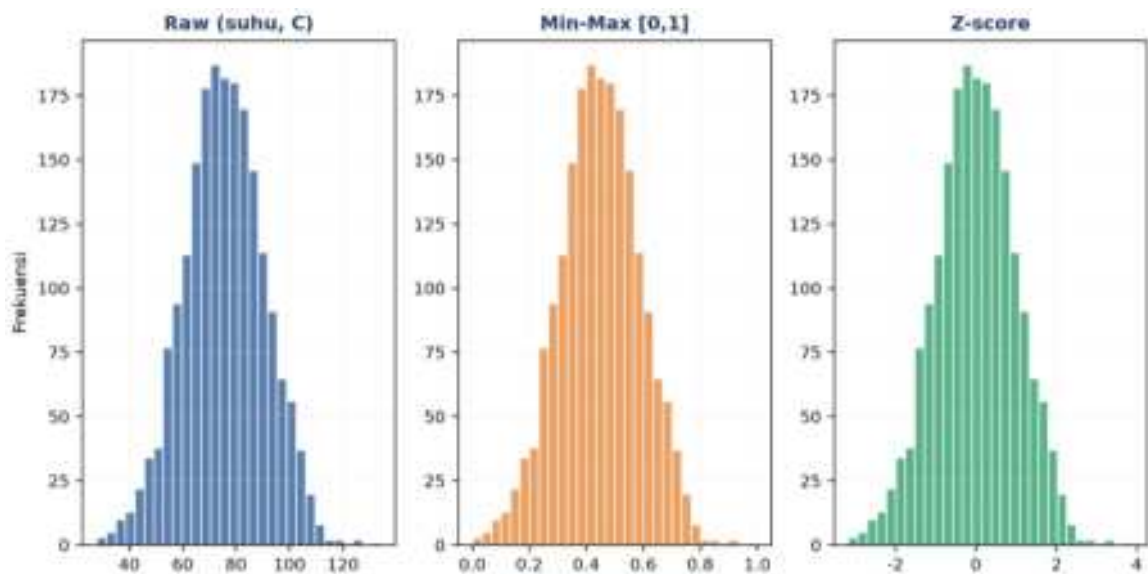
Tabel 3. Konteks aplikasi dan teknik resampling/normalisasi yang sesuai.

Konteks	Teknik Disarankan	Alasan
Multi-sensor join (suhu + getaran)	Resample ke laju sama + Z-score	Skala fisik berbeda, distribusi tidak diketahui
Dashboard monitoring real-time	Downsample (mean) ke 1 Hz	Hemat bandwidth, tidak butuh resolusi tinggi
Training neural network	Min-max scaling [0,1]	Konvergensi lebih cepat, gradient lebih stabil
FFT/spectrum analysis	Resample uniform + jangan normalisasi	FFT butuh sampling uniform; normalisasi mengubah amplitudo asli
Anomaly detection berbasis statistik	Z-score per-window (rolling)	Adaptif terhadap perubahan baseline (drift)

Downsampling untuk Join Multi-Sensor: Downsampling multi-sensor perlu dilakukan hati-hati saat sensor-sensor tersebut akan digabungkan (joined) untuk analisis korelasi: menyamakan seluruh sensor ke sampling rate terendah di antara mereka (mis. suhu 1 Hz vs getaran 1000 Hz disamakan ke 1 Hz) memang menyederhanakan join, tetapi membuang informasi frekuensi tinggi getaran yang mungkin justru relevan. Alternatif yang lebih baik: hitung agregat statistik (RMS, peak, kurtosis) dari sinyal getaran resolusi penuh pada tiap jendela waktu yang sesuai dengan sampling rate sensor suhu, sehingga informasi frekuensi tinggi tidak hilang begitu saja melainkan terkondensasi menjadi fitur yang tetap bermakna secara statistik.



Gambar 6. Ilustrasi downsampling ($fs=20$ Hz, dekat batas Nyquist) dan upsampling via interpolasi pada sinyal 8 Hz.



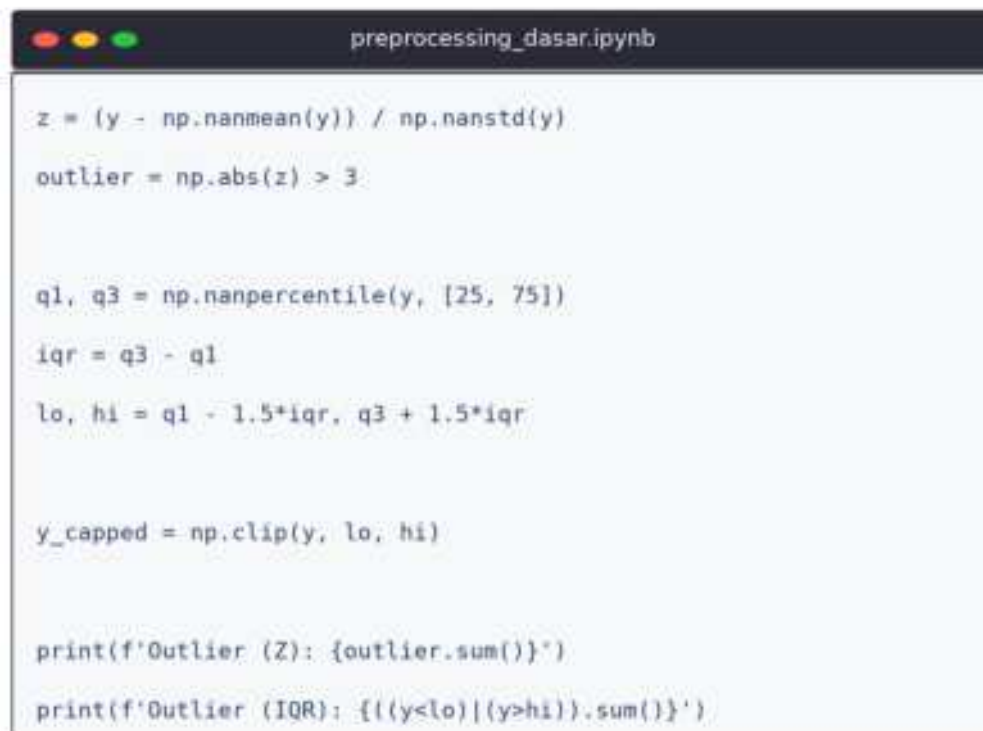
Gambar 7. Distribusi data sebelum (raw) dan sesudah normalisasi Min-Max serta Z-score.

Perhatikan pada Gambar 7 bahwa bentuk distribusi (skewness, kurtosis relatif) tidak berubah setelah normalisasi, hanya rentang nilainya yang berbeda: Min-Max menghasilkan rentang [0,1] sedangkan Z-score menghasilkan mean nol dengan sebaran simetris di sekitarnya.

Validasi akhir wajib: Validasi akhir sebelum data dianggap siap dianalisis mencakup empat pemeriksaan: (1) plot perbandingan raw vs cleaned; (2) bandingkan statistik dasar sebelum dan sesudah; (3) pastikan `df.isnull().sum()` bernilai nol; (4) pastikan indeks waktu monoton naik tanpa duplikat.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada data sintesis yang meniru sensor accelerometer pada spindle mesin CNC milling. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, matplotlib; notebook `preprocessing_dasar.ipynb`. Gambar 8 menunjukkan cuplikan Cell 3 yang mendeteksi outlier dengan Z-score dan IQR sekaligus.



```
z = (y - np.nanmean(y)) / np.nanstd(y)
outlier = np.abs(z) > 3

q1, q3 = np.nanpercentile(y, [25, 75])
iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr

y_capped = np.clip(y, lo, hi)

print(f'Outlier (Z): {outlier.sum()}')
print(f'Outlier (IQR): {(y<lo)|(y>hi)}.sum()')
```

Gambar 8. Cuplikan Cell 3: deteksi outlier Z-score dan IQR sekaligus, dilanjutkan dengan capping nilai ekstrem.

- **Cell 1:** generate sinyal sintesis ($f_s=10$ Hz, durasi 120 detik), trend+osilasi+noise, injeksi sekitar 5% missing dan 1% outlier (spike ± 8).

- **Cell 2:** bandingkan 3 metode imputasi (forward-fill, interpolasi time, mean fill), plot 3 subplot, cetak std masing-masing.
- **Cell 3:** deteksi outlier Z-score dan IQR pada hasil interpolasi, capping ke batas IQR (`np.clip`), plot sebelum dan sesudah capping.
- **Cell 4:** pipeline lengkap: Savitzky-Golay smoothing (window=21, polyorder=3), resample ke 1 Hz, min-max normalize [0,1], plot 2 subplot.

TUGAS PERTEMUAN 4

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.



Gambar 9. Distribusi 50 poin Tugas Pertemuan 4 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Sebuah sensor suhu pada motor mengalami gap missing pendek 1-3 sampel akibat noise komunikasi sesaat. Teknik imputasi mana yang paling tepat dan aman untuk situasi ini?

- A. Drop semua baris yang mengandung NaN, kehilangan kontinuitas waktu
- B. Interpolasi linear/forward-fill, mempertahankan kontinuitas dan mendekati nilai sebenarnya
- C. Isi dengan mean seluruh kolom, menumpulkan dinamika lokal
- D. Biarkan NaN dan teruskan ke model, model akan error

2. Sensor accelerometer pada bearing pompa sentrifugal menunjukkan spike amplitudo besar yang berulang dengan periode tetap. Mengapa membuang spike ini sebagai outlier Z-score bisa berbahaya untuk monitoring kondisi mesin?

- A. Karena algoritma deteksi outlier menjadi lambat
- B. Spike berulang adalah impulse dari bearing yang mulai retak, sinyal kerusakan dini, bukan noise
- C. Karena membuat distribusi tidak Gaussian
- D. Karena membuat standar deviasi data berkurang signifikan

3. Untuk dataset dengan distribusi non-Gaussian dan banyak outlier ekstrem, metode mana yang paling robust untuk mendeteksi outlier?

- A. Z-score standar ($|z| > 3$), karena mean dan std terdistorsi oleh outlier itu sendiri
- B. IQR ($Q1 - 1,5 \cdot IQR$, $Q3 + 1,5 \cdot IQR$), bebas asumsi distribusi, tidak terdistorsi
- C. Mean $\pm 2\sigma$, sama sensitifnya dengan Z-score
- D. Min/Max scaling, bukan metode deteksi outlier

4. Apa konsekuensi memilih window size yang terlalu besar pada moving average untuk smoothing sinyal vibrasi?

- A. Sinyal hasil menjadi lebih noisy daripada sinyal mentah
- B. Tidak ada konsekuensi, semakin besar window selalu lebih baik
- C. Puncak/transien tertumpul atau terhapus, indikator kerusakan dini bisa hilang
- D. Kode menjadi error karena window melebihi panjang data

5. Pada resampling time series dari 100 Hz menjadi 10 Hz menggunakan rata-rata blok, syarat Nyquist criterion agar tidak terjadi aliasing adalah...

- A. Sinyal harus memiliki rata-rata nol
- B. Window resampling harus berukuran ganjil
- C. Frekuensi tertinggi sinyal harus di bawah 5 Hz, yaitu setengah dari sampling rate baru
- D. Harus menggunakan interpolasi cubic spline

6. Saat membersihkan data sensor, ditemukan banyak baris dengan timestamp identik (duplikat). Pendekatan terbaik untuk menanganinya adalah...

- A. Biarkan saja, duplikat tidak memengaruhi analisis time series
- B. Hapus dengan `df.drop_duplicates()` atau agregasi (mean/last) per timestamp
- C. Tambahkan 1 milidetik ke setiap timestamp duplikat agar unik
- D. Konversi indeks waktu menjadi string untuk mengabaikan duplikasi

7. Untuk dataset dengan beberapa outlier ekstrem, mengapa Modified Z-score (berbasis MAD) lebih disarankan daripada Z-score standar?

- A. Modified Z-score lebih cepat dihitung secara komputasi
- B. Median dan MAD (median absolute deviation) tidak terdistorsi oleh outlier itu sendiri
- C. Modified Z-score selalu menghasilkan nilai threshold $|z| > 2$

- D. MAD memerlukan asumsi distribusi normal yang ketat

8. Saat downsampling sensor 1 kHz menjadi 10 Hz menggunakan agregasi rata-rata blok 100 sampel, efek apa yang akan terjadi pada sinyal?

- A. Memperbesar amplitudo spike pendek karena akumulasi
- B. Spike pendek tertumpul dan baseline level tetap, tetapi komponen frekuensi tinggi hilang
- C. Pasti menyebabkan aliasing tanpa peduli isi sinyal
- D. Membuat seluruh hasil menjadi NaN karena pembagian nol

9. Rolling Z-score (Z-score dihitung dalam window bergerak) lebih unggul dari Z-score global untuk time series dengan karakteristik...

- A. Distribusi Gaussian sempurna dan stasioner
- B. Tren/drift/perubahan baseline lambat, adaptif terhadap kondisi lokal
- C. Dataset sangat kecil (< 100 sampel)
- D. Hanya satu kolom data tanpa indeks waktu

10. Min-Max Scaling $[0,1]$ lebih disarankan dibanding Z-score Standardization ketika...

- A. Algoritma butuh batas terdefinisi, misalnya k-NN atau neural network dengan aktivasi sigmoid
- B. Distribusi data sangat skewed dengan outlier ekstrem
- C. Banyak outlier yang akan mengompresi rentang fitur lain
- D. Dataset memiliki jutaan baris dan butuh komputasi cepat

Bagian B: Komputasi Easy-Medium (10 soal $\times$ 2 poin)

C1. Sebuah sensor merekam dengan laju 10 Hz selama 120 detik. Hitung jumlah baris data yang dihasilkan (tidak termasuk header).

- 💡 **HINT:** Jumlah baris $N = \text{laju sampling} \times \text{durasi}$. Substitusikan nilai dari soal.

C2. Hitung jumlah bins histogram untuk dataset $N=1500$ sampel menggunakan aturan Sturges: $\text{bins} = \text{ceil}(\log_2(N) + 1)$.

- 💡 **HINT:** Aturan Sturges: $\text{bins} = \text{ceil}(\log_2(N) + 1)$. Pakai `np.log2` dan `np.ceil`, lalu bulatkan ke integer.

C3. Sebuah sensor memiliki distribusi nilai dengan mean $\mu=2,5$ dan std $\sigma=1,2$. Hitung Z-score untuk pembacaan ekstrem $x=8,5$.

- 💡 **HINT:** $Z\text{-score} = (x - \text{mean}) / \text{std}$. Substitusikan x , mean, dan std dari soal.

C4. Diberi data sensor $[10,12,11,14,100,13,11,12,14,13]$. Hitung upper bound IQR untuk deteksi outlier: $\text{upper} = Q3 + 1,5 \times IQR$.

- 💡 **HINT:** Cari $Q1$ dan $Q3$ dengan `np.percentile`, lalu $IQR = Q3 - Q1$ dan $\text{upper bound} = Q3 + 1,5 \times IQR$.

C5. Diberi sinyal [1,2,3,4,5,6,7,8,9,10]. Hitung rolling mean dengan window 5 menggunakan pandas dan ambil nilai rolling mean di indeks terakhir.

- 💡 **HINT:** Gunakan `Series.rolling(window=5).mean()` lalu ambil nilai di indeks terakhir dengan `.iloc[-1]`.

C6. Sebuah dataset memiliki 1500 baris dan 75 baris mengandung NaN. Hitung persentase missing (dalam %).

- 💡 **HINT:** Persentase missing = (jumlah baris NaN / total baris) x 100. Substitusikan angka dari soal.

C7. Saat resampling sinyal dari 100 Hz menjadi 4 Hz, hitung frekuensi Nyquist (Hz) pada laju sampling baru.

- 💡 **HINT:** Frekuensi Nyquist = $fs_baru / 2$. Substitusikan laju sampling baru dari soal.

C8. Diberi dataset suhu dengan min=10C dan max=50C. Lakukan min-max scaling (normalisasi ke [0,1]) untuk nilai x=35C.

- 💡 **HINT:** Min-max scaling = $(x - min) / (max - min)$. Substitusikan x, min, dan max dari soal.

C9. Pada Z-score standardization dataset dengan mean=20 dan std=5, berapa nilai standardized untuk pembacaan x=27,5?

- 💡 **HINT:** Standardisasi = $(x - \mu) / \sigma$. Substitusikan x, mean, dan std dari soal.

C10. Diberi sinyal [2,4,6,NaN,NaN,12,14]. Lakukan interpolasi linear menggunakan pandas, kemudian ambil nilai pada indeks ke-4 (NaN kedua) setelah interpolasi.

- 💡 **HINT:** Buat `pd.Series` dari data, panggil `.interpolate()` untuk mengisi NaN, lalu ambil nilai pada indeks yang diminta dengan `.iloc`.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan deteksi outlier Modified Z-score berbasis MAD (Median Absolute Deviation) tanpa `scipy.stats`. Data: [12.1,12.3,12.0,12.2,45.0,12.1,12.4,12.2,-8.0,12.0,12.1]. Formula: $MAD = \text{median}(|x_i - \text{median}(x)|)$;

$z_mod = 0,6745 \times (x_i - \text{median}(x)) / MAD$; outlier jika $|z_mod| > 3,5$. Hitung jumlah outlier.

- 💡 **HINT:** Hitung median data, lalu $MAD = \text{median}(|x_i - \text{median}(x)|)$; tiap $z_mod = 0,6745 \times (x_i - \text{median}(x)) / MAD$, lalu hitung berapa yang $|z_mod| > 3,5$.

C12 (Hard). Implementasikan Hampel Filter (rolling MAD outlier detection) dengan `window=5` (`centered`) dan `threshold=3x1,4826xMAD_lokal`. Data:

[5.1,5.2,5.0,5.3,25.0,5.2,5.1,5.3,5.0,-10.0,5.2,5.1,5.3,5.2,5.0]. Cek hanya indeks dengan window penuh. Hitung total outlier terdeteksi.

- 💡 **HINT:** Loop tiap titik dengan window penuh; ambil window 5 titik, hitung median dan MAD lokal, lalu flag jika $|x_i - \text{median_lokal}|$ melebihi $3 \times 1,4826 \times \text{MAD_lokal}$.

C13 (Hard). Implementasikan interpolasi linear manual (tanpa `pd.Series.interpolate()` atau `np.interp()`) untuk mengisi NaN. Data: [10.0,11.0,NaN,NaN,14.0,15.0,NaN,17.0,18.0]. Formula: $\text{value}[i] = \text{value}[i_prev] + (i - i_prev) / (i_next - i_prev) \times (\text{value}[i_next] - \text{value}[i_prev])$. Hitung `sum()` dari array hasil interpolasi.

- 💡 **HINT:** Untuk tiap NaN, cari indeks valid sebelum (`i_prev`) dan sesudah (`i_next`), lalu terapkan rumus interpolasi linear pada soal; akhirnya jumlahkan array dengan `sum()`.

C14 (Hard). Implementasikan Exponential Moving Average (EMA) dari awal (tanpa `pd.Series.ewm()`). Data: [10.0,12.0,11.0,13.0,14.0,13.0,15.0], $\alpha=0,3$. $\text{EMA}_0 = \text{data}_0$; $\text{EMA}_i = \alpha \times \text{data}_i + (1 - \alpha) \times \text{EMA}_{i-1}$ untuk $i \geq 1$. Cetak nilai EMA terakhir, dibulatkan 2 desimal.

- 💡 **HINT:** Inisialisasi EMA dengan titik pertama, lalu iterasi $\text{EMA}_i = \alpha \times \text{data}_i + (1 - \alpha) \times \text{EMA}_{i-1}$; ambil nilai EMA terakhir dan bulatkan 2 desimal.

C15 (Hard). Implementasikan pipeline 3-tahap deteksi + replacement outlier dengan IQR pada data: [5.0,5.1,5.2,50.0,5.3,5.4,5.5,-20.0,5.6,5.7,5.8,5.9,100.0,6.0,6.1]. Tahap: (1) hitung $Q1/Q3/IQR$ dan identifikasi outlier, (2) ganti outlier dengan median data bersih (median setelah outlier dikeluarkan), (3) hitung mean array hasil replacement, dibulatkan 2 desimal.

- 💡 **HINT:** Tahap 1: cari $Q1/Q3$ dengan `np.percentile` dan batas $Q1 - 1,5 \times IQR$ / $Q3 + 1,5 \times IQR$; Tahap 2: ganti outlier dengan median data bersih; Tahap 3: hitung mean array hasil dan bulatkan 2 desimal.

DAFTAR PUSTAKA

Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. IEEE Transactions on Knowledge and Data Engineering, 35(10), 10339–10350.

<https://doi.org/10.1109/TKDE.2023.3268125>

Hyndman, R. J., & Athanasopoulos, G. (2021). Forecasting: Principles and practice (3rd ed.). OTexts. <https://otexts.com/fpp3/>

- Khan, S., & Lee, D.-H. (2017). An adaptive dynamically weighted median filter for impulse noise removal. *EURASIP Journal on Advances in Signal Processing*, 2017, 67. <https://doi.org/10.1186/s13634-017-0502-z>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 5

Preprocessing Lanjutan: Transformasi

Abstrak

Pertemuan ini melanjutkan pembersihan data dengan teknik transformasi (log, Box-Cox, differencing, normalisasi) untuk

Sub - CPMK

Sub-CPMK 2.2 – Mampu merancang sistem otomasi menggunakan perangkat lunak simulasi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-5.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Data yang sudah “bersih” dari Pertemuan 4 belum tentu siap dimodelkan: masih ada tren, musiman, dan varians tidak konstan yang melanggar asumsi algoritma statistik dan machine learning. Pertemuan kelima ini membahas teknik transformasi untuk menstabilkan varians serta teknik dekomposisi untuk memisahkan komponen tren, musiman, dan residual, dirancang dan diuji menggunakan perangkat lunak simulasi (Python, NumPy, statsmodels) sebelum diterapkan pada sistem produksi. Gambar 1 menunjukkan posisi Pertemuan 5 dalam alur mata kuliah.



Gambar 1. Posisi Pertemuan 5 (Transformasi & Dekomposisi) dalam alur mata kuliah, menjembatani preprocessing dengan ekstraksi fitur pada Pertemuan 6-7.

Materi mencakup transformasi variance-stabilizing (log, Box-Cox), differencing untuk menghilangkan tren, normalisasi/standarisasi skala fitur, dekomposisi klasik dan STL, serta smoothing lanjutan. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan latihan komputasi.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 2.2:** Mampu merancang sistem otomasi menggunakan perangkat lunak simulasi, yaitu merancang dan menguji pipeline transformasi serta dekomposisi deret waktu memakai Python sebelum diterapkan pada sistem monitoring produksi (CPMK 2).
- Setelah pertemuan ini mahasiswa dapat: menerapkan transformasi log, Box-Cox, differencing, dan normalisasi sesuai kebutuhan data; menjelaskan perbedaan dekomposisi aditif dan multiplikatif; menerapkan dekomposisi STL; serta memilih teknik smoothing lanjutan yang sesuai konteks.

TRANSFORMASI DATA: LOG, BOX-COX, DIFFERENCING & NORMALISASI

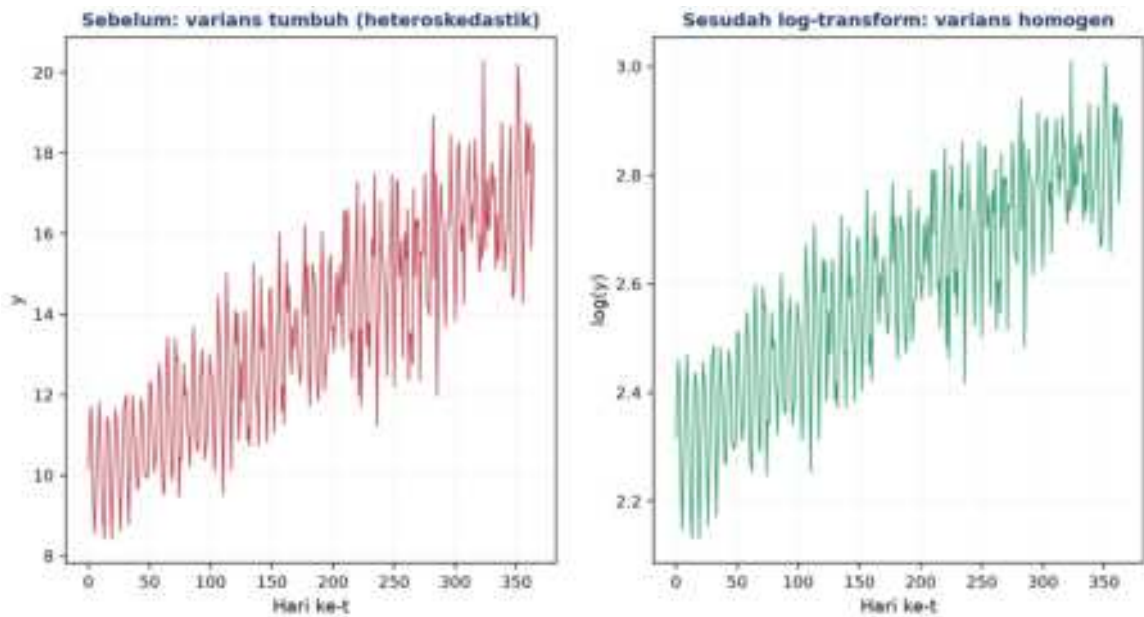
Box-Cox: $y(\lambda) = (x\lambda - 1)/\lambda$ untuk $\lambda \neq 0$; $y(\lambda) = \log(x)$ untuk $\lambda = 0$

- **Log Transform:** $y' = \log(y+\epsilon)$; dipakai untuk data dengan pola varians yang membesar seiring mean (fan-shaped variance), mensyaratkan data positif.
- **Box-Cox (λ optimal):** λ optimal dicari melalui maximum likelihood estimation; nilai umum $\lambda \in \{-1$ (reciprocal), 0 (log), 0,5 (sqrt), 1 (identity)}. Diimplementasikan via ``scipy.stats.boxcox``, mensyaratkan $y > 0$ (pakai Yeo-Johnson bila data bisa nol/negatif).
- **Differencing:** orde-1 ($\nabla x_t = x_t - x_{t-1}$) menghilangkan tren linear; orde-2 menghilangkan tren kuadratik; seasonal differencing ($x_t - x_{t-s}$) menghilangkan pola musiman berperiode s .
- **Z-score & Min-Max:** Z-score untuk algoritma seperti k-NN, SVM, PCA; Min-Max untuk rentang 0-1 (jaringan saraf); keduanya harus di-fit hanya pada data train untuk menghindari data leakage.

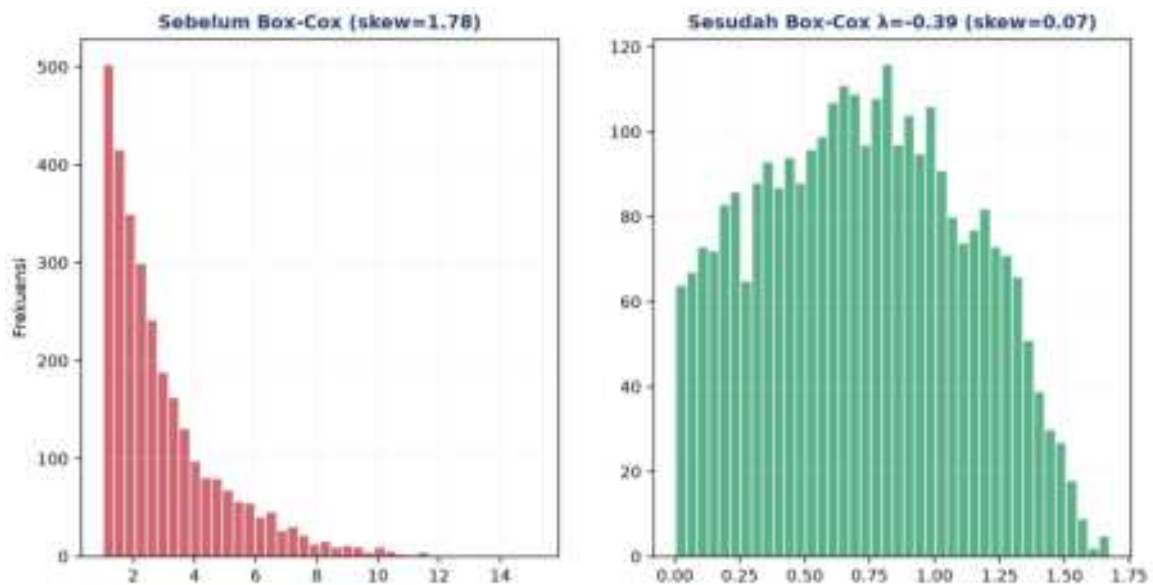
Tabel 1 merangkum enam teknik transformasi beserta kapan dipakai, sifat reversibilitas, dan catatan penting.

Tabel 1. Perbandingan enam teknik transformasi data: kapan dipakai, reversibilitas, dan catatan.

Transformasi	Kapan Dipakai	Reversible?	Catatan
Log	Varians tumbuh proporsional terhadap mean	Ya, $y = \exp(y')$	Data harus positif; tambahkan epsilon kecil jika ada nol
Box-Cox	Ingin mendekati normal; varians tidak stabil	Ya, inverse tersedia	Hanya untuk $y > 0$; pakai Yeo-Johnson untuk $y \leq 0$
Differencing	Menghilangkan tren dan mencapai stasioneritas	Ya, integrasi kumulatif	Kurangi 1 observasi per orde; perbesar noise
Z-score	Algoritma sensitif jarak (k-NN, SVM, PCA, clustering)	Ya, $x = z \cdot \sigma + \mu$	Output unbounded; sensitif outlier
Min-Max	Jaringan saraf (aktivasi sigmoid/tanh)	Ya, $x = z \cdot (\max - \min) + \min$	Output 0-1 ketat; sangat sensitif outlier
Robust Scaler	Data banyak outlier yang belum dibersihkan	Ya	Pakai median dan IQR; tahan outlier



Gambar 2. Efek log-transform terhadap variance stabilization: sinyal asli dengan varians tumbuh (heteroskedastik) menjadi homogen setelah log-transform.

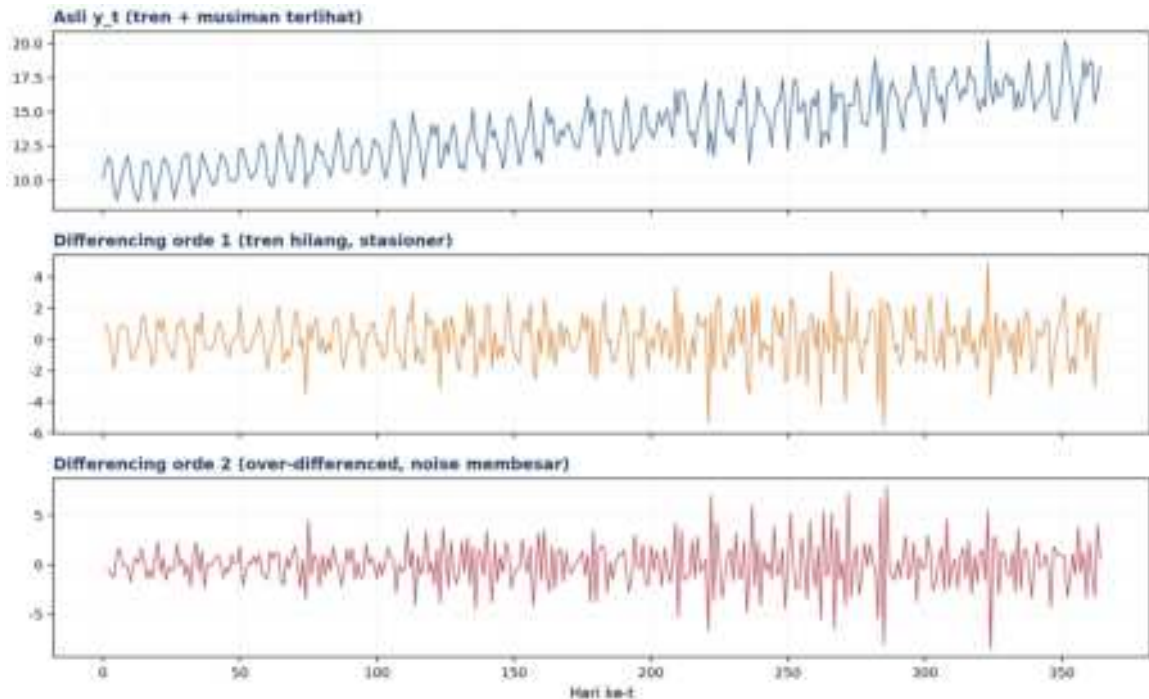


Gambar 3. Distribusi data sebelum dan sesudah Box-Cox transform; skewness turun dari 1,78 menjadi 0,07 mendekati distribusi normal.

Info penting: Aturan fit-transform harus dipatuhi untuk menghindari data leakage: scaler di-fit sekali pada data train (`scaler.fit(X_train)`), lalu dipakai berulang untuk transform data test (`scaler.transform(X_test)`), tidak pernah sebaliknya.

Uji Stasioneritas dengan ADF Test: Uji stasioneritas formal sebaiknya melengkapi inspeksi visual sebelum memutuskan orde differencing yang dipakai. Augmented Dickey-Fuller (ADF) test menguji hipotesis nol bahwa deret memiliki unit root (non-stasioner); p-value di bawah 0,05 mengindikasikan deret sudah cukup stasioner untuk dianalisis lebih lanjut. Praktik yang disarankan: mulai dari differencing orde 0 (data asli), jalankan ADF test,

bila masih non-stasioner naikan ke orde 1, uji lagi, dan seterusnya, lalu berhenti begitu p-value ADF signifikan, karena melanjutkan differencing lebih jauh dari yang diperlukan hanya memperbesar noise tanpa manfaat tambahan.



Gambar 4. Efek differencing: deret asli dengan tren dan musiman (atas), hasil differencing orde 1 yang stasioner (tengah), dan orde 2 yang over-differenced dengan noise membesar (bawah).

Peringatan: Lupa melakukan inverse transform saat menampilkan hasil prediksi menyebabkan error dilaporkan dalam skala yang salah, sebuah kesalahan praktis yang sering luput dari perhatian.

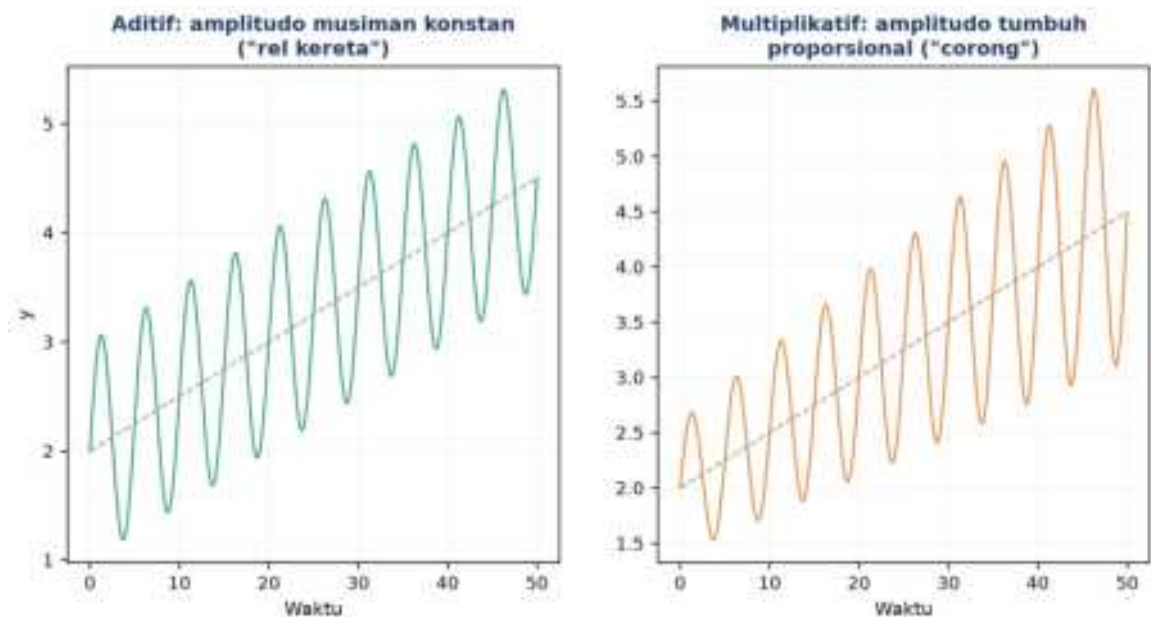
DEKOMPOSISI DERET WAKTU: TREN, MUSIMAN, RESIDUAL

Aditif: $y_t = T_t + S_t + R_t$

Multiplikatif: $y_t = T_t \times S_t \times R_t$

- **Kapan Aditif?:** amplitudo pola musiman relatif konstan terlepas dari tren, contoh suhu pabrik harian berfluktuasi $\pm 3^\circ\text{C}$ tanpa terpengaruh tren jangka panjang, mengikuti pola "rel kereta".
- **Kapan Multiplikatif?:** amplitudo pola musiman tumbuh proporsional dengan tren, contoh konsumsi energi pabrik naik 10% per tahun sehingga fluktuasi musimannya turut membesar, mengikuti pola "corong". Trik praktis: $\log(y)$ mengubah pola multiplikatif menjadi aditif.
- **Komponen Tren:** dihitung via moving average centered dengan window sepanjang periode musiman ($T_t = \text{MA}_k(y_t)$).

- **Komponen Musiman:** dihitung sebagai rata-rata data yang sudah di-detrend per posisi dalam periode ($St = \text{rata-rata fase}(y-T)$).



Gambar 5. Perbandingan konseptual pola aditif (amplitudo musiman konstan, "rel kereta") dan multiplikatif (amplitudo tumbuh proporsional, "corong").

Diagnosa cepat aditif vs multiplikatif dapat dilakukan dengan mengamati plot: jika amplitudo musiman tetap konstan sepanjang deret waktu, pilih model aditif; jika amplitudo membesar seiring naiknya tren, pilih model multiplikatif.

STL DECOMPOSITION & SMOOTHING LANJUTAN

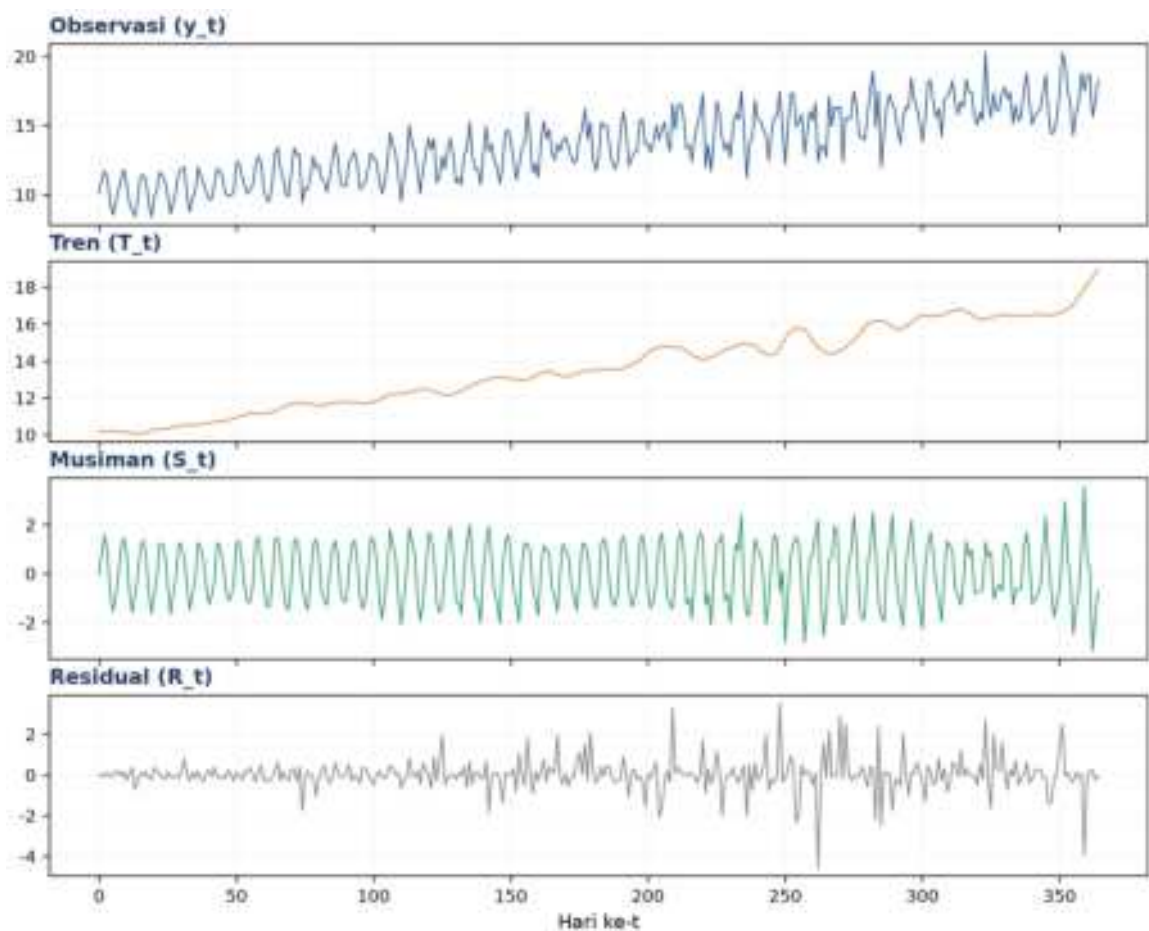
$$MA_k(y_t) = (1/k) \cdot \sum_{i=1}^k y_{t-i} \quad EMA_\alpha(y_t) = \alpha \cdot y_t + (1-\alpha) \cdot EMA_\alpha(y_{t-1})$$

- **STL, Keunggulan:** berbasis LOESS, mampu menangani musiman yang berubah secara bertahap, robust terhadap outlier; diimplementasikan via ``statsmodels.tsa.seasonal.STL``.
- **Moving Average (MA):** varian centered (tanpa lag, offline) vs trailing (causal, real-time); trade-off window besar (halus tetapi lag besar) versus window kecil (responsif tetapi berisik).
- **Exponential Moving Average (EMA):** populer untuk alerting real-time karena bersifat one-pass dan hanya butuh nilai sebelumnya.
- **Savitzky-Golay Filter:** fitting polinomial per window, menjaga puncak dan lembah sehingga cocok untuk data getaran yang bentuknya bermakna.

Tabel 2 merangkum lima metode smoothing/dekomposisi beserta keunggulan, kekurangan, dan kapan dipakai.

Tabel 2. Perbandingan lima metode smoothing/dekomposisi: keunggulan, kekurangan, dan kapan dipakai.

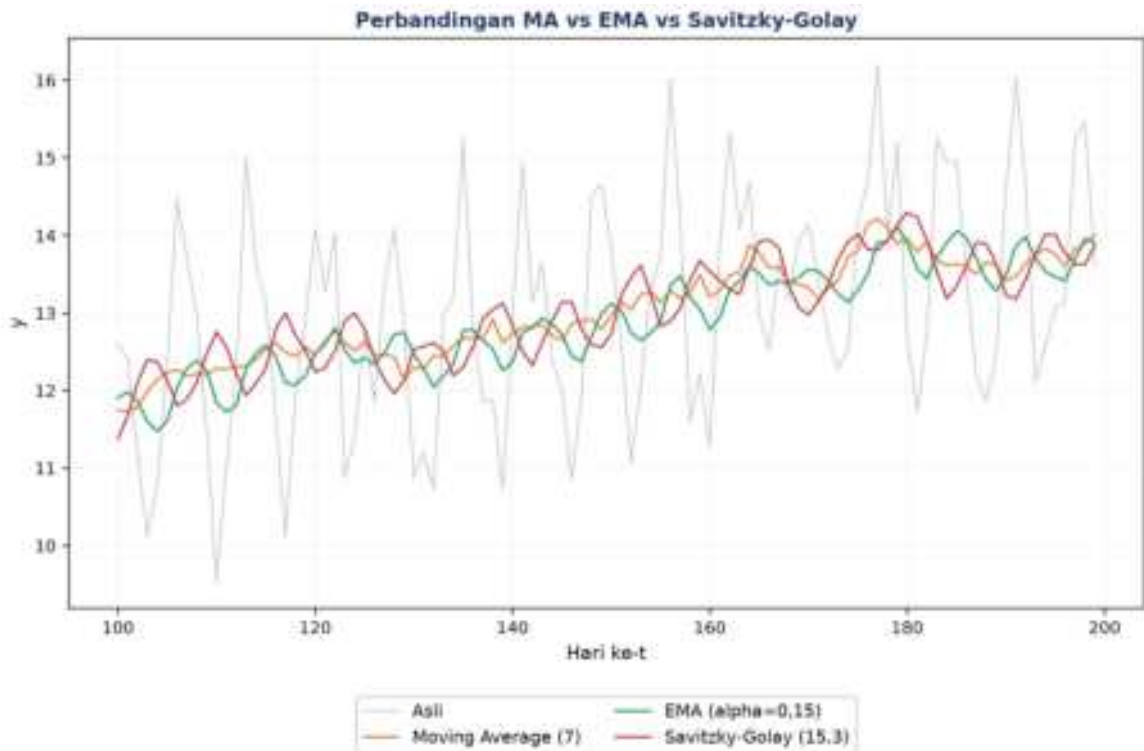
Metode	Keunggulan	Kekurangan	Kapan Dipakai
MA (Moving Average)	Sederhana, cepat, mudah diinterpretasi	Lag, smoothing sama untuk semua titik	Dekomposisi klasik, baseline cepat
EMA (Exp. MA)	Responsif, lag kecil, one-pass	Sulit menentukan alpha optimal	Alerting real-time, streaming
STL	Seasonal dinamis, robust outlier	Komputasi lebih mahal, butuh statsmodels	Forecasting industri serius
Savitzky-Golay	Jaga bentuk peak/trough	Butuh tuning (w,p); asumsi data smooth	Sinyal getaran, spektroskopi
LOESS/LOWESS	Non-parametrik, fleksibel, robust	Sensitif terhadap bandwidth	Eksplorasi tren non-linear



Gambar 6. Dekomposisi STL deret harian sintesis menjadi observasi, tren, musiman, dan residual, dengan periode musiman 7 hari.

LOESS untuk Musiman yang Berubah: LOESS/LOWESS layak dipertimbangkan sebagai alternatif STL saat pola musiman tidak konstan sepanjang waktu, misalnya pabrik yang mengubah pola shift kerja di pertengahan tahun sehingga periode musiman mingguan berubah karakternya. Parameter bandwidth pada LOESS mengontrol trade-off serupa dengan window size pada MA: bandwidth kecil mengikuti fluktuasi lokal secara ketat (risiko

overfitting ke noise), sedangkan bandwidth besar menghasilkan kurva tren yang lebih halus namun berpotensi kurang responsif terhadap perubahan struktural yang legitimate dalam data.



Gambar 7. Perbandingan Moving Average, EMA, dan Savitzky-Golay pada segmen data yang sama.

Rekomendasi pipeline: Pipeline preprocessing industrial yang direkomendasikan: (1) plot data untuk inspeksi awal; (2) terapkan log/Box-Cox bila diperlukan; (3) terapkan STL atau differencing; (4) periksa residual; (5) uji stasioneritas dengan ADF test; (6) simpan parameter transformasi untuk keperluan inverse.

Peringatan: Jebakan industri yang sering terjadi: lupa menyimpan parameter scaler sehingga terjadi data leakage; $\log(0)$ menghasilkan NaN (solusinya pakai $\log(x+1)$ atau Yeo-Johnson); periode musiman yang salah dimasukkan ke STL; serta over-differencing yang justru memperbesar noise.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Volume Konser & Skala Desibel:** telinga manusia bekerja secara logaritmik terhadap intensitas suara ($\text{dB} = 10 \cdot \log(P/P_0)$), analog langsung dengan log-transform pada sinyal.
- **Saham & Return Harian:** log-return harga saham ($r_t = \log(P_t/P_{t-1})$) menyetarakan saham dengan skala harga yang sangat berbeda agar bisa dibandingkan.

- **Penjualan Ritel & Weekend Spike:** dekomposisi memisahkan tren, musiman (lonjakan akhir pekan), dan residual; residual yang anomali bisa jadi sinyal aktivitas kompetitor.
- **Termometer Medis & Z-score:** Z-score suhu tubuh ($z = (37,8 - \mu) / \sigma$) dengan $z > 2$ dianggap abnormal, analog langsung dengan normalisasi multi-sensor.
- **Stok Gudang & Differencing:** differencing pada data stok kumulatif mengubahnya menjadi pola penerimaan harian yang lebih stabil dan mudah dianalisis.
- **Tinggi & Berat Badan:** tanpa scaling, algoritma seperti k-NN didominasi oleh fitur berskala besar (misal berat badan dalam gram vs tinggi dalam meter); Min-Max/Z-score menyetarakan kontribusi tiap fitur.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

- **Condition Monitoring Bearing:** pipeline $\log(\text{RMS})$ lalu STL menghasilkan sinyal tren murni sebagai indikator kesehatan bearing, terpisah dari fluktuasi musiman harian.
- **Energy Monitoring Pabrik:** model $E = T_y + S_w + S_d + R$ (tren tahunan + musiman mingguan + musiman harian + residual); residual yang anomali menjadi sinyal deteksi kebocoran energi.
- **Temperature Sensor Oven:** EMA dengan $\alpha = 0,05$ sebagai baseline, alert dipicu jika $|y - \text{EMA}| > 3\sigma$.
- **SCADA Multi-Sensor:** normalisasi Z-score ($z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$) wajib dilakukan sebelum data multi-sensor dimasukkan ke Isolation Forest, One-Class SVM, atau autoencoder.

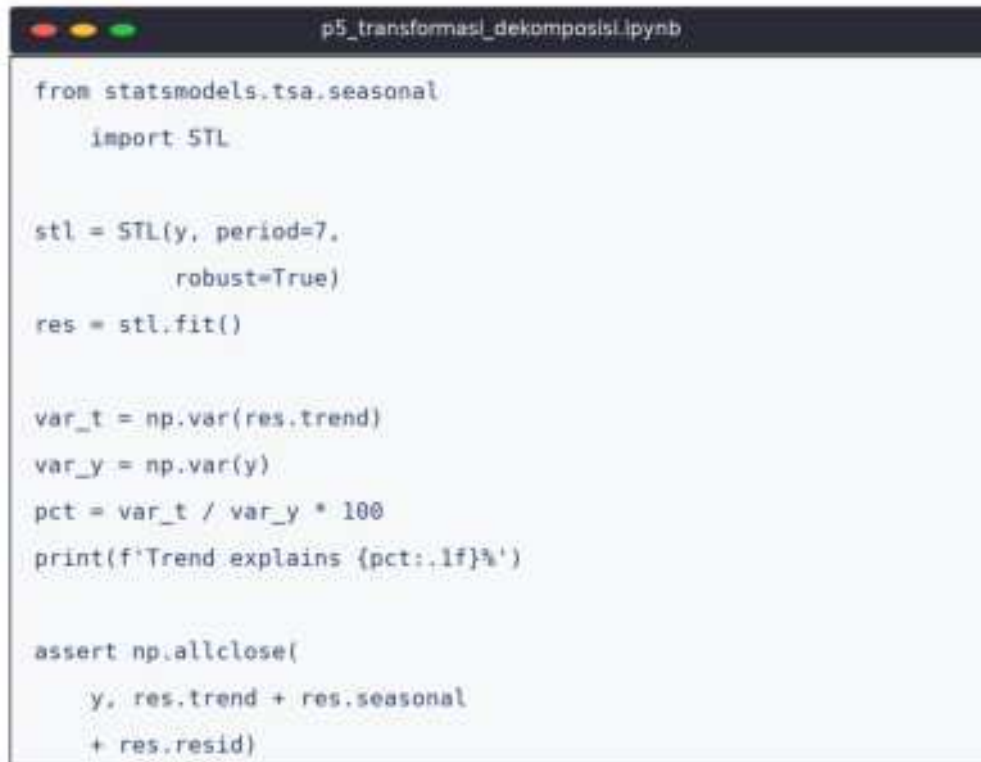
Kaitan dengan pertemuan berikutnya: Materi ini menjadi fondasi bagi Pertemuan 6 (fitur domain waktu yang sensitif terhadap varians) dan Pertemuan 7 (FFT, di mana musiman yang tidak didekomposisi dapat menghasilkan peak frekuensi palsu).

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada deret sintesis. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, statsmodels, matplotlib; notebook p5_transformasi_dekomposisi.ipynb. Gambar 8 menunjukkan cuplikan Cell 4 yang menjalankan dekomposisi STL dan memverifikasi rekonstruksinya.

Sanity-Check Rekonstruksi STL: Verifikasi rekonstruksi (memastikan observasi $\approx$ trend + seasonal + residual dalam toleransi numerik kecil) merupakan langkah sanity-check yang

sering diabaikan tetapi penting: kegagalan rekonstruksi biasanya mengindikasikan periode musiman yang salah dimasukkan ke STL, atau data mengandung missing values yang belum ditangani sebelum dekomposisi dijalankan. Cell 4 pada notebook ini secara eksplisit menghitung selisih antara data asli dan hasil penjumlahan ketiga komponen, mencetak nilai maksimum residual rekonstruksi sebagai indikator kebenaran proses.



```
from statsmodels.tsa.seasonal
    import STL

stl = STL(y, period=7,
          robust=True)
res = stl.fit()

var_t = np.var(res.trend)
var_y = np.var(y)
pct = var_t / var_y * 100
print(f'Trend explains {pct:.1f}%')

assert np.allclose(
    y, res.trend + res.seasonal
    + res.resid)
```

Gambar 8. Cuplikan Cell 4: dekomposisi STL penuh dan verifikasi rekonstruksi $y \approx \text{trend} + \text{seasonal} + \text{residual}$.

- **Cell 1:** generate deret sintetis 365 hari (tren linear + musiman mingguan + noise heteroskedastik), plot dan statistik deskriptif.
- **Cell 2:** log transform + Z-score (`StandardScaler`, fit pada 80% data train) + Min-Max scaling, plot perbandingan 2x2.
- **Cell 3:** differencing orde 1 dan 2 (`np.diff`) plus ADF test (`statsmodels.tsa.stattools.adfuller`, $p < 0,05$ berarti stasioner), plot 3-panel.
- **Cell 4:** dekomposisi STL penuh (period=7, robust=True), cetak persentase variance explained per komponen, plot 4-panel, verifikasi rekonstruksi.

TUGAS PERTEMUAN 5

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.



Gambar 9. Distribusi 50 poin Tugas Pertemuan 5 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal $\times$ 1 poin)

1. Kondisi stasioneritas pada deret waktu berarti...

- A. Data dikumpulkan pada interval waktu yang tetap
- B. Mean, variansi, dan struktur kovariansnya konstan sepanjang waktu
- C. Deretnya berbentuk garis lurus (linear)
- D. Semua nilai dalam deret adalah sama (tidak berfluktuasi)

2. Bila sebuah deret waktu menunjukkan varians yang tumbuh seiring mean (plot berbentuk corong), transformasi yang paling tepat adalah...

- A. Min-Max scaling ke $[0, 1]$
- B. Differencing orde 2
- C. Log transform (atau Box-Cox)
- D. Menghapus semua observasi dengan nilai ekstrem

3. Pada transformasi Box-Cox dengan parameter $\lambda=0$, formula yang diterapkan setara dengan...

- A. Identitas, tidak ada transformasi
- B. Log natural, $y' = \log(x)$
- C. Akar kuadrat, $y' = \sqrt{x}$
- D. Inverse, $y' = 1/x$

4. Operasi differencing orde 1 ($\nabla x_t = x_t - x_{t-1}$) pada sebuah deret waktu bertujuan utama untuk...

- A. Mengurangi jumlah observasi agar model lebih cepat dilatih
- B. Menghilangkan tren (linear) sehingga deret menjadi lebih stasioner
- C. Menyetarakan skala antar fitur sensor yang berbeda
- D. Menghilangkan outlier secara otomatis

5. Setelah menerapkan Z-score normalization dengan benar pada sebuah fitur, mean dan standar deviasi hasilnya menjadi...

- A. mean = 1, std = 0
- B. mean = 0, std = 1
- C. min = 0, max = 1
- D. mean = 0, std = 0

6. Dalam dekomposisi deret waktu, model multiplikatif ($y_t = T_t \times S_t \times R_t$) paling tepat digunakan saat...

- A. Amplitudo pola musiman konstan terlepas dari tren
- B. Amplitudo pola musiman tumbuh proporsional dengan tren
- C. Deret tidak memiliki komponen musiman sama sekali
- D. Data bernilai negatif di banyak titik

7. Pada uji Augmented Dickey-Fuller (ADF) untuk stasioneritas, p-value < 0,05 artinya...

- A. Null hypothesis "non-stasioner" ditolak, deret dianggap stasioner
- B. Deret pasti memiliki tren linear yang kuat
- C. Deret pasti non-stasioner, butuh differencing tambahan
- D. Model tidak valid dan harus diulang dengan data baru

8. Dibandingkan dengan Moving Average, keunggulan utama Exponential Moving Average (EMA) untuk monitoring sensor real-time adalah...

- A. EMA selalu menghasilkan varians lebih kecil
- B. EMA lebih responsif (lag lebih kecil) dan bersifat one-pass, cocok untuk streaming
- C. EMA tidak perlu parameter apapun
- D. EMA menghasilkan output 0-1 secara otomatis

9. Praktik fit-transform yang benar untuk StandardScaler pada pipeline train/test split adalah...

- A. fit_transform pada train dan fit_transform pada test juga
- B. fit_transform pada train, lalu transform saja pada test (pakai parameter train)
- C. Gabungkan train + test, lalu fit_transform keseluruhan
- D. Hitung mu dan sigma per batch, masing-masing terpisah

10. Keunggulan utama STL (Seasonal-Trend decomposition using LOESS) dibandingkan dekomposisi klasik berbasis moving average adalah...

- A. STL selalu jauh lebih cepat dari moving average
- B. Komponen musiman boleh berubah bertahap sepanjang waktu dan lebih robust terhadap outlier
- C. STL tidak memerlukan penentuan periode musiman
- D. Hasil STL otomatis stasioner tanpa perlu differencing

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Dataset referensi (dipakai C1-C15, dibuat sekali di Jupyter):

```
import numpy as np, pandas as pd
np.random.seed(42)
N = 365
t = np.arange(N)
trend = 0.02 * t
seasonal = 1.5 * np.sin(2*np.pi*t/7)
noise = np.random.normal(0, 0.3, N)
y = 10 + trend + seasonal + noise
series = pd.Series(y, index=pd.date_range('2025-01-01', periods=N, freq='D'))
```

C1. Dari dataset referensi series, hitung mean sepanjang seluruh deret dengan `series.mean()`.

- 💡 **HINT:** Gunakan `series.mean()` untuk menghitung rata-rata deret tersebut.

C2. Hitung standar deviasi dari dataset referensi dengan `series.std()` (ingat pandas memakai default `ddof=1`).

- 💡 **HINT:** Gunakan `series.std()`; ingat pandas memakai default `ddof=1` (sample std).

C3. Terapkan log transform pada dataset referensi: `y_log = np.log(y)`. Hitung mean hasil log.

- 💡 **HINT:** Transformasi log dengan `np.log(y)`, lalu hitung rata-ratanya dengan `np.mean`.

C4. Terapkan Z-score normalization manual: $z = (y - \text{mean})/\text{std}$. Hitung standar deviasi dari `z` (harus mendekati 1,0).

- 💡 **HINT:** Standardisasi $z = (y - \text{mean})/\text{std}$, lalu hitung std dari `z` dengan `np.std(..., ddof=1)`.

C5. Terapkan Min-Max scaling manual ke rentang `[0,1]`: $y_{\text{mm}} = (y - \text{min})/(\text{max} - \text{min})$. Cetak nilai maksimum dari `y_mm`.

- 💡 **HINT:** Min-max scaling $= (y - \text{min})/(\text{max} - \text{min})$; ambil nilai maksimum hasilnya dengan `.max()`.

C6. Hitung first difference: `d1 = np.diff(y)`. Cetak mean dari `d1` (harus dekat dengan nilai tren rata-rata per hari).

- 💡 **HINT:** Hitung first difference dengan `np.diff(y)`, lalu rata-ratakan dengan `np.mean`.

C7. Hitung moving average window 7 (mingguan) via pandas: `ma7 = series.rolling(7).mean()`. Cetak nilai pada `ma7.iloc[50]`.

- 💡 **HINT:** Gunakan `series.rolling(window=7).mean()`, lalu ambil nilai pada indeks yang diminta dengan `.iloc`.

C8. Hitung Exponential Moving Average $\alpha=0,1$ via pandas: `ema = series.ewm(alpha=0.1, adjust=False).mean()`. Cetak nilai pada `ema.iloc[-1]`.

- 💡 **HINT:** Gunakan `series.ewm(alpha=..., adjust=False).mean()`, lalu ambil nilai pada indeks terakhir dengan `.iloc[-1]`.

C9. Terapkan Box-Cox transform dengan `scipy.stats.boxcox` pada dataset referensi. Cetak nilai parameter lambda optimal.

- 💡 **HINT:** Pakai `scipy.stats.boxcox`; fungsi mengembalikan data transformasi dan parameter lambda optimal sebagai nilai kedua.

C10. Jalankan Augmented Dickey-Fuller test pada deret hasil differencing orde-1 (`np.diff(y)`). Cetak p-value hasilnya.

- 💡 **HINT:** Gunakan `statsmodels.tsa.stattools.adfuller` pada deret hasil differencing; p-value adalah elemen indeks ke-1 dari tuple hasilnya.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan dekomposisi klasik aditif dari awal (tanpa `statsmodels`). Untuk dataset referensi dengan `periode=7`: (1) hitung tren T dengan centered moving average window 7, (2) hitung detrended $= y - T$, (3) hitung komponen musiman S dengan rata-rata detrended per posisi hari ($\text{mod } 7$), (4) hitung residual $R = y - T - S$. Laporkan standar deviasi residual.

- 💡 **HINT:** Hitung tren T dengan centered moving average periode 7, detrended $= y - T$; komponen musiman S = rata-rata detrended per posisi hari ($\text{mod } 7$); residual $= y - T - S$, lalu hitung std-nya (abaikan NaN ujung).

C12 (Hard). Implementasikan Savitzky-Golay filter dari awal (tanpa `scipy.signal.savgol_filter`) untuk `window=11` dan orde polinomial $p=2$. Terapkan pada dataset referensi. Laporkan nilai filtered pada indeks 200.

- 💡 **HINT:** Untuk tiap pusat window, selesaikan least-squares polinomial orde p dengan `np.polyfit` pada `y[i-5:i+6]`, lalu ambil nilai polinomial di pusat. Di tepi, ambil nilai asli. Bandingkan dengan `scipy.signal.savgol_filter` untuk validasi.

C13 (Hard). Jalankan STL decomposition dengan statsmodels pada dataset referensi (periode=7, robust=True). Laporkan persentase varians yang dijelaskan komponen tren: $\text{var}(\text{trend})/\text{var}(y) \times 100$.

- 💡 **HINT:** Jalankan STL dari statsmodels.tsa.seasonal, ambil komponen .trend, lalu hitung rasio $\text{var}(\text{trend}) / \text{var}(y) \times 100$.

C14 (Hard). Implementasikan deteksi anomali berbasis EMA: (1) hitung EMA dengan $\alpha=0,1$, (2) hitung residual = $y - \text{EMA}$, (3) hitung sigma_residual, (4) hitung jumlah titik dimana $|\text{residual}| > 3 \cdot \text{sigma_residual}$. Sebelum hitung, injeksikan 5 anomali buatan pada indeks [50,100,150,200,300] dengan $y_{\text{mod}} += 10$. Laporkan jumlah anomali terdeteksi pada y_{mod} .

- 💡 **HINT:** Hitung EMA dengan `ewm(...).mean()`, residual = $y_{\text{mod}} - \text{EMA}$, lalu hitung jumlah titik dengan $|\text{residual}|$ melebihi $3 \times \text{std residual}$.

C15 (Hard). Implementasikan pipeline preprocessing lengkap: (1) log transform, (2) Z-score scaling dengan fit hanya pada 80% data training pertama, (3) differencing orde 1, (4) ADF test pada hasil akhir. Laporkan nilai mean absolut dari hasil akhir pipeline (seharusnya sangat kecil karena differencing + standardisasi).

- 💡 **HINT:** Urutan: (1) log transform dengan `np.log`; (2) fit scaler pada 80% data awal lalu transform semua; (3) differencing dengan `np.diff`; (4) laporkan nilai absolut dari mean hasil differencing.

DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. *IEEE Transactions on Knowledge and Data Engineering*, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>

Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972.

<https://doi.org/10.3390/app12030972>

Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. <https://doi.org/10.1609/aaai.v37i9.26317>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 6

Ekstraksi Fitur Domain Waktu

Abstrak

Pertemuan ini membahas ekstraksi fitur domain waktu (time-domain feature extraction) dari sinyal vibrasi mesin,

Sub - CPMK

Sub-CPMK 2.3 – Mampu mengimplementasikan dan menguji model sistem dengan simulasi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-6.html>. Pada layar masuk, pilih tombol “👁️ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Setelah sinyal dibersihkan dan ditransformasi (Pertemuan 3-5), sinyal masih berupa jutaan sampel mentah yang perlu dipadatkan menjadi belasan angka representatif sebelum bisa dianalisis atau dimasukkan ke model machine learning. Pertemuan keenam ini membahas ekstraksi fitur domain waktu, dirancang dan diuji melalui simulasi Python pada sinyal sintesis sebelum diterapkan pada data sensor sungguhan. Gambar 1 menunjukkan posisi Pertemuan 6 dalam alur mata kuliah.



Gambar 1. Posisi Pertemuan 6 (Fitur Domain Waktu) dalam alur mata kuliah, menjembatani preprocessing dengan fitur domain frekuensi pada Pertemuan 7.

Contoh skala data: accelerometer 25 kHz yang merekam 4 detik menghasilkan 100.000 sampel, yang kemudian dipadatkan menjadi vektor 12 fitur untuk klasifikasi healthy/outer-race/inner-race fault. Fitur domain waktu bersifat $O(N)$, tidak butuh FFT, dan real-time ready, dengan tiap fitur sensitif terhadap jenis kerusakan yang berbeda. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan latihan komputasi.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 2.3:** Mampu mengimplementasikan dan menguji model sistem dengan simulasi, yaitu membangun fungsi ekstraksi fitur domain waktu dan mengujinya pada sinyal sintesis sebelum diterapkan pada sistem monitoring produksi (CPMK 2).
- Setelah pertemuan ini mahasiswa dapat: menghitung statistik dasar (mean, variance, std, RMS); menghitung fitur bentuk sinyal (peak, peak-to-peak, mean absolute); menghitung fitur orde tinggi (crest factor, kurtosis, skewness, shape factor, impulse factor); serta menginterpretasikan fenomena U-shape crest factor sepanjang lifetime bearing.

STATISTIK DASAR: MEAN, VARIANCE, STD, RMS

$$\mu = (1/N) \cdot \sum x_i$$

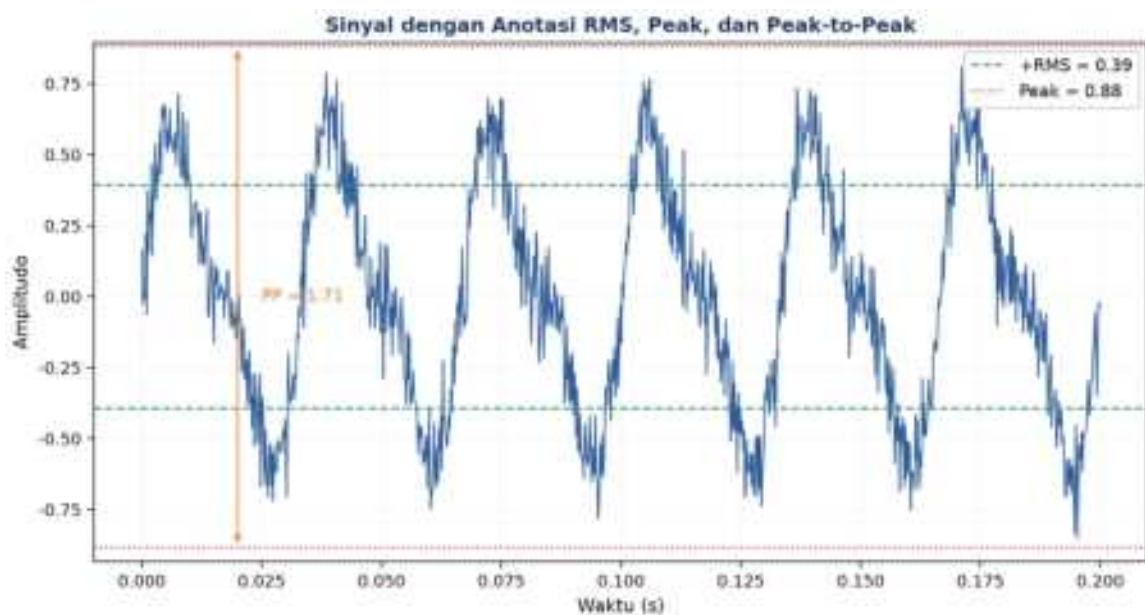
$$\sigma = \sqrt{(1/N) \cdot \sum (x_i - \mu)^2}$$

$$RMS = \sqrt{(1/N) \cdot \sum x_i^2}$$

Mean idealnya bernilai 0 untuk sinyal AC-coupled seperti getaran; mean yang bergeser dari nol menandakan bias DC dari sensor, bukan kondisi mesin. Variance/std mengukur sebaran; std yang naik berarti amplitudo getaran membesar. RMS adalah fitur paling fundamental dalam industri vibrasi karena merepresentasikan energi sinyal per satuan waktu; ISO 10816 memakai RMS velocity (mm/s) sebagai basis klasifikasi severity.

Identitas penting: $RMS^2 = \mu^2 + \sigma^2$ (untuk sinyal AC-coupled dengan $\mu \approx 0$, $RMS \approx \sigma$)

RMS lebih disukai dibanding peak untuk monitoring karena tahan terhadap spike noise: satu impact palsu bisa melonjakkan peak hingga 10 kali lipat, tetapi hampir tidak mengubah RMS. Inilah sebabnya standar ISO 10816, ISO 13373, dan API 670 semuanya memakai RMS sebagai basis klasifikasi.



Gambar 2. Sinyal getaran dengan anotasi RMS (garis putus-putus hijau), Peak (garis titik-titik merah), dan Peak-to-Peak (panah oranye).

BENTUK SINYAL: PEAK, PEAK-TO-PEAK, MEAN ABSOLUTE

$$x_{peak} = \max|x_i|$$

$$PP = \max(x) - \min(x)$$

$$|x|_{avg} = (1/N) \cdot \sum |x_i|$$

- **Peak:** sangat sensitif terhadap impact transien tetapi juga rentan terhadap spike noise; memerlukan pre-filter median atau Hampel sebelum dihitung.
- **Peak-to-Peak (PP):** menangkap rentang amplitudo penuh; untuk sinyal AC-coupled yang simetris, PP mendekati 2 kali Peak. Banyak sensor displacement memakai satuan mils PP atau mikrometer PP.

- **Mean Absolute:** mirip std tetapi lebih murah secara komputasi karena tanpa operasi kuadrat; untuk sinyal Gaussian, $|x|_{avg}$ mendekati $0,798 \cdot RMS$.

Hubungan antarfitur (sinyal Gaussian): Untuk sinyal noise Gaussian murni berlaku hubungan pendekatan: $|x|_{avg} \approx 0,798 \cdot RMS$, $x_{peak} \approx 3 \cdot RMS$ (aturan 3-sigma), dan $PP \approx 6 \cdot RMS$. Deviasi signifikan dari rasio-rasio ini menandakan sinyal bersifat non-Gaussian, biasanya karena ada komponen impulsif atau periodik.

Tabel 1 merangkum lima fitur bentuk sinyal beserta sensitivitas terhadap dampak, ketahanan terhadap outlier, dan aplikasi tipikalnya.

Tabel 1. Fitur bentuk sinyal domain waktu terhadap sensitivitas dampak, ketahanan outlier, dan aplikasi tipikal.

Fitur	Sensitivitas terhadap Dampak	Robust Outlier?	Aplikasi Tipikal
Mean (μ)	Tidak (untuk AC)	Tidak, sensitif outlier	DC offset detection, suhu, tekanan
RMS / Std	Sedang	Ya, energi rata-rata	ISO 10816 vibration severity
Peak	Sangat tinggi	Tidak, single sample dominan	Impact detection (bearing fault)
Peak-to-Peak	Tinggi	Tidak, dipengaruhi extreme	Displacement monitoring (mils PP)
Mean Absolute	Sedang	Ya, mirip std	Cheap hardware, streaming

FITUR ORDE TINGGI: CREST FACTOR, KURTOSIS, SKEWNESS

$$Crest = x_{peak}/RMS \quad Kurt = E[(x-\mu)^4]/\sigma^4 \quad Skew = E[(x-\mu)^3]/\sigma^3$$

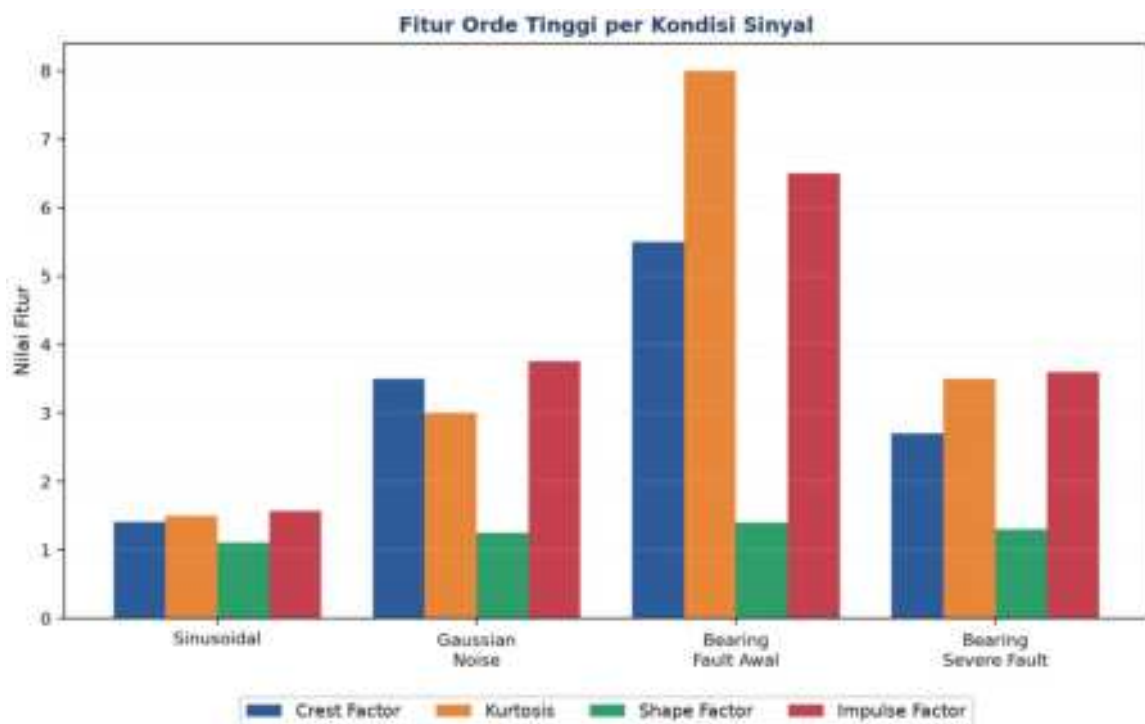
- **Crest Factor:** sinusoidal murni menghasilkan $CF = \sqrt{2} \approx 1,414$; noise Gaussian menghasilkan $CF \approx 3-4$; $CF > 4-5$ menandakan dampak transien atau kerusakan bearing tahap awal; CF merupakan leading indicator yang naik sebelum RMS ikut naik.
- **Kurtosis:** momen orde-4; sinyal Gaussian menghasilkan $K=3$; sinyal impulsif menghasilkan K jauh lebih besar dari 3. Kurtosis adalah fitur paling sensitif untuk deteksi bearing fault tahap awal, bisa naik 5-10 kali sebelum terlihat pada RMS.
- **Skewness:** momen orde-3; sinyal sehat menghasilkan Skew mendekati 0; nilai yang menyimpang dari nol menandakan asimetri, misalnya akibat clipping sensor atau dampak satu arah pada gear meshing.
- **Shape Factor & Impulse Factor:** $SF = RMS/|x|_{avg}$ (Gaussian $SF \approx 1,253$); $IF = x_{peak}/|x|_{avg}$ (Gaussian $IF \approx 3,76$, naik tajam untuk dampak singkat dan jarang); Margin Factor tahan terhadap clipping/outlier meski jarang dipakai.

Peringatan: Perhatikan parameter `fisher` pada `scipy.stats.kurtosis`: excess kurtosis (default scipy, Gaussian=0) berbeda dari plain kurtosis (Gaussian=3). Selalu periksa parameter `fisher=True/False` sebelum membandingkan nilai dengan referensi literatur.

Tabel 2 membandingkan lima fitur orde tinggi pada empat jenis sinyal, dari sinusoidal murni hingga bearing severe fault.

Tabel 2. Perbandingan lima fitur orde tinggi pada empat jenis sinyal, dari sinusoidal murni hingga bearing severe fault.

Fitur	Sinusoidal	Gaussian Noise	Bearing Fault Awal	Bearing Severe Fault
Crest Factor	1,414	~3-4	4-6 (naik)	2-3 (turun, RMS naik)
Kurtosis	1,5	3,0	5-10	3-4 (turun, impak konstan)
Skewness	0	0	~0 (simetris)	tidak nol (jika clipping)
Shape Factor	1,111	1,253	1,3-1,5	1,2-1,4
Impulse Factor	1,571	~3,76	5-8	3-4



Gambar 3. Perbandingan nilai empat fitur orde tinggi (Crest Factor, Kurtosis, Shape Factor, Impulse Factor) pada empat kondisi sinyal.

Fenomena U-shaped Crest Factor: Fenomena U-shaped crest factor terjadi sepanjang lifetime bearing: kondisi sehat menghasilkan CF sekitar 3-4; early defect menaikkan CF ke 5-8 karena impak masih singkat dan belum mengangkat RMS; sedangkan severe fault menurunkan CF kembali ke 3-4 karena defect sudah menjadi konstan/berisik dan RMS ikut

naik proporsional. Kesimpulannya, kombinasi CF, RMS, dan Kurtosis diperlukan bersama-sama, tidak cukup satu fitur saja.



Gambar 4. Trajectory RMS (naik monoton, sumbu kiri) dibandingkan Crest Factor dan Kurtosis (pola U-shape, sumbu kanan) sepanjang simulasi lifetime bearing.

Gambar 4 menegaskan mengapa RMS saja tidak cukup: RMS terus naik secara monoton sepanjang degradasi, sementara Crest Factor dan Kurtosis justru naik tajam pada fase awal kerusakan (sekitar progress 0,2-0,4) lalu turun kembali mendekati level normal pada fase severe (mendekati progress 1,0), meskipun kerusakan sebenarnya semakin parah.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Listrik AC Rumah 220V RMS:** nilai "220V RMS" pada listrik rumah sebenarnya berasal dari tegangan puncak sekitar $\pm 311V$; $RMS = 311 / \sqrt{2} = 220V$, merepresentasikan energi efektif, bukan nilai sesaat.
- **EKG & Irama Jantung Tidak Beraturan:** kurtosis tinggi pada interval RR (jarak antar detak) menandakan aritmia atau fibrilasi atrial; skewness bukan nol menandakan pola detak yang tidak teratur.
- **Drummer vs Pianist:** audio drum memiliki crest factor jauh lebih tinggi (sekitar 10-15) dibanding piano (sekitar 3); kompresor audio bekerja dengan menurunkan crest factor sinyal.

- **Driving Style Detection via Telematics:** skewness dan kurtosis pada data akselerasi mobil dari asuransi telematik dapat mengidentifikasi gaya mengemudi; pengemudi agresif cenderung menghasilkan skewness positif dan kurtosis tinggi.
- **Tinggi Gelombang Signifikan Laut:** tinggi gelombang signifikan laut didefinisikan sebagai $H_s=4\sigma$, bukan berdasarkan nilai peak atau mean, konsisten dengan representasi energi gelombang dan tahan terhadap rogue wave tunggal.
- **Voice Activity Detection HP:** deteksi aktivitas suara pada HP memakai threshold peak-to-peak amplitude ($PP>0,05$ berarti sedang bicara, $PP<0,01$ berarti diam).

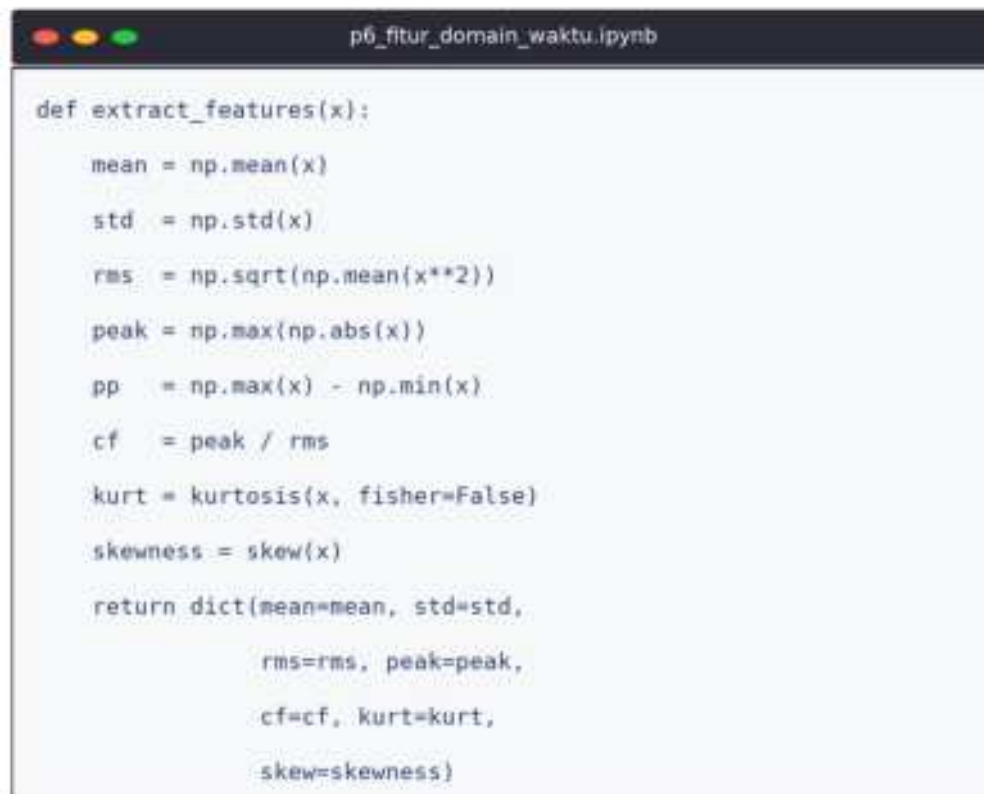
APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

- **Bearing Condition Monitoring:** accelerometer 25 kHz merekam 4 detik, 12 fitur diekstrak untuk mengklasifikasikan kondisi healthy/inner-race/outer-race/ball fault, mengikuti pendekatan yang dipakai SKF Multilog, Bently Nevada, dan GE SmartSignal.
- **Gearbox Tooth Fault Detection:** peak-to-peak dan kurtosis naik tajam saat terjadi chipped atau cracked tooth, seperti terlihat pada dataset NREL gearbox.
- **Wind Turbine Blade Anomaly:** skewness strain gauge naik saat terjadi icing asimetris pada blade, memicu sistem SCADA untuk mengaktifkan proses ice-removal.
- **Railway Wheel-Track Health:** RMS yang naik menandakan wheel flat atau rail corrugation; impulse factor yang naik menandakan rail joint defect; 12 fitur dihitung per segmen 100 meter.

Peringatan: Kesalahan praktis yang sering terjadi: window terlalu pendek sehingga statistik tidak representatif; DC offset tidak dibuang sebelum menghitung fitur; definisi kurtosis (fisher vs plain) tercampur aduk; sampling rate tidak konsisten antar sesi pengukuran; serta lupa pre-filter spike sebelum menghitung fitur orde tinggi.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada sinyal bearing sintetis. Lingkungan: conda env optoauto dengan paket numpy, scipy, matplotlib; notebook p6_fitur_domain_waktu.ipynb. Gambar 5 menunjukkan cuplikan fungsi `extract_features()` yang dipakai berulang sepanjang modul.



```

def extract_features(x):
    mean = np.mean(x)
    std = np.std(x)
    rms = np.sqrt(np.mean(x**2))
    peak = np.max(np.abs(x))
    pp = np.max(x) - np.min(x)
    cf = peak / rms
    kurt = kurtosis(x, fisher=False)
    skewness = skew(x)
    return dict(mean=mean, std=std,
                rms=rms, peak=peak,
                cf=cf, kurt=kurt,
                skew=skewness)

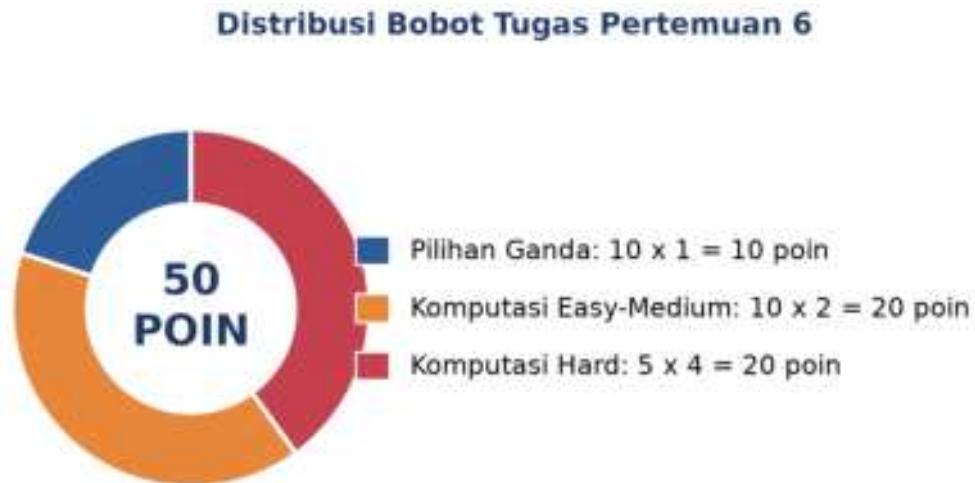
```

Gambar 5. Cuplikan fungsi `extract_features()` yang menghitung 12 fitur domain waktu sekaligus dalam satu dictionary.

- **Cell 1:** generate sinyal bearing healthy (30 Hz rotasi + harmonik + noise Gaussian) vs faulty (ditambah impak periodik BPFO 104 Hz, exponential decay).
- **Cell 2:** fungsi ``basic_stats(x)`` menghitung mean, std (ddof=0), RMS; verifikasi identitas $RMS^2 = \mu^2 + \sigma^2$; plot dengan overlay $\pm RMS / \pm std$.
- **Cell 3:** fungsi ``advanced_features(x)`` menghitung peak, pp, cf, sf, if, kurtosis (fisher=False), skewness (scipy.stats); bar chart healthy vs faulty.
- **Cell 4:** ``extract_features(x)`` reusable (12 fitur dict) plus ``simulate_segment(progress)`` simulasi degradasi bearing 50 segmen lifetime; plot trajectory RMS vs CF/Kurtosis dengan twin-axis.

TUGAS PERTEMUAN 6

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 6 merangkum distribusi bobotnya.



Gambar 6. Distribusi 50 poin Tugas Pertemuan 6 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Formula RMS (Root Mean Square) untuk N sampel sinyal $x_1..x_N$ adalah...

- A. $(1/N) \sum |x_i|$
- B. $\text{akar}((1/N) \sum x_i^2)$
- C. $\max|x_i| - \min|x_i|$
- D. $(1/N) \sum x_i$

2. Untuk sinyal AC-coupled (mean mendekati 0), hubungan antara RMS dan standar deviasi adalah...

- A. $\text{RMS} > \text{std}$ (selalu)
- B. RMS mendekati std, karena $\text{RMS}^2 = \mu^2 + \sigma^2$ dan μ mendekati 0
- C. $\text{RMS} = 2 \times \text{std}$
- D. RMS dan std tidak terkait sama sekali

3. Crest Factor didefinisikan sebagai...

- A. Peak-to-peak / Mean
- B. $\text{Peak}(|x|_{\max}) / \text{RMS}$
- C. $\text{RMS} / \text{Standard deviation}$
- D. $\text{Variance} / \text{Mean kuadrat}$

4. Untuk sinyal sinusoidal murni, nilai crest factor secara teoretis adalah...

- A. 1,0
- B. $\text{akar}(2)$ mendekati 1,414
- C. 3,0
- D. π

5. Kurtosis sebuah sinyal noise Gaussian murni (tanpa impak) bernilai sekitar...

- A. 0 (selalu nol untuk Gaussian)
- B. 1,5
- C. 3 (definisi plain kurtosis Gaussian)
- D. 10 (Gaussian itu tinggi kurtosisnya)

6. Saat bearing mengalami defect awal (early-stage spalling), fitur yang paling cepat melonjak adalah...

- A. Mean, naik karena ada DC offset baru
- B. RMS, naik dramatis seketika
- C. Crest Factor dan Kurtosis, sensitif terhadap dampak transien
- D. Skewness, selalu turun di kerusakan awal

7. Pada severe bearing fault (kerusakan parah, dampak tidak lagi singkat-singkat tapi konstan), kurva crest factor dan kurtosis cenderung...

- A. Terus naik tanpa batas
- B. Turun lagi (U-shape), RMS naik proporsional dengan peak
- C. Tetap di level early-fault
- D. Naik turun secara acak tanpa pola

8. Standar internasional ISO 10816 menggunakan ukuran apa untuk klasifikasi vibration severity mesin?

- A. Peak velocity (mm/s)
- B. Peak-to-peak displacement (mikrometer PP)
- C. RMS velocity (mm/s), tahan spike noise
- D. Maximum kurtosis

9. Sebuah sinyal memiliki distribusi sample yang asimetris dengan ekor lebih panjang ke kanan (positif). Nilai skewness-nya...

- A. Negatif (skewness selalu berlawanan arah ekor)
- B. Positif, ekor kanan berarti skew positif
- C. Tepat 0, skewness tidak peduli arah
- D. Selalu lebih besar dari 3

10. Mengapa praktik industri menggabungkan banyak fitur sekaligus (RMS + CF + Kurtosis + Skewness) untuk monitoring bearing?

- A. Lebih banyak fitur otomatis bikin model lebih akurat
- B. Tiap fitur sensitif terhadap jenis kerusakan berbeda; satu fitur saja tidak cukup karena fenomena U-shape
- C. Untuk membuang fitur orde rendah yang tidak akurat
- D. Karena tidak ada satu fitur pun yang berfungsi

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Dataset referensi (dipakai C1-C10, deterministik untuk auto-grading):

```
import numpy as np
N = 1000
i = np.arange(N)
x = np.sin(2*np.pi*i/100) + 0.5*((-1)**i)
```

C1. Hitung mean sinyal x dari dataset referensi. Untuk vibrasi AC-coupled, ekspektasi mean mendekati 0.

-  **HINT:** Gunakan `np.mean(x)`; untuk komponen sinusoidal + alternating yang simetris, rata-ratanya mendekati nol.

C2. Hitung standar deviasi populasi (`ddof=0`) sinyal x dengan `np.std(x, ddof=0)`.

-  **HINT:** Gunakan `np.std(x, ddof=0)` untuk standar deviasi populasi sinyal.

C3. Hitung RMS sinyal x menggunakan formula akar($(1/N)\sum x_i^2$). Untuk sinyal AC-coupled, RMS harus mendekati std.

-  **HINT:** RMS = akar dari rata-rata kuadrat sinyal; gunakan `np.sqrt(np.mean(x**2))`. Untuk mean mendekati 0 nilainya mendekati std.

C4. Hitung peak ($|x|_{\max}$) sinyal x, nilai absolut maksimum.

-  **HINT:** Peak = nilai absolut maksimum; kombinasikan `np.abs` dengan `np.max` atas x.

C5. Hitung peak-to-peak (PP) sinyal x: $PP = \max(x) - \min(x)$.

-  **HINT:** Peak-to-peak = `np.max(x) - np.min(x)` (selisih nilai maksimum dan minimum sinyal).

C6. Hitung Crest Factor sinyal x: $CF = \text{peak}/\text{RMS}$.

-  **HINT:** Crest Factor = Peak dibagi RMS; bagi nilai absolut maksimum dengan RMS sinyal.

C7. Hitung Mean Absolute sinyal x: $|x|_{\text{avg}} = (1/N)\sum |x_i|$.

-  **HINT:** Absolute average = rata-rata nilai absolut sinyal; gunakan `np.mean(np.abs(x))`.

C8. Hitung Shape Factor sinyal x: $SF = \text{RMS}/|x|_{\text{avg}}$.

-  **HINT:** Shape Factor = RMS dibagi absolute average; bagi RMS dengan rata-rata nilai absolut sinyal.

C9. Hitung Skewness (asimetri distribusi) sinyal x. Karena sinyal ini simetris, skewness mendekati 0.

-  **HINT:** Gunakan `scipy.stats.skew` atau hitung manual: rata-rata pangkat tiga deviasi dibagi std pangkat tiga.

C10. Hitung Kurtosis (plain, fisher=False) sinyal x. Untuk sinyal Gaussian nilainya 3; untuk sinyal ini sedikit di bawah karena mix sinusoidal+alternating bersifat platykurtic.

- 💡 **HINT:** Gunakan `scipy.stats.kurtosis` dengan `fisher=False` untuk kurtosis (Pearson).

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan fungsi `extract_features(x)` reusable yang mengembalikan dictionary berisi 12 fitur domain waktu: mean, std, rms, peak, pp, abs_mean, square_mean, crest_factor, shape_factor, impulse_factor, kurtosis, skewness. Pakai `scipy.stats.kurtosis(x, fisher=False)` dan `scipy.stats.skew(x)`. Terapkan pada dataset referensi. Laporkan nilai `impulse_factor`.

- 💡 **HINT:** Bangun fungsi yang mengembalikan dict berisi tiap fitur; Impulse Factor = peak dibagi absolute average (nilai absolut maksimum dibagi rata-rata nilai absolut).

C12 (Hard). Implementasikan simulasi degradasi bearing: untuk progress p dalam {0.0, 0.5, 1.0}, generate sinyal $\text{sig} = \sin(2\pi \cdot 30 \cdot t) + 0,4 \cdot (1+2p) \cdot \text{np.random.randn}(N)$ dengan tambahan $\text{int}(100 \cdot p)$ impak transien ($\text{decay} \times \sin(2\pi \cdot 2000 \cdot t)$) di lokasi acak. Pakai `np.random.seed(42)`, `fs=25600`, `N=fs*1`, `T=1` detik. Hitung RMS pada `p=1,0` (severe).

- 💡 **HINT:** Loop tiap tingkat severity p, bangun sinyal lengkap lalu hitung RMS-nya dengan `np.sqrt(np.mean(sig**2))`; severity tinggi menghasilkan RMS yang lebih besar dari kondisi healthy.

C13 (Hard). Implementasikan U-shape detection algorithm: dari trajectory crest factor sepanjang lifetime bearing (50 segmen), deteksi indeks puncak (early-fault) dan akhir trajectory (severe). Generate trajectory dengan formula $\text{CF}(p) = 3 + 8 \cdot p \cdot \exp(-4p) + \text{noise}(0,0.2)$, p dalam `linspace(0,1,50)`, `seed=42`. Laporkan indeks segmen dengan CF puncak (`argmax` dari trajectory).

- 💡 **HINT:** Bangun trajectory crest factor sesuai model di soal, lalu cari indeks puncaknya dengan `np.argmax`; puncak teoretis ada di turunan nol dari suku $p \cdot \exp(-4p)$.

C14 (Hard). Implementasikan klasifikasi healthy vs faulty berbasis threshold: untuk dataset referensi, injeksikan 5 impak besar di indeks [100,250,400,600,850] dengan amplitudo +5,0 (single sample) menjadi `x_faulty`. Hitung kurtosis x dan `x_faulty`. Laporkan selisih kurtosis (faulty – healthy), harus positif besar karena impak menambah ekor distribusi.

- 💡 **HINT:** Injeksikan spike sesuai soal ke salinan sinyal, lalu hitung selisih kurtosis(..., fisher=False) antara sinyal faulty dan baseline; spike besar menaikkan kurtosis tajam.

C15 (Hard). Implementasikan feature pipeline lengkap untuk segmen multi-channel: simulasikan 3 sensor (acc_x , acc_y , acc_z) dari dataset referensi tetapi $acc_y = x \cdot 1,2$, $acc_z = x \cdot 0,8$ (faktor channel). Untuk setiap channel hitung 12 fitur. Buat feature matrix dengan shape (3,12). Laporkan jumlah total semua RMS dari 3 channel: $rms_total = \sum(rms_channel \text{ for each})$.

- 💡 **HINT:** RMS bersifat linear terhadap skala: $RMS(\alpha \cdot x) = \alpha \cdot RMS(x)$, jadi total = (jumlah koefisien) $\times$ $RMS(x)$. Implementasikan dengan loop atau operasi vektor.

DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. <https://doi.org/10.1609/aaai.v37i9.26317>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 7

Ekstraksi Fitur Domain Frekuensi

Abstrak

Pertemuan ini membahas ekstraksi fitur domain frekuensi dari sinyal vibrasi mesin sebagai pelengkap fitur domain

Sub - CPMK

Sub-CPMK 3.1 – Mampu menggunakan metode simulasi yang tepat dalam desain sistem

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-7.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Fitur domain waktu (Pertemuan 6) memadatkan sinyal menjadi statistik deskriptif seperti RMS dan crest factor, tetapi banyak pola kerusakan mesin bersifat periodik dan hanya tampak jelas ketika sinyal ditransformasi ke domain frekuensi. Pertemuan ketujuh ini membahas ekstraksi fitur domain frekuensi, dirancang dan diuji melalui simulasi Python pada sinyal bearing sintesis sebelum diterapkan pada data sensor sungguhan.

Posisi Pertemuan 7 dalam Alur Mata Kuliah Optimalisasi & Otomasi



Gambar 1. Posisi Pertemuan 7 (Fitur Domain Frekuensi) dalam alur mata kuliah, melengkapi fitur domain waktu Pertemuan 6 sebelum masuk ke materi optimasi Pertemuan 9-13.

Seperti terlihat pada Gambar 1, Pertemuan 7 menjembatani fitur domain waktu dengan materi optimasi (Linear Programming, optimasi non-linear, metaheuristik) yang dimulai di Pertemuan 9. Kombinasi fitur waktu dan frekuensi menghasilkan feature vector 24 dimensi yang menjadi input model machine learning condition monitoring di Pertemuan 14-15.

Capaian Pembelajaran (Sub-CPMK)

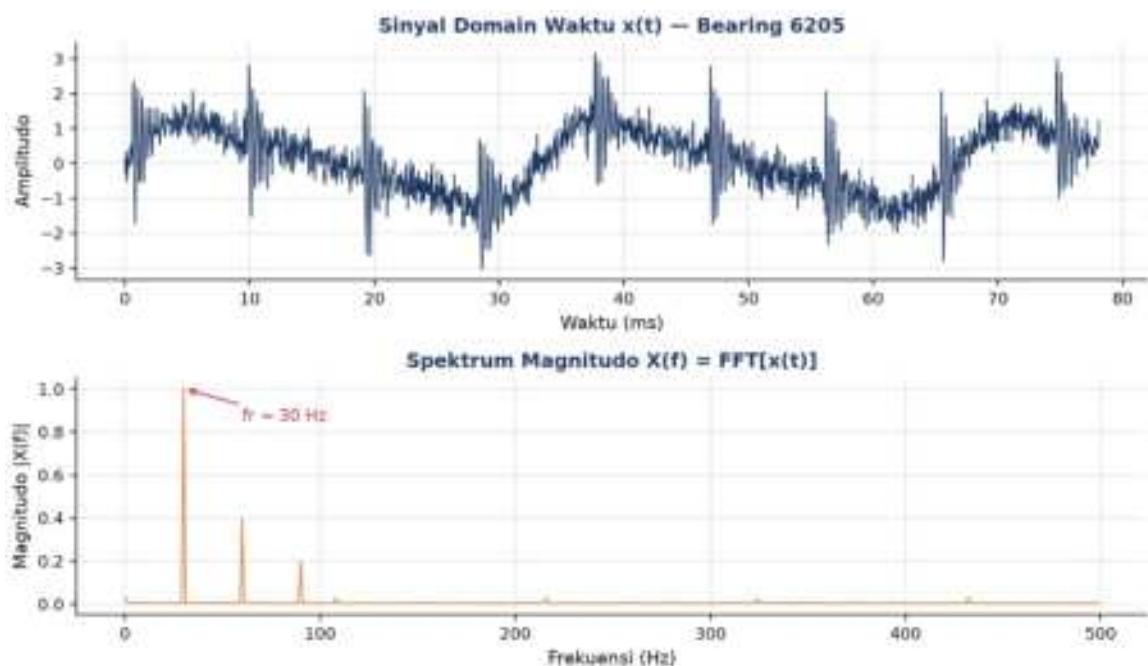
- **Sub-CPMK 3.1:** Mampu menggunakan metode simulasi yang tepat dalam desain sistem, yaitu memilih dan menerapkan metode ekstraksi fitur domain frekuensi (FFT, PSD, envelope analysis) yang tepat melalui simulasi Python sebagai dasar desain sistem monitoring kondisi mesin (CPMK 3).
- Setelah pertemuan ini mahasiswa dapat: menghitung dan menginterpretasikan DFT/FFT beserta resolusi frekuensinya; membedakan periodogram dan Welch PSD; menghitung frekuensi karakteristik kerusakan bearing (BPFO, BPFI, BSF, FTF); serta menerapkan envelope analysis untuk mendeteksi fault dini.

DFT & FFT: DARI SAMPEL DISKRIT KE SPEKTRUM FREKUENSI

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N} \quad f_k = k \cdot f_s/N \quad \Delta f = f_s/N \quad |X[k]| = \sqrt{(\text{Re}^2 + \text{Im}^2)}$$

DFT vs FFT: DFT (Discrete Fourier Transform) menghitung spektrum secara langsung dengan kompleksitas $O(N^2)$; untuk $N=8192$ sampel dibutuhkan sekitar 67 juta operasi. FFT (Fast Fourier Transform, algoritma Cooley-Tukey 1965) menghitung hasil identik dengan kompleksitas $O(N \log N)$, untuk N yang sama hanya sekitar 100 ribu operasi, atau sekitar 660 kali lebih cepat. Inilah sebabnya FFT menjadi standar praktis, bukan DFT langsung.

- **Window Function:** sebelum FFT diterapkan, sinyal dikalikan dengan window function (Hann, Hamming, Blackman, flat-top) agar potongan sinyal non-periodik di batas segmen tidak menimbulkan spectral leakage (kebocoran energi ke bin frekuensi tetangga).

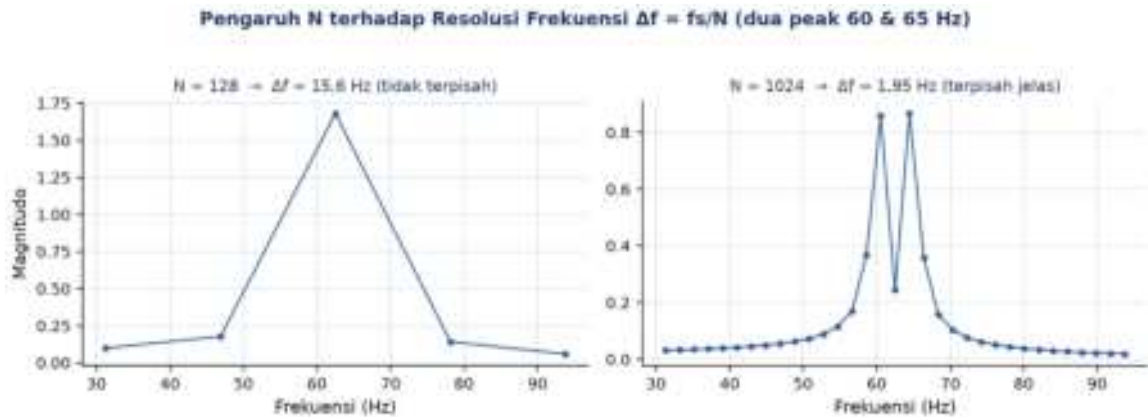


Gambar 2. Sinyal domain waktu $x(t)$ bearing 6205 (atas) dan spektrum magnitudo hasil FFT $X(f)$ (bawah), dengan peak $f_r=30$ Hz.

Gambar 2 menunjukkan transformasi sinyal domain waktu menjadi spektrum magnitudo; peak paling dominan muncul pada frekuensi rotasi shaft $f_r=30$ Hz. Untuk spektrum one-sided (fungsi ``numpy.fft.rfft``), setiap bin dikalikan faktor skala $2/N$ agar merepresentasikan amplitudo fisik yang benar, kecuali bin DC dan Nyquist yang memakai faktor $1/N$.

Resolusi Frekuensi: Resolusi frekuensi $\Delta f=f_s/N$ diperbaiki dengan menambah durasi rekaman $T=N/f_s$, bukan sekadar menaikkan f_s . N sebaiknya berupa pangkat 2 agar FFT efisien; $N=1024-2048$ cukup untuk observasi umum, sedangkan $N=8192-32768$ diperlukan

untuk memisahkan BPFO/BPFI yang berdekatan dengan frekuensi line 50/60 Hz. Zero-padding menghasilkan interpolasi visual pada spektrum, bukan resolusi tambahan yang sesungguhnya.



Gambar 3. Pengaruh N terhadap resolusi frekuensi $\Delta f = f_s/N$: dua peak berdekatan (60 & 65 Hz) tidak terpisah pada $N=128$ tetapi jelas terpisah pada $N=1024$.

Gambar 3 membuktikan secara visual bahwa menaikkan N (memperpanjang durasi rekaman) memperhalus Δf sehingga dua peak yang berdekatan dapat dibedakan secara diagnostik.

MAGNITUDE SPECTRUM & POWER SPECTRAL DENSITY

$|X[k]|$ (magnitude) $|X[k]|^2$ (power) $PSD = |X[k]|^2/(f_s \cdot N)$ Welch PSD = rata-rata antar-segmen tumpang-tindih

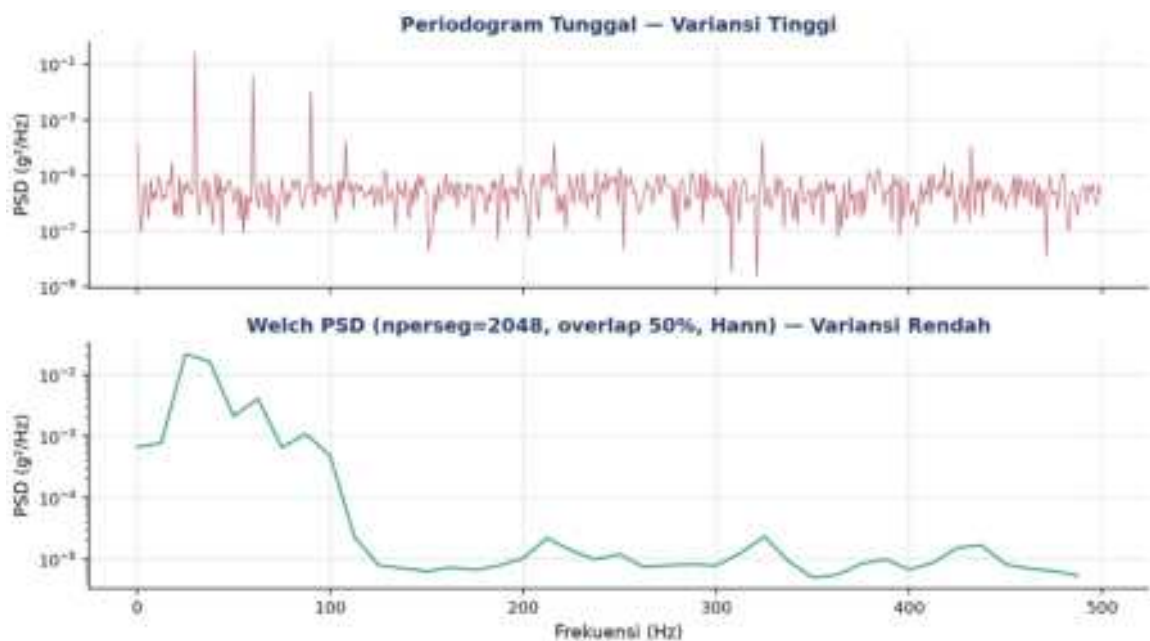
Trade-off Variansi vs Resolusi: Periodogram tunggal (power spectrum dari satu blok FFT penuh) memiliki resolusi frekuensi paling halus tetapi variansi estimasi yang tinggi, sehingga rawan disalahartikan sebagai peak diagnostik padahal hanya derau statistik. Welch's method mengatasi ini dengan membagi sinyal menjadi beberapa segmen tumpang-tindih (biasanya 50%), menerapkan window Hann pada tiap segmen, menghitung periodogram tiap segmen, lalu merata-ratakannya; hasilnya variansi jauh lebih rendah dengan konsekuensi resolusi frekuensi sedikit lebih kasar ($\Delta f = f_s/n_{\text{perseg}}$).

Pemilihan Window Function: Pemilihan window function pada tiap segmen Welch juga memengaruhi trade-off antara spectral leakage dan resolusi: window Hann (default) memberi keseimbangan yang baik untuk kebanyakan aplikasi vibrasi, window Hamming sedikit lebih tajam pada main lobe tetapi sidelobe lebih tinggi, sedangkan window Flat-Top memberi akurasi amplitudo terbaik namun resolusi frekuensi yang paling kasar di antara ketiganya, sehingga cocok khusus untuk kalibrasi amplitudo, bukan untuk pemisahan peak frekuensi berdekatan.

Tabel 1. Perbandingan estimator spektrum berdasarkan variansi, resolusi, dan kapan masing-masing dipakai.

Estimator Spektrum	Variansi	Resolusi	Kapan Dipakai
Periodogram tunggal	Tinggi	$\Delta f = fs/N$ (paling halus)	Quick-look saja, bukan diagnostik
Welch (50% overlap, Hann)	Rendah	$\Delta f = fs/nperseg$	Default: bearing, gear, motor
Bartlett (tanpa overlap)	Rendah	Sama seperti Welch	Sinyal panjang, implementasi simpel
Multitaper	Sangat rendah	$\Delta f = fs/N$ (lebih baik dari Welch)	Penelitian, sinyal pendek kritikal
Lomb-Scargle	Bergantung data	Variabel	Sampling tidak seragam (non-uniform)

Tabel 1 merangkum lima estimator spektrum yang umum dipakai; `scipy.signal.welch(x, fs, nperseg=2048, noverlap=1024, window='hann')` menjadi pilihan default untuk diagnosa bearing, gear, dan motor.



Gambar 4. Periodogram tunggal (atas, variansi tinggi) dibandingkan Welch PSD dengan `nperseg=2048` dan `overlap 50%` (bawah, variansi rendah).

Gambar 4 memperlihatkan perbedaan visual: periodogram tunggal tampak berisik di sekujur spektrum, sedangkan Welch PSD menghasilkan kurva yang jauh lebih halus sehingga peak diagnostik lebih mudah dibedakan dari noise floor.

Teorema Parseval: Teorema Parseval menyatakan energi total domain waktu sama dengan energi total domain frekuensi: $\sum |x[n]|^2 = (1/N) \cdot \sum |X[k]|^2$, atau ekuivalen $\sum x^2 = \int \text{PSD} df$. Identitas ini berguna untuk memverifikasi implementasi FFT/PSD; jika kedua sisi tidak sama, kemungkinan ada kesalahan skala.

Skala dB: Spektrum sering ditampilkan dalam skala desibel (dB) untuk memampatkan rentang dinamis yang lebar: $\text{dB} = 10 \cdot \log_{10}(P/P_{\text{ref}})$. Rasio power 0,50 terhadap 0,05 misalnya setara dengan 10 dB.

FREKUENSI KARAKTERISTIK BEARING: BPFO, BPFI, BSF, FTF

$$\begin{aligned} \text{BPFO} &= (n/2) \cdot \text{fr} \cdot (1 - (d/D) \cdot \cos \alpha) & \text{BPFI} &= (n/2) \cdot \text{fr} \cdot (1 + (d/D) \cdot \cos \alpha) & \text{BSF} &= \\ & (D/2d) \cdot \text{fr} \cdot (1 - (d/D)^2 \cdot \cos^2 \alpha) & \text{FTF} &= (\text{fr}/2) \cdot (1 - (d/D) \cdot \cos \alpha) \end{aligned}$$

- **BPFO (Ball Pass Frequency Outer race):** untuk bearing 6205 ($n=9$, $d=7,94\text{mm}$, $D=39,04\text{mm}$, $\alpha=0^\circ$) pada shaft 1480 rpm ($\text{fr} \approx 24,67\text{ Hz}$), BPFO menandakan kerusakan pada outer race; sensitivitas deteksi paling tinggi karena posisi outer race statis relatif terhadap sensor.
- **BPFI (Ball Pass Frequency Inner race):** selalu lebih besar dari BPFO pada geometri bearing yang sama karena formula memakai tanda plus ($1 + (d/D) \cos \alpha$); dimodulasi oleh fr sehingga muncul sideband $\text{BPFI} \pm \text{fr}$ di spektrum.
- **BSF (Ball Spin Frequency):** menandakan kerusakan pada elemen bola/roller; deteksi lebih sulit karena posisi bola yang rusak terus berputar relatif terhadap sensor, sering disertai harmonik $2 \cdot \text{BSF}$.
- **FTF (Fundamental Train Frequency):** selalu lebih kecil dari $\text{fr}/2$; merupakan indikator akhir umur pakai cage/sangkar bearing, paling jarang terdeteksi karena amplitudonya rendah.
- **Harmoni & Sideband:** kerusakan lokal menghasilkan harmonik $k \cdot \text{BPFI}$ atau $k \cdot \text{BPFO}$ dan sideband $\pm m \cdot \text{fr}$ di sekitarnya; pola sideband inilah yang membedakan fault bearing dari sekadar unbalance atau misalignment.

Tabel 2. Lokasi defect bearing dan pola frekuensi diagnostiknya di spektrum, beserta sensitivitas terhadap deteksi stadium awal.

Lokasi Defect	Frekuensi Diagnostik	Pola di Spektrum	Sensitivitas Stadium Awal
Outer race	BPFO + harmoni	Peak diskrit, sedikit sideband	Tinggi, paling mudah dideteksi
Inner race	BPFI $\pm k \cdot \text{fr}$ (sideband)	Peak + sideband $\pm \text{fr}$	Tinggi, terutama di envelope
Ball / roller	BSF + harmoni $2 \cdot \text{BSF}$	Peak ganda + modulasi cage	Sedang, posisi bola berputar
Cage	FTF	Peak frekuensi rendah	Rendah, indikasi akhir umur bearing
Lubrication poor	Broadband 1-10 kHz	Tidak ada peak diskrit	Bukan via FFT, pakai stress wave/acoustic emission

Tabel 2 merangkum keterkaitan lokasi kerusakan bearing dengan pola spektrum yang dihasilkan.



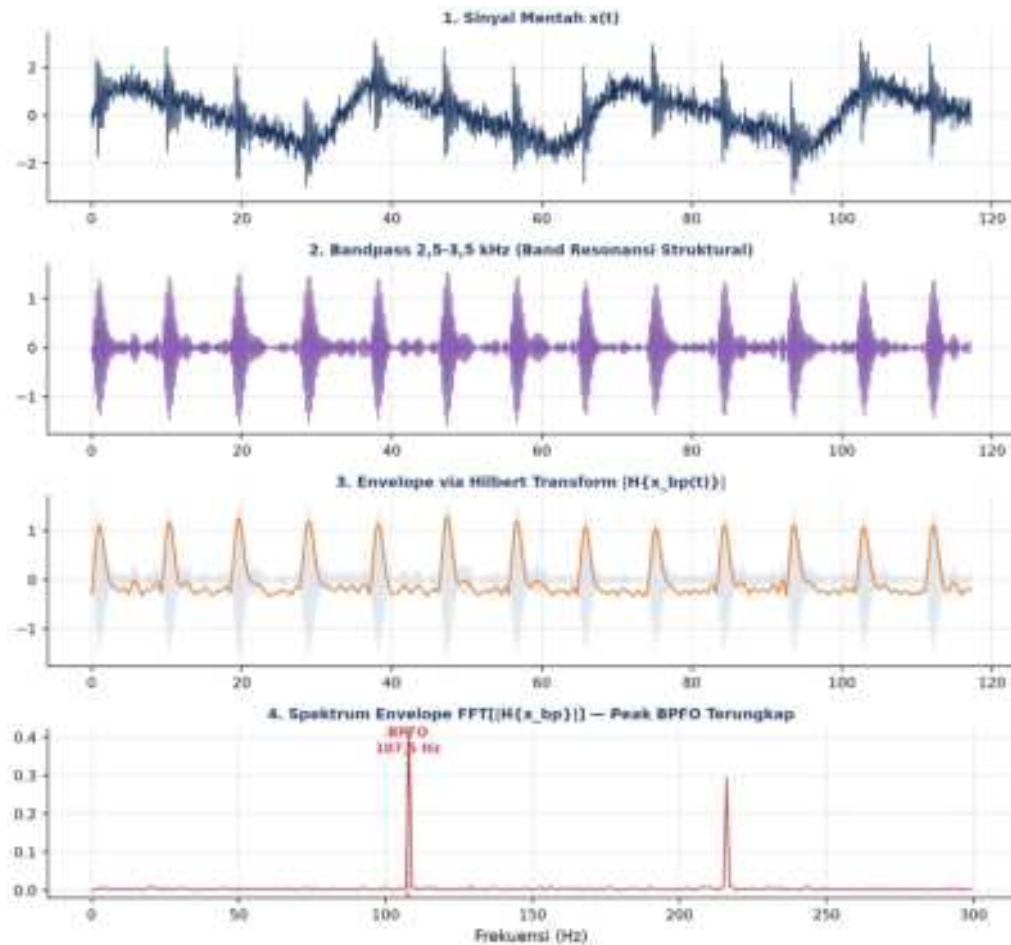
Gambar 5. Spektrum sinyal bearing 6205 dengan overlay garis frekuensi karakteristik FTF, fr, BSF, BPFO, dan BPFI (fr=30 Hz).

Gambar 5 menunjukkan keempat frekuensi karakteristik bearing pada satu spektrum yang sama, memperlihatkan bagaimana peak fault (di sini BPFO≈107,5 Hz) dapat dibedakan dari peak harmonik shaft (fr dan kelipatannya).

Slip dalam Realita: Frekuensi yang teramati pada mesin sungguhan biasanya 1-3% lebih rendah dari prediksi teoretis akibat slip antara bola dan race; disarankan memberi toleransi pencocokan sekitar $\pm 5\%$ saat mencocokkan peak spektrum dengan frekuensi karakteristik hasil perhitungan.

ENVELOPE ANALYSIS: MENGUNGKAP FAULT TERSEMBUNYI DI FREKUENSI RESONANSI

Mengapa Perlu Envelope Analysis: Energi dampak akibat defect bearing seringkali tidak muncul langsung pada frekuensi BPFO/BPFI di spektrum FFT biasa, melainkan memodulasi (menumpangi) frekuensi resonansi struktural bearing/housing yang jauh lebih tinggi (umumnya 2-20 kHz). Envelope analysis (High Frequency Resonance Technique/HFRT) mengekstrak modulasi tersebut lewat pipeline empat tahap: bandpass filter pada band resonansi, transformasi Hilbert untuk mendapatkan envelope (selubung amplitudo), lalu FFT pada envelope tersebut untuk mengungkap peak BPFO/BPFI yang sebelumnya tersembunyi.



Gambar 6. Pipeline envelope analysis empat tahap: (1) sinyal mentah, (2) hasil bandpass 2,5-3,5 kHz, (3) envelope via Hilbert transform, (4) spektrum FFT envelope dengan peak BPFO terungkap jelas.

Gambar 6 mendemonstrasikan pipeline tersebut pada sinyal bearing 6205: impak periodik yang samar di sinyal mentah (panel 1) menjadi jelas berulang secara periodik setelah bandpass filtering (panel 2), envelope Hilbert (panel 3) menangkap pola amplop dari tiap impak, dan spektrum FFT dari envelope (panel 4) menunjukkan peak tajam tepat di frekuensi BPFO. Teknik ini dipakai secara luas oleh sistem monitoring komersial seperti SKF Multilog, Bently Nevada, dan IFM Octavis.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Equalizer Audio & Spektrum Frekuensi Live:** bar-bar equalizer pada aplikasi streaming musik pada dasarnya adalah magnitude spectrum real-time; window FFT sekitar 20ms dipakai agar update terasa responsif secara visual.
- **Shazam & Constellation Map:** aplikasi pengenalan lagu membangun constellation map dari peak dominan hasil STFT lalu mencocokkannya via hashing terhadap database,

analog dengan bagaimana peak diskrit BPFO/BPFI berfungsi sebagai sidik jari kerusakan bearing.

- **Radio Tuning & Selektivitas Frekuensi:** tuner radio FM memakai bandpass filter untuk menyeleksi satu stasiun dari spektrum yang padat, sama seperti envelope analysis membandpass sinyal ke band resonansi 5-20kHz sebelum ekstraksi envelope.
- **Prisma Newton & Dekomposisi Cahaya Putih:** cahaya putih adalah superposisi berbagai warna yang diuraikan prisma; sinyal getaran adalah superposisi berbagai komponen sinusoidal yang diuraikan FFT, ibarat prisma numerik.
- **Heart Rate Variability (HRV) Spectrum:** rasio daya low-frequency (0,04-0,15 Hz) terhadap high-frequency (0,15-0,4 Hz) pada spektrum denyut jantung dipakai mendiagnosis keseimbangan sistem saraf otonom; mesin sehat menghasilkan spektrum broadband datar, sedangkan fault menghasilkan peak diskrit yang tajam.
- **Spektrogram Suara & Voice Print:** ahli forensik suara mengidentifikasi formant F1/F2/F3 (200-3500Hz) dari spektrogram suara; sinyal non-stasioner seperti run-up motor atau perpindahan gigi transmisi juga memerlukan spektrogram (STFT), bukan FFT tunggal.

Pola Umum: Pola yang sama muncul di semua analogi di atas: transformasi dari domain waktu ke domain frekuensi mengungkap struktur tersembunyi yang tidak tampak di sinyal mentah. Kerusakan yang bersifat periodik menghasilkan peak diskrit, sedangkan kerusakan yang bersifat stokastik/acak menghasilkan spektrum broadband tanpa peak yang jelas.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

- **Envelope Analysis Bearing:** accelerometer 25kHz dengan pipeline bandpass 5-10kHz, Hilbert, lalu FFT dipakai secara luas oleh sistem monitoring komersial (SKF Multilog, Bently Nevada, IFM Octavis) untuk deteksi dini kerusakan bearing di lapangan.
- **Gear Mesh Frequency & Sideband:** Gear Mesh Frequency ($GMF = \text{jumlah gigi} \times fr$) dan sideband $\pm fr$ di sekitarnya dipakai mendeteksi chipped/cracked tooth pada gearbox; threshold rasio sideband $>5\%$ memicu flag maintenance, dipakai oleh Volvo dan Caterpillar, mengikuti pendekatan dataset NREL gearbox.
- **Motor Current Signature Analysis (MCSA):** FFT arus stator motor mendeteksi broken rotor bar lewat sideband pada $f_{sb} = (1 \pm 2s) \cdot f_{supply}$; untuk slip 2-5% dan supply 50Hz, sideband muncul di sekitar 47-53Hz dan 99-101Hz.

- **Wind Turbine Blade-Pass & Tower Resonance:** blade-pass frequency

$f_{BP} = \text{jumlah_blade} \times f_r$ ($3 \times f_r$ untuk rotor 3-blade) dipantau bersama resonansi tower ($\sim 0,3\text{Hz}$); ice buildup asimetris pada blade menaikkan amplitudo blade-pass secara tidak simetris, memicu sistem SCADA mengaktifkan proses ice-removal.

Peringatan, Jebakan Praktis FFT & PSD: Kesalahan praktis yang sering terjadi: (1) lupa menerapkan window function sehingga terjadi spectral leakage; (2) salah skala amplitudo, lupa mengalikan faktor $2/N$; (3) menerapkan FFT tunggal pada sinyal non-stasioner sehingga peak menjadi kabur (smear), seharusnya memakai STFT/order tracking; (4) resolusi frekuensi terlalu kasar, perlu memperbesar N atau memakai zoom-FFT; (5) aliasing karena $f_s < 2 \cdot f_{\text{max}}$, perlu anti-aliasing filter sebelum ADC; (6) hanya mengandalkan periodogram tunggal untuk diagnostik, seharusnya memakai Welch PSD.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada sinyal bearing sintesis. Lingkungan: conda env optoauto dengan paket numpy, scipy, matplotlib; notebook p7_frequency_domain_features.ipynb.

```
p7_frequency_domain_features.ipynb — Cell 4
def extract_features_freq(x, fs):
    """Ekstrak 12 fitur domain frekuensi dari sinyal x."""
    N = len(x)
    X = np.fft.rfft(x)
    freqs = np.fft.rfftfreq(N, d=1/fs)
    P = (np.abs(X)**2) / N
    p_norm = P / P.sum()

    peak_freq = freqs[np.argmax(P)]
    mean_freq = np.sum(freqs * p_norm)
    var_freq = np.sum(((freqs-mean_freq)**2)*p_norm)
    sk_spec = np.sum((((freqs-mean_freq)**3)*p_norm) / var_freq**1.5
    ku_spec = np.sum((((freqs-mean_freq)**4)*p_norm) / var_freq**2
    sp_ent = entropy(p_norm)

    return {'peak_freq': peak_freq, 'mean_freq': mean_freq,
            'spectral_kurtosis': ku_spec, 'spectral_entropy': sp_ent, ...}
```

Gambar 7. Cuplikan fungsi `extract_features_freq()` yang menghitung 12 fitur domain frekuensi dalam satu dictionary.

Gambar 7 menunjukkan cuplikan fungsi `extract_features_freq()` yang dipakai berulang sepanjang modul.

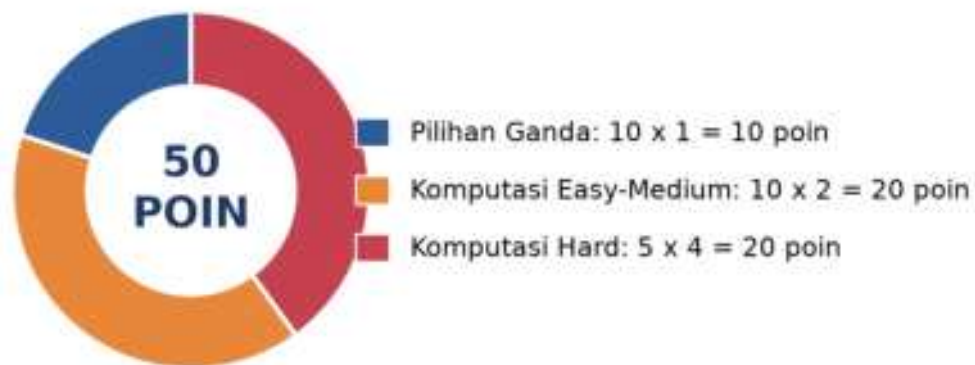
- **Cell 1:** men-generate sinyal bearing 6205 (rotasi 30Hz + 2 harmonik + noise Gaussian) dengan dampak periodik BPFO 107,7Hz (decay $\tau \approx 1\text{ms}$, carrier 3000Hz); mencetak Δf , Nyquist, dan statistik sinyal.

- **Cell 2:** menghitung FFT magnitude spectrum (`scipy.fft.rfft`) dan Welch PSD (`scipy.signal.welch`, `nperseg=4096`, `overlap 2048`, `window Hann`); deteksi peak di bawah 500Hz dengan `scipy.signal.find_peaks`.
- **Cell 3:** menghitung frekuensi fault bearing 6205 (BPFO/BPFI/FTF/BSF) dari geometri dan `fr=30Hz`, lalu overlay sebagai garis vertikal berwarna pada spektrum Cell 2.
- **Cell 4:** mendefinisikan fungsi `extract_features_freq(x, fs)` reusable (12 fitur: `peak_freq`, `mean_freq`, `rms_freq`, `spectral_skewness`, `spectral_kurtosis`, `spectral_entropy`, `total_power`, `band_power_low/mid/high`, `ratio_high_to_low`, `parseval_check`), plus envelope analysis (bandpass Butterworth 2500-3500Hz → Hilbert → FFT envelope) dan verifikasi Parseval.

TUGAS PERTEMUAN 7

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 7



Gambar 8. Distribusi 50 poin Tugas Pertemuan 7 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Pernyataan yang BENAR mengenai DFT vs FFT adalah...

- A. DFT dan FFT memberi hasil numerik yang berbeda
- B. FFT adalah algoritma efisien yang menghitung DFT, kompleksitas turun dari $O(N^2)$ ke $O(N \cdot \log N)$
- C. FFT hanya bekerja untuk sinyal sinusoidal murni
- D. DFT lebih cepat dari FFT karena tidak butuh padding

2. Resolusi frekuensi Δf dari spektrum FFT ditentukan oleh...

- A. $\Delta f = f_s \cdot N$ (semakin tinggi sampling, semakin jelek resolusi)
- B. $\Delta f = f_s / N$, diperbaiki dengan menambah durasi rekaman $T = N/f_s$
- C. Δf hanya tergantung window function, tidak f_s/N
- D. $\Delta f = 1$ Hz selalu, terlepas dari f_s dan N

3. Tujuan utama menerapkan window function (Hann, Hamming, Blackman) sebelum FFT adalah...

- A. Mempercepat algoritma FFT
- B. Mengurangi spectral leakage akibat truncation sinyal non-periodik di batas segmen
- C. Menambahkan harmonik palsu agar peak lebih jelas
- D. Menghilangkan komponen DC otomatis

4. Untuk mendapatkan amplitudo fisik yang benar dari spektrum one-sided (rfft), faktor skala yang dipakai pada $|X(k)|$ adalah...

- A. $1/N$
- B. $2/N$ (untuk bin di antara DC dan Nyquist)
- C. $1/\text{akar}(N)$
- D. $N/2$ (memperbesar dengan jumlah sample)

5. Trade-off antara periodogram raw dan Welch method adalah...

- A. Welch sama persis dengan periodogram tetapi lebih lambat
- B. Welch menurunkan variansi estimasi PSD dengan cost menurunkan resolusi frekuensi (segmenting + averaging)
- C. Periodogram tidak perlu window, Welch wajib pakai Blackman saja
- D. Welch hanya bisa untuk sinyal stasioner Gaussian

6. Untuk geometri bearing yang sama dan kecepatan shaft tetap, hubungan antara BPFI dan BPFO adalah...

- A. BPFI = BPFO selalu (geometri tidak berpengaruh)
- B. BPFI lebih besar dari BPFO karena formula BPFI memakai $(1 + (d/D)\cos\alpha)$ sedangkan BPFO memakai $(1 - (d/D)\cos\alpha)$
- C. BPFI lebih kecil dari BPFO karena inner race lebih kecil
- D. BPFI = $2 \times$ BPFO selalu (rasio konstan)

7. Pipeline envelope analysis standar untuk diagnosa bearing fault adalah...

- A. FFT raw, lalu low-pass filter, lalu cari peak BPFO langsung
- B. Bandpass filter (band resonansi), Hilbert transform (envelope), FFT envelope, lalu cari peak BPFO/BPFI
- C. Squaring sinyal saja, lalu plot time-domain
- D. STFT, ambil maximum spectrogram, langsung jadi diagnosis

8. Kriteria Nyquist-Shannon menyatakan bahwa untuk merekonstruksi sinyal dengan komponen frekuensi maksimum $f_{\max}$, sampling rate f_s harus...

- A. $f_s = f_{\max}$ (sama persis)
- B. $f_s > 2 \cdot f_{\max}$; jika tidak akan terjadi aliasing (frekuensi tinggi muncul palsu di low-band)
- C. $f_s = f_{\max}/2$ (Nyquist = setengah)
- D. f_s bebas, aliasing tidak pernah terjadi

9. Teorema Parseval untuk DFT menyatakan bahwa...

- A. $\sum |x_n|^2$ selalu = 1 (normalisasi otomatis)
- B. Energi total domain waktu sama dengan energi total domain frekuensi: $\sum |x_n|^2 = (1/N) \cdot \sum |X_k|^2$
- C. Hanya berlaku untuk sinyal sinusoidal murni
- D. Hubungan antara amplitudo dan fase saja

10. Saat motor mengalami run-up (kecepatan berubah dari 0 ke 1500 rpm), tool analisis frekuensi yang paling tepat adalah...

- A. FFT tunggal pada seluruh segmen, peak akan jelas
- B. Spectrogram (STFT) atau order tracking, karena sinyal non-stasioner dan frekuensi berubah seiring waktu
- C. Welch PSD averaging, variansi rendah sudah cukup
- D. Hilbert envelope saja sudah cukup

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Konstanta bersama (dipakai C1-C15, deterministik untuk auto-grading):

```
import numpy as np
# Geometri bearing 6205
n_balls, d, D, alpha = 9, 7.94, 39.04, 0.0
ratio = d / D                                # sekitar 0,2034

# Formula bearing fault frequency
# BPF0 = (n/2) * fr * (1 - (d/D)*cos(alpha))
# BRFI = (n/2) * fr * (1 + (d/D)*cos(alpha))
# FTF = (fr/2) * (1 - (d/D)*cos(alpha))
# BSF = (D/(2d))* fr * (1 - ((d/D)*cos(alpha))**2)
```

C1. Hitung resolusi frekuensi Δf untuk $f_s = 10000$ Hz dan $N = 8192$ sample. Formula: $\Delta f = f_s / N$. Print nilainya (Hz, presisi 4 desimal).

- 💡 **HINT:** Resolusi frekuensi $\Delta f = f_s$ dibagi N . Resolusi makin halus dengan N besar (durasi rekaman lebih panjang). Substitusikan f_s dan N dari soal.

C2. Generate sinyal sinusoidal murni 60 Hz: $f_s=2000$, $N=4000$, $t=np.arange(N)/f_s$, $x=np.sin(2*np.pi*60*t)$. Hitung peak frequency dengan `scipy.fft.rfft + rfftfreq`. Print nilainya (Hz).

- 💡 **HINT:** Hitung spektrum dengan rfft dan frekuensinya dengan rfftfreq, lalu ambil frekuensi pada magnitudo terbesar via `np.argmax(np.abs(X))`.

C3. Hitung BPFO (Ball Pass Frequency Outer race) untuk bearing 6205 ($n=9$, $d=7,94$, $D=39,04$, $\alpha=0^\circ$) pada shaft 1480 rpm. Formula: $BPFO = (n/2) \cdot fr \cdot (1 - (d/D) \cdot \cos\alpha)$. Print dalam Hz.

- 💡 **HINT:** $fr = \text{rpm} \text{ dibagi } 60$; $BPFO = (n/2) \cdot fr \cdot (1 - (d/D) \cdot \cos\alpha)$. Substitusikan jumlah bola, diameter, dan sudut kontak dari soal.

C4. Untuk parameter sama dengan soal sebelumnya (bearing 6205, 1480 rpm), hitung BPFI (Ball Pass Frequency Inner race). Formula: $BPFI = (n/2) \cdot fr \cdot (1 + (d/D) \cdot \cos\alpha)$. Print dalam Hz.

- 💡 **HINT:** $BPFI = (n/2) \cdot fr \cdot (1 + (d/D) \cdot \cos\alpha)$. Perhatikan tanda plus membuat BPFI selalu lebih besar dari BPFO pada bearing yang sama.

C5. Untuk bearing 6205 di 1480 rpm, hitung FTF (Fundamental Train Frequency / cage frequency). Formula: $FTF = (fr/2) \cdot (1 - (d/D) \cdot \cos\alpha)$. Print dalam Hz.

- 💡 **HINT:** $FTF = (fr/2) \cdot (1 - (d/D) \cdot \cos\alpha)$. FTF selalu paling rendah di antara frekuensi fault bearing.

C6. Verifikasi Parseval: untuk $fs=1000$, $N=1024$, $t=np.arange(N)/fs$, $x=2.0 \cdot np.\sin(2 \cdot np.\pi \cdot 50 \cdot t)$, hitung $N \cdot np.mean(x^2)$ (energi domain waktu). Print hasilnya.

- 💡 **HINT:** Untuk sinusoid amplitudo A , $RMS^2 = A^2/2$; energi total = $N \cdot \text{mean}(x^2)$, yang menurut teorema Parseval sama dengan $\sum |X_k|^2/N$. Substitusikan A dan N dari soal.

C7. Hitung spectral centroid (mean frequency) untuk dua-peak power: $P[100 \text{ Hz}] = 0,6$, $P[400 \text{ Hz}] = 0,4$, sisanya nol. Formula: $f_bar = \sum (f \cdot P) / \sum (P)$. Print Hz.

- 💡 **HINT:** Spectral centroid = $\sum (f_i \cdot P_i)$ dibagi $\sum (P_i)$, yaitu "pusat massa" PSD berbobot power. Substitusikan frekuensi dan power dari soal.

C8. Berapa N minimum agar peak 50 Hz bisa dipisahkan dari 53 Hz pada $fs = 10000 \text{ Hz}$? Δf harus kurang dari sama dengan $(53-50) = 3 \text{ Hz}$. Cari N integer terkecil. Print nilainya.

- 💡 **HINT:** $N_min = \text{ceil}(fs \text{ dibagi } \Delta f_max)$. Pakai `np.ceil`; praktiknya bulatkan ke pangkat 2 terdekat agar FFT efisien.

C9. Power peak $A = 0,50 \text{ (g}^2/\text{Hz)}$, peak $B = 0,05$. Hitung rasio dB antara A dan B : $dB = 10 \cdot \log_{10}(P_A / P_B)$. Print nilainya.

- 💡 **HINT:** Rasio dalam dB = $10 \cdot \log_{10}(P_peak \text{ dibagi } P_floor)$; gunakan `np.log10` dengan nilai power dari soal.

C10. Hitung frekuensi Nyquist untuk sampling rate $fs = 25000 \text{ Hz}$. Formula: $f_Nyg = fs/2$. Print dalam Hz.

- 💡 **HINT:** Frekuensi Nyquist = $fs \text{ dibagi } 2$. Komponen di atasnya akan ter-alias; pakai anti-alias low-pass filter sebelum ADC. Substitusikan fs dari soal.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Hitung sideband frequency atas ($f_{sb,upper}$) untuk diagnosa broken rotor bar motor induksi. Diketahui frekuensi supply $f_s = 50$ Hz dan slip $s = 0,03$ (3%). Sideband muncul pada $f_s \cdot (1 \pm 2s)$. Print nilai sideband UPPER ($f_s + 2 \cdot s \cdot f_s$).

- 💡 **HINT:** Sideband broken rotor bar = $f_s \cdot (1 \pm 2s)$; pasangan $\pm 2s \cdot f_s$ khas pada motor squirrel-cage. Substitusikan frekuensi suplai dan slip dari soal.

C12 (Hard). Hitung Gear Mesh Frequency (GMF) untuk pinion 25 gigi pada input shaft 1800 rpm. Formula: $GMF = N_{teeth} \cdot fr_{pinion}$. Print nilainya (Hz).

- 💡 **HINT:** $fr_{pinion} = rpm$ dibagi 60; $GMF = \text{jumlah gigi} \cdot fr_{pinion}$. Diagnosa gear melihat GMF beserta sideband $\pm fr$. Substitusikan rpm dan jumlah gigi dari soal.

C13 (Hard). Hitung spectral kurtosis (SK) untuk lima nilai PSD dalam satu band: $P = [1, 2, 3, 4, 5]$. Normalisasi P jadi probabilitas, hitung mean & var atas index $k = [0, 1, 2, 3, 4]$, lalu $SK = E[(k - \mu)^4] / \text{var}^2$ (plain kurtosis, fisher=False). Print nilainya.

- 💡 **HINT:** Hitung $\mu = \sum(k \cdot p)$, variansi = $\sum((k - \mu)^2 \cdot p)$, momen ke-4 = $\sum((k - \mu)^4 \cdot p)$; spectral kurtosis = m4 dibagi var^2 . Bisa diverifikasi dengan `scipy.stats.kurtosis(..., fisher=False)`.

C14 (Hard). Untuk Welch PSD pada $f_s = 12800$ Hz, hitung nperseg minimum berupa pangkat 2 agar $\Delta f \leq 0,5$ Hz. $\Delta f_{welch} = f_s / nperseg$, sehingga $nperseg \geq f_s / \Delta f$. Print pangkat 2 terkecil yang lebih besar sama dengan 25600.

- 💡 **HINT:** $N_{minimum} = \text{ceil}(f_s \text{ dibagi } \Delta f)$, lalu bulatkan ke pangkat 2 terkecil yang lebih besar sama dengan nilai itu; pakai pola $2^{**\text{int}(\text{np.ceil}(\text{np.log2}(f_s / \Delta f)))}$.

C15 (Hard). Order tracking: shaft berputar 60 Hz, bearing 6205 ($n=9$, $d/D=0,2034$, $\alpha=0^\circ$). Hitung order BPFO = $BPFO / fr$, invariant terhadap kecepatan shaft, dipakai untuk variable-speed monitoring. Print nilainya (3 desimal).

- 💡 **HINT:** $BPFO / fr = (n/2) \cdot (1 - (d/D) \cdot \cos \alpha)$, order BPFO tidak bergantung rpm. Substitusikan parameter geometri bearing dari soal; berguna untuk order tracking saat run-up/coast-down.

DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(9), 11121–11128. <https://doi.org/10.1609/aaai.v37i9.26317>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 8

Optimasi Linear: Linear Programming

Abstrak

Pertemuan ini memperkenalkan Linear Programming (LP) sebagai fondasi optimasi keputusan dengan sumber

Sub - CPMK

Sub-CPMK 3.2 – Mampu mengintegrasikan hasil simulasi untuk mendukung keputusan desain

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-8.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Setelah Ujian Tengah Semester, mata kuliah memasuki bagian kedua: optimasi. Pertemuan kesembilan ini membahas Linear Programming (LP), teknik optimasi klasik untuk memaksimalkan profit atau meminimalkan biaya di bawah kendala sumber daya yang linier. LP menjadi fondasi sebelum masuk ke optimasi non-linear (Pertemuan 10) dan metaheuristik (Pertemuan 11).



Gambar 1. Posisi Pertemuan 9 (Linear Programming) dalam alur mata kuliah, mengawali blok materi optimasi setelah UTS.

Gambar 1 menunjukkan posisi Pertemuan 9 dalam alur mata kuliah, menjembatani materi analisis sinyal (Pertemuan 1-7) dengan materi optimasi dan monitoring cerdas (Pertemuan 10-15).

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 3.2:** Mampu mengintegrasikan hasil simulasi untuk mendukung keputusan desain, yaitu menggunakan hasil solve Linear Programming (nilai optimal, shadow price, sensitivity analysis) sebagai dasar pengambilan keputusan alokasi sumber daya dalam desain sistem produksi (CPMK 3).
- Setelah pertemuan ini mahasiswa dapat: memformulasikan masalah rekayasa sebagai LP standar; menyelesaikan LP dua variabel secara grafis; memahami prinsip kerja algoritma Simpleks; serta menginterpretasikan shadow price dari sensitivity analysis.

MENGAPA LINEAR PROGRAMMING

$$Z = c_1x_1 + c_2x_2 + \dots + c_nx_n \quad P = \{x \mid Ax \leq b, x \geq 0\} \quad x^* \in \text{vertices}(P)$$

Asumsi Fundamental LP: LP mengasumsikan baik fungsi objektif maupun seluruh kendala bersifat linier terhadap variabel keputusan; variabel bersifat kontinu dan umumnya non-negatif. Daerah feasibel yang terbentuk dari irisan kendala linier selalu berupa poligon konveks (polihedron di dimensi lebih tinggi).

Teorema Dasar LP: Teorema Dasar Linear Programming (Dantzig, 1947) menyatakan bahwa solusi optimal LP, jika daerah feasibelnya terbatas, selalu berada di setidaknya satu vertex (titik sudut) daerah feasibel, bukan di titik interior. Inilah dasar mengapa algoritma Simpleks cukup menelusuri vertex demi vertex, bukan seluruh titik di dalam daerah feasibel.

Implementasi Python: Implementasi praktis LP di Python memakai ``scipy.optimize.linprog``, yang secara default meminimumkan fungsi objektif; untuk maksimisasi, koefisien c harus dinegasikan terlebih dahulu.

FORMULASI STANDAR LP

Bentuk Kanonik: $\max c^T x$ s.t. $Ax \leq b, x \geq 0$ *Bentuk Standar (Slack): $Ax + Is = b, x, s \geq 0$*
Bounds: $lb \leq x \leq ub$

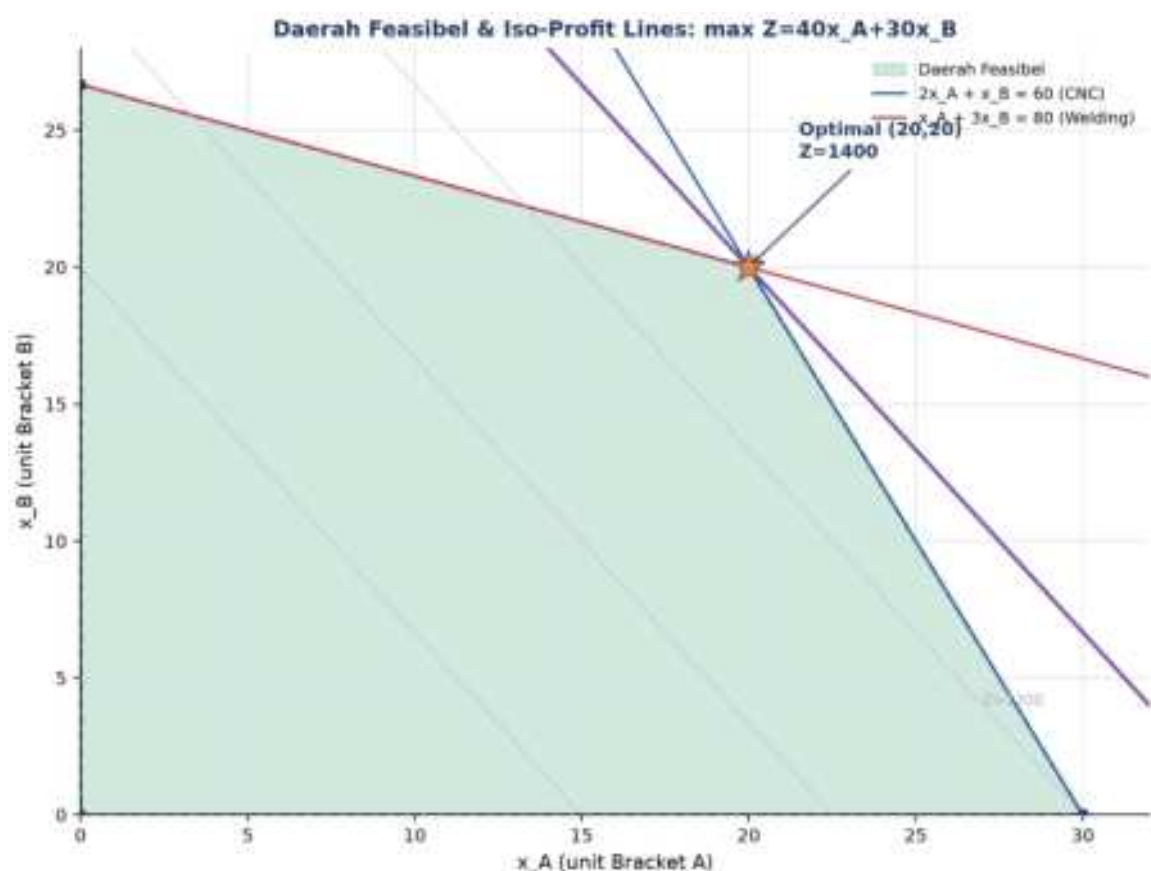
- **Bentuk Standar (Slack Variables):** kendala inequality ($\leq$) diubah menjadi equality dengan menambah slack variable non-negatif; ``Ax + Is = b``.
- **Bounds: Variabel Berbatas:** batas atas/bawah tiap variabel diset lewat parameter ``bounds`` di `linprog`, terpisah dari kendala ``A_ub`/`b_ub``.
- **Tiga Status Solusi LP:** status 0 = optimal ditemukan; status 2 = infeasible (daerah feasibel kosong); status 3 = unbounded (Z tak terbatas).

Contoh Kasus: Pabrik Komponen Mesin. Pabrik komponen mesin memproduksi dua jenis bracket: Bracket A memberi profit Rp40 ribu/unit, membutuhkan 2 jam CNC dan 1 jam welding; Bracket B memberi profit Rp30 ribu/unit, membutuhkan 1 jam CNC dan 3 jam welding. Kapasitas CNC 60 jam/minggu, kapasitas welding 80 jam/minggu. Formulasi: maksimalkan $Z = 40x_A + 30x_B$ dengan kendala $2x_A + x_B \leq 60$ dan $x_A + 3x_B \leq 80, x \geq 0$. Solusi optimal berada di vertex (20,20) dengan $Z = \text{Rp}1.400$ ribu.

Cheat Sheet Konversi: Konversi bentuk umum ke bentuk standar linprog: $\min \leftrightarrow \max$ dengan negasi c ; kendala $\geq$ dikonversi ke $\leq$ dengan negasi kedua sisi; kendala $=$ tetap dipisah sebagai ``A_eq`/`b_eq``; variabel bebas tanda dipecah menjadi dua variabel non-negatif; selalu periksa konsistensi dimensi A, b , dan bounds sebelum menjalankan solver.

DAERAH FEASIBEL & METODE GRAFIS

- **Plot Constraint Lines:** gambar tiap kendala sebagai garis batas di bidang 2D.
- **Identifikasi Daerah Feasibel:** irisan seluruh half-plane kendala membentuk daerah feasibel berupa poligon konveks.
- **Iso-Profit Line & Optimasi:** garis $Z=\text{konstan}$ digeser sejajar ke arah peningkatan objektif hingga menyentuh vertex terakhir daerah feasibel; itulah solusi optimal.
- **Enumerasi Vertex (Brute Force):** cara brute-force: evaluasi Z di setiap vertex daerah feasibel, lalu pilih yang terbesar (maksimisasi) atau terkecil (minimisasi).



Gambar 2. Daerah feasibel (hijau) dan garis iso-profit untuk masalah pabrik bracket; solusi optimal di vertex $(20, 20)$ dengan $Z=1400$.

Gambar 2 mengilustrasikan metode grafis: garis iso-profit digeser ke kanan-atas hingga menyentuh vertex terakhir sebelum keluar dari daerah feasibel, yaitu titik $(20, 20)$.

Keterbatasan Metode Grafis: Metode grafis hanya praktis untuk masalah dua variabel keputusan karena keterbatasan visualisasi bidang; begitu jumlah variabel mencapai tiga, daerah feasibel menjadi polihedron tiga dimensi yang masih dapat digambar namun sudah sulit dibaca secara visual, dan untuk empat variabel atau lebih, representasi geometris langsung sudah tidak lagi memungkinkan. Untuk kasus itulah metode Simpleks (bergerak

antar vertex secara aljabar tanpa perlu visualisasi) dan solver numerik seperti HiGHS menjadi pendekatan standar industri, sementara metode grafis tetap bernilai pedagogis penting karena memberi intuisi visual tentang MENGAPA solusi optimal LP selalu berada di salah satu vertex daerah feasibel, bukan di titik interior sembarang.

Tabel 1. Tiga status utama daerah feasibel LP beserta karakteristik, solusi, dan aksi yang perlu diambil engineer.

Status Daerah Feasibel	Karakteristik	Solusi LP	Aksi Engineer
Bounded & non-empty	Poligon tertutup terbatas	Optimal unik (atau edge multiple)	Lanjut solve dengan Simpleks/HiGHS
Unbounded	Membentang ke tak hingga pada arah c	Z menuju tak hingga (unbounded)	Tambah kendala, formulasi kurang lengkap
Empty (infeasible)	Tidak ada x yang memenuhi semua kendala	Tidak ada solusi	Cek kontradiksi kendala
Single point	Titik tunggal (semua kendala equality)	Solusi tunggal trivial	Tidak perlu optimasi, substitusi langsung
Degenerate vertex	3 atau lebih kendala bertemu di 1 titik	Optimal tapi berisiko cycling	Pakai aturan Bland di Simpleks

Tabel 1 merangkum status yang mungkin terjadi pada daerah feasibel LP.



Gambar 3. Ilustrasi visual tiga status utama daerah feasibel: bounded & non-empty, unbounded, dan infeasible.

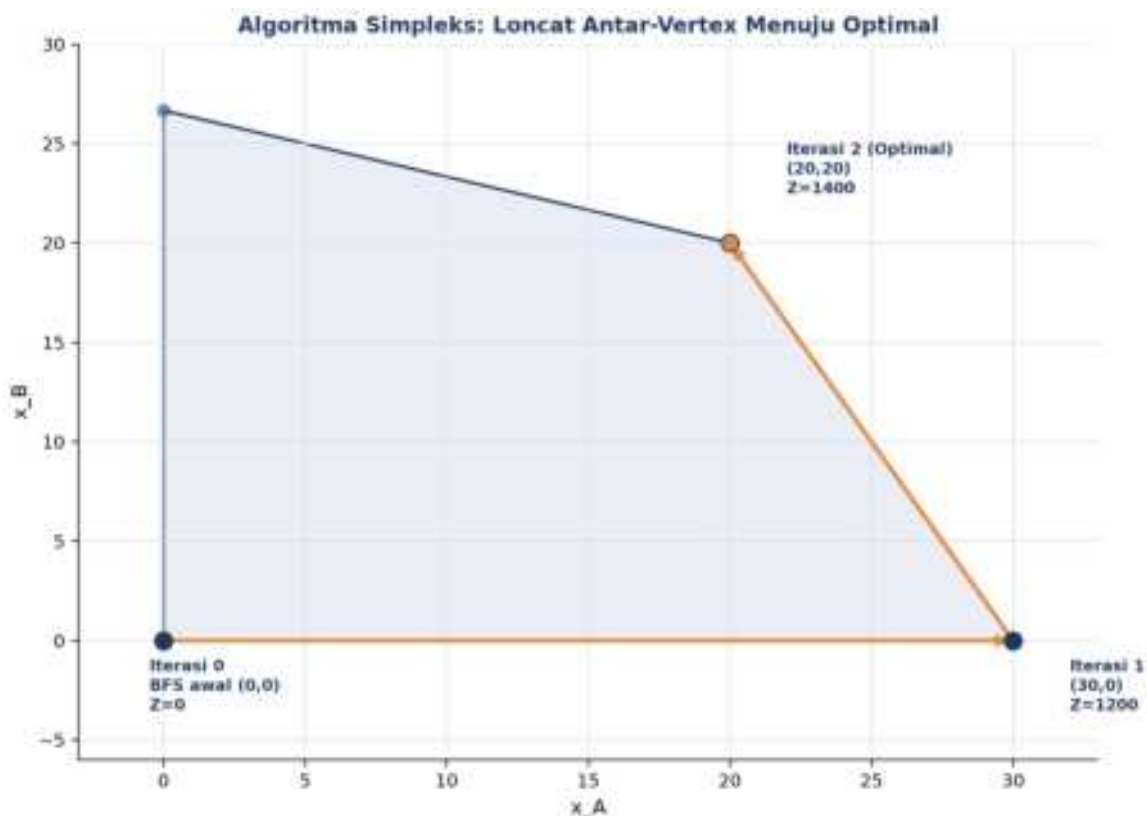
Gambar 3 memvisualisasikan tiga status tersebut secara geometris; kasus unbounded dan infeasible biasanya menandakan formulasi kendala yang kurang lengkap atau kontradiktif.

METODE SIMPLEKS

Algoritma Simpleks dikembangkan oleh George Dantzig pada tahun 1947 untuk kebutuhan logistik militer Angkatan Udara AS. Meski Klee-Minty (1972) menunjukkan kompleksitas worst-case eksponensial, dalam praktik Simpleks sangat cepat dan tetap dipakai luas hingga kini.

Initial BFS Pricing: $\bar{z}_j = c_j - c_{BV} \cdot B^{-1} \cdot a_j$ Ratio Test: $\theta^ = \min\{b_i/a_{ij} : a_{ij} > 0\}$*
Pivot (Gauss-Jordan)

- **Tableau Awal & BFS:** mulai dari basic feasible solution awal, biasanya origin dengan semua slack variable sebagai variabel basis.
- **Pricing: Pilih Variabel Masuk:** pilih variabel non-basis dengan reduced cost paling menguntungkan sebagai variabel masuk.
- **Ratio Test: Pilih Variabel Keluar:** tentukan variabel keluar lewat rasio minimum b_i/a_{ij} agar tetap feasibel.
- **Pivot: Update Tableau:** update tableau via eliminasi Gauss-Jordan; nilai Z tidak pernah menurun di tiap iterasi.
- **Big-M & Two-Phase Method:** untuk kendala equality/ $\geq$ yang tidak punya BFS trivial, dipakai variabel artifisial dengan penalti besar M, atau pendekatan dua fase.
- **Revised Simplex & Modern Solver:** solver modern (HiGHS, dipakai `scipy.optimize.linprog(method='highs')`) memakai revised Simplex atau interior-point method yang jauh lebih efisien untuk masalah skala besar.



Gambar 4. Algoritma Simpleks bergerak dari vertex ke vertex tetangga: (0,0) dengan $Z=0$, ke (30,0) dengan $Z=1200$, ke (20,20) dengan $Z=1400$ (optimal).

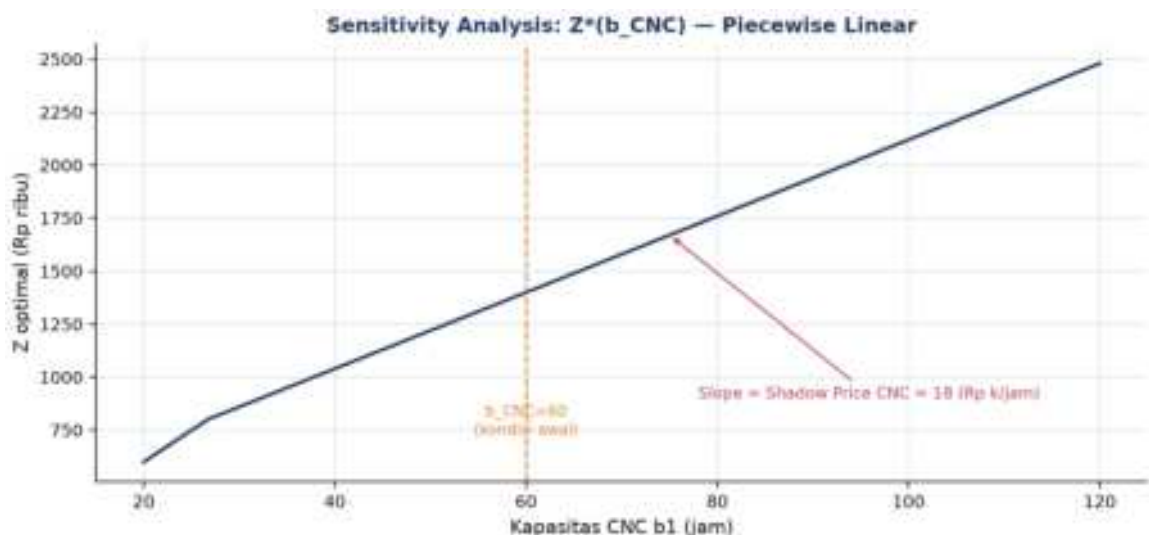
Gambar 4 mengilustrasikan bagaimana Simpleks meloncat dari vertex awal (0,0) menuju vertex optimal (20,20) melalui satu vertex perantara, dengan nilai Z yang selalu meningkat setiap iterasi.

Tabel 2. Perbandingan tiga pendekatan solver LP: Simpleks, Interior Point, dan Network Simplex.

Aspek	Simpleks	Interior Point	Network Simplex
Worst-case	Eksponensial (Klee-Minty)	$O(n^3 L)$ polynomial	$O(mn)$ untuk network LP
Praktik	Sekitar 2m-3m iterasi, sangat cepat	Sekitar 30-50 iterasi, scalable	10-100 kali lebih cepat untuk transport
Solusi	Vertex (sparse)	Interior (dense)	Vertex jaringan
Cocok untuk	LP kecil-sedang, warm start	LP besar, dense matrix	Transportation, shortest path
Library Python	linprog(method='simplex')	linprog(method='highs-ipm')	networkx / linear_sum_assignment

Tabel 2 membandingkan tiga pendekatan solver LP dari sisi kompleksitas, karakteristik solusi, dan library Python yang relevan.

Dual Simplex & Sensitivity Analysis: Untuk masalah pabrik bracket, dual value (shadow price) dari sistem persamaan $2y_1 + y_2 = 40$ dan $y_1 + 3y_2 = 30$ menghasilkan $y_{\text{CNC}} = 18$ dan $y_{\text{welding}} = 4$; artinya tiap tambahan 1 jam kapasitas CNC menaikkan profit optimal sekitar Rp18 ribu, sedangkan tambahan 1 jam welding hanya menaikkan sekitar Rp4 ribu.



Gambar 5. Sensitivity analysis $Z^*(b_{\text{CNC}})$: fungsi piecewise linear terhadap kapasitas CNC, dengan slope sama dengan shadow price CNC.

Gambar 5 menunjukkan bagaimana Z optimal berubah secara piecewise linear terhadap kapasitas CNC; kemiringan garis pada rentang di sekitar $b_{\text{CNC}} = 60$ sama dengan shadow price CNC.

Peringatan, Degenerasi & Cycling: Degenerasi terjadi ketika lebih dari n kendala aktif bertemu di satu vertex, berisiko menyebabkan cycling (Simpleks berputar tanpa progres). Aturan Bland (memilih variabel dengan indeks terkecil saat terjadi seri) menjamin terminasi meski memperlambat konvergensi.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Diet Plan & Stigler's Diet Problem:** Stigler (1939) menyelesaikan manual diet termurah seharga \$39.93/tahun; setelah Simpleks ditemukan Dantzig (1947) menghitung ulang menjadi \$39.69/tahun.
- **Logistik & Transportation Problem:** rute pengiriman gudang Amazon adalah LP klasik dengan variabel keputusan jumlah barang yang dikirim per rute.
- **Refinery Blending & Petroleum LP:** Shell pada 1950-an menjadi pionir LP untuk blending fraksi minyak mentah menjadi produk bahan bakar.
- **Penjadwalan Mesin di Pabrik:** penjadwalan 10 mesin CNC untuk 50 jenis produk adalah LP/MILP skala industri, kunci Industry 4.0.
- **Portfolio & Markowitz's Model:** model Markowitz (1952) mengalokasikan portofolio investasi via LP/QP, mendasari robo-advisor modern.
- **Power Grid & Economic Dispatch:** PLN P2B menjalankan LP setiap 5 menit untuk economic dispatch pembangkit listrik.

Pola Umum: Pola yang sama muncul di semua analogi: sumber daya terbatas, banyak pilihan alokasi, dan kebutuhan mencari titik optimal secara sistematis, bukan coba-coba.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

- **Production Planning & Mix Optimization:** Toyota, Bosch, dan Astra memakai LP untuk menentukan bauran produksi optimal dari 8 jenis bracket dengan software SAP IBP.
- **Material Blending: Steel & Alloy:** ArcelorMittal, Krakatau Steel, dan Posco memakai LP untuk blending scrap/pig iron/ferro-alloy pada tanur EAF agar komposisi kimia sesuai target dengan biaya minimum.
- **Logistik & Vehicle Routing:** perusahaan logistik seperti Logivan dan Amazon FBA memakai LP untuk mengalokasikan 500 ton/minggu dari 3 gudang ke 12 pabrik dengan biaya transportasi minimum.
- **Energy Optimization Smart Factory:** smart factory dengan solar PV 200kW, genset, dan grid PLN (kontrak maksimum 150kW) memakai LP untuk menjadwalkan sumber

energi termurah tiap jam, mengikuti pendekatan Siemens MindSphere dan GE Predix.

Peringatan, Jebakan Praktis Formulasi LP: Kesalahan praktis yang sering terjadi: lupa mengecek satuan konsisten antar kendala; kendala yang saling kontradiktif menghasilkan infeasible tanpa disadari; lupa menambah bounds non-negatif; menganggap hasil LP selalu integer padahal variabel kontinu; tidak melakukan sensitivity analysis meski keputusan bisnis sensitif terhadap kapasitas; serta salah interpretasi shadow price pada kendala yang tidak binding ($\text{slack} > 0$).

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada studi kasus pabrik bracket dan perluasannya ke skala industri. Lingkungan: conda env optoauto dengan paket numpy, scipy, matplotlib, pandas; notebook p9_linear_programming.ipynb.



```
p9_linear_programming.ipynb — Cell 1
from scipy.optimize import linprog

c = [-40, -30] # maksimisasi: negasikan
A_ub = [[2, 1], [1, 3]] # CNC, Welding
b_ub = [60, 80]
bounds = [(0, None), (0, None)]

res = linprog(c, A_ub=A_ub, b_ub=b_ub,
              bounds=bounds, method='highs')

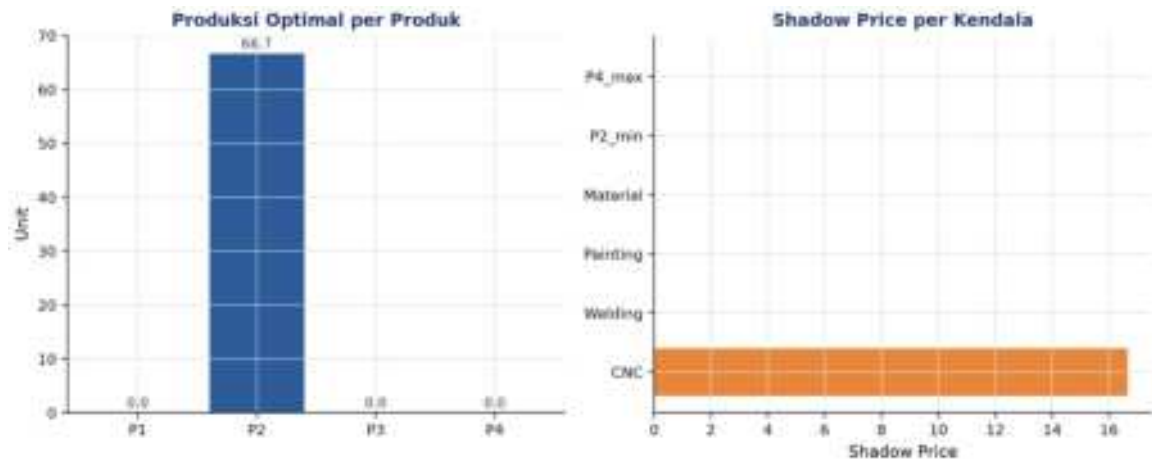
print(f'x_A={res.x[0]:.1f}, x_B={res.x[1]:.1f}')
print(f'Z_max = {-res.fun:.0f}')
# output: x_A=20.0, x_B=20.0, Z_max=1400
```

Gambar 6. Cuplikan kode formulasi dan penyelesaian LP pabrik bracket menggunakan `scipy.optimize.linprog`.

Gambar 6 menunjukkan cuplikan kode inti Cell 1.

- **Cell 1:** formulasi LP pabrik bracket: $\max 40x_A + 30x_B$ s.t. kendala CNC dan welding; solve dengan `linprog(method='highs')`; cetak `x_A`, `x_B`, `Z_max`, dan utilisasi tiap sumber daya.
- **Cell 2:** visualisasi daerah feasibel via meshgrid, garis kendala, garis iso-profit, enumerasi 4 vertex, dan penanda vertex optimal.
- **Cell 3:** LP skala industri 4 produk x 6 kendala (CNC, welding, painting, material, batas minimum P2, batas maksimum P4) plus sensitivity analysis: shadow price dari `result.ineqlin.marginals` dan slack dari `result.ineqlin.residual`.

- **Cell 4:** fungsi ``solve_production_lp()`` reusable mengembalikan dict berisi status, Z, produksi, shadow price, utilisasi, dan slack sebagai pandas Series; divisualisasikan sebagai bar chart produksi dan shadow price.



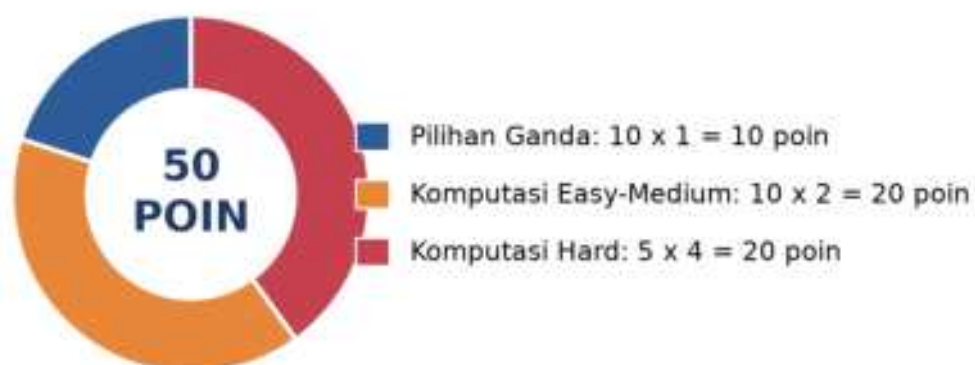
Gambar 7. Hasil LP 4-produk skala industri: produksi optimal per produk (kiri) dan shadow price per kendala (kanan) dari Cell 3-4.

Gambar 7 menunjukkan hasil LP 4-produk dari Cell 3-4: hanya produk P2 yang diproduksi pada solusi optimal, dan hanya kendala CNC yang binding (shadow price positif), sedangkan kendala lain memiliki slack sehingga shadow price-nya nol.

TUGAS PERTEMUAN 9

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 9



Gambar 8. Distribusi 50 poin Tugas Pertemuan 9 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Pernyataan yang BENAR mengenai asumsi fundamental Linear Programming adalah...

- A. Fungsi objektif boleh kuadratik tapi kendala harus linier
- B. Baik fungsi objektif maupun kendala harus linier dalam variabel keputusan; variabel kontinu dan biasanya non-negatif
- C. Variabel keputusan harus integer, tidak boleh pecahan
- D. LP hanya bisa diselesaikan untuk masalah 2 variabel

2. Teorema Dasar Linear Programming menyatakan bahwa solusi optimal LP (jika terbatas) selalu berada di...

- A. Titik tengah daerah feasibel
- B. Setidaknya satu vertex (titik sudut) daerah feasibel
- C. Origin $x = 0$ selalu
- D. Bidang dengan luasan terbesar

3. Tujuan menambahkan slack variable $s_i \geq 0$ ke kendala $a_i^T x \leq b_i$ adalah...

- A. Memperkenalkan ketidakpastian probabilistik ke LP
- B. Mengkonversi kendala inequality menjadi equality $a_i^T x + s_i = b_i$ agar bisa diolah algoritma Simpleks
- C. Membuat masalah jadi non-linier
- D. Memperbesar daerah feasibel dengan tambahan dimensi

4. Pada `scipy.optimize.linprog`, untuk memecahkan masalah maksimisasi $\max Z = c^T x$, yang harus dilakukan adalah...

- A. Memberi parameter `direction='max'` ke `linprog`
- B. Negasikan koefisien c (kirim $-c$) karena `linprog` selalu meminimumkan; nilai Z dikembalikan dengan `-result.fun`
- C. Tidak bisa, `scipy` tidak mendukung maksimisasi sama sekali
- D. Membalikkan tanda `b_ub` dan `A_ub` saja

5. Setelah solver LP berjalan, status "unbounded" berarti...

- A. Solusi optimal sudah ditemukan dengan akurasi tinggi
- B. Daerah feasibel terbentang ke arah peningkatan Z tanpa batas, biasanya kurang kendala fisik; harus ditinjau ulang formulasinya
- C. Daerah feasibel kosong, kendala kontradiktif
- D. Solver belum konvergen, perlu iterasi lebih banyak

6. Daerah feasibel sebuah masalah LP selalu berbentuk...

- A. Lingkaran (lingkup bola di dimensi tinggi)
- B. Polihedron (poligon konveks di 2D, polihedron di nD), irisan setengah-bidang
- C. Selalu cembung tetapi tidak pernah terbatas (bounded)

- D. Bentuk fraktal yang kompleks

7. Algoritma Simpleks menyelesaikan LP dengan cara...

- A. Memeriksa semua titik di dalam daerah feasibel satu per satu
- B. Bergerak dari satu vertex ke vertex tetangga yang memberi nilai objektif lebih baik, hingga tidak ada peningkatan lagi
- C. Mengambil rata-rata dari semua kendala
- D. Menebak solusi awal lalu menggunakan gradient descent kontinu

8. Shadow price (dual variable) sebuah kendala mengindikasikan...

- A. Total biaya yang dikeluarkan untuk kendala tersebut
- B. Berapa peningkatan nilai Z optimal kalau RHS kendala bi ditambah 1 unit, sebagai panduan investasi sumber daya
- C. Probabilitas kendala tidak terpenuhi
- D. Bobot kendala dalam fungsi objektif

9. Untuk LP min $Z = 3x_1 + 5x_2$ s.t. $x_1 + x_2 \geq 4$, $x \geq 0$, agar bisa diolah linprog, kendala $\geq$ harus dikonversi menjadi...

- A. $x_1 + x_2 = 4$ (langsung dijadikan equality)
- B. $-x_1 - x_2 \leq -4$ (negasikan kedua sisi membalik tanda inequality)
- C. Menambah variabel kuadratik di sisi kiri
- D. Tidak perlu konversi, kendala $\geq$ langsung diterima

10. Untuk masalah pabrik dengan kebutuhan jumlah unit produksi harus integer (misal tidak bisa produksi 3,7 mesin), tool yang paling tepat adalah...

- A. Tetap pakai LP biasa, lalu pembulatan hasil ke integer terdekat
- B. Mixed-Integer Linear Programming (MILP) dengan variabel bertipe integer (integrality=1 di scipy 1.9+); pembulatan LP biasa bisa infeasible atau sub-optimal
- C. Quadratic Programming saja
- D. LP dengan kendala tambahan x anggota bilangan real

Bagian B: Komputasi Easy-Medium (10 soal $\times$ 2 poin)

Catatan: gunakan `scipy.optimize.linprog` dengan `method='highs'`. Ingat scipy selalu meminimumkan, untuk maksimisasi negasikan koefisien c. Hasil profit dikembalikan dengan -result.fun. Pastikan cek result.success sebelum membaca result.x.

```
from scipy.optimize import linprog
# Bentuk umum: minimize c.x s.t. A_ub.x <= b_ub, bounds
# Maksimisasi: kirim -c, ambil Z = -result.fun
```

C1. Selesaikan LP 1D sederhana: max $Z = 5x$ s.t. $2x \leq 10$, $x \geq 0$. Print nilai Z optimal.

-  **HINT:** linprog meminimalkan, jadi untuk maksimisasi negasikan vektor c; nilai Z optimal didapat dari -res.fun.

C2. Pabrik bracket 2-produk: $\max Z = 40x_A + 30x_B$ s.t. $2x_A + x_B \leq 60$ (CNC), $x_A + 3x_B \leq 80$ (welding), $x \geq 0$. Selesaikan dengan linprog dan print Z optimal (Rp ribu).

- 💡 **HINT:** Susun c (negasi karena maksimisasi), A_ub dari koefisien kendala, dan b_ub dari batas kapasitas; ambil Z dari -res.fun.

C3. Untuk LP yang sama dengan soal sebelumnya, print x_A optimal (jumlah unit Bracket A).

- 💡 **HINT:** Vertex optimal terjadi di perpotongan kedua kendala binding (CNC dan welding); nilai x_A diambil dari elemen pertama res.x.

C4. LP minimisasi diet: $\min Z = 2x_1 + 3x_2$ (biaya per unit) s.t. $x_1 + 2x_2 \geq 10$ (protein), $3x_1 + x_2 \geq 15$ (kalori), $x \geq 0$. Print Z minimum.

- 💡 **HINT:** Konversi kendala $\geq$ menjadi $\leq$ dengan menegasikan kedua sisi; karena ini minimisasi, c dipakai langsung tanpa negasi dan $Z = \text{res.fun}$.

C5. LP 3 variabel: $\max Z = 10x_1 + 6x_2 + 4x_3$ s.t. $x_1 + x_2 + x_3 \leq 100$, $10x_1 + 4x_2 + 5x_3 \leq 600$, $2x_1 + 2x_2 + 6x_3 \leq 300$, $x \geq 0$. Print Z optimal.

- 💡 **HINT:** Susun c (negasi untuk maksimisasi) dan tiga baris A_ub/b_ub dari kendala; serahkan ke linprog dan ambil Z dari -res.fun.

C6. Slack di vertex optimal pabrik bracket 2-produk: hitung slack kendala CNC $s_{\text{CNC}} = 60 - (2x_A + x_B)$ di solusi optimal (selesaikan LP tersebut lebih dulu). Print nilainya.

- 💡 **HINT:** Slack = kapasitas dikurangi pemakaian di solusi optimal; slack = 0 berarti kendala binding. Bisa juga dibaca dari res.ineqlin.residual.

C7. Hitung shadow price untuk kendala CNC pada masalah pabrik bracket 2-produk. Pakai res.ineqlin.marginals lalu negasikan (karena scipy negasi internal). Print nilainya (Rp ribu/jam).

- 💡 **HINT:** Shadow price = perubahan Z per unit kenaikan kapasitas kendala; baca dari res.ineqlin.marginals lalu negasikan karena konvensi internal scipy.

C8. LP dengan kendala equality: $\min Z = x_1 + 2x_2$ s.t. $x_1 + x_2 = 10$, $x_1 \geq 0$, $x_2 \geq 0$. Pakai parameter A_eq, b_eq di linprog. Print Z minimum.

- 💡 **HINT:** Gunakan parameter A_eq dan b_eq untuk kendala equality; karena minimisasi, solver memilih variabel berkoefisien kecil. Ambil Z dari res.fun.

C9. Vertex enumeration manual: untuk LP $\max Z = 3x_1 + 2x_2$ s.t. $x_1 + x_2 \leq 4$, $x_1 \leq 3$, $x \geq 0$. Vertex feasibel: (0,0), (3,0), (3,1), (0,4). Hitung Z maksimal dengan iterasi loop di Python.

- 💡 **HINT:** Loop tiap vertex feasibel yang diberikan, evaluasi fungsi tujuan $Z = 3x_1 + 2x_2$, lalu ambil nilai maksimum dengan max.

C10. Pabrik dengan bounds variabel: $\max Z = 5x_1 + 4x_2$ s.t. $x_1 + x_2 \leq 50$, $0 \leq x_1 \leq 20$, $0 \leq x_2 \leq 35$. Pakai parameter bounds di linprog. Print Z optimal.

- 💡 **HINT:** Masukkan batas atas/bawah variabel lewat parameter bounds; negasikan c untuk maksimisasi dan ambil Z dari -res.fun.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal, mahasiswa diharapkan meneliti dan menulis sendiri. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong + submit = 0 poin dan tidak terkunci (bisa retry).

C11 (Hard). LP 4-Produk dengan 6 Kendala: max $Z = 30x_1 + 50x_2 + 40x_3 + 60x_4$ s.t. CNC: $2x_1 + 3x_2 + 2,5x_3 + 4x_4 \leq 200$; Welding: $x_1 + 2x_2 + 1,5x_3 + 3x_4 \leq 150$; Painting: $0,5x_1 + x_2 + x_3 + 2x_4 \leq 120$; Material: $5x_1 + 8x_2 + 6x_3 + 10x_4 \leq 800$; $x_2 \geq 5$; $x_4 \leq 30$. Print Z optimal.

- 💡 **HINT:** Negasikan c untuk maksimisasi; kendala $x_2 \geq 5$ ditulis sebagai $-x_2 \leq -5$ dan $x_4 \leq 30$ sebagai baris biasa. Ambil Z dari -res.fun.

C12 (Hard). Transportation Problem: 2 gudang (kapasitas 30, 40) kirim ke 3 kota (demand 20, 25, 25). Cost matrix (Rp ribu per unit): $C = \begin{bmatrix} 8, 6, 10 \\ 9, 12, 13 \end{bmatrix}$. Variabel x_{ij} = unit dari gudang i ke kota j (6 variabel). Print total cost minimum.

- 💡 **HINT:** Flatten cost matrix menjadi vektor c, susun baris kendala supply per gudang dan demand per kota; karena total supply = total demand, pakai kendala equality. Ambil biaya dari res.fun.

C13 (Hard). Blending Problem (Steel Alloy): campur 3 raw material x_1, x_2, x_3 (harga Rp/kg: 10, 8, 12) untuk hasilkan 100 kg alloy. Kandungan karbon (%): [0,5; 0,3; 0,7]; target karbon $\geq 0,5\%$. Minimalkan total biaya. Print biaya minimum (Rp/100 kg).

- 💡 **HINT:** Kendala total berat = 100 kg pakai equality; kendala karbon $\geq$ target dikonversi ke $\leq$ dengan negasi. Karena minimisasi, biaya diambil dari res.fun.

C14 (Hard). Vertex enumerasi LP 2D otomatis: untuk LP max $Z = 4x_1 + 3x_2$ s.t. $2x_1 + x_2 \leq 8$, $x_1 + 2x_2 \leq 8$, $x \geq 0$: tulis algoritma yang (1) cari semua titik perpotongan tiap pasangan kendala (termasuk sumbu), (2) filter feasibel, (3) evaluasi Z di tiap vertex feasibel, (4) pilih max. Print Z optimal.

- 💡 **HINT:** Cari titik perpotongan tiap pasang garis kendala (termasuk sumbu) dengan np.linalg.solve, filter yang feasibel, lalu evaluasi Z di tiap vertex dan ambil maksimum.

C15 (Hard). Sensitivity Analysis: untuk pabrik bracket 2-produk (max $40x_A + 30x_B$, kendala $CNC \leq 60$, $Welding \leq 80$, $x \geq 0$). Hitung perubahan Z optimal kalau kapasitas CNC ditambah dari 60 menjadi 65 (lebih 5 jam). Bandingkan dengan prediksi shadow price ($\Delta \text{kapasitas} \times \text{shadow_price_CNC}$). Print Z_{baru} minus Z_{lama} .

- 💡 **HINT:** Selesaikan LP dua kali (kapasitas CNC lama vs baru), lalu hitung selisih Z; bandingkan dengan prediksi shadow price dikali Δb sebagai validasi sensitivity analysis.

DAFTAR PUSTAKA

- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Oancea, G. (2022). Machining parameters optimization based on objective function linearization. *Mathematics*, 10(5), 803. <https://doi.org/10.3390/math10050803>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 9

Optimasi Non-Linear: Gradient &

Abstrak

Pertemuan ini membahas Non-Linear Programming (NLP) untuk masalah rekayasa tanpa dan dengan kendala:

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 3.3 – Mampu menggunakan teknologi 4.0 untuk mengoptimalkan sistem otomatisasi

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-9.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Linear Programming (Pertemuan 9) mengasumsikan fungsi objektif dan kendala bersifat linier, padahal banyak fenomena rekayasa (gaya drag proporsional v^2 , energi kinetik proporsional v^2) bersifat non-linier. Pertemuan kesepuluh ini membahas Non-Linear Programming (NLP): metode gradient-based untuk masalah tanpa kendala, serta KKT conditions dan penalty/barrier method untuk masalah berkendala.



Gambar 1. Posisi Pertemuan 10 (Optimasi Non-Linear) dalam alur mata kuliah, di antara Linear Programming (P9) dan Metaheuristik (P11).

Gambar 1 menunjukkan posisi Pertemuan 10 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 3.3:** Mampu menggunakan teknologi 4.0 untuk mengoptimalkan sistem otomasi, yaitu menerapkan metode gradient-based dan constraint handling (scipy.optimize) sebagai teknologi komputasi 4.0 untuk mengoptimalkan parameter desain sistem otomasi non-linier (CPMK 3).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan syarat optimalitas orde-pertama dan orde-kedua; membedakan gradient descent, Newton's method, dan BFGS; menjelaskan KKT conditions; serta menerapkan `scipy.optimize.minimize` pada masalah desain berkendala.

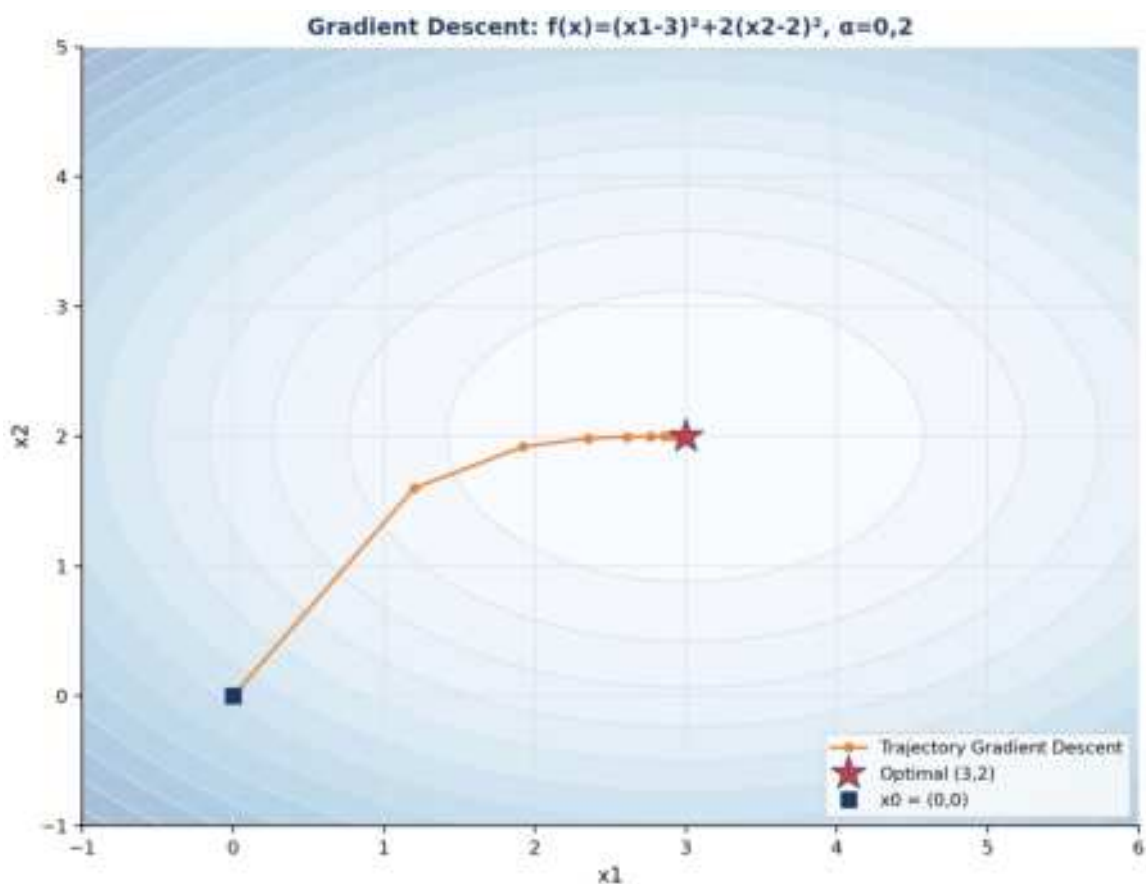
GRADIENT DESCENT & LINE SEARCH

x_0 (initial guess) $dk = -\nabla f(x_k)$ (direction) ak (step size) berhenti saat $\|\nabla f\| < \varepsilon$

Steepest Descent: Syarat orde-pertama untuk local minimum tanpa kendala adalah $\nabla f(x^*)=0$ (titik stasioner). Gradient descent bergerak dari x_0 mengikuti arah negative gradient: $x_{k+1}=x_k-\alpha \cdot \nabla f(x_k)$. Untuk $f(x_1,x_2)=x_1^2+2x_2^2$ dari $x_0=(3,2)$ dengan $\nabla f=(6,8)$ dan $\alpha=0,1$, iterasi pertama menghasilkan $x_1=(2,4; 1,2)$.

Backtracking Line Search: Backtracking line search (Armijo rule) memilih α secara adaptif: $f(x_k+\alpha d_k) \leq f(x_k)+c \cdot \alpha \cdot \nabla f^T d_k$ dengan $c \approx 10^{-4}$, lalu α dikurangi bertahap (faktor $\rho=0,5$) sampai kondisi terpenuhi. Ini mencegah overshoot tanpa harus mencoba-coba α secara manual.

Conjugate Gradient: Conjugate Gradient (Hestenes-Stiefel, 1952) memodifikasi arah pencarian $d_{k+1}=-\nabla f+\beta \cdot d_k$ sehingga untuk fungsi kuadratik konvergen dalam maksimal n iterasi, jauh lebih cepat dari steepest descent murni.



Gambar 2. Trajectory gradient descent pada kontur $f(x)=(x_1-3)^2+2(x_2-2)^2$ dari $x_0=(0,0)$ menuju optimal $(3,2)$, $\alpha=0,2$.

Gambar 2 mengilustrasikan trajectory gradient descent menuju titik optimal; arah pergerakan selalu tegak lurus terhadap garis kontur di tiap titik.

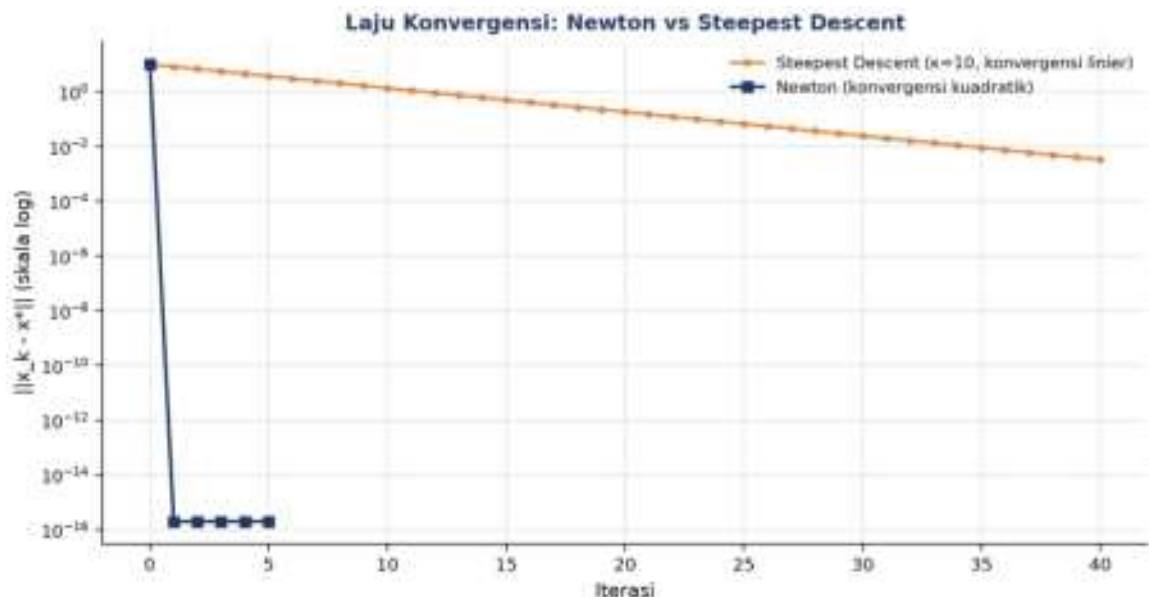
Contoh: Optimasi Tinggi Antena. Optimasi tinggi antena $f(h)=1000/h+50h^2$ untuk meminimalkan biaya kabel dan material tiang: $f'(h)=-1000/h^2+100h=0$ menghasilkan

$h^* = \text{akar_pangkat_tiga}(10) \approx 2,154$ m; uji konveksitas $f''(h) = 2000/h^3 + 100 > 0$ menegaskan ini adalah minimum global.

NEWTON'S METHOD & QUASI-NEWTON

Newton: $dk = -[\nabla^2 f]^{-1} \cdot \nabla f$ **BFGS update:** $B_{k+1} \approx B_k + \text{koreksi}(sk, yk)$ **L-BFGS:** simpan m pasangan terakhir

Pure Newton: Newton's method memakai informasi Hessian (turunan kedua) untuk membangun model kuadratik lokal, arah pencarian $d = -H^{-1} \cdot \nabla f$. Konvergensi kuadratik: $\|x_{k+1} - x^*\| \leq C \cdot \|x_k - x^*\|^2$, jauh lebih cepat dari gradient descent yang hanya konvergen linier, tetapi butuh biaya $O(n^3)$ untuk invers Hessian tiap iterasi.



Gambar 3. Perbandingan laju konvergensi Newton (kuadratik) vs Steepest Descent (linier, condition number $\kappa=10$) pada fungsi kuadratik.

Gambar 3 membandingkan laju konvergensi kedua metode secara log-skala; Newton mencapai presisi tinggi hanya dalam beberapa iterasi, sedangkan steepest descent memerlukan puluhan iterasi terutama saat condition number tinggi.

- **BFGS:** BFGS (Broyden-Fletcher-Goldfarb-Shanno, 1970) mengaproksimasi Hessian hanya dari informasi gradient via secant condition $sk = x_{k+1} - x_k$, $yk = \nabla f_{k+1} - \nabla f_k$; konvergensi super-linier tanpa perlu menghitung Hessian eksak.
- **L-BFGS:** menyimpan hanya $m=5-20$ pasangan (s,y) terakhir sehingga kebutuhan memori $O(mn)$, cocok untuk masalah berdimensi besar.
- **Trust Region:** membatasi langkah d dalam radius kepercayaan $\|d\| \leq \Delta_k$; lebih robust dibanding line search murni untuk fungsi non-konveks.

Tabel 1. Perbandingan lima algoritma optimasi non-linear berdasarkan laju konvergensi, biaya per iterasi, dan kebutuhan turunan.

Algoritma	Konvergensi	Cost/Iterasi	Kebutuhan	Cocok Untuk
Steepest Descent	Linier	$O(n)$	Gradient saja	Konveks well-conditioned, sederhana
Conjugate Gradient	Super-linier (kasus kuadratik)	$O(n)$	Gradient saja	Sparse, skala besar, mirip kuadratik
Newton	Kuadratik	$O(n^3)$	Hessian eksak	n kecil, Hessian tersedia analitik
BFGS (Quasi-Newton)	Super-linier	$O(n^2)$	Gradient saja	Default unconstrained NLP, n hingga 1000
L-BFGS-B	Super-linier	$O(mn)$	Gradient, bounds OK	Skala besar, ML, identifikasi parameter

Tabel 1 merangkum trade-off lima algoritma optimasi non-linear yang umum dipakai.

Tip Praktis: Sebelum memakai gradient analitik dalam kode produksi, validasi dulu dengan ``scipy.optimize.check_grad(f, jac, x0)`` untuk memastikan turunan yang diimplementasikan benar.

KKT CONDITIONS & CONSTRAINT HANDLING

Stationarity: $\nabla f + \sum \lambda_i \nabla h_i + \sum \mu_j \nabla g_j = 0$ **Primal feasibility:** $g(x) \leq 0, h(x) = 0$ **Dual feasibility:** $\mu_j \geq 0$ **Complementary slackness:** $\mu_j \cdot g_j(x) = 0$

- **Lagrangian Function:** $L(x, \lambda) = f(x) + \lambda^T \cdot h(x) + \mu^T \cdot g(x)$; turunan Lagrangian terhadap x memberi syarat stasioner, dan $\partial f^*/\partial b = -\lambda^*$ menghubungkan multiplier dengan sensitivitas terhadap RHS kendala.
- **Complementary Slackness:** $\mu_j \cdot g_j(x^*) = 0$ berarti kendala aktif ($g=0$) boleh punya $\mu > 0$, sedangkan kendala tidak aktif ($g < 0$) harus punya $\mu = 0$.
- **Penalty Method:** mengubah masalah berkendala menjadi tanpa kendala dengan menambah term $p \cdot \sum \max(0, g_j(x))^2$ ke objektif; saat p menuju tak hingga, solusi mendekati solusi constrained sesungguhnya, tetapi p besar menyebabkan ill-conditioning.
- **Barrier Method:** menambah term $-\mu \cdot \sum \log(-g_j(x))$ yang menuju tak hingga saat mendekati batas kendala, menjaga iterasi selalu berada di interior; mendasari interior-point method (Karmarkar, 1984).

- **Sequential Quadratic Programming (SQP):** menyelesaikan serangkaian sub-masalah quadratic programming yang mengaproksimasi NLP asli secara lokal; efisien untuk kendala non-linier umum.
- **Augmented Lagrangian:** $L_A = f + \lambda^T h + (p/2) \|h\|^2$; menggabungkan penalty dan multiplier Lagrange (Powell 1969, Hestenes 1969) sehingga p tidak perlu sangat besar untuk akurasi tinggi.

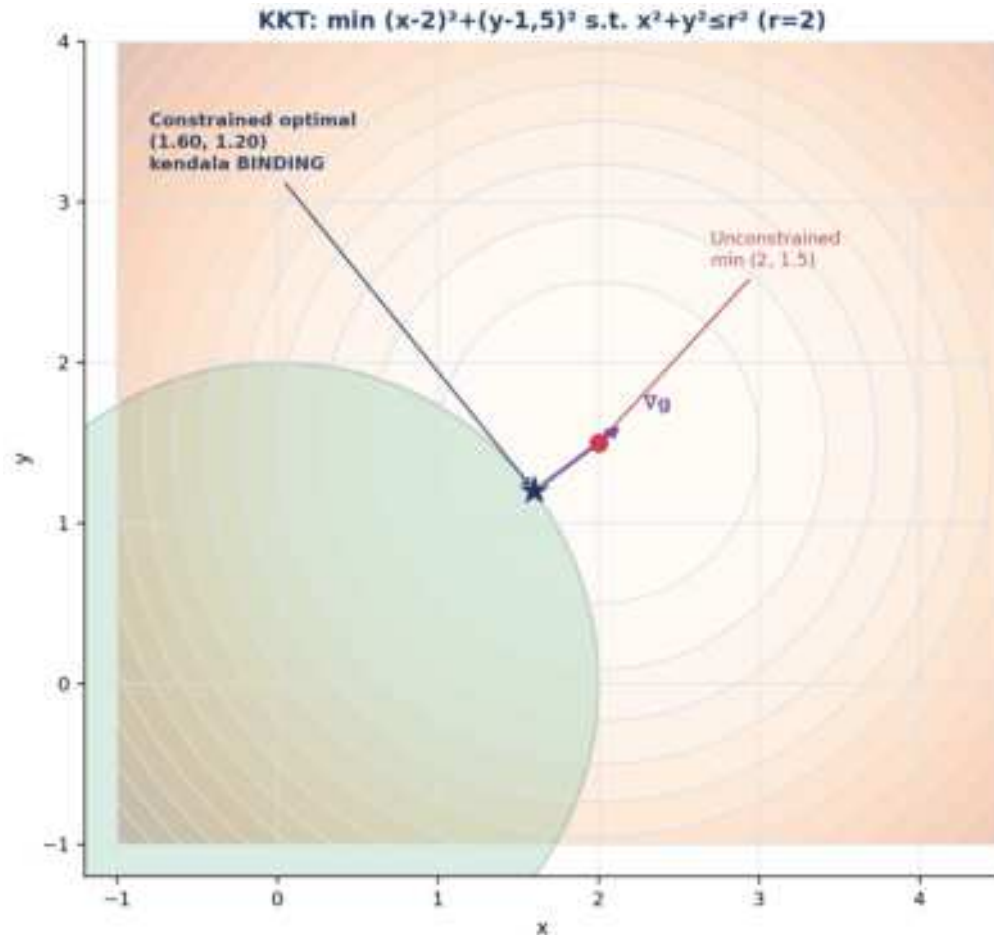
Tabel 2. Perbandingan lima pendekatan constraint handling di `scipy.optimize` berdasarkan tipe kendala yang didukung dan robustness.

Method	Tipe Kendala	Robustness	Cocok Untuk
L-BFGS-B	Hanya bounds	Tinggi	Bounds saja, scalable
SLSQP	Equality, inequality, bounds	Tinggi	Default NLP berkendala, $n < 100$
trust-constr	Equality, inequality, bounds, sparse	Sangat tinggi	Modern, robust, skala besar
COBYLA	Hanya inequality (tanpa equality)	Sedang	Derivative-free, kendala $\leq$ saja
Penalty manual	Apapun	Rendah (ill-conditioning)	Belajar konsep, prototyping cepat

Tabel 2 membandingkan lima metode constraint handling yang tersedia di `scipy.optimize` beserta karakteristik kecocokannya.

Heuristik Pemilihan Solver dalam Praktik Rekayasa: Pemilihan solver dari Tabel 2 dalam praktik rekayasa sering mengikuti heuristik sederhana: mulai dengan SLSQP sebagai default karena dukungannya yang luas terhadap tipe kendala dan kecepatan konvergensi yang baik pada kebanyakan masalah desain berdimensi sedang ($n < 100$), lalu beralih ke `trust-constr` bila SLSQP gagal konvergen atau masalah melibatkan kendala sparse berskala besar. COBYLA dicadangkan khusus untuk kasus di mana turunan fungsi objektif tidak tersedia secara analitik maupun numerik (misalnya objektif berasal dari simulasi black-box seperti FEM atau CFD), sementara penalty method manual umumnya hanya dipakai untuk tujuan pembelajaran konsep, bukan produksi, karena rawan ill-conditioning saat parameter p perlu dibesarkan untuk mencapai akurasi tinggi.

Interpretasi Geometris Kondisi KKT: Interpretasi geometris KKT conditions menjadi jauh lebih intuitif saat divisualisasikan: pada titik optimal constrained, gradient fungsi objektif ∇f harus berupa kombinasi linear dari gradient kendala aktif ∇g (dengan koefisien non-negatif untuk kendala inequality), yang secara geometris berarti tidak ada arah pergerakan yang SEKALIGUS meningkatkan feasibilitas dan memperbaiki nilai objektif. Bila kondisi ini gagal terpenuhi pada suatu titik, maka titik tersebut BUKAN optimal, karena selalu ada arah perbaikan (feasible descent direction) yang belum dieksplorasi oleh solver.



Gambar 4. Visualisasi KKT: $\min (x-2)^2+(y-1,5)^2$ dengan kendala $x^2+y^2 \leq 4$; optimal tanpa kendala (2; 1,5) berada di luar daerah feasibel, sehingga solusi constrained bergeser ke batas lingkaran dengan kendala BINDING.

Gambar 4 menunjukkan kasus di mana optimum tanpa kendala berada di luar daerah feasibel; solusi constrained bergeser ke titik pada batas lingkaran di mana gradient kendala ∇g sejajar dengan arah menuju optimum tanpa kendala, sesuai kondisi stasioner KKT.

Contoh: Tangki Silinder Volume Tetap. Untuk desain tangki silinder dengan volume tetap V_0 , minimisasi luas permukaan $f=2\pi R^2+2\pi RH$ dengan kendala $\pi R^2H=V_0$ menghasilkan solusi analitik $H=2R$, $R^*=\text{akar_pangkat_tiga}(V_0/2\pi)\approx 0,542$ m dan $H\approx 1,084$ m (untuk $V_0=1$ m³).

Peringatan, Trap KKT yang Sering Diabaikan: Empat jebakan KKT yang sering diabaikan: constraint qualification yang tidak terpenuhi membuat KKT tidak valid; untuk NLP non-konveks, KKT hanya syarat perlu bukan cukup (bisa jadi saddle point); penalty dengan p terlalu besar menyebabkan ill-conditioning numerik; serta lupa memeriksa `result.constr\_violation` untuk memastikan solusi benar-benar feasible.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Mendaki Gunung di Kabut Tebal:** mendaki gunung dalam kabut tebal, hanya bisa merasakan kemiringan tanah di bawah kaki dan melangkah ke arah yang paling menurun/menaik, persis seperti gradient descent yang hanya memakai informasi lokal.
- **Naik Sepeda & Local Stability:** pengendara sepeda terus melakukan koreksi kecil untuk menjaga keseimbangan (feedback iteratif); sepeda fixed-gear di tanjakan curam adalah analogi jebakan non-konveks (local trap).
- **Audio Compressor & Knee Curve:** mastering engineer mengatur threshold, ratio, attack, dan release sehingga loudness tetap di atas -14 LUFS, sebuah optimasi berkendala interaktif.
- **Cruise Control Modern (MPC):** cruise control modern (MPC, dipakai Tesla Autopilot dan Mercedes EQS) menyelesaikan NLP berkendala setiap 100 milidetik untuk menjaga jarak aman sambil meminimalkan konsumsi bahan bakar.
- **Training Neural Network (Deep Learning):** model bahasa besar seperti GPT-4 dan AlphaFold dilatih via SGD/Adam/AdamW pada landscape loss yang sangat non-konveks berdimensi miliaran.
- **Camera Autofocus (Phase & Contrast Detection):** autofocus kamera memakai hill-climbing berbasis gradient finite-difference dari nilai kontras/fase untuk menemukan posisi lensa optimal.

Pola Umum: Pola yang sama di semua analogi: keputusan diambil berulang berdasarkan informasi lokal (slope/gradient) tanpa memerlukan gambaran global sistem, dan iterasi terus dilakukan hingga tidak ada perbaikan berarti lagi.

APLIKASI DI TEKNIK MESIN & INDUSTRI 4.0

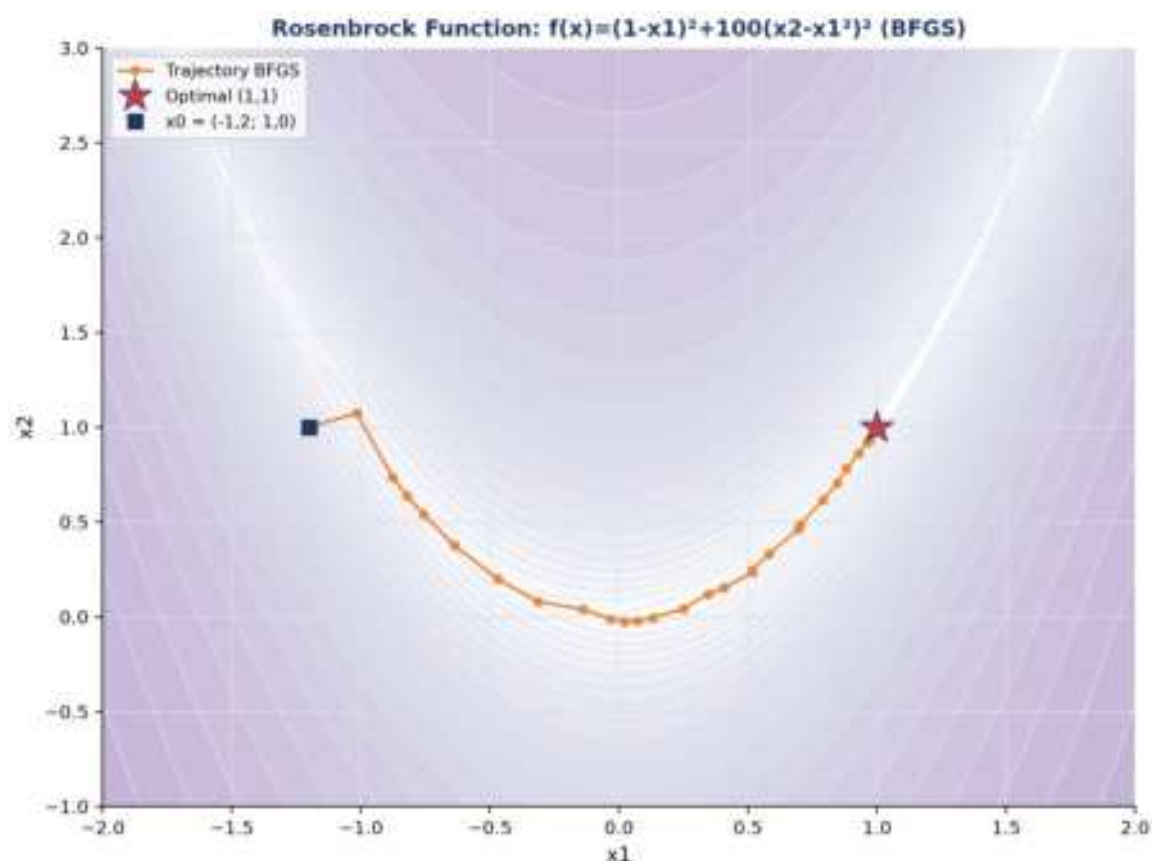
- **Design Parameter Optimization:** desain parameter pegas coil suspensi otomotif (diameter kawat d , jumlah lilitan N , panjang L) dioptimasi untuk minimisasi massa dengan kendala tegangan luluh, dipakai Mercedes, BMW, dan Toyota.
- **PID/MPC Controller Tuning:** tuning parameter PID/MPC untuk minimisasi ITAE/ISE, diimplementasikan dengan library `do-mpc` dan `CasADi`.
- **Topology Optimization (FEM):** topology optimization berbasis FEM (metode SIMP) dipakai Boeing 787 untuk bracket, BMW iX untuk gearbox, dan Tesla untuk giga-casting demi minimisasi berat struktur.

- **Robot Trajectory & Optimal Control:** perencanaan trajectory robot industri (KUKA, Universal Robots) meminimalkan waktu tempuh dengan kendala dinamika dan batas sendi.

Peringatan, Jebakan Praktis Formulasi NLP: Enam jebakan praktis formulasi NLP: lupa memvalidasi gradient analitik sebelum dipakai solver; memilih titik awal yang jauh dari solusi pada masalah non-konveks; mengabaikan scaling variabel yang berbeda orde besaran; menganggap hasil SLSQP tunggal sudah global optimal; lupa memeriksa constraint violation pada solusi akhir; serta tidak melakukan multi-start pada masalah dengan banyak local minimum.

IMPLEMENTASI PYTHON NLP DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini, berpuncak pada studi kasus desain pegas coil suspensi. Lingkungan: conda env optoauto dengan paket numpy, scipy, matplotlib, pandas, sympy; notebook p10_nonlinear_programming.ipynb.

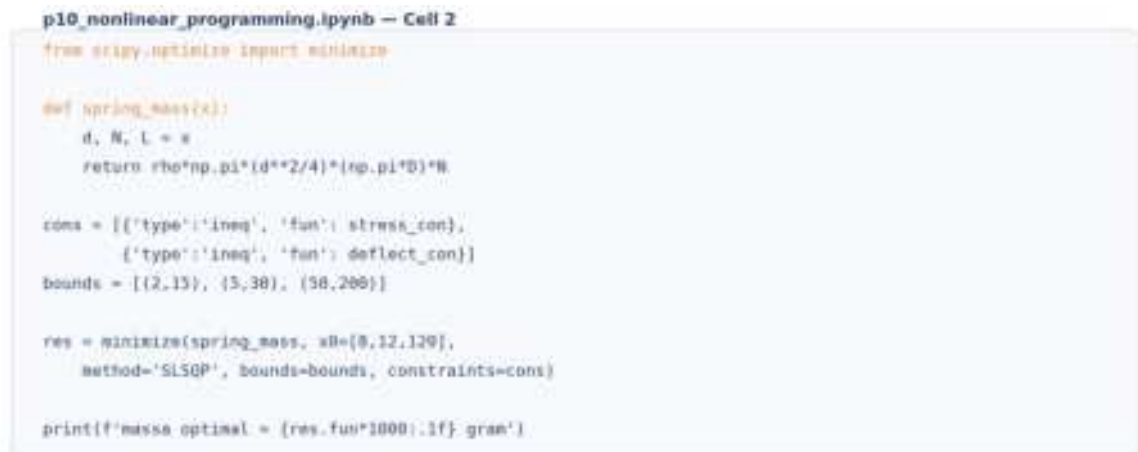


Gambar 5. Trajectory BFGS pada Rosenbrock function $f(x)=(1-x_1)^2+100(x_2-x_1^2)^2$, benchmark klasik NLP dengan lembah sempit melengkung.

Gambar 5 menunjukkan trajectory BFGS pada fungsi Rosenbrock, benchmark klasik yang menguji kemampuan solver menavigasi lembah sempit melengkung menuju optimal (1,1).

- **Cell 1:** gradient descent manual dan `scipy.optimize.minimize(method='BFGS')` untuk $\min f(x,y)=(x-3)^2+2(y-2)^2$ (solusi analitik $x^*=3$, $y^*=2$, $f^*=0$); plot kontur dan trajectory.
- **Cell 2:** constrained NLP desain pegas coil: minimisasi massa dengan kendala tegangan geser dan defleksi, memakai `method='SLSQP'`, konstanta $\rho=7,85e-6$ kg/mm³, $G=79300$ MPa, $\tau_y=600$ MPa, $F=500$ N, $D=50$ mm, $\delta_{\text{target}}=25$ mm, bounds $d \in [2,15]$, $N \in [5,30]$, $L \in [50,200]$, $x0=[8,12,120]$.
- **Cell 3:** KKT dan Lagrangian tangki silinder volume tetap ($V0=1,0$ m³): solusi simbolik via `sympy` diverifikasi numerik dengan `scipy.minimize(method='SLSQP')`.
- **Cell 4:** fungsi `solve_design_nlp()` reusable dengan validasi gradient (`check_grad`) dan multi-start (`np.random.seed(42)`) untuk masalah non-konveks; didemonstrasikan ulang pada masalah pegas Cell 2 dengan `n_start=10`.

Validasi Solusi via Multi-Start: Cell 4 menegaskan pentingnya validasi solusi NLP secara sistematis sebelum dipakai sebagai keputusan desain final: fungsi `solve_design_nlp()` tidak hanya menjalankan solver sekali, tetapi mengulanginya dari sepuluh titik awal acak berbeda (multi-start) dan membandingkan hasil `result.fun` dari tiap run. Bila seluruh sepuluh run konvergen ke nilai objektif yang sama (dalam toleransi numerik kecil), keyakinan bahwa solusi tersebut adalah optimum global (bukan sekadar local minimum) menjadi jauh lebih tinggi, meski tetap tidak ada jaminan matematis mutlak untuk masalah non-konveks umum.



```
p10_nonlinear_programming.ipynb — Cell 2
from scipy.optimize import minimize

def spring_mass(x):
    d, N, L = x
    return rho*np.pi*(d**2/4)*(np.pi*D)*N

cons = [{'type':'ineq', 'fun': stress_con},
        {'type':'ineq', 'fun': deflect_con}]
bounds = [(2,15), (3,30), (50,200)]

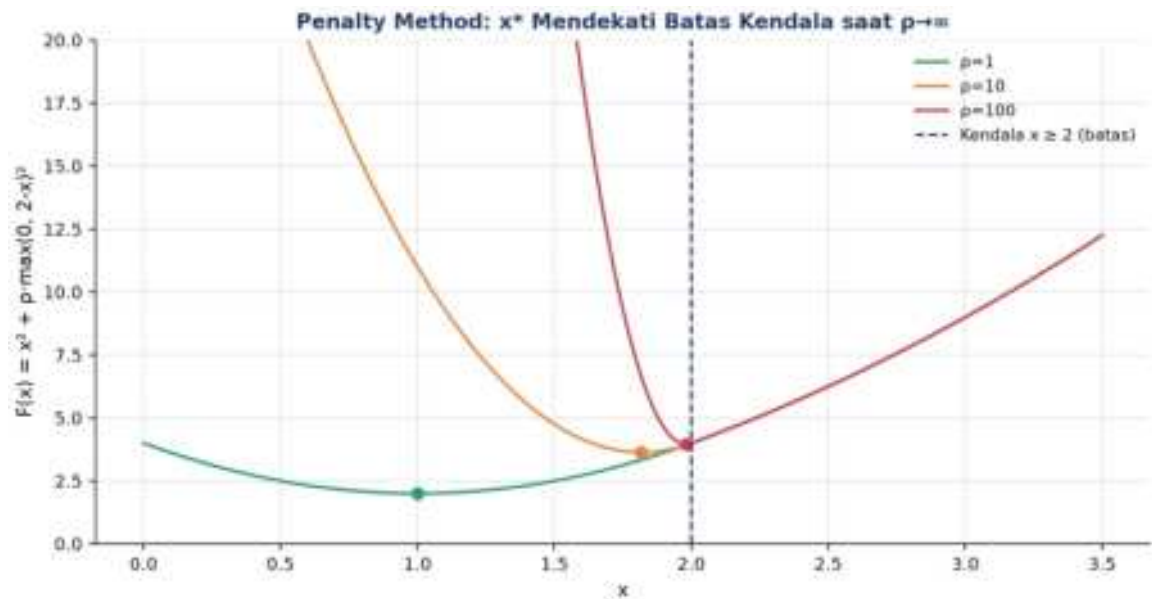
res = minimize(spring_mass, x0=[8,12,120],
               method='SLSQP', bounds=bounds, constraints=cons)

print(f'massa optimal = {res.fun*1000:.1f} gram')
```

Gambar 6. Cuplikan kode Cell 2: formulasi dan penyelesaian desain pegas coil dengan `scipy.optimize.minimize method SLSQP`.

Gambar 6 menunjukkan cuplikan kode inti Cell 2.

Penalty Method dalam Praktik: Untuk masalah non-konveks, satu kali menjalankan SLSQP dari satu titik awal berisiko terjebak local minimum. Multi-start (mencoba banyak titik awal acak dan mengambil hasil terbaik) atau metaheuristik (GA/PSO, dibahas Pertemuan 11) lebih realistis untuk mendekati solusi global.



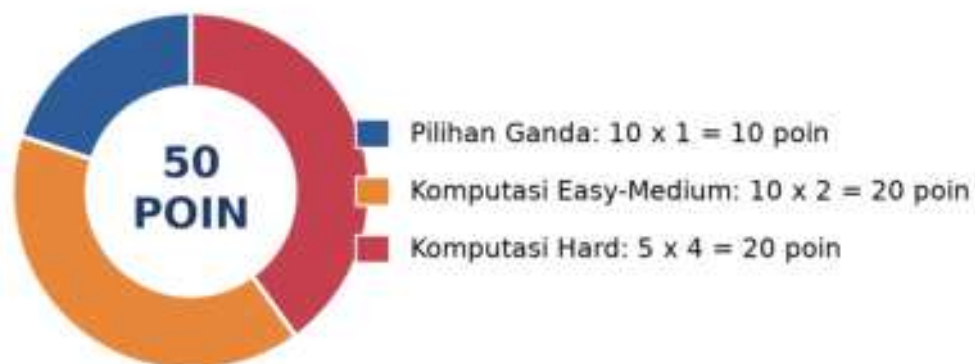
Gambar 7. Penalty method: $F(x)=x^2+p \cdot \max(0,2-x)^2$ untuk $p=1, 10, 100$; solusi x^* semakin mendekati batas kendala $x=2$ saat p membesar.

Gambar 7 menunjukkan bagaimana penalty method secara bertahap mendorong solusi menuju batas kendala saat parameter penalty p diperbesar.

TUGAS PERTEMUAN 10

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 10



Gambar 8. Distribusi 50 poin Tugas Pertemuan 10 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Pernyataan paling tepat tentang syarat orde-pertama (necessary condition) bagi local minimum fungsi $f(x)$ tanpa kendala adalah...

- A. $f(x^*) = 0$ (nilai fungsi nol)
- B. $\nabla f(x^*) = 0$ (gradient nol, disebut titik stasioner)
- C. $\nabla^2 f(x^*) = 0$ (Hessian nol)
- D. $x^* = 0$ (variabel keputusan nol)

2. Pada gradient descent dengan formula $x_{k+1} = x_k - \alpha \cdot \nabla f(x_k)$, peran step size α adalah...

- A. Menentukan jumlah dimensi variabel
- B. Mengontrol seberapa jauh melangkah ke arah negative gradient; terlalu besar overshoot/divergen, terlalu kecil konvergen lambat
- C. Memberi penalty terhadap kendala
- D. Selalu harus diset $\alpha = 1,0$ saja

3. Bedanya Newton's method dari steepest descent adalah...

- A. Newton tidak butuh gradient, hanya pakai nilai fungsi
- B. Newton pakai informasi Hessian (turunan kedua) untuk model lokal kuadratik, arah $d = -H^{-1} \nabla f$; konvergensi kuadratik (jauh lebih cepat dari linier)
- C. Newton hanya untuk fungsi linier; steepest descent untuk non-linier
- D. Newton tidak menjamin konvergensi sama sekali

4. Method 'BFGS' di `scipy.optimize.minimize` termasuk dalam keluarga...

- A. Gradient-free derivative-free method (seperti Nelder-Mead)
- B. Quasi-Newton, aproksimasi Hessian dari gradient saja, konvergensi super-linier, default `scipy` untuk unconstrained NLP
- C. Gradient-only method seperti steepest descent murni
- D. Linear programming solver simpleks variant

5. Untuk masalah NLP berkendala $\min f(x)$ s.t. $g(x) \leq 0$, $h(x) = 0$, Lagrangian didefinisikan sebagai...

- A. $L = f(x) - g(x) - h(x)$
- B. $L(x, \lambda, \mu) = f(x) + \lambda^T \cdot h(x) + \mu^T \cdot g(x)$ dengan $\mu \geq 0$ untuk inequality
- C. $L = f(x) \cdot g(x) \cdot h(x)$ (perkalian)
- D. $L = \nabla f(x)$ saja, tanpa multiplier

6. Complementary slackness dalam KKT conditions menyatakan untuk tiap kendala $g_j(x) \leq 0$ dengan multiplier $\mu_j \geq 0$...

- A. $\mu_j + g_j(x) = 0$ selalu

- B. $\mu_j \cdot g_j(x^*) = 0$; kendala aktif ($g=0$) bisa punya $\mu>0$, kendala tidak aktif ($g<0$) harus $\mu=0$
- C. $\mu_j = g_j(x)$ selalu sama nilainya
- D. $\mu_j > g_j(x)$ tanpa kondisi tambahan

7. Penalty method mengkonversi NLP berkendala menjadi unconstrained dengan cara...

- A. Menghapus semua kendala dari masalah
- B. Menambah term $p \cdot \sum \max(0, g_j)^2$ ke objektif; saat p menuju tak hingga, solusi unconstrained menuju solusi constrained
- C. Mengubah variabel x menjadi $\log(x)$
- D. Hanya untuk LP, tidak berlaku untuk NLP

8. Untuk NLP berkendala umum (equality dan inequality) di scipy, method yang paling tepat dari minimize adalah...

- A. method='BFGS' (hanya unconstrained)
- B. method='SLSQP' atau 'trust-constr', mendukung equality, inequality, dan bounds
- C. method='Nelder-Mead' (derivative-free, tidak handle constraints langsung)
- D. method='CG' (conjugate gradient, unconstrained)

9. Untuk fungsi konveks, sembarang local minimum juga merupakan...

- A. Saddle point
- B. Global minimum, sifat fundamental fungsi konveks (Hessian semi-definit positif)
- C. Maksimum lokal di sisi lain
- D. Tidak ada hubungan sama sekali dengan global

10. Untuk masalah NLP non-konveks (banyak local minimum), strategi paling realistis untuk mendapat solusi mendekati global adalah...

- A. Pakai BFGS sekali saja dari titik default, pasti dapat global
- B. Multi-start dengan banyak titik awal random lalu ambil hasil terbaik, atau pakai metaheuristik (GA/PSO, Pertemuan 11)
- C. Ubah kendala secara acak hingga masalah jadi linier
- D. Tidak perlu metode khusus, non-konveks selalu solvable dengan gradient descent biasa

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: gunakan `scipy.optimize.minimize`. Method 'BFGS' untuk unconstrained, 'SLSQP' untuk berkendala. Sediakan gradient analitik via `'jac=...'` bila bisa. Pastikan cek `result.success`.

C1. Selesaikan unconstrained NLP $\min f(x) = (x-4)^2$ dengan `scipy.optimize.minimize` method BFGS, $x_0=0$. Print nilai x optimal.

- 💡 **HINT:** Definisikan fungsi tujuan sebagai lambda, panggil minimize(..., method='BFGS'), lalu ambil x optimal dari res.x. Cek analitik: turunan nol di $x=4$.

C2. Untuk fungsi quadratic 2D $f(x,y)=(x-3)^2+2(y-2)^2$, hitung nilai gradient di titik (1,1) sebagai vektor norma: $\|\nabla f(1,1)\|=\text{akar}(\nabla f_x^2+\nabla f_y^2)$. Print nilainya.

- 💡 **HINT:** Gradien $\nabla f = (2(x-3), 4(y-2))$; evaluasi di titik yang diminta lalu hitung norma $= \text{akar}(\nabla f_x^2+\nabla f_y^2)$ dengan np.sqrt.

C3. Selesaikan min $f(x,y)=x^2+2y^2$ dengan scipy.minimize method BFGS, $x_0=[3,2]$. Print nilai f optimal ($f(x^*)$).

- 💡 **HINT:** Panggil minimize(..., method='BFGS') dengan fungsi tujuan sebagai lambda; nilai f optimal dibaca dari res.fun (bisa sangat kecil karena floating point).

C4. Selesaikan NLP berkendala min $f(x,y)=x^2+y^2$ s.t. $x+y=4$. Pakai method='SLSQP' dengan constraints={'type':'eq', 'fun': lambda x: x[0]+x[1]-4}. Print x optimal (x[0]).

- 💡 **HINT:** Pakai method='SLSQP' dengan kendala equality lewat dict {'type':'eq', 'fun': ...}; mulai dari x_0 feasible dan baca x optimal dari res.x.

C5. Untuk masalah sebelumnya (min x^2+y^2 s.t. $x+y=4$), hitung multiplier λ dari Lagrangian $L=x^2+y^2+\lambda(x+y-4)$. Solusi: $\partial L/\partial x=2x+\lambda=0$, $\partial L/\partial y=2y+\lambda=0$. Print nilai λ .

- 💡 **HINT:** Dari kondisi stasioner Lagrangian $\partial L/\partial x=2x+\lambda=0$, sehingga $\lambda=-2x$ dievaluasi di solusi optimal.

C6. Lakukan 1 iterasi gradient descent untuk $f(x)=x^2$ mulai dari $x_0=5$ dengan step size $\alpha=0,3$. Formula: $x_1=x_0-\alpha \cdot f'(x_0)$. Print x_1 .

- 💡 **HINT:** Turunan $f'(x)=2x$; terapkan rumus update $x_1=x_0-\alpha \cdot f'(x_0)$ dengan nilai x_0 dan α dari soal.

C7. Untuk fungsi $f(x)=x^4-4x^2+5$, hitung 1 step Newton's method mulai dari $x_0=3$ dengan formula $x_1=x_0-f'(x_0)/f''(x_0)$. Print x_1 .

- 💡 **HINT:** Turunkan $f'(x)=4x^3-8x$ dan $f''(x)=12x^2-8$, evaluasi di x_0 , lalu terapkan update Newton $x_1=x_0-f'(x_0)/f''(x_0)$.

C8. Validasi konveksitas $f(x)=x^4+2x^2+1$ di $x=1$ via Hessian (turunan kedua). Hitung $f''(1)$. Print nilainya. Jika $f''(x) \geq 0$ di seluruh domain maka konveks.

- 💡 **HINT:** Hessian 1D = turunan kedua $f''(x)=12x^2+4$; evaluasi di x yang diminta. Karena selalu lebih besar dari 0, fungsi bersifat strictly convex.

C9. Selesaikan NLP berkendala dengan bounds: min $f(x,y)=(x-2)^2+(y-3)^2$ s.t. $0 \leq x \leq 1$, $0 \leq y \leq 5$. Pakai method='L-BFGS-B', $x_0=[0.5,0.5]$. Print x optimal.

- 💡 **HINT:** Pakai method='L-BFGS-B' dengan parameter bounds; jika optimum tak berkendala di luar batas, solver memproyeksikannya ke boundary. Baca x optimal dari res.x.

C10. Penalty method: untuk $\min f(x)=x^2$ s.t. $x \geq 2$, formulasi penalty $F(x)=x^2+p \cdot \max(0, 2-x)^2$. Hitung $F(1.5)$ dengan $p=100$. Print nilainya.

- 💡 **HINT:** Substitusikan x ke $F(x)=x^2+p \cdot \max(0, 2-x)^2$; suku penalty hanya aktif saat kendala dilanggar. Saat p menuju tak hingga, solusi mendekati constraint binding ($x=2$).

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong + submit = 0 poin dan tidak terkunci (bisa retry).

C11 (Hard). Rosenbrock Function, benchmark NLP klasik: $f(x,y)=(1-x)^2+100(y-x^2)^2$. Selesaikan dengan `scipy.minimize` method BFGS, $x_0=[-1.2, 1.0]$. Print $x^* + y^*$ (jumlah dari koordinat minimum yang ditemukan optimizer).

- 💡 **HINT:** Rosenbrock ("banana function") punya solusi unik di $(1,1)$; selesaikan dengan `minimize(..., method='BFGS')` lalu jumlahkan komponen `res.x` dengan `.sum()`.

C12 (Hard). Tangki Silinder Volume Tetap: cari R, H untuk min surface area $f=2\pi R^2+2\pi RH$ s.t. volume $\pi R^2 H=1,0 \text{ m}^3$. Pakai method=SLSQP. Print R^* (radius optimal, m).

- 💡 **HINT:** Pakai method='SLSQP' dengan kendala volume sebagai dict equality `{'type':'eq', 'fun': lambda x: pi*R**2*H - V}`; mulai dari x_0 feasible dan baca R dari `res.x`. Solusi analitik optimal: $H=2R$.

C13 (Hard). Penalty Method Convergence: untuk $\min f(x)=x^2$ s.t. $x \geq 2$, formulasi penalty $F(x)=x^2+p \cdot \max(0, 2-x)^2$. Solve untuk $p=1, 10, 100$ dengan `scipy.minimize`. Print x^* untuk $p=100$.

- 💡 **HINT:** Untuk tiap p , minimkan $F(x)=x^2+p \cdot \max(0, 2-x)^2$ dengan `minimize` lalu baca `res.x`; makin besar p , x^* makin mendekati constraint binding $x=2$.

C14 (Hard). Multi-Start untuk Non-Konveks: untuk $f(x)=x^4-8x^2+5$ (dua minimum di $x=\pm 2$, satu lokal di $x=0$), jalankan 10 kali `minimize` BFGS dari x_0 random uniform di $[-5,5]$ (`seed=42`). Print nilai f minimum global yang ditemukan.

- 💡 **HINT:** Jalankan `minimize` berkali-kali dari x_0 acak (set seed), kumpulkan tiap `res.fun`, lalu ambil yang terkecil dengan `min` untuk menemukan minimum global.

C15 (Hard). Pegas Coil Engineering Design: min mass $m=\rho \cdot \pi \cdot (d^2/4) \cdot (\pi D) \cdot N$ dengan $\rho=7,85e-6 \text{ kg/mm}^3$, $D=50 \text{ mm}$, $F=500 \text{ N}$. Kendala: $\tau_{\max}=8FD/(\pi d^3) \leq 600 \text{ MPa}$, $\delta=8FD^3N/(Gd^4) \geq 25 \text{ mm}$ ($G=79300 \text{ MPa}$), $L \geq N \cdot d$. Bounds: $d \in [2,15]$, $N \in [5,30]$, $L \in [50,200]$. Pakai SLSQP. Print massa minimum (gram, $x_0=[8,12,120]$).

-  **HINT:** Formulasikan fungsi massa dan tiga kendala (tegangan, defleksi, panjang) sebagai dict inequality, lalu selesaikan dengan method='SLSQP' memakai bounds dan x0 dari soal; massa dalam gram = res.fun x 1000.

DAFTAR PUSTAKA

- Usha, R. (2022). A mathematical model for nonlinear optimization which attempts membership functions to address the uncertainties. *Mathematics*, 10(10), 1743. <https://doi.org/10.3390/math10101743>
- Oancea, G. (2022). Machining parameters optimization based on objective function linearization. *Mathematics*, 10(5), 803. <https://doi.org/10.3390/math10050803>
- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Optimasi Berbasis ANOVA: Analisis

Abstrak

Pertemuan ini memperkenalkan ANOVA (Analysis of Variance) sebagai metode optimasi berbasis eksperimen fisik:

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 4.1 – Mampu menganalisis penggunaan teknologi 4.0 untuk meningkatkan kinerja

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-10.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

LP (Pertemuan 9) dan NLP (Pertemuan 10) berbasis formula matematis yang dapat dievaluasi secara eksak. Namun banyak masalah rekayasa harus dioptimasi lewat eksperimen fisik yang mahal dan berisik (noisy): tiap trial memakan waktu dan biaya, dan hasil pengukuran selalu bercampur noise. Pertemuan kesebelas ini membahas ANOVA (Analysis of Variance), metode optimasi berbasis eksperimen yang menjawab pertanyaan: apakah perbedaan rata-rata antar level faktor signifikan secara statistik, atau hanya kebetulan (noise)?



Gambar 1. Posisi Pertemuan 11 (Optimasi Berbasis ANOVA) dalam alur mata kuliah, antara optimasi non-linear (P10) dan monitoring rule-based (P12-P13).

Gambar 1 menunjukkan posisi Pertemuan 11 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

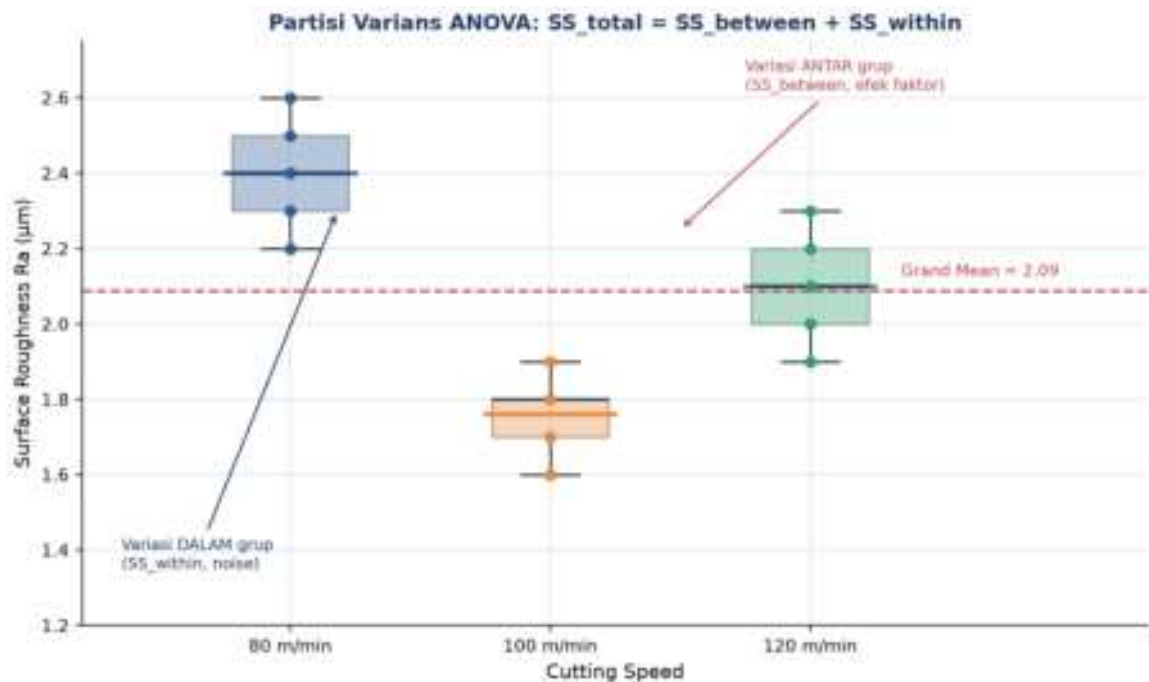
- **Sub-CPMK 4.1:** Mampu menganalisis penggunaan teknologi 4.0 untuk meningkatkan kinerja, yaitu menganalisis hasil eksperimen berbasis ANOVA sebagai basis pengambilan keputusan berbasis bukti statistik sebelum parameter optimal diimplementasikan pada sistem otomasi (CPMK 4).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan partisi varians dan statistik F; menjalankan workflow one-way dan two-way ANOVA lengkap; menginterpretasikan interaction effect; serta menerapkan Taguchi orthogonal array untuk efisiensi eksperimen.

ANOVA CORE: PARTISI VARIANS, F-STATISTIC & P-VALUE

ANOVA dikembangkan oleh Ronald A. Fisher (1925) di Rothamsted Experimental Station. Ide dasarnya: total variasi data dipecah menjadi variasi ANTAR grup (disebabkan faktor yang diuji) dan variasi DALAM grup (noise pengukuran).

$$H_0: \mu_1 = \mu_2 = \dots = \mu_k \quad SS_{total} = SS_{between} + SS_{within} \quad df_b = k-1, df_w = N-k$$

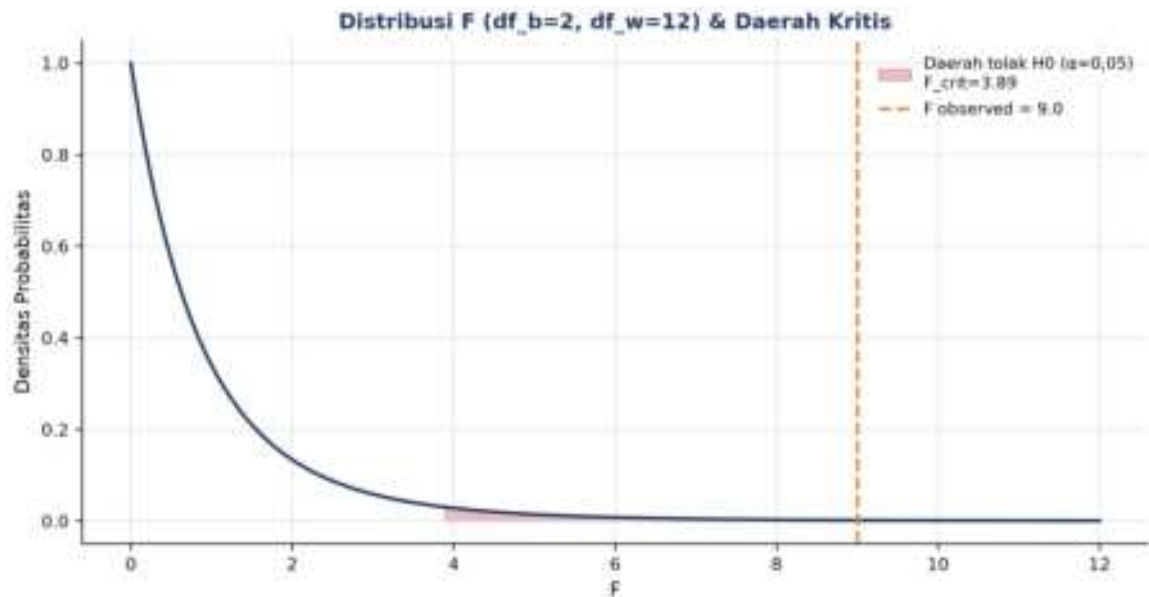
$$MS_b = SS_b / df_b \quad F = MS_b / MS_w \quad p < \alpha \rightarrow \text{tolak } H_0$$



Gambar 2. Partisi varians pada studi kasus CNC cutting speed vs surface roughness: variasi antar grup (efek faktor) vs variasi dalam grup (noise).

Gambar 2 mengilustrasikan partisi varians pada studi kasus cutting speed CNC (3 level: 80, 100, 120 m/min, $n=5$ replikasi). Variasi dalam grup (SS_{within}) mencerminkan noise pengukuran, sedangkan variasi antar grup ($SS_{between}$) mencerminkan efek nyata dari perubahan cutting speed.

Contoh: Cutting Speed CNC vs Surface Roughness. Untuk data cutting speed di atas: $SS_{between}=0,96$, $SS_{within}=0,64$, $df_b=2$, $df_w=12$, $MS_b=0,48$, $MS_w=0,053$, sehingga $F=(0,48/0,053)=9,0$ dengan $p=0,004$. Karena $p<0,05$, H_0 ditolak: cutting speed berpengaruh signifikan terhadap surface roughness. Uji Tukey HSD lebih lanjut menunjukkan level 100 vs 80 berbeda signifikan ($p=0,003$), sedangkan 120 vs 80 tidak ($p=0,21$).



Gambar 3. Distribusi F ($df_b=2$, $df_w=12$) dengan daerah kritis $\alpha=0,05$; $F_{\text{observed}}=9,0$ jauh melewati $F_{\text{crit}}=3,89$ sehingga H_0 ditolak.

Gambar 3 memperlihatkan posisi F_{observed} relatif terhadap daerah kritis distribusi F; semakin jauh F_{observed} berada di kanan F_{crit} , semakin kuat bukti bahwa faktor yang diuji berpengaruh nyata.

Tools Python: Implementasi praktis memakai `scipy.stats.f_oneway` untuk uji cepat, `statsmodels.formula.api.ols` beserta `anova_lm` untuk tabel ANOVA lengkap, dan library `pingouin` untuk laporan ANOVA siap pakai termasuk effect size.

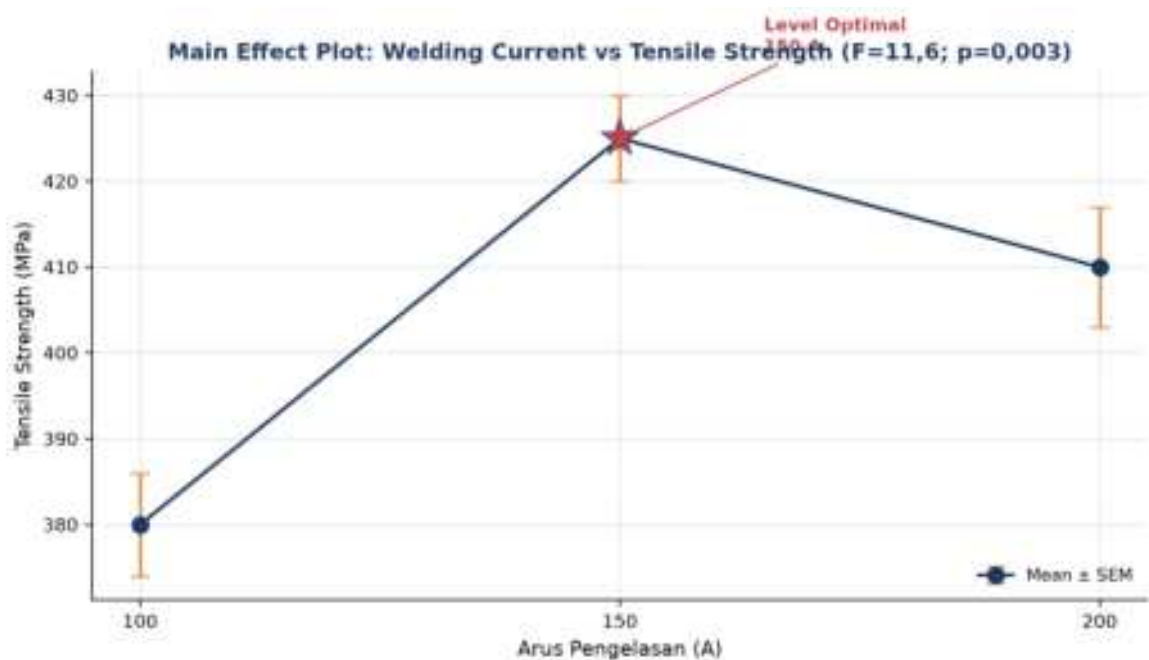
ONE-WAY ANOVA WORKFLOW: DARI DESAIN EKSPERIMEN KE PEMILIHAN LEVEL OPTIMUM

Tabel 1. Tujuh tahap workflow one-way ANOVA, dari perumusan hipotesis hingga pemilihan level optimum.

Tahap Workflow	Aksi	Tools/Function	Output
1. Hipotesis	Tetapkan H_0 , H_1 , α , faktor, level	-	Protokol eksperimen
2. Desain (CRD)	Randomize urutan trial	<code>np.random.permutation</code>	List urutan trial
3. Cek asumsi	Normality + equal variance	<code>scipy.stats.shapiro</code> / <code>levene</code>	$p \geq \alpha$ (asumsi OK)
4. Tabel ANOVA	Hitung SS, df, MS	<code>statsmodels.anova_lm</code>	Tabel ANOVA
5. F & p-value	Bandingkan p ke α	<code>scipy.stats.f_oneway</code>	F, p, keputusan
6. Post-hoc	Pairwise compare (Tukey HSD)	<code>pairwise_tukeyhsd</code>	Pasangan signifikan
7. Effect size	$\eta^2_{\text{kuadrat}} = SS_b / SS_{\text{total}}$	Manual atau <code>pingouin.anova</code>	η^2_{kuadrat} (kecil/sedang/besar)

Tabel 1 merangkum tujuh tahap workflow one-way ANOVA yang lengkap, dari perumusan hipotesis hingga effect size.

Contoh: Welding Current vs Tensile Strength. Studi kasus Welding Current vs Tensile Strength: 3 level arus (100, 150, 200 A), n=4 replikasi per level, N=12. Uji Shapiro-Wilk ($p=0,42$) dan Levene ($p=0,78$) mengonfirmasi asumsi normality dan homoscedasticity terpenuhi. $SS_{\text{between}}=4180$, $SS_{\text{within}}=1620$, $df_b=2$, $df_w=9$, sehingga $F=(4180/2)/(1620/9)=2090/180=11,6$ dengan $p=0,003$.



Gambar 4. Main effect plot: rata-rata tensile strength per level arus pengelasan, dengan error bar SEM; level optimal 150 A.

Gambar 4 menunjukkan main effect plot arus pengelasan terhadap tensile strength; uji Tukey HSD mengonfirmasi 150 A vs 100 A berbeda signifikan ($p=0,002$) sedangkan 150 A vs 200 A tidak ($p=0,18$), sehingga 150 A dipilih sebagai level optimal.

TWO-WAY ANOVA & FAKTORIAL DESIGN: DUA FAKTOR, EFEK UTAMA & INTERAKSI

Model aditif: $y_{ij} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \varepsilon$ $F_A = MS_A/MS_{\text{error}}$ $F_{AB} \rightarrow$
garis tidak paralel = interaksi

- **Model Aditif & Full Factorial:** kuadrat total dipecah menjadi kontribusi faktor A, faktor B, interaksi A×B, dan error; desain full factorial membutuhkan $k_1 \times k_2 \times \dots \times k_p$ trial, tumbuh eksponensial seiring jumlah faktor.
- **Taguchi Orthogonal Arrays:** Genichi Taguchi (1986) memperkenalkan orthogonal array (L4/L8/L9/L16/L18/L27) yang memungkinkan estimasi efek utama dengan jauh

lebih sedikit trial dibanding full factorial; L9 misalnya mengakomodasi 4 faktor x 3 level hanya dengan 9 trial.

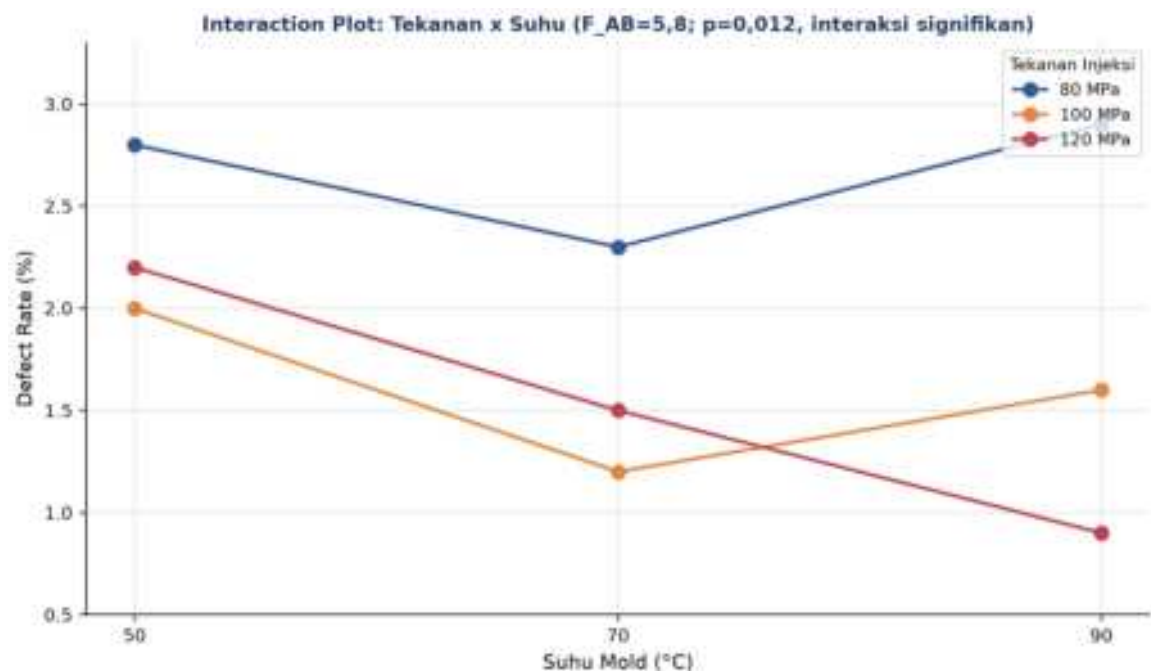
- **Signal-to-Noise Ratio:** $S/N = -10 \cdot \log_{10}(\text{MSD})$; tiga tipe: Larger-the-better, Smaller-the-better, Nominal-the-best, dipakai Taguchi untuk mengukur robustness solusi terhadap variasi noise.

Tabel 2. Tabel ANOVA two-way untuk dua faktor A dan B beserta interaksinya.

Sumber	SS	df	MS	F
Faktor A (main)	SS_A	a-1	SS_A/(a-1)	MS_A/MS_e
Faktor B (main)	SS_B	b-1	SS_B/(b-1)	MS_B/MS_e
Interaksi A x B	SS_AB	(a-1)(b-1)	SS_AB/df_AB	MS_AB/MS_e
Error (within)	SS_e	ab(n-1)	MS_e	-
Total	SS_A+SS_B+SS_AB+SS_e	abn-1	-	-

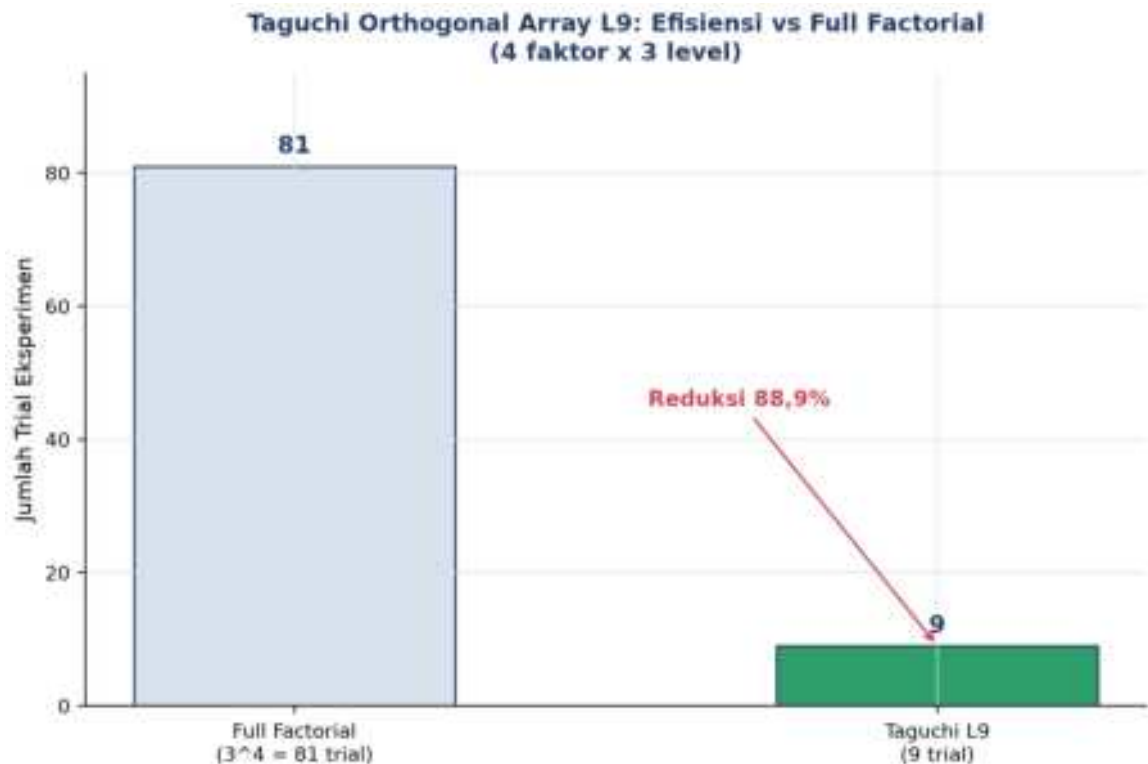
Tabel 2 merangkum struktur tabel ANOVA two-way, memisahkan kontribusi faktor A, faktor B, interaksi A×B, dan error.

Contoh: Injection Molding (Tekanan x Suhu). Studi kasus Injection Molding: Faktor A = tekanan injeksi (80, 100, 120 MPa), Faktor B = suhu mold (50, 70, 90°C), n=2 replikasi, total $3 \times 3 \times 2 = 18$ trial. Hasil: $F_A = 28,4$ ($p < 0,001$), $F_B = 12,1$ ($p = 0,003$), dan $F_{AB} = 5,8$ ($p = 0,012$), interaksi signifikan, artinya efek tekanan bergantung pada level suhu dan tidak bisa diinterpretasikan terpisah.



Gambar 5. Interaction plot tekanan injeksi x suhu mold: garis tidak paralel menandakan interaksi signifikan; kombinasi optimal 120 MPa & 90°C (defect terendah).

Gambar 5 menunjukkan garis yang saling menyilang, ciri khas interaksi signifikan; kombinasi 120 MPa dan 90°C menghasilkan defect rate terendah (sekitar 0,9%), berbeda dari kombinasi optimal 100 MPa pada 70°C jika hanya melihat efek utama tekanan secara terpisah.



Gambar 6. Efisiensi Taguchi orthogonal array L9 dibanding full factorial untuk 4 faktor x 3 level: reduksi 88,9% jumlah trial.

Gambar 6 menunjukkan penghematan jumlah trial signifikan yang ditawarkan Taguchi L9 dibanding pendekatan full factorial untuk masalah 4 faktor.

Peringatan, Trap ANOVA yang Sering Diabaikan: Tujuh jebakan ANOVA yang sering diabaikan: p-hacking (mengulang uji sampai signifikan); mengabaikan pelanggaran asumsi normality/homoscedasticity; lupa melakukan post-hoc setelah ANOVA signifikan; mengabaikan effect size padahal p-value bergantung pada sample size; lurking variable yang tidak terkontrol; tidak melakukan randomisasi trial; serta sample size terlalu kecil (butuh sekitar $n=28$ /grup untuk Cohen's $d=0,5$, $k=3$, $power=0,80$).

ANALOGI KEHIDUPAN SEHARI-HARI

- **Mencicipi Kopi dari 3 Roaster:** membandingkan rating rasa kopi dari 3 roaster berbeda (masing-masing 5 cup) adalah one-way ANOVA sederhana.

- **Eksperimen Fisher di Rothamsted:** Fisher merancang eksperimen 5 jenis pupuk di Rothamsted (1919) dengan tiga prinsip: randomization, replication, blocking, dituangkan dalam buku "Statistical Methods for Research Workers" (1925).
- **Quality Control Lini Produksi Toyota (Six Sigma):** Toyota memakai ANOVA untuk quality control diameter shaft antar-operator dalam program Six Sigma, menurunkan defect rate dari 6700 ppm menjadi kurang dari 3,4 ppm.
- **Taguchi di Toyota (Robust Design):** Taguchi mempopulerkan orthogonal array di Toyota tahun 1950-an: L9 (4 faktor x 3 level) hanya butuh 9 trial dibanding 81 pada full factorial, dituangkan dalam buku "The Quality Revolution".
- **Uji Klinis 3 Dosis Obat:** uji klinis 3 dosis obat (50, 100, 200mg) dengan n=30/grup memakai ANOVA; FDA mensyaratkan $p < 0,05$ sebelum obat disetujui.
- **A/B/C Testing Tombol Aplikasi Mobile:** A/B/C testing warna tombol aplikasi (hijau/oranye/biru) dengan 1000 user/grup adalah ANOVA untuk conversion rate; pendekatan modern memakai multi-armed bandit untuk optimasi berkelanjutan.

Pelajaran Umum: Semua analogi berbagi pelajaran yang sama: perbedaan yang tampak besar secara visual belum tentu signifikan secara statistik, dan sebaliknya; ANOVA memberi bukti kuantitatif sebelum keputusan diambil.

APLIKASI ANOVA DI TEKNIK MESIN & MANUFAKTUR

- **Tuning Parameter CNC Machining (Sandvik):** Sandvik Coromant memakai Taguchi L9 + ANOVA untuk tuning cutting speed, feed rate, dan depth of cut pada mesin CNC AISI 4340; hasil menunjukkan cutting speed paling berpengaruh ($\eta_{kuadrat}=0,62$), diikuti feed rate (0,24), sedangkan depth of cut tidak signifikan (0,05).
- **Welding Parameter Optimization (Lincoln Electric):** Lincoln Electric memakai two-way ANOVA (arus x voltage) untuk optimasi parameter pengelasan; arus signifikan ($F=28$, $p < 0,001$), voltage signifikan ($F=12$, $p=0,003$), interaksi tidak signifikan ($p=0,18$), mengikuti standar ISO 3834-1.
- **Quality Control Pabrik Turbin (GE Aviation):** GE Aviation memakai SPC + ANOVA untuk quality control blade turbin lintas 3 shift kerja, bagian dari program Six Sigma DMAIC, menurunkan defect rate dari 850 menjadi 12 ppm.
- **Additive Manufacturing Parameter Validation (GE Additive):** GE Additive memakai full factorial ANOVA untuk validasi parameter Selective Laser Melting (SLM) pada material Inconel 718 (laser power, scan speed, hatch spacing, preheat temp), mengikuti standar ASTM F3301, mencapai kepadatan 99,5%.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan ANOVA pada studi kasus manufaktur. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, statsmodels, matplotlib, seaborn; notebook p11_anova.ipynb.

```
p11_anova.ipynb — Cell 1
from scipy import stats

group_80 = [2.5, 2.4, 2.6, 2.3, 2.2]
group_100 = [1.8, 1.7, 1.9, 1.8, 1.6]
group_120 = [2.0, 2.2, 2.1, 2.3, 1.9]

F, p = stats.f_oneway(group_80, group_100, group_120)
print('F = {F:.2f}, p = {p:.4f}')

if p < 0.05:
    print('Tolak H0: cutting speed berpengaruh signifikan')
# output: F = 9.08, p = 0.0046
```

Gambar 7. Cuplikan kode one-way ANOVA cutting speed CNC menggunakan `scipy.stats.f_oneway`.

Gambar 7 menunjukkan cuplikan kode inti Cell 1.

- **Cell 1:** one-way ANOVA dari scratch (SS, df, MS, F, p manual) untuk data cutting speed CNC, diverifikasi dengan `scipy.stats.f_oneway`.
- **Cell 2:** two-way ANOVA statsmodels untuk data welding arus x voltage: `ols('strength ~ C(arus) * C(voltage)', data=df).fit()` diikuti `anova_lm`; visualisasi heatmap mean per sel dan interaction plot.
- **Cell 3:** Tukey HSD post-hoc dan cek asumsi (Shapiro-Wilk dan Levene) untuk data injection molding; `pairwise_tukeyhsd` mengidentifikasi pasangan level yang berbeda signifikan.
- **Cell 4:** fungsi `run_anova_workflow()` reusable: cek asumsi, ANOVA, effect size dengan interpretasi Cohen, Tukey HSD kondisional; didemonstrasikan pada data heat treatment temperature vs Rockwell hardness.

TUGAS PERTEMUAN 11

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.



Gambar 8. Distribusi 50 poin Tugas Pertemuan 11 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Hipotesis nol (H_0) untuk one-way ANOVA dengan k level faktor adalah...

- A. $H_0: \mu_1 \neq \mu_2$ (semua mean berbeda)
- B. $H_0: \mu_1 = \mu_2 = \dots = \mu_k$ (semua mean grup sama, faktor tidak berpengaruh)
- C. $H_0: \sigma_1 = \sigma_2 = \dots = \sigma_k$ (semua varians sama)
- D. $H_0: F = 0$ (statistik F nol)

2. Identitas fundamental ANOVA "partition of variance" adalah...

- A. $SS_{\text{total}} = SS_{\text{between}} \times SS_{\text{within}}$
- B. $SS_{\text{total}} = SS_{\text{between}} + SS_{\text{within}}$, total varians sama dengan varians antar grup (efek faktor) ditambah varians dalam grup (noise)
- C. $SS_{\text{total}} = SS_{\text{between}} - SS_{\text{within}}$
- D. $SS_{\text{total}} = \text{akar}(SS_{\text{between}}^2 + SS_{\text{within}}^2)$

3. Untuk one-way ANOVA dengan $k=4$ level dan total $N=24$ observasi, degrees of freedom adalah...

- A. $df_{\text{between}} = 4$, $df_{\text{within}} = 24$
- B. $df_{\text{between}} = k-1 = 3$, $df_{\text{within}} = N-k = 20$, $df_{\text{total}} = N-1 = 23$
- C. $df_{\text{between}} = 24$, $df_{\text{within}} = 4$
- D. $df_{\text{between}} = 1$, $df_{\text{within}} = 23$

4. Statistik F dalam ANOVA didefinisikan sebagai...

- A. $F = SS_{\text{between}} - SS_{\text{within}}$
- B. $F = MS_{\text{between}} / MS_{\text{within}} = (SS_b/df_b) / (SS_w/df_w)$, rasio dua estimasi varians

- C. $F = \text{grand_mean} / \text{std}$
 - D. $F = N - k$
5. Saat hasil ANOVA menunjukkan $p=0,003 < \alpha=0,05$, keputusan yang benar adalah...
- A. Terima H_0 karena p kecil
 - B. Tolak H_0 , terdapat bukti statistik bahwa paling tidak satu pasangan mean berbeda; lanjut dengan post-hoc Tukey HSD untuk identifikasi pasangan mana
 - C. Tidak ada cukup bukti untuk keputusan
 - D. Ubah α ke 0,001 dan ulangi analisis
6. Tiga asumsi utama ANOVA yang harus dicek sebelum interpretasi hasil adalah...
- A. Linearitas, kontinuitas, monotonisitas
 - B. Normality residual (Shapiro-Wilk), homoscedasticity/equal variance antar grup (Levene's test), independence trial (dijamin oleh randomisasi)
 - C. Konveksitas, smoothness, differentiability
 - D. H_0 , H_1 , α
7. Tukey HSD post-hoc test dipakai setelah ANOVA signifikan karena...
- A. F-statistic-nya rendah
 - B. ANOVA hanya bilang "ada efek" tanpa identifikasi pasangan; Tukey HSD membandingkan semua pasangan dengan mengontrol family-wise error rate, pasangan dengan $p_{\text{adj}} < \alpha$ signifikan berbeda
 - C. Untuk menambah sample size secara post-hoc
 - D. Untuk mengubah α dari 0,05 ke 0,10
8. Pada two-way ANOVA, interaction effect A x B signifikan ketika...
- A. Garis-garis di interaction plot saling paralel
 - B. F_{AB} signifikan ($p < \alpha$), efek faktor A bergantung pada level B; di interaction plot garis tidak paralel atau saling menyilang. Main effect tidak boleh diinterpretasi terpisah
 - C. Sample size sama besar untuk semua kombinasi
 - D. F_A dan F_B sama nilainya
9. Saat melaporkan hasil ANOVA di paper/laporan, praktik yang BENAR adalah...
- A. Cukup laporkan p-value saja, yang penting signifikan
 - B. Laporkan $F(df_b, df_w)=X.XX$, $p=X.XXX$, $\eta^2_{\text{kuadrat}}=X.XX$; F-statistic, df, p-value, dan effect size. Effect size penting karena p bergantung pada sample size
 - C. Set $\alpha=0,001$ supaya p selalu signifikan
 - D. Hanya laporkan grand mean
10. Taguchi orthogonal arrays (misal L9) digunakan untuk...
- A. Menggantikan ANOVA, hanya pakai S/N ratio

- B. Mengurangi jumlah trial eksperimen; $L_9=4$ faktor x 3 level dalam 9 trial (vs full factorial 81). Tiap pasangan level muncul frekuensi sama (orthogonal & balance) sehingga identifikasi level optimum efisien
- C. Menambah jumlah trial dari full factorial
- D. Hanya bekerja untuk 1 faktor

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: gunakan `scipy.stats.f_oneway`, `statsmodels.formula.api.ols` + `anova_lm`, `statsmodels.stats.multicomp.pairwise_tukeyhsd`. Formula:

$SS_{total}=SS_{between}+SS_{within}$; $F=MS_{between}/MS_{within}=(SS_b/df_b)/(SS_w/df_w)$; $\eta^2_{kuadrat}=SS_{between}/SS_{total}$; $df_b=k-1$, $df_w=N-k$, $df_{total}=N-1$.

C1. Hitung SS_{total} (total sum of squares) untuk dataset 3 grup: A=[5,6,7], B=[8,9,10], C=[11,12,13]. Formula: $SS_{total}=\sum(y_{ij}-\bar{y})^2$ dengan $\bar{y}$ =grand mean. Print nilainya.

- 💡 **HINT:** SS_{total} = jumlah $(y_i - \text{grand mean})$ kuadrat; gabungkan semua grup ke satu array lalu jumlahkan kuadrat deviasi terhadap meannya.

C2. Untuk dataset di soal sebelumnya, hitung $SS_{between} = \sum n_j \cdot (\bar{y}_j - \bar{y})^2$. Print nilainya.

- 💡 **HINT:** $SS_{between}$ = jumlah $n_j \cdot (\text{mean grup} - \text{grand mean})$ kuadrat; hitung mean tiap grup dan grand mean, lalu jumlahkan deviasi kuadrat berbobot ukuran grup.

C3. Untuk dataset yang sama, hitung $SS_{within} = \sum \sum (y_{ij} - \bar{y}_j)^2$ (deviasi data dari mean grupnya, jumlah semua grup). Print nilainya.

- 💡 **HINT:** SS_{within} = jumlah kuadrat deviasi tiap data terhadap mean grupnya, dijumlahkan atas semua grup. Verifikasi: $SS_{between} + SS_{within} = SS_{total}$.

C4. Diketahui $SS_{between}=120$, $SS_{within}=36$, $df_b=3$, $df_w=16$. Hitung statistik $F=(SS_b/df_b)/(SS_w/df_w)$. Print nilainya.

- 💡 **HINT:** $MS_b = SS_b/df_b$ dan $MS_w = SS_w/df_w$; $F = MS_b$ dibagi MS_w . F besar menandakan efek faktor yang kuat.

C5. Untuk one-way ANOVA dengan $k=4$ level dan $n=5$ replikasi per level, hitung df_{total} . Formula: $df_{total}=N-1$ dengan $N=k \cdot n$. Print nilainya.

- 💡 **HINT:** $N = k \times n$ (total observasi), lalu $df_{total} = N - 1$. Verifikasi: $df_{total} = df_{between} (k-1) + df_{within} (N-k)$.

C6. Gunakan `scipy.stats.f_oneway` pada 3 grup A=[5,6,7], B=[8,9,10], C=[11,12,13]. Print nilai F-statistic hasil scipy.

- 💡 **HINT:** Gunakan `scipy.stats.f_oneway` dengan tiap grup sebagai argumen; fungsi mengembalikan F-statistic dan p-value (F sama dengan hasil manual sebelumnya).

C7. Effect size $\eta^2_{kuadrat}$: untuk $SS_{between}=120$ dan $SS_{total}=156$, hitung $\eta^2_{kuadrat}=SS_b/SS_{total}$. Print nilainya (desimal).

- 💡 **HINT:** $\eta^2_{kuadrat} = SS_{between} / SS_{total}$, proporsi varians yang dijelaskan faktor. Menurut Cohen, $\eta^2_{kuadrat} \geq 0,14$ tergolong large effect.

C8. F-critical value: untuk $\alpha=0,05$, $df_b=2$, $df_w=12$, hitung F_{crit} menggunakan `scipy.stats.f.ppf(0.95, dfn=2, dfd=12)`. Print nilainya.

- 💡 **HINT:** Gunakan `scipy.stats.f.ppf(1- $\alpha$ , dfn, dfd)` untuk F-critical; jika $F_{observed}$ melebihinya, tolak H_0 . Makin besar df , F_{crit} makin kecil.

C9. Shapiro-Wilk normality test: untuk grup data [2.5, 2.4, 2.6, 2.3, 2.2], gunakan `scipy.stats.shapiro` untuk dapat W-statistic. Print nilai W (statistik pertama dari output, bukan p-value).

- 💡 **HINT:** Gunakan `scipy.stats.shapiro`; nilai pertama yang dikembalikan adalah W-statistic (mendekati 1 berarti makin normal).

C10. Total trial Taguchi L9: orthogonal array L9 mengakomodasi 4 faktor x 3 level dengan hanya 9 baris (vs full factorial $3^4=81$). Hitung reduction ratio= $(81-9)/81 \times 100$ (persen pengurangan trial). Print nilainya.

- 💡 **HINT:** Reduction ratio = $(trial_{full} - trial_{L9}) / trial_{full} \times 100$, dengan $trial_{full} = 3^4$. Substitusikan nilai dari soal.

Bagian C: Komputasi Hard (5 soal $\times$ 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong + submit = 0 poin dan tidak terkunci (bisa retry).

C11 (Hard). One-Way ANOVA dari scratch, CNC Cutting Speed vs Surface Roughness: gunakan data 3 grup $A=[2.5, 2.4, 2.6, 2.3, 2.2]$, $B=[1.8, 1.7, 1.9, 1.8, 1.6]$, $C=[2.0, 2.2, 2.1, 2.3, 1.9]$ (R_a μm pada speed 80, 100, 120 m/min). Hitung F-statistic manual via $SS_{between}$, SS_{within} , df , MS , tanpa `scipy.stats.f.oneway`. Print F.

- 💡 **HINT:** Hitung mean tiap grup dan grand mean, lalu $SS_{between}$, SS_{within} , MS , dan $F=MS_b/MS_w$ secara manual; verifikasi dengan `scipy.stats.f.oneway`.

C12 (Hard). Two-Way ANOVA Welding (statsmodels): dataset arus x voltage. Generate via `seed=42`, 3 level arus (100, 150, 200 A), 3 level voltage (22, 26, 30 V), 3 replikasi/cell. Mean cell: $arus_idx \times 25 + volt_idx \times 10 + arus_idx \times volt_idx \times 3 + N(0,8)$. Pakai `ols('strength ~ C(arus)*C(voltage)', df).fit() + anova_lm`. Print p-value untuk faktor arus (main effect).

- 💡 **HINT:** Bangun DataFrame (set seed sesuai soal), fit model `ols('strength ~ C(arus)*C(voltage)')`, jalankan `anova_lm`, lalu ambil p-value baris C(arus) dari kolom `PR(>F)`.

C13 (Hard). Tukey HSD Post-Hoc + Asumsi Check, Heat Treatment: 4 level temperatur (450, 500, 550, 600 C), 5 spesimen/level, target HRC: {450:52, 500:58, 550:62, 600:55} plus

minus 2. seed=42. (1) Cek normality (Shapiro per grup), (2) cek equal variance (Levene), (3) Run ANOVA, (4) Run pairwise_tukeyhsd. Print jumlah pasangan signifikan (kolom reject=True) dari Tukey HSD.

- 💡 **HINT:** Jalankan pairwise_tukeyhsd, lalu hitung berapa baris dengan kolom reject == True (pasangan yang berbeda signifikan); 4 level menghasilkan $C(4,2)=6$ pasangan.

C14 (Hard). Power Analysis ANOVA: gunakan statsmodels.stats.power.FTestAnovaPower untuk hitung sample size per grup yang dibutuhkan untuk deteksi effect size $f=0,4$ (large effect by Cohen) dengan $k=4$ grup, $\alpha=0,05$, power=0,80. solve_power(effect_size=0.4, alpha=0.05, power=0.80, k_groups=4). Print n per grup (round ke integer).

- 💡 **HINT:** Gunakan FTestAnovaPower().solve_power(effect_size, alpha, power, k_groups) dengan parameter dari soal, lalu bulatkan ke atas dengan np.ceil untuk n per grup.

C15 (Hard). Engineering ANOVA Workflow, Injection Molding 2-Faktor: faktor A=tekanan (3 level: 80, 100, 120 MPa), B=suhu mold (3 level: 50, 70, 90 C), 3 replikasi/cell. Mean cell: defect persen = $4 - 0,02.A - 0,01.B + 0,001.A.B + N(0, 0.3)$. Total 27 trial, seed=42. Two-way ANOVA. Print F-statistic untuk interaction effect (A x B).

- 💡 **HINT:** Fit model ols('defect ~ C(A)*C(B)') lalu jalankan anova_lm; ambil F-statistic dari baris interaksi C(A):C(B). Jika signifikan, main effect tidak boleh diinterpretasi terpisah.

DAFTAR PUSTAKA

- Nguyen, T. T., Phi, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>
- Mohsin, I., He, K., Li, Z., Zhang, F., & Du, R. (2020). Optimization of the polishing efficiency and torque by using Taguchi method and ANOVA in robotic polishing. *Applied Sciences*, 10(3), 824. <https://doi.org/10.3390/app10030824>
- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>

Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972.

<https://doi.org/10.3390/app12030972>

Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Otomatisasi Monitoring: Sensor,

Abstrak

Pertemuan ini membahas fondasi sistem monitoring kondisi mesin berbasis rule-based: pemilihan sensor (accelerometer

Sub - CPMK

Sub-CPMK 4.2 – Mampu merancang sistem otomasi berbasis teknologi 4.0 untuk efisiensi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-11.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan 9-11 membahas optimasi berbasis formula matematis dan eksperimen statistik untuk menemukan parameter terbaik. Pertemuan kedua belas ini beralih ke topik baru: bagaimana parameter dan kondisi mesin dipantau secara real-time di lapangan menggunakan sistem monitoring rule-based, yang menjadi standar industri (ABB, Siemens, GE, Honeywell) karena sifatnya yang transparan, deterministik, dan hemat komputasi.



Gambar 1. Posisi Pertemuan 12 (Monitoring Sensor & ISO 10816) dalam alur mata kuliah, mengawali blok materi monitoring rule-based sebelum Pertemuan 13 (Tier Alarm & FSM).

Gambar 1 menunjukkan posisi Pertemuan 12 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 4.2:** Mampu merancang sistem otomasi berbasis teknologi 4.0 untuk efisiensi, yaitu merancang pipeline sensor-akuisisi-threshold untuk sistem monitoring kondisi mesin yang efisien dan sesuai standar ISO 10816 (CPMK 4).
- Setelah pertemuan ini mahasiswa dapat: mengklasifikasikan kondisi vibrasi mesin menurut zona ISO 10816-3; menerapkan hysteresis untuk mencegah chattering; memilih sensor yang tepat untuk kebutuhan monitoring; serta memahami kriteria Nyquist dalam akuisisi data.

THRESHOLD & ISO 10816 ZONE CLASSIFICATION

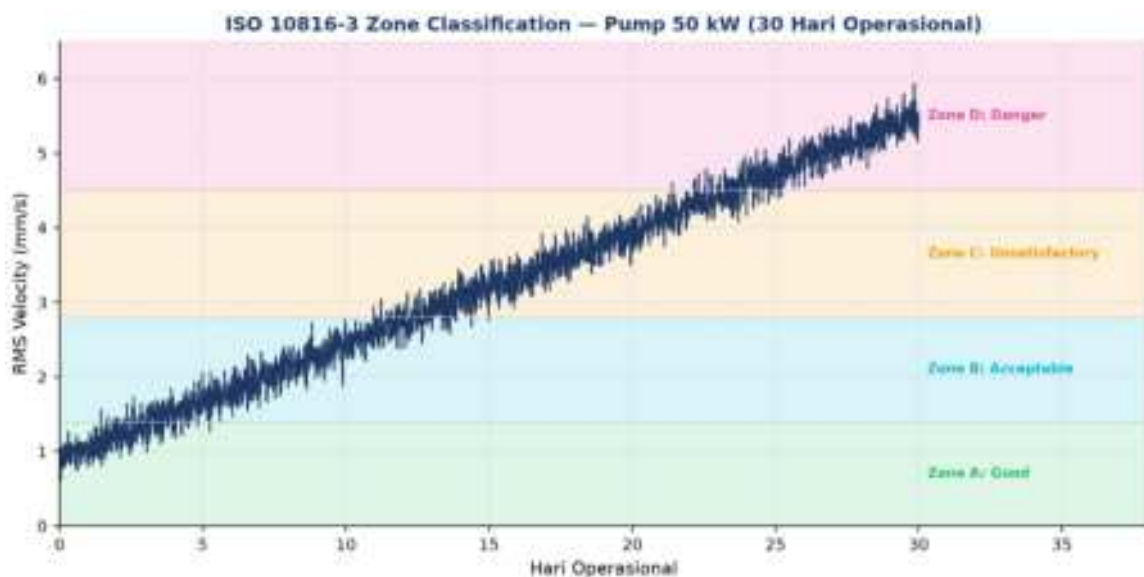
*IF condition THEN action ISO 10816-3 Zona A|B|C|D $RMS_v = akar(mean(v^2))$
di band 10Hz-1kHz*

Sumber Threshold: Threshold monitoring bisa bersumber dari standar internasional (ISO 10816, API 670), datasheet vendor, atau baseline empiris ($\mu+3\sigma$). Standar ISO 10816-3 mengklasifikasikan RMS velocity mesin rotasi ke 4 zona: A (Good), B (Acceptable), C (Unsatisfactory), D (Danger), bergantung pada kelas mesin dan jenis fondasi.

Tabel 1. Klasifikasi zona ISO 10816-3 Group 2 (mesin 15-300 kW, fondasi rigid) berdasarkan RMS velocity.

ISO 10816-3 Group 2 (15-300 kW, rigid)	RMS Velocity (mm/s)	Zone	Action
Newly commissioned, healthy	$\leq 1,4$	A - Good	Continuous operation OK
Acceptable for long-term	1,4 - 2,8	B - Acceptable	Operate, monitor
Suitable for short-term only	2,8 - 4,5	C - Unsatisfactory	Plan repair within weeks
Damage occurring	$> 4,5$	D - Danger	Shutdown / immediate repair

Tabel 1 merangkum klasifikasi zona ISO 10816-3 Group 2 yang menjadi acuan sepanjang modul ini.



Gambar 2. Simulasi streaming RMS velocity pompa 50kW selama 30 hari operasional, dengan overlay zona ISO 10816-3 A-D; sinyal berdegradasi dari zona A menuju D.

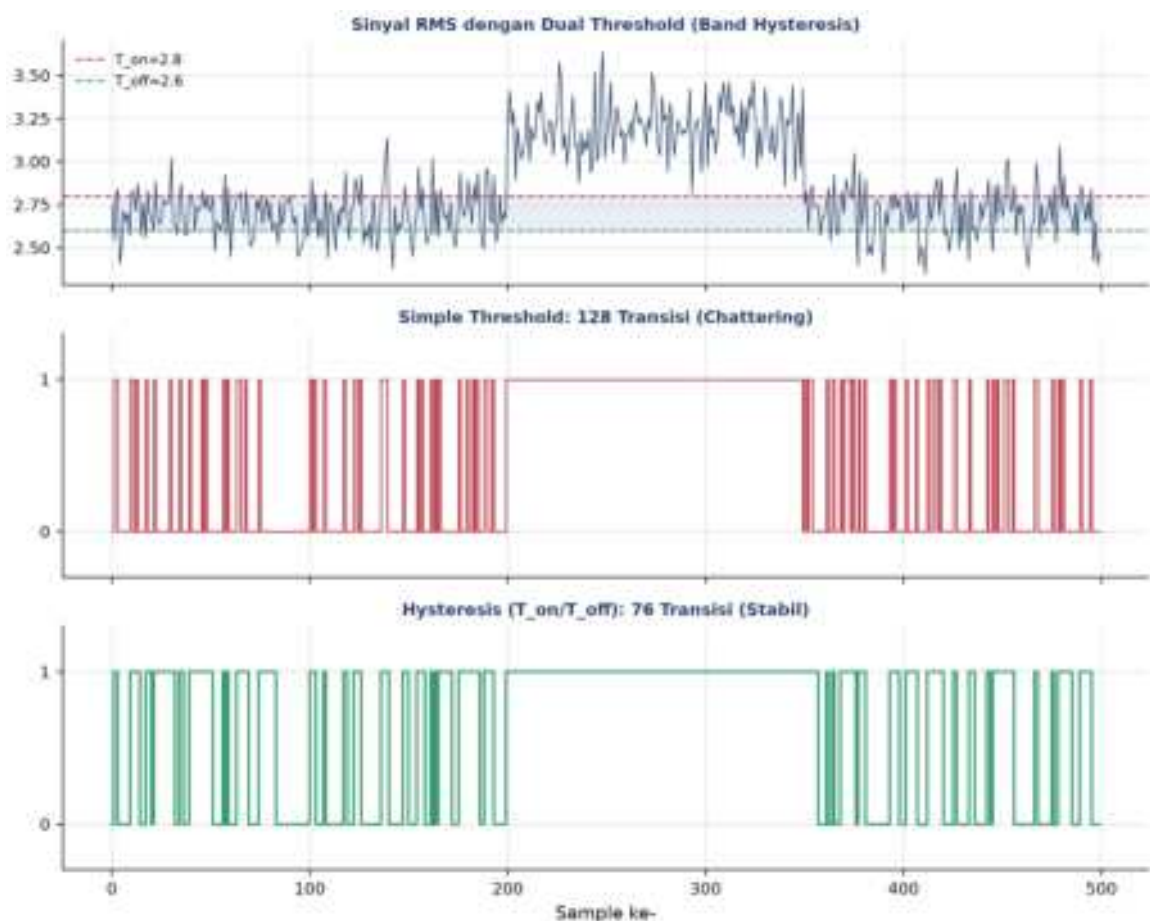
Gambar 2 mengilustrasikan bagaimana sinyal RMS yang berdegradasi secara bertahap melewati keempat zona ISO 10816-3; studi kasus pompa 50kW Class II bergerak dari 0,9 mm/s (commissioning) menjadi 5,2 mm/s (9 bulan kemudian) hingga terdiagnosis inner-race spalling pada bearing.

Multi-Feature Threshold: Selain RMS tunggal, sistem monitoring modern sering menggabungkan beberapa fitur sekaligus (RMS, peak, kurtosis>4) untuk menaikkan sensitivitas deteksi fault dini, sesuai materi Pertemuan 6.

HYSTERESIS (ANTI-CHATTERING): SCHMITT TRIGGER UNTUK THRESHOLD YANG STABIL

Hysteresis: $T_{on} > T_{off}$ Tier alarm: $T_{warn} < T_{alert} < T_{crit}$ Alarm flood prevention: ≤ 1 alarm/10 menit

Masalah Chattering & Solusi Hysteresis: Ketika sinyal berosilasi tepat di sekitar satu threshold tunggal, sistem alerting dapat memicu ratusan alert per menit (chattering), membanjiri operator dengan notifikasi palsu. Solusinya adalah hysteresis (Otto Schmitt, 1934): memakai dua threshold, T_{on} untuk memicu status ON dan $T_{off} < T_{on}$ untuk kembali ke status OFF, sehingga terbentuk zona penyangga yang mencegah osilasi di sekitar satu titik memicu transisi berulang.



Gambar 3. Perbandingan simple threshold (chattering, banyak transisi) vs hysteresis dual-threshold (stabil, transisi minimal) pada sinyal RMS yang sama.

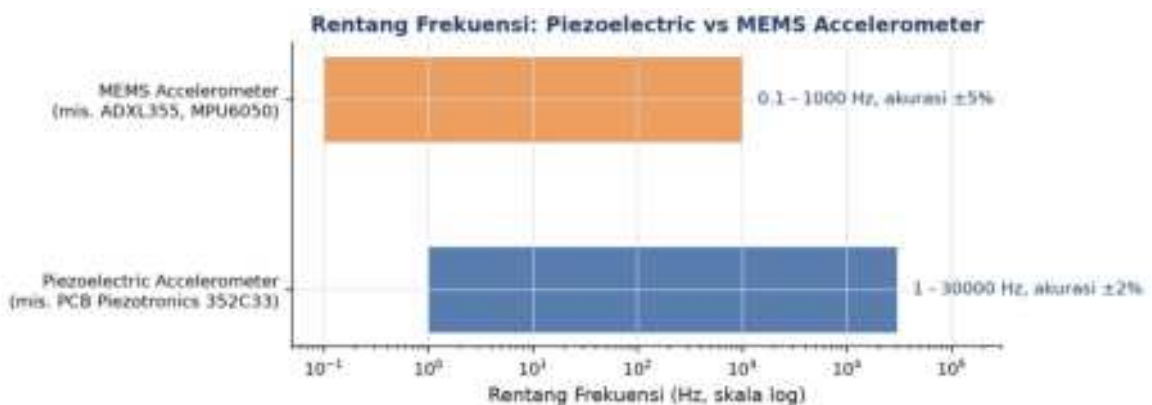
Gambar 3 membuktikan secara kuantitatif bahwa hysteresis secara drastis menurunkan jumlah transisi status dibanding simple threshold pada sinyal RMS yang identik; margin $T_{on}-T_{off}$ yang lebih besar dari standar deviasi noise sinyal menjamin stabilitas.

Menuju Tier Alarm (Pertemuan 13): Sistem monitoring produksi biasanya memakai tier alarm bertingkat (Normal, Warning, Alert, Critical) dengan aksi berbeda per tier, dan membatasi laju alarm sesuai ANSI/ISA 18.2 (maksimal 1 alarm per 10 menit per operator) untuk mencegah alarm flood. Topik ini dibahas lebih dalam di Pertemuan 13.

SENSOR SELECTION & AKUISISI DATA: HARDWARE FOUNDATION MONITORING SYSTEM

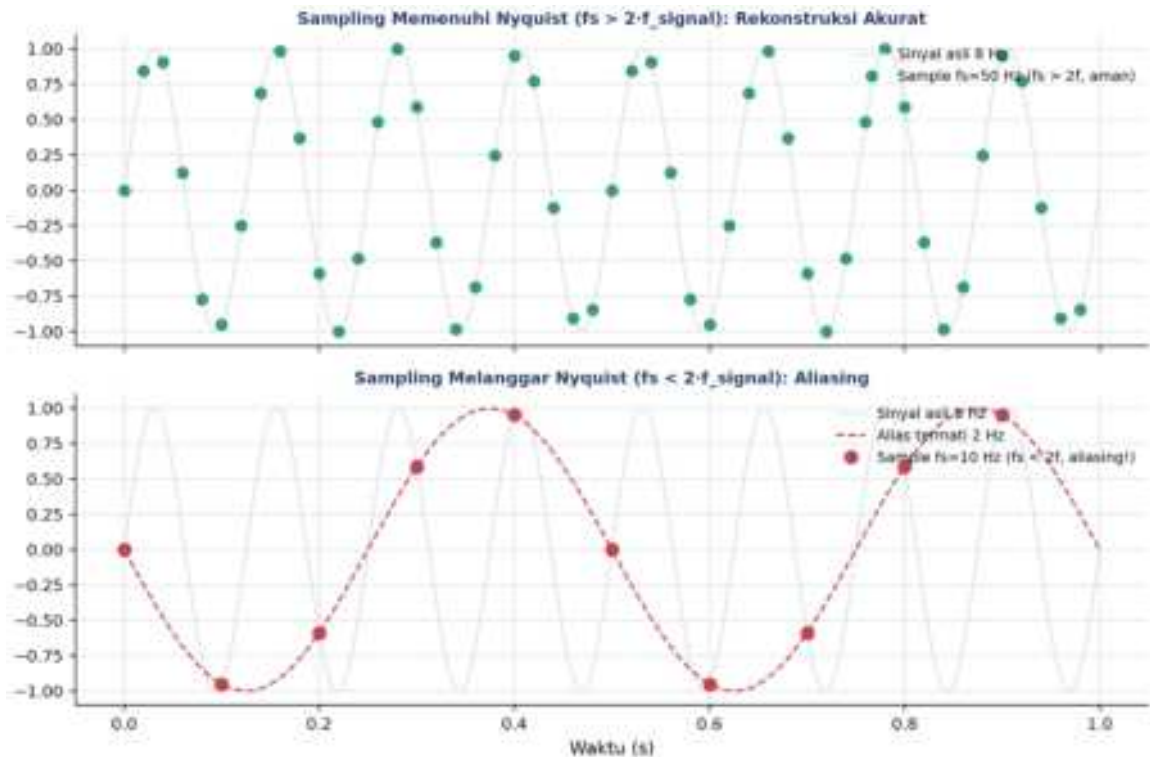
Sensor (transducer) Signal conditioning ADC resolution Sampling rate
(Nyquist): $f_s \geq 2 \cdot f_{max}$ Protokol komunikasi

- **Accelerometer Vibrasi:** IEPE (1Hz-30kHz), contoh PCB Piezotronics 352C33, akurat namun mahal; MEMS (DC-1kHz), contoh ADXL355/MPU6050, murah dan cocok untuk IoT skala besar meski akurasi lebih rendah.
- **Temperature RTD & Thermocouple:** RTD Pt100 (akurasi $\pm 0,1^\circ\text{C}$, -200 hingga 850°C) untuk pengukuran presisi; Thermocouple Type K (akurasi $\pm 1^\circ\text{C}$, -270 hingga 1372°C) untuk rentang suhu ekstrem.
- **Current Transformer (CT) Motor:** Current Transformer mengukur arus motor untuk Motor Current Signature Analysis (MCSA), formula $I_{secondary} = I_{primary}/N$.
- **Sampling Rate Nyquist Criterion:** kriteria Nyquist-Shannon: sampling rate f_s harus lebih besar dari 2 kali frekuensi maksimum sinyal ($f_s \geq 2 \cdot f_{max}$); jika dilanggar, komponen frekuensi tinggi akan "terlipat" menjadi frekuensi rendah yang salah (aliasing).



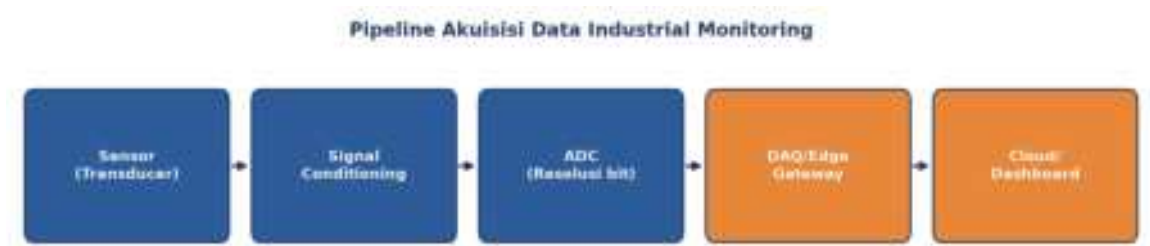
Gambar 4. Perbandingan rentang frekuensi accelerometer piezoelectric (1-30000 Hz, akurasi $\pm 2\%$) vs MEMS (0,1-1000 Hz, akurasi $\pm 5\%$).

Gambar 4 menunjukkan bahwa accelerometer piezoelectric mencakup rentang frekuensi jauh lebih lebar dibanding MEMS, menjadikannya pilihan utama untuk monitoring bearing/gear yang butuh deteksi frekuensi tinggi (BPFO/BPFI hingga puluhan kHz via envelope analysis).



Gambar 5. Ilustrasi kriteria Nyquist: sampling $f_s=50\text{Hz}$ pada sinyal 8Hz merekonstruksi sinyal dengan akurat (atas), sedangkan sampling $f_s=10\text{Hz}$ melanggar Nyquist dan menghasilkan aliasing, sinyal 8Hz "muncul" seolah 2Hz (bawah).

Gambar 5 mendemonstrasikan konsekuensi pelanggaran kriteria Nyquist: sinyal 8Hz yang di-sample pada 10Hz (kurang dari $2 \times 8 = 16\text{Hz}$) menghasilkan alias yang tampak seperti sinyal 2Hz , sebuah kesalahan diagnostik serius jika tidak diantisipasi dengan anti-aliasing filter sebelum ADC.



Gambar 6. Pipeline akuisisi data industrial monitoring: sensor, signal conditioning, ADC, DAQ/edge gateway, hingga cloud/dashboard.

Gambar 6 menunjukkan pipeline akuisisi data lengkap. DAQ hardware terbagi tiga tier: Tier 1 (lab, NI USB-6009/cDAQ), Tier 2 (industrial, Bently Nevada 3500/Emerson AMS), dan Tier 3 (hobby/IoT, Raspberry Pi+ADS1115, Arduino+MPU6050, ESP32); komunikasi data industri umumnya memakai protokol Modbus RTU/TCP, OPC UA, MQTT, atau HART.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Termostat AC (Hysteresis Klasik):** AC rumah dengan setpoint 24°C dan margin 2°C adalah contoh hysteresis klasik, mencegah kompresor menyala-mati berulang; ditemukan Albert Butz (Honeywell, 1885).
- **Indikator Bensin Mobil (Tier Alarm):** indikator bensin mobil memakai tier alarm 3 level: ¼ tank (warning), E-line (alert), buzzer merah (critical).
- **Alarm Asap di Rumah (Debouncing):** alarm asap rumah (standar UL 217/EN 14604) mensyaratkan konsentrasi asap terdeteksi kontinu minimal 30 detik sebelum alarm berbunyi, mencegah false alarm dari asap sesaat (debouncing).
- **Smartwatch Heart Rate Alarm (Multi-Threshold):** Apple Watch mendeteksi HR>120bpm selama 10 menit tanpa olahraga sebagai kondisi high, dan HR<40bpm selama 10 menit sebagai kondisi low; FDA menyetujui fitur deteksi AFib pada 2018.
- **Lampu Lalu Lintas (FSM Klasik):** lampu lalu lintas adalah FSM klasik 3-state (Hijau 30 detik, Kuning 5 detik, Merah 30 detik) dengan interlock antar-arrah.
- **DevOps Server Monitoring (Production-Grade):** Netflix, Google, dan AWS memantau CPU>80% selama 5 menit sebagai warning dan response time>5 detik sebagai critical, menggunakan tools seperti Prometheus, Grafana, Datadog, dan New Relic.

Pola Umum & Keterbatasan: Tiga elemen yang sama muncul di semua analogi: threshold dengan referensi yang jelas, filter noise (hysteresis/debouncing), dan eskalasi bertahap. Namun rule-based memiliki batas skala; ketika jumlah fitur melebihi sekitar 50, pendekatan machine learning (Pertemuan 14-15) menjadi lebih tepat.

APLIKASI RULE-BASED MONITORING DI INDUSTRI 4.0 & IOT MANUFAKTUR

- **Wind Turbine SCADA Monitoring (Vestas):** Vestas memasang 200+ sensor per turbin mengikuti standar ISO 10816-21, dengan tools OSIsoft PI System dan Bachmann Webmonitor, menurunkan MTTR sebesar 30%.

- **Gedung Tinggi Smart HVAC (Siemens Building):** Siemens Building System memantau 10.000+ titik data di gedung seperti Burj Khalifa dan 1WTC, dengan tier CO2 800/1000/1500 ppm, menghemat energi 15-25% mengikuti standar ASHRAE 90.1.
- **Power Grid Substation Monitoring (PLN, ABB):** PLN dan ABB memantau gardu induk dengan protokol IEC 61850 dan rule rate-of-change (kenaikan suhu oli >5°C/menit = critical), mendeteksi 90% gangguan sebelum blackout.
- **F1 Race Car Telemetry (Mercedes-AMG, McLaren):** Mercedes-AMG dan McLaren memantau 300+ sensor telemetry mobil F1 dengan tier suhu rem (warning 700°C, critical 850°C) dan latency di bawah 50 milidetik.

Peringatan, Jebakan Praktis Rule-Based Monitoring: Tujuh jebakan praktis rule-based monitoring di industri: threshold ditebak tanpa referensi standar; lupa menerapkan hysteresis; hanya memantau satu fitur (single-feature monitoring); tidak ada pengecekan kesehatan sensor (sensor health check); mengabaikan risiko alarm flood; tidak melakukan alarm rationalization; serta threshold statis yang tidak disesuaikan seiring waktu (threshold drift).

IMPLEMENTASI PYTHON RULE-BASED MONITORING DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada studi kasus pompa sentrifugal. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib; notebook p12_monitoring.ipynb.

```
p12_monitoring.ipynb - Cell 1
def iso10816_zone(rms_ms):
    """Klasifikasi zona ISO 10816-3 Group 2."""
    if rms_ms <= 1.4:
        return 'A', 'GOOD', '#22c55e'
    elif rms_ms <= 2.8:
        return 'B', 'ACCEPTABLE', '#00bcd4'
    elif rms_ms <= 4.3:
        return 'C', 'UNSATISFACTORY', '#f59e0b'
    else:
        return 'D', 'DANGER', '#e54899'

zone, label, color = iso10816_zone(3.2)
print(f'{zone}: {label}')
# output: C: UNSATISFACTORY
```

Gambar 7. Cuplikan fungsi `iso10816_zone()` yang mengklasifikasikan RMS velocity ke zona A-D.

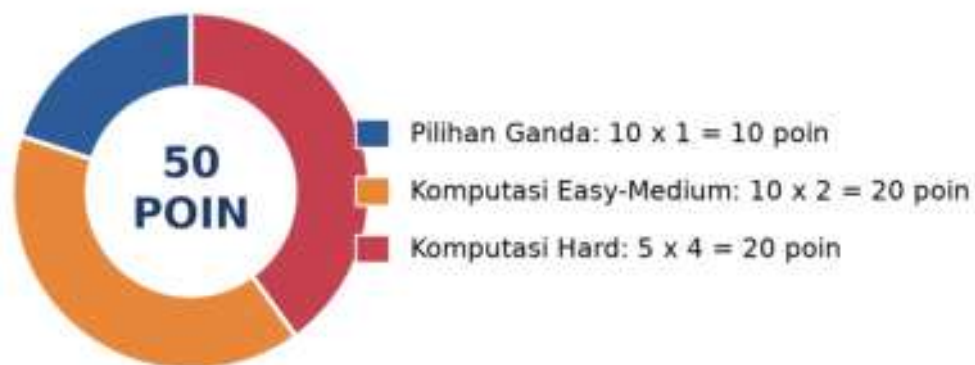
Gambar 7 menunjukkan cuplikan kode inti Cell 1.

- **Cell 1:** fungsi ``iso10816_zone(rms_mm_s)`` yang mengklasifikasikan RMS ke zona A-D; diterapkan pada sinyal degradasi 30 hari sintetis pompa 50kW, dengan deteksi first-crossing tiap batas zona.
- **Cell 2:** class ``HysteresisAlarm`` (T_{on}/T_{off}) dibandingkan dengan class ``SimpleThreshold``; pada sinyal RMS 500 sample dengan elevasi sesaat, hysteresis menekan jumlah transisi secara signifikan.
- **Cell 3:** finite state machine 5-state (NORMAL/WARNING/ALERT/CRITICAL/FAULT) dengan debouncing $N=3$ sample dan audit log; sinyal 12 jam (1440 sample) dengan 3 spike noise yang berhasil difilter oleh debouncing.
- **Cell 4:** fungsi ``run_monitoring_pipeline()`` reusable menggabungkan zona ISO, FSM dengan hysteresis dan debounce, audit log, statistik (persentase waktu per state), dan dashboard 3-panel.

TUGAS PERTEMUAN 12

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 12



Gambar 8. Distribusi 50 poin Tugas Pertemuan 12 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal $\times$ 1 poin)

1. Karakteristik utama yang membuat rule-based monitoring menjadi standar industri untuk safety-critical systems adalah...

- A. Rule-based selalu lebih akurat dari machine learning

- **B.** Transparan/auditable, deterministik, real-time (latency < 100 ms), edge-friendly, dan compliance-ready (ISO/API). Setiap keputusan bisa dijelaskan tanpa explainability tools
- **C.** Tidak butuh threshold sama sekali
- **D.** Lebih murah dari machine learning karena tidak butuh GPU

2. Standar ISO 10816-3 mengklasifikasi vibrasi mesin rotasi 15-300 kW (rigid foundation) ke 4 zona berdasarkan...

- **A.** Acceleration peak (g) di band 100 Hz-10 kHz
- **B.** RMS velocity (mm/s) di band 10 Hz-1 kHz: A (Good $\leq 1,4$), B (Acceptable $\leq 2,8$), C (Unsatisfactory $\leq 4,5$), D (Danger $> 4,5$)
- **C.** Displacement micrometer di band < 10 Hz
- **D.** Hanya temperature bearing

3. Tujuan hysteresis dalam threshold-based alerting adalah...

- **A.** Membuat threshold tunggal lebih ketat
- **B.** Hindari chattering (alarm berflicker rapid) saat sinyal di sekitar threshold, dengan menggunakan dua threshold: T_on (ke ON) lebih besar dari T_off (ke OFF). Margin biasanya 5-15% dari setpoint
- **C.** Mengganti threshold dengan ML model
- **D.** Menambah jumlah sensor

4. Tier alarm (multi-level escalation) lebih baik daripada binary OK/FAIL karena...

- **A.** Tier alarm lebih sederhana implementasinya
- **B.** Menyediakan gradual escalation dengan aksi berbeda per tier, WARNING (log+email), ALERT (SMS supervisor), CRITICAL (auto-shutdown). Operator bisa proactive sebelum critical
- **C.** Tier alarm tidak butuh threshold
- **D.** Tier alarm hanya pakai 1 sensor

5. Debouncing N=3 consecutive samples dipakai dalam FSM monitoring untuk...

- **A.** Memperlambat sistem secara umum
- **B.** Filter noise transient, butuh 3 sample berturut-turut di kondisi alert sebelum transition state. Single spike dari noise tidak trigger transition. Trade-off: latency vs false-alarm rate
- **C.** Mempercepat sampling rate sensor
- **D.** Mengganti hysteresis

6. Dalam finite state machine (FSM), perbedaan utama antara Mealy dan Moore machine adalah...

- **A.** Mealy lebih cepat dari Moore

- **B.** Moore: output bergantung HANYA pada state saat ini ($f(s)$). Mealy: output bergantung pada state + input ($f(s, \sigma)$). Mealy lebih ekspresif, Moore lebih sederhana
- **C.** Moore tidak bisa pakai Python
- **D.** Mealy hanya untuk hardware

7. Heartbeat/watchdog timer dalam monitoring system dipakai untuk...

- **A.** Mengukur detak jantung manusia
- **B.** Detect sensor failure/disconnect: kalau tidak ada sample baru dalam timeout period (misal $2 \times$ sample period), pindah ke FAULT state. Output sensor frozen tidak dibiarkan dianggap "NORMAL"
- **C.** Mempercepat sampling rate
- **D.** Hanya untuk Apple Watch

8. ANSI/ISA 18.2 (Alarm Management Standard) menetapkan target alarm rate untuk operator monitoring sebesar...

- **A.** ≥ 100 alarm/menit untuk awareness penuh
- **B.** ≤ 1 alarm per 10 menit per operator (steady state); ≤ 10 alarm dalam 10 menit pertama post-disturbance. Lebih dari itu = alarm flood, operator alarm fatigue
- **C.** Tidak ada batasan
- **D.** 1000 alarm per shift normal

9. Audit log dari setiap state transition dalam FSM monitoring penting untuk...

- **A.** Hanya untuk debug awal, tidak perlu di production
- **B.** Post-mortem analysis (kenapa shutdown?), compliance audit (ISO/API), trend analysis (frequency naik = degradasi), dan ML training data (Pertemuan 14). Source of truth untuk monitoring system
- **C.** Hanya untuk billing
- **D.** Tidak perlu jika pakai dashboard real-time

10. Stack industrial monitoring production yang umum dipakai untuk implementasi rule-based + dashboard di pabrik adalah...

- **A.** Hanya Excel + email manual
- **B.** MQTT (sensor pub/sub) -> TimescaleDB/InfluxDB (time-series storage) -> Streamlit/Grafana (dashboard real-time). Open-source, edge-friendly, scalable
- **C.** HTTP polling tiap 10 menit
- **D.** Tidak ada standar, tiap pabrik berbeda

Bagian B: Komputasi Easy-Medium (10 soal $\times$ 2 poin)

Catatan: ISO 10816-3 Group 2, rigid foundation: Zone A $\leq 1,4$ mm/s, Zone B 1,4-2,8, Zone C 2,8-4,5, Zone D $> 4,5$. Hysteresis: $T_{off} = T_{on} - \text{margin}$. Tier alarm: WARNING < ALERT < CRITICAL. Debouncing: N consecutive sample untuk transisi.

C1. Klasifikasi ISO 10816-3 zone untuk RMS=3,2 mm/s. Print integer kode zone: 0=A, 1=B, 2=C, 3=D.

- 💡 **HINT:** Threshold ISO 10816-3: $A \leq 1,4$, $B \leq 2,8$, $C \leq 4,5$, sisanya D. Bandingkan RMS terhadap batas-batas ini secara berurutan untuk menentukan kode zone (0-3).

C2. Untuk threshold $T_{on}=2,8$ dan margin hysteresis 0,2, hitung T_{off} (threshold untuk transisi kembali ke OFF). Formula: $T_{off}=T_{on}-margin$. Print nilainya.

- 💡 **HINT:** $T_{off} = T_{on} - margin$. Sinyal harus turun di bawah T_{off} untuk kembali OFF, mencegah chattering di sekitar threshold. Substitusikan nilai dari soal.

C3. Hitung jumlah sample per jam untuk sensor dengan sampling period 30 detik. Print integer.

- 💡 **HINT:** Jumlah sample per jam = 3600 detik dibagi sampling period (detik). Substitusikan period dari soal.

C4. Untuk debounce $N=5$ consecutive samples dengan sampling period 1 detik, hitung minimum latency deteksi (detik) sebelum transition state. Print nilainya.

- 💡 **HINT:** Latency deteksi = $N \times \text{sampling period}$. Trade-off: N besar lebih robust terhadap noise tapi latency tinggi. Substitusikan N dan period dari soal.

C5. ANSI/ISA 18.2 menetapkan max 1 alarm/10 menit per operator. Hitung max alarm/jam (steady state) yang masih compliant. Print nilainya.

- 💡 **HINT:** Max alarm/jam = 60 menit dibagi (menit per alarm yang diizinkan). Melebihi ini menyebabkan alarm flood. Substitusikan batas dari soal.

C6. Untuk RMS=[1.0, 1.5, 2.0, 2.9, 3.5, 4.2, 4.7, 5.0], hitung jumlah sample yang masuk Zone D (Danger, $>4,5$). Print integer.

- 💡 **HINT:** Buat mask boolean $rms > 4,5$ (batas Zone D) lalu jumlahkan dengan `.sum()` untuk menghitung sample yang masuk.

C7. Hitung persentase uptime (persen time di Zone A "Good") dari 1000 sample dimana 850 sample punya $RMS \leq 1,4$. Print persentase desimal (misal 85.0).

- 💡 **HINT:** $\text{Uptime\%} = (\text{jumlah sample di Zone A} / \text{total sample}) \times 100$. Substitusikan angka dari soal.

C8. Untuk monitoring 24 jam dengan sampling period 15 detik, hitung total sample. Print integer.

- 💡 **HINT:** Total sample = total durasi dalam detik dibagi sampling period; konversi 24 jam ke detik ($\times 3600$) lebih dulu. Substitusikan period dari soal.

C9. Hysteresis state simulation: dengan $T_{on}=2,8$ dan $T_{off}=2,6$, RMS sequence=[2.5, 2.7, 2.9, 2.7, 2.5, 2.7, 2.9]. Awal state OFF. Hitung jumlah transitions total. Print integer.

- 💡 **HINT:** Telusuri sequence sample per sample: state berubah ke ON saat melewati T_{on} dan ke OFF saat turun di bawah T_{off} ; hitung berapa kali state berubah.

C10. Hitung MTTR improvement (persen) kalau MTTR sebelum monitoring=8 jam, setelah monitoring=5,6 jam. Formula: $(MTTR_before - MTTR_after) / MTTR_before \times 100$. Print persentase.

- 💡 **HINT:** $MTTR\ improvement\% = (MTTR_before - MTTR_after) \text{ dibagi } MTTR_before \times 100$. Substitusikan nilai dari soal.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil salah/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

C11 (Hard). HysteresisAlarm Class, Anti-Chattering Demo: implementasi class HysteresisAlarm dengan $T_{on}=2,8$, $T_{off}=2,6$. Generate sinyal RMS noisy: `np.random.seed(42); rms=2.7+0.15*np.random.randn(500)`. Apply class ke seluruh sinyal. Print jumlah transitions (state change).

- 💡 **HINT:** Implementasikan class dengan dua threshold (T_{on}/T_{off}); margin hysteresis yang lebih besar dari std noise membuat jumlah transition kecil. Hitung berapa kali state berubah di seluruh sinyal.

C12 (Hard). FSM Monitoring Pipeline, RMS Streaming dengan Tier Alarm: bangun FSM dengan 4 state (NORMAL, WARNING, ALERT, CRITICAL), thresholds (1.4, 2.8, 4.5), debounce $N=3$. Apply ke sinyal RMS sintesis: `np.random.seed(42); rms=0.7+np.linspace(0,4.0,1000)+0.2*np.random.randn(1000)`. Print jumlah state transitions total.

- 💡 **HINT:** Bangun FSM 4-state dengan thresholds dan debounce N ; sinyal yang naik bertahap akan melewati tiap tier. Hitung jumlah transition state di seluruh sinyal.

C13 (Hard). Multi-Feature Threshold Combination: 100 sample dengan fitur RMS, peak, kurtosis. Generate: `np.random.seed(42); rms=np.random.uniform(0.5,4.0,100); peak=rms*3+np.random.randn(100); kurt=3+np.random.uniform(-1,4,100)`. Hitung jumlah sample yang trigger ALERT dengan rule: $(rms>2.8) \text{ OR } (peak>8) \text{ OR } (kurt>4)$. Print integer.

- 💡 **HINT:** Gabungkan tiga kondisi dengan operator OR, lalu jumlahkan dengan `.sum()`; sample trigger jika melanggar setidaknya satu threshold.

C14 (Hard). Persistence Timer/Escalation Logic: untuk sinyal yang tetap di Zone B selama 12 menit (sample period 30 detik), dengan eskalasi rule "Warning persist > 5 menit -> Alert", hitung kapan eskalasi ke ALERT terjadi (sample index ke berapa). Print integer.

- 💡 **HINT:** Konversi durasi persist (menit) ke jumlah sample = (menit x 60) dibagi sampling period; eskalasi terjadi saat persist mencapai ambang itu.

C15 (Hard). Engineering Pipeline, End-to-End Monitoring 24 jam: simulasi 24 jam (sampling 30 detik), generate sinyal RMS dengan `np.random.seed(42); n=2880; rms=0.7+np.linspace(0,3.5,n)+0.2*np.random.randn(n)`. Apply ISO 10816 zone classification + FSM tier alarm. Print persentase waktu di Zone D (Danger).

- 💡 **HINT:** Klasifikasikan tiap sample ke zone-nya, lalu hitung persentase Zone D = (jumlah sample Zone D dibagi total sample) x 100.

DAFTAR PUSTAKA

- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>
- Mohsin, I., He, K., Li, Z., Zhang, F., & Du, R. (2020). Optimization of the polishing efficiency and torque by using Taguchi method and ANOVA in robotic polishing. *Applied Sciences*, 10(3), 824. <https://doi.org/10.3390/app10030824>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Otomatisasi Monitoring: Tier Alarm,

Abstrak

Pertemuan ini memperluas monitoring rule-based (Pertemuan 12) menjadi arsitektur alarm management

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

Sub - CPMK

Sub-CPMK 4.3 – Mampu mengevaluasi dampak teknologi 4.0 terhadap kinerja dan kualitas

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-12.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan 12 membahas fondasi monitoring: sensor, akuisisi, threshold ISO 10816, dan hysteresis. Namun sistem monitoring produksi sungguhan menghadapi tiga masalah tambahan: keputusan biner OK/FAIL terlalu kasar, sistem tanpa memori tidak bisa membedakan spike sesaat dari degradasi nyata, dan terlalu banyak alarm justru membanjiri operator (alarm flood). Pertemuan ketiga belas ini membahas solusinya: tier alarm bertingkat, finite state machine (FSM) dengan memori, dan standar ANSI/ISA 18.2 untuk manajemen alarm.



Gambar 1. Posisi Pertemuan 13 (Tier Alarm, FSM & ANSI/ISA 18.2) dalam alur mata kuliah, melengkapi monitoring Pertemuan 12 sebelum masuk ke machine learning Pertemuan 14-15.

Gambar 1 menunjukkan posisi Pertemuan 13 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 4.3:** Mampu mengevaluasi dampak teknologi 4.0 terhadap kinerja dan kualitas, yaitu mengevaluasi efektivitas arsitektur tier alarm dan FSM terhadap metrik kinerja sistem (availability, MTBF/MTTR, OEE) dan kualitas manajemen alarm (ANSI/ISA 18.2) (CPMK 4).
- Setelah pertemuan ini mahasiswa dapat: merancang arsitektur tier alarm 4-level; membangun finite state machine dengan debouncing; menerapkan prinsip ANSI/ISA 18.2 untuk mencegah alarm flood; serta menghitung metrik keandalan sistem (availability, OEE, redundansi).

TIER ALARM ARCHITECTURE: 4-LEVEL ESCALATION DARI NORMAL HINGGA CRITICAL

4 tier (N/W/A/C) Visual color & sound Response SLA per tier Auto-escalation rules Audit log dengan timestamp

Empat Level Eskalasi: Alih-alih keputusan biner, sistem tier alarm mendefinisikan 4 level: Normal (tidak ada aksi), Warning/Low (log dan email), Alert/Medium (SMS supervisor dan work order), dan Critical/High (auto-shutdown, page operator, eskalasi manajemen).



Gambar 2. Arsitektur tier alarm 4-level: Normal, Warning, Alert, Critical, dengan threshold ISO 10816-3 dan aksi operator masing-masing.

Gambar 2 menunjukkan keempat tier beserta threshold dan aksi operator yang berbeda di tiap level, konsisten dengan zona ISO 10816-3 yang dibahas di Pertemuan 12.

Tabel 1. Tier alarm, threshold, warna HMI, aksi operator, dan aturan eskalasi berbasis persistence timer.

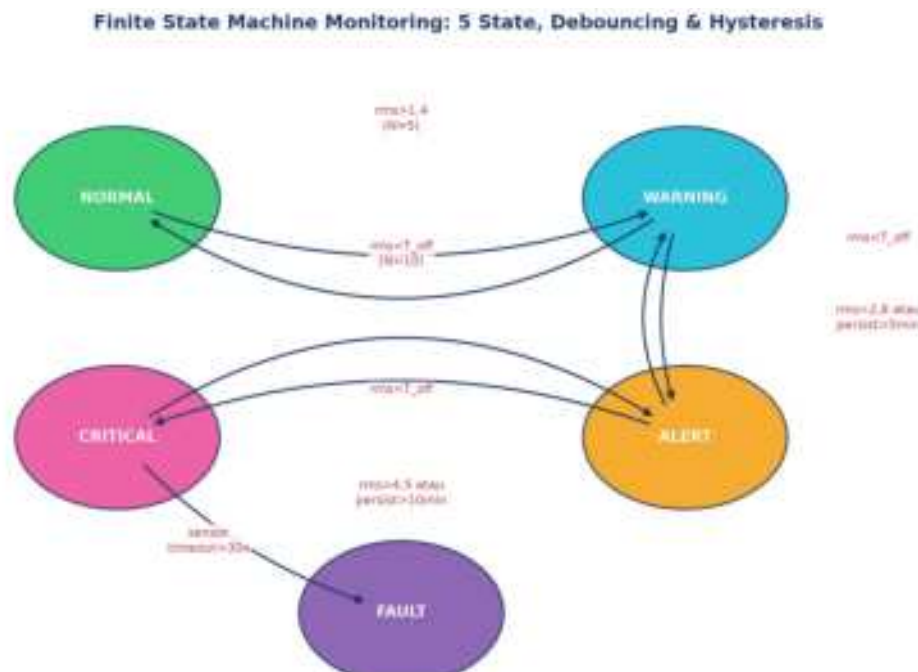
Tier	Threshold (RMS mm/s)	Color (HMI)	Action Operator	Eskalasi (Persist)
Normal	<= 1,4 (Zone A)	Hijau	Tidak ada	-
Warning	1,4 - 2,8 (Zone B)	Kuning	Log + email	5 menit -> Alert
Alert	2,8 - 4,5 (Zone C)	Oranye	SMS supervisor	10 menit -> Critical
Critical	> 4,5 (Zone D)	Merah	Auto-shutdown + page	Immediate

Tabel 1 merangkum keempat tier alarm beserta threshold, warna HMI standar, aksi operator, dan aturan eskalasi berbasis persistence timer.

Contoh: Chattering Demo. Studi kasus: $RMS=2,79$ mm/s berosilasi tepat di sekitar $T_{warn}=2,8$ dengan noise $\pm 0,05$ menghasilkan hingga 600 alert/menit tanpa hysteresis; dengan hysteresis ($T_{on}=2,8$, $T_{off}=2,6$, margin 0,2), chattering ini hilang sepenuhnya.

FINITE STATE MACHINE (FSM): MEMORI & TRANSITION LOGIC

Formalisme FSM: FSM diformalkan sebagai 5-tuple ($S, s_0, \text{Sigma}, \text{delta}, F$): $S=\{\text{NORMAL}, \text{WARNING}, \text{ALERT}, \text{CRITICAL}, \text{FAULT}\}$, $s_0=\text{NORMAL}$, Sigma =pembacaan sensor dan event timer, delta =aturan transisi. Moore machine menghasilkan output hanya berdasarkan state saat ini $f(s)$; Mealy machine menghasilkan output berdasarkan state dan input $f(s, \text{sigma})$, lebih ekspresif tetapi lebih kompleks.



Gambar 3. Diagram state FSM monitoring: 5 state (NORMAL, WARNING, ALERT, CRITICAL, FAULT) dengan transisi berbasis threshold, hysteresis, debouncing, dan sensor timeout.

Gambar 3 menunjukkan diagram lengkap FSM monitoring dengan 5 state; transisi naik (eskalasi) membutuhkan threshold terlampaui ATAU persistence timer terlampaui, sedangkan transisi turun (recovery) membutuhkan sinyal berada di bawah T_{off} (hysteresis) selama beberapa sample berturut-turut.

Moore vs Mealy dalam Implementasi Praktis: Pemilihan antara Moore dan Mealy machine untuk implementasi FSM monitoring berpengaruh pada bagaimana aksi (seperti pengiriman notifikasi) dipetakan terhadap state: pada Moore machine, aksi hanya bergantung pada state tujuan sehingga transisi WARNING → ALERT dan transisi langsung NORMAL → ALERT (melewati WARNING akibat kenaikan mendadak) akan memicu aksi

ALERT yang identik. Pada Mealy machine, aksi dapat dibedakan berdasarkan transisi spesifik yang terjadi, memungkinkan sistem mengirim notifikasi berbeda untuk "eskalasi bertahap terdeteksi dini" versus "lonjakan mendadak terdeteksi", informasi diagnostik tambahan yang berharga bagi tim maintenance dalam menentukan urgensi respons.

Tabel 2. Lima transisi FSM monitoring: kondisi pemicu, aksi on_entry, dan jumlah debounce sample yang dibutuhkan.

Transition	Trigger Condition	Action (on_entry)	Debounce N
NORMAL -> WARNING	rms > T_warn (1,4)	Log + email	5 sample
WARNING -> ALERT	rms > T_alert (2,8) atau persist > 5 menit	SMS supervisor	3 sample
ALERT -> CRITICAL	rms > T_crit (4,5) atau persist > 10 menit	Auto-shutdown + page	2 sample
CRITICAL -> FAULT	sensor timeout > 30 detik	Mark sensor as faulty	1 timeout
Any -> NORMAL	rms < T_off (hysteresis) dan persist > 1 menit	Log recovery	10 sample

Tabel 2 merangkum lima transisi utama FSM beserta kondisi pemicu, aksi yang dijalankan, dan jumlah sample debounce yang dibutuhkan sebelum transisi disahkan.

Peringatan, Trap FSM & Debouncing: Tujuh jebakan FSM dan debouncing yang sering diabaikan: state explosion (terlalu banyak state membuat sistem sulit dianalisis); race condition antar-thread yang membaca/menulis state bersamaan; stuck state karena bug logic; debounce N yang terlalu besar menyebabkan deteksi lambat; heartbeat/watchdog timer yang terlupa (sensor mati dianggap tetap "NORMAL"); audit log yang membengkak tanpa retention policy; serta tidak adanya diagram state formal sebagai dokumentasi.

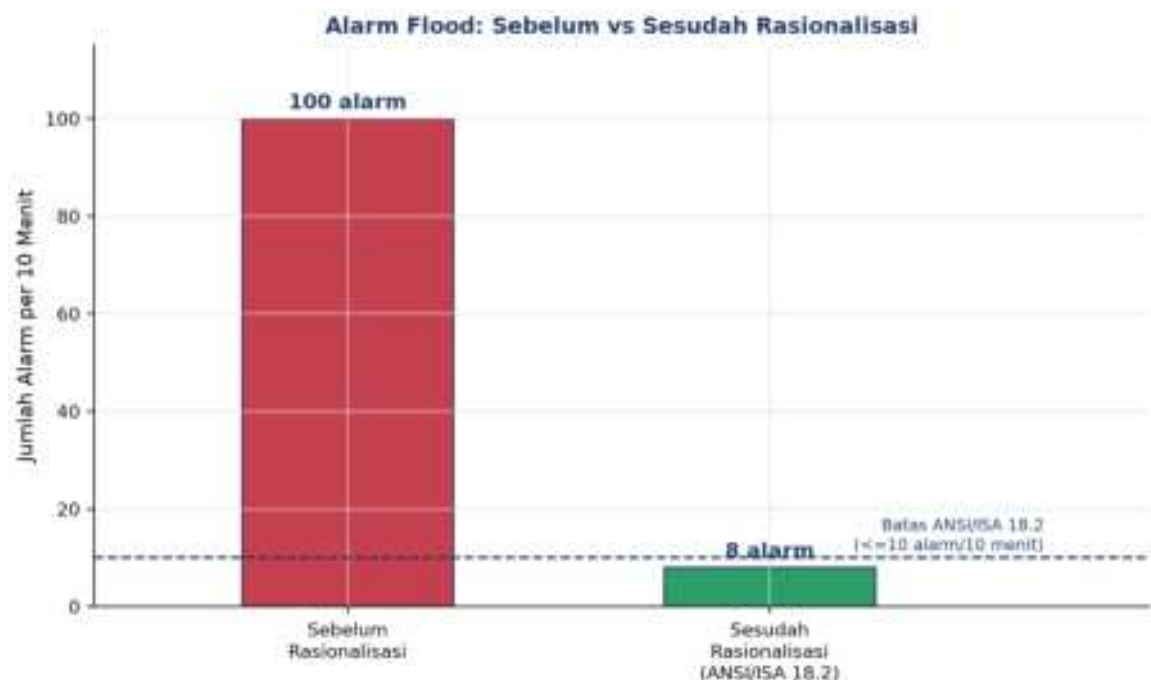
ANSI/ISA 18.2: ALARM MANAGEMENT STANDARD

Latar Belakang & Lifecycle: ANSI/ISA-18.2-2016 mendefinisikan lifecycle manajemen alarm 7 fase, dari identifikasi hingga audit berkala. Standar ini lahir sebagian dari pembelajaran insiden industri seperti ledakan kilang BP Texas City (2005) yang menewaskan 15 orang, di mana alarm flood berkontribusi pada kegagalan operator merespons kondisi kritis tepat waktu.

- **Alarm Rationalization:** memastikan setiap alarm unik, benar-benar memerlukan respons operator, serta memiliki prioritas dan setpoint yang terdokumentasi; alarm tanpa aksi jelas dihapus dari sistem.

- **Alarm Shelving:** menunda/menyembunyikan sementara alarm yang sudah diketahui (operator-initiated, auto-unshelve setelah timeout) agar tidak menambah beban alarm flood.
- **Suppression Rules (Parent-Child):** menekan alarm turunan (child) ketika alarm induk (parent) sudah aktif, karena keduanya disebabkan oleh akar masalah yang sama.

Target Alarm Rate: Target ANSI/ISA 18.2: kondisi steady-state maksimal 1 alarm per 10 menit per operator; dalam 10 menit pertama pasca-gangguan, maksimal 10 alarm masih dapat diterima. Melebihi ini disebut alarm flood, memicu alarm fatigue yang membuat operator cenderung mengabaikan semua alarm, termasuk yang genuinely kritis.



Gambar 4. Perbandingan jumlah alarm per 10 menit sebelum dan sesudah alarm rationalization; batas ANSI/ISA 18.2 adalah 10 alarm per 10 menit.

Gambar 4 menunjukkan dampak alarm rationalization: dari 100 alarm per 10 menit (jauh melampaui batas alarm flood) menjadi 8 alarm per 10 menit (di bawah batas ANSI/ISA 18.2), tercapai lewat penghapusan alarm duplikat dan penerapan suppression rules.

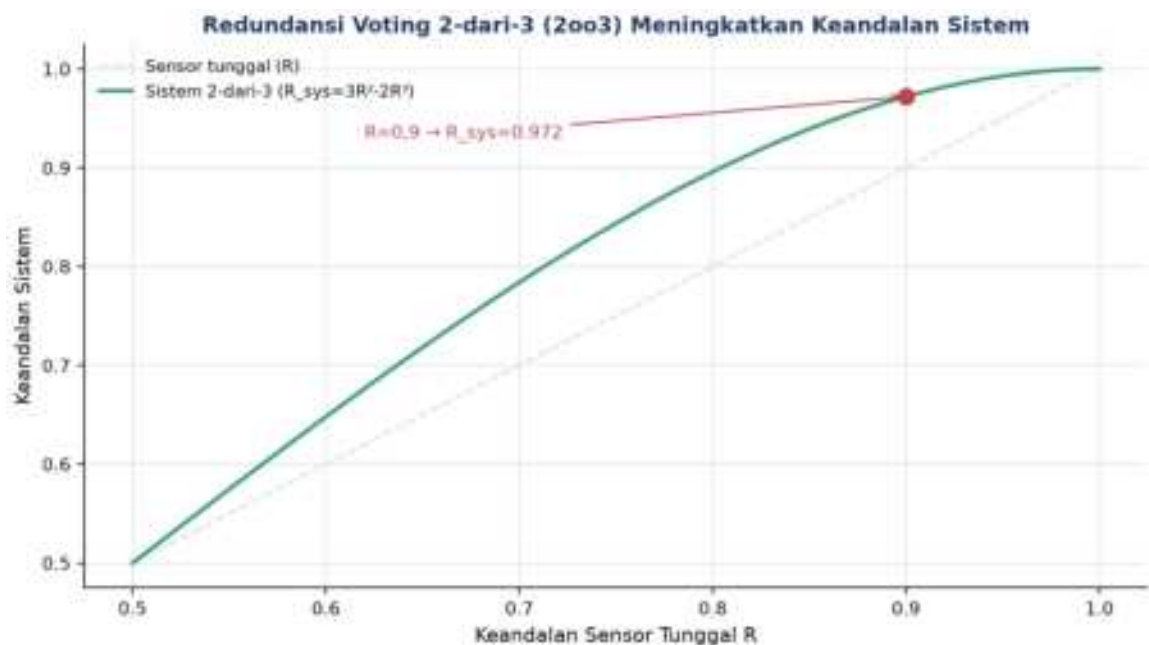
METRIK KEANDALAN SISTEM: AVAILABILITY, OEE & REDUNDANSI

$$\text{Availability} = \text{MTBF} / (\text{MTBF} + \text{MTTR}) \quad \text{MTBF} = \text{total jam operasi} / \text{jumlah kegagalan}$$

$$\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality} \quad 2003: R_{\text{sys}} = 3R^2 - 2R^3$$

Availability & MTBF/MTTR: Availability (ketersediaan) mengukur proporsi waktu mesin dapat beroperasi normal, dihitung dari Mean Time Between Failures (MTBF) dan Mean Time To Repair (MTTR).

Contoh Perhitungan Availability: Sebagai contoh perhitungan konkret: mesin dengan MTBF=720 jam (30 hari operasi rata-rata antar kegagalan) dan MTTR=8 jam (waktu perbaikan rata-rata) menghasilkan $\text{Availability} = 720 / (720 + 8) = 98,9\%$, angka yang tampak tinggi namun setara dengan sekitar 96 jam downtime per tahun, cukup signifikan untuk lini produksi kontinu bernilai tinggi. Menurunkan MTTR (respons perbaikan lebih cepat, biasanya via monitoring kondisi yang memberi peringatan dini) umumnya jauh lebih murah dan lebih cepat diimplementasikan dibanding menaikkan MTBF (yang membutuhkan overhaul desain atau penggantian komponen berkualitas lebih tinggi).



Gambar 5. Keandalan sistem redundan 2-dari-3 (2oo3) dibanding sensor tunggal: $R_{\text{sys}} = 3R^2 - 2R^3$ selalu lebih tinggi dari R untuk $R > 0,5$.

Gambar 5 menunjukkan bagaimana skema voting 2-dari-3 (sistem trip hanya bila minimal 2 dari 3 sensor sepakat) meningkatkan keandalan sistem secara signifikan dibanding mengandalkan satu sensor saja; untuk keandalan sensor tunggal $R=0,9$, keandalan sistem 2oo3 naik menjadi 0,972, sekaligus toleran terhadap kegagalan 1 sensor tanpa menyebabkan false trip.

Mengapa OEE sebagai Metrik Komposit: OEE dipilih sebagai metrik komposit tunggal karena sifat perkaliannya memaksa tim maintenance mengoptimalkan seluruh rantai produktivitas secara seimbang, bukan hanya salah satu aspek. Pabrik yang hanya berfokus meningkatkan Availability (misalnya lewat preventive maintenance agresif) tanpa

memperhatikan Performance (kecepatan aktual dibanding kecepatan ideal) atau Quality (proporsi produk sesuai standar) tetap akan memiliki OEE rendah meski mesinnya jarang berhenti, karena downtime bukan satu-satunya sumber kerugian produktivitas.



Gambar 6. Overall Equipment Effectiveness (OEE) sebagai perkalian tiga faktor: Availability, Performance, dan Quality.

Gambar 6 mengilustrasikan bagaimana $OEE = Availability \times Performance \times Quality$; meskipun setiap faktor individual terlihat tinggi (90%, 95%, 99%), OEE gabungan turun menjadi 84,6% karena sifat perkalian, menegaskan pentingnya optimasi ketiga faktor secara bersamaan bukan hanya salah satu.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Termostat AC (Hysteresis Klasik):** setpoint 24°C dengan hysteresis 1°C mencegah kompresor AC menyala-mati berulang; mengacu pada prinsip Schmitt trigger yang sama dengan Pertemuan 12.
- **Indikator Bensin Mobil (Tier Alarm):** indikator bensin mobil: $\frac{1}{4}$ tank (warning), E-line dan ikon pompa (alert, sekitar 10km), buzzer dan lampu merah (critical, sekitar 5km); mobil modern seperti Tesla memakai FSM untuk logika ini.
- **Alarm Asap di Rumah (Debouncing):** standar UL 217/EN 14604 mensyaratkan konsentrasi asap terdeteksi kontinu minimal 30 detik (debouncing) sebelum alarm berbunyi, plus bunyi "beep" heartbeat baterai lemah setiap 30 detik.

- **Smartwatch Heart Rate Alarm (Multi-Threshold):** Apple Watch dan Fitbit mendeteksi HR>120bpm tanpa olahraga selama 10 menit (high) dan HR<40bpm selama 10 menit (low); FDA menyetujui deteksi AFib dengan klaim false-positive di bawah 0,5%.
- **Lampu Lalu Lintas (FSM Klasik):** lampu lalu lintas 3-state (Hijau 30 detik, Kuning 5 detik, Merah 30 detik) dengan interlock lintas-arah adalah FSM klasik paling sederhana.
- **DevOps Server Monitoring (Production-Grade):** Netflix, Google, dan AWS memakai tier CPU>80% selama 5 menit (warning ke PagerDuty/Slack), CPU>95% selama 2 menit (alert ke on-call), dan response time>5 detik (critical, auto-failover), dengan tools Prometheus+Alertmanager, Grafana, Datadog, dan New Relic.

Pola Umum & Keterbatasan: Tiga elemen yang sama muncul di semua analogi: threshold dengan referensi jelas, filter noise (hysteresis/debouncing), dan eskalasi bertahap. Rule-based memiliki batas skala; ketika kompleksitas sistem melebihi puluhan fitur atau ratusan mode kegagalan, pendekatan machine learning (Pertemuan 14-15) menjadi lebih tepat.

APLIKASI RULE-BASED MONITORING DI INDUSTRI 4.0 & IOT MANUFAKTUR

- **Wind Turbine SCADA Monitoring (Vestas):** 200+ sensor per turbin mengikuti standar ISO 10816-21, memakai OSIsoft PI System dan Bachmann Webmonitor, menurunkan MTTR sebesar 30%.
- **Gedung Tinggi Smart HVAC (Siemens Building):** 10.000+ titik data pada gedung seperti Burj Khalifa dan 1WTC, tier CO2 800/1000/1500 ppm dengan hysteresis 100ppm, menghemat energi 15-25% mengikuti standar ASHRAE 90.1 dan EN 15251.
- **Power Grid Substation Monitoring (PLN, ABB):** gardu induk 150kV/500kV dengan protokol IEC 61850, rule rate-of-change (kenaikan suhu oli >5°C/menit=critical), mendeteksi 90% gangguan sebelum blackout; vendor ABB Relion, Siemens Reyrolle, Schneider MiCOM.
- **F1 Race Car Telemetry (Mercedes-AMG, McLaren):** 300+ sensor telemetri dengan latency di bawah 50 milidetik, tier suhu rem (warning 700°C, critical 850°C), mengikuti regulasi retensi data FIA.

IMPLEMENTASI PYTHON RULE-BASED MONITORING DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada studi kasus pompa sentrifugal. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib; notebook p13_tier_alarm_fsm.ipynb.

p13_tier_alarm_fsm.ipynb — Cell 3 (MonitoringFSM)

```
class MonitoringFSM:
    STATES = ['NORMAL', 'WARNING', 'ALERT',
              'CRITICAL', 'FAULT']
    THRESHOLDS = {'WARNING':1.4, 'ALERT':2.8,
                  'CRITICAL':4.5}
    HYSTERESIS = 0.2
    DEBOUNCE_N = 3

    def update(self, x, ts=None):
        target = self._target_state(x)
        # debounced up/down counter...
        if self.up_counter == self.DEBOUNCE_N:
            self._transition(target, x, ts)
```

Gambar 7. Cuplikan class MonitoringFSM dengan definisi state, threshold, hysteresis, dan debounce.

Gambar 7 menunjukkan cuplikan kode inti class `MonitoringFSM`.

- **Cell 1:** fungsi `iso10816\_zone()` untuk klasifikasi zona diterapkan pada sinyal degradasi 30 hari sintetis pompa 50kW.
- **Cell 2:** class `HysteresisAlarm` (T_{on}/T_{off}) dibandingkan `SimpleThreshold` pada sinyal RMS 500 sample; hysteresis menekan chattering secara signifikan.
- **Cell 3:** class `MonitoringFSM` lengkap: 5 state, threshold {WARNING:1,4; ALERT:2,8; CRITICAL:4,5}, hysteresis 0,2, debounce N=3, audit log ke pandas DataFrame; diuji pada sinyal 12 jam (1440 sample) dengan 3 spike noise yang berhasil difilter debouncing.
- **Cell 4:** fungsi `run\_monitoring\_pipeline()` reusable menggabungkan klasifikasi zona, FSM, audit log, statistik persentase waktu per state, dan dashboard 3-panel; didemonstrasikan pada simulasi 24 jam (2880 sample).

Fungsi `run\_monitoring\_pipeline()` pada Cell 4 dirancang reusable agar dapat langsung diadaptasi ke sensor lain hanya dengan mengganti parameter threshold dan hysteresis, tanpa perlu menulis ulang logika FSM dan audit log dari nol, mencerminkan praktik rekayasa perangkat lunak yang baik untuk sistem monitoring produksi yang harus dipasang pada puluhan hingga ratusan titik sensor berbeda.

TUGAS PERTEMUAN 13

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 13



Gambar 8. Distribusi 50 poin Tugas Pertemuan 13 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Dalam lifecycle ANSI/ISA 18.2, tujuan tahap alarm rationalization adalah...

- A. Menambah sebanyak mungkin alarm agar operator selalu waspada
- B. Memastikan setiap alarm unik, benar-benar perlu respons operator, serta punya prioritas dan setpoint terdokumentasi; alarm tanpa aksi jelas dihapus
- C. Mengganti semua alarm dengan model machine learning
- D. Menonaktifkan seluruh alarm low-priority secara permanen

2. Kondisi alarm flood menurut ANSI/ISA 18.2 terjadi ketika...

- A. Operator menerima kurang dari 1 alarm per jam
- B. Semua alarm berprioritas rendah
- C. Lebih dari 10 alarm dalam 10 menit per operator, melampaui kapasitas kognitif sehingga operator cenderung mengabaikan semua alarm (alarm fatigue)
- D. Alarm muncul tepat 1 kali per 10 menit

3. Ketersediaan (availability) sebuah mesin dinyatakan sebagai...

- A. $A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$
- B. $A = \text{MTTR} / \text{MTBF}$
- C. $A = \text{MTBF} \times \text{MTTR}$
- D. $A = 1 - \text{MTBF}$

4. Skema sensor redundan 2-out-of-3 (2oo3) meningkatkan keandalan dengan cara...

- A. Memakai 1 sensor saja agar sederhana
- B. Mengambil rata-rata semua sensor tanpa logika voting
- C. Sistem trip hanya bila ketiga sensor setuju
- D. Keputusan diambil bila minimal 2 dari 3 sensor setuju, toleran terhadap 1 sensor gagal sekaligus menahan false trip

5. Fitur alarm shelving dalam alarm management berfungsi untuk...

- A. Menghapus alarm secara permanen dari sistem
- B. Menunda/menyembunyikan alarm yang sudah diketahui untuk sementara (operator-initiated, auto-unshelve setelah timeout) agar tidak menambah flood
- C. Menaikkan prioritas semua alarm
- D. Mengirim alarm ke semua operator sekaligus

6. Overall Equipment Effectiveness (OEE) dihitung sebagai...

- A. $OEE = Availability + Performance + Quality$
- B. $OEE = Availability - Downtime$
- C. $OEE = Availability \times Performance \times Quality$
- D. $OEE = MTBF / MTTR$

7. Penentuan prioritas alarm (ANSI/ISA 18.2) idealnya didasarkan pada...

- A. Urutan abjad nama tag
- B. Tingkat konsekuensi (safety > environmental > economic) dan waktu yang tersedia untuk merespons
- C. Warna favorit operator
- D. Jumlah karakter pada pesan alarm

8. Time-series database (misal TimescaleDB / InfluxDB) cocok untuk data sensor karena...

- A. Mendukung hypertable/partisi waktu, continuous aggregate (downsampling otomatis), kompresi dan retention policy untuk volume tinggi
- B. Hanya menyimpan satu nilai terakhir
- C. Tidak mendukung query rentang waktu
- D. Memerlukan input spreadsheet manual

9. Keunggulan utama edge computing untuk monitoring getaran di pabrik adalah...

- A. Selalu butuh koneksi cloud agar berfungsi
- B. Menghapus kebutuhan sensor
- C. Latency lebih tinggi daripada cloud
- D. Latency rendah, hemat bandwidth (kirim fitur, bukan raw), dan tetap berjalan saat koneksi cloud putus (resilient)

10. Metrik kinerja sistem alarm (alarm system performance) yang dipantau rutin mencakup...

- A. Hanya jumlah sensor terpasang
- B. Warna dashboard
- C. Rata-rata laju alarm ter-annunciate, jumlah standing/stale alarm, dan persen waktu dalam kondisi flood
- D. Harga lisensi software

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: $\text{Availability} = \text{MTBF} / (\text{MTBF} + \text{MTTR})$; MTBF = total jam operasi/jumlah kegagalan; $\text{OEE} = \text{Availability} \times \text{Performance} \times \text{Quality}$; redundansi 2oo3 voting: $R_{\text{sys}} = 3R^2 - 2R^3$; ANSI/ISA 18.2: target ≤ 1 alarm/10 menit/operator, alarm flood $\Rightarrow 10$ alarm/10 menit/operator.

C1. Hitung availability (persen) sebuah mesin dengan MTBF=200 jam dan MTTR=8 jam.
Formula: $A = \text{MTBF} / (\text{MTBF} + \text{MTTR}) \times 100$.

- 💡 **HINT:** Availability = uptime rata-rata dibagi total siklus (uptime+repair), dikali 100.

C2. Target ANSI/ISA 18.2 ≤ 1 alarm/10 menit per operator (steady state). Hitung jumlah alarm per hari maksimum yang masih compliant.

- 💡 **HINT:** 1 alarm/10 menit $\rightarrow$ per jam, lalu dikali 24.

C3. Hitung OEE (persen) untuk Availability=0,90, Performance=0,95, Quality=0,99.
Formula: $\text{OEE} = A \times P \times Q \times 100$.

- 💡 **HINT:** Kalikan ketiga faktor lalu dikali 100.

C4. Keandalan sistem 2oo3 (dua dari tiga) bila keandalan tiap sensor $R=0,9$: $R_{\text{sys}} = 3R^2 - 2R^3$. Hitung R_{sys} .

- 💡 **HINT:** Substitusi $R=0,9$ ke $3R^2 - 2R^3$.

C5. Sebuah mesin mengalami 5 kegagalan dalam 10000 jam operasi. Hitung MTBF (jam) = total jam operasi / jumlah kegagalan.

- 💡 **HINT:** Bagi total jam dengan jumlah kegagalan.

C6. Durasi alarm aktif (menit): [2, 45, 5, 120, 30]. Hitung jumlah standing alarm (durasi > 30 menit).

- 💡 **HINT:** Hitung elemen yang nilainya lebih besar dari 30.

C7. Data sensor 1 Hz selama 1 jam di-downsample menjadi rata-rata per 1 menit (continuous aggregate). Hitung jumlah titik output.

- 💡 **HINT:** 3600 detik dibagi 60 detik per bucket.

C8. Untuk target availability 99,5% selama 1 tahun (8760 jam), hitung maksimum downtime (jam) = $8760 \times (1 - 0,995)$.

- 💡 **HINT:** Downtime budget = total jam $\times$ (1 - target).

C9. Daftar prioritas alarm: ['L','H','C','C','M','C']. Hitung jumlah alarm berprioritas Critical ('C').

- 💡 **HINT:** Hitung kemunculan 'C' di list.

C10. Sebelum rasionalisasi: 500 alarm/hari; sesudah: 120 alarm/hari. Hitung pengurangan (persen) = $(500 - 120) / 500 \times 100$.

- 💡 **HINT:** Selisih dibagi nilai awal, dikali 100.

Bagian C: Komputasi Hard (5 soal $\times$ 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal, hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

C11 (Hard). Availability dari event log: diberi waktu antar-kegagalan up=[100,150,200] jam dan waktu perbaikan down=[5,8,6] jam. Implementasikan perhitungan MTBF=mean(up), MTTR=mean(down), lalu print availability (persen)= $MTBF / (MTBF + MTTR) \times 100$.

- 💡 **HINT:** Rata-ratakan up dan down, lalu terapkan formula availability.

C12 (Hard). Monte Carlo keandalan 2oo3: np.random.seed(42); 10000 trial, tiap trial 3 sensor dengan probabilitas gagal 0,1 (np.random.rand(10000,3)<0.1). Sistem GAGAL bila ≥ 2 sensor gagal. Print keandalan sistem (fraksi trial yang sistemnya OK).

- 💡 **HINT:** Hitung jumlah sensor gagal per trial; sistem OK bila gagal < 2; rata-ratakan.

C13 (Hard). Deteksi alarm flood (sliding window): np.random.seed(42); timestamp alarm (detik)=np.sort(np.random.uniform(0,3600,80)). Dengan window 10 menit (600 detik), hitung jumlah alarm maksimum dalam satu window (geser tiap alarm sebagai awal window).

- 💡 **HINT:** Untuk tiap alarm t, hitung alarm pada [t, t+600); ambil maksimum.

C14 (Hard). OEE dari time-series: np.random.seed(42);
 $A = np.random.uniform(0.85, 0.98, 30)$; $P = np.random.uniform(0.80, 0.95, 30)$;
 $Q = np.random.uniform(0.95, 1.0, 30)$. Print OEE (persen)= $mean(A) \times mean(P) \times mean(Q) \times 100$.

- 💡 **HINT:** Rata-ratakan tiap faktor 30-hari, kalikan, dikali 100.

C15 (Hard). End-to-end availability 24 jam: simulasi 24 jam (sampling 30 detik, n=2880). np.random.seed(42); state_up=np.random.rand(2880) > 0.03 (mesin down 3% waktu). Print persentase downtime (persen)= $100 \times mean(\sim state_up)$.

- 💡 **HINT:** Hitung fraksi sample yang state_up False, dikali 100.

DAFTAR PUSTAKA

- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Nguyen, T. T., Phi, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>
- Bayá, G., Canale, E., Nesmachnow, S., De Sousa, A., & Sartor, P. (2022). Production optimization in a grain facility through mixed-integer linear programming. *Applied Sciences*, 12(16), 8212. <https://doi.org/10.3390/app12168212>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Otomatisasi Prediktif: Supervised

Abstrak

Pertemuan ini memperkenalkan machine learning sebagai pelengkap monitoring rule-based (Pertemuan 12-13) untuk

Sub - CPMK

Sub-CPMK 5.1 – Mampu melakukan pengukuran kinerja sistem dan menganalisis hasilnya

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-13.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Rule-based monitoring (Pertemuan 12-13) sangat transparan dan reliable, tetapi memiliki tiga keterbatasan: hanya bekerja baik untuk batas keputusan linier (single/few threshold), threshold tetap 0,5 tidak selalu optimal, dan satu kali split data rawan bias. Pertemuan keempat belas ini memperkenalkan machine learning sebagai pelengkap: model belajar pola dari data historis berlabel, dan mampu menangani kombinasi banyak fitur sekaligus yang sulit ditangani rule tunggal.



Gambar 1. Posisi Pertemuan 14 (Supervised Learning & Logistic Regression) dalam alur mata kuliah, mengawali blok materi machine learning sebelum Pertemuan 15.

Gambar 1 menunjukkan posisi Pertemuan 14 dalam alur mata kuliah.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 5.1:** Mampu melakukan pengukuran kinerja sistem dan menganalisis hasilnya, yaitu mengukur kinerja model klasifikasi (accuracy, precision, recall, F1) dan menganalisis hasilnya untuk menentukan kelayakan model pada sistem prediktif (CPMK 5).
- Setelah pertemuan ini mahasiswa dapat: membangun dataset X/y dan melakukan train/test split stratified; menjelaskan cara kerja Logistic Regression (sigmoid, decision boundary); menginterpretasikan confusion matrix; serta menghitung precision, recall, dan F1-score.

SUPERVISED LEARNING FOUNDATIONS: DATASET, FEATURES, LABELS & TRAINING WORKFLOW

$X[n,d], y[n]$ raw signal -> 20+ fitur `train_test_split(X,y,0.2)`
`model.fit(X,y).predict(X_baru)`

Tabel 1. Tujuh tahap workflow supervised learning, dari load data hingga deploy ke production.

Tahap Workflow	Aksi	scikit-learn API	Output
1. Load data	Read CSV/Parquet -> DataFrame	<code>pd.read_csv()</code>	X, y arrays
2. Preprocess	Standardize, encode, impute	StandardScaler, OneHotEncoder	X_scaled
3. Split	Train/test 80/20 stratified	<code>train_test_split(stratify=y)</code>	X_train, X_test, y_train, y_test
4. Train	Fit model di train	<code>model.fit(X_train, y_train)</code>	Trained model object
5. Evaluate	Predict + metrics di test	<code>classification_report</code>	Accuracy, F1, confusion matrix
6. Tune	Grid search hyperparams	<code>GridSearchCV(cv=5)</code>	Best model + params
7. Deploy	Save model, integrasi API	<code>joblib.dump(model)</code>	File .pkl untuk production

Tabel 1 merangkum tujuh tahap workflow supervised learning standar menggunakan scikit-learn.



Gambar 2. Stratified train/test split: proporsi kelas 90:10 (healthy:fault) dipertahankan baik di train set (80%) maupun test set (20%).

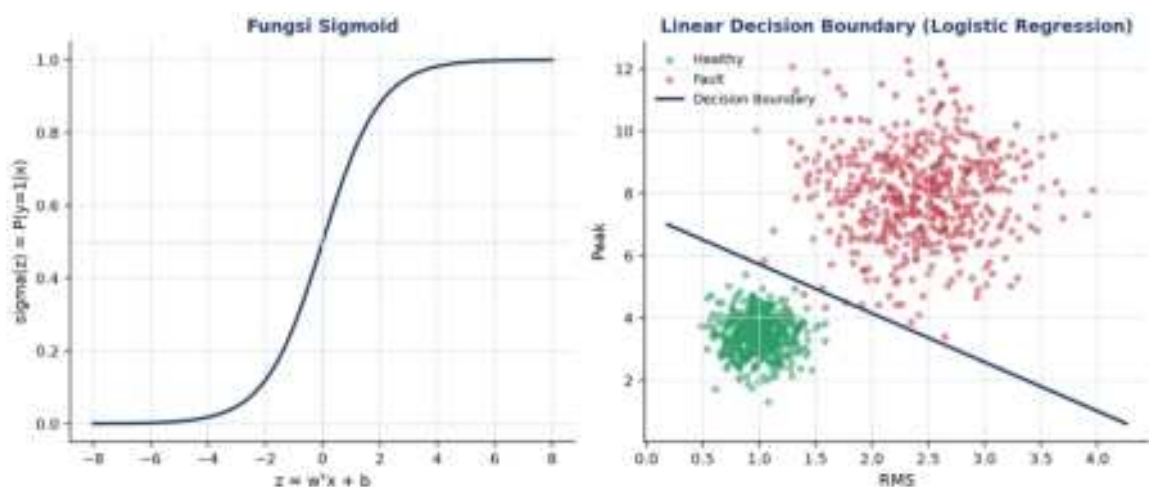
Gambar 2 menunjukkan pentingnya parameter ``stratify=y`` pada ``train_test_split``: tanpa stratify, test set berisiko kebetulan tidak memiliki sampel kelas minoritas sama sekali, terutama pada data yang sangat imbalanced.

Contoh Konkret: Bearing Fault Detection (CWRU Dataset). Benchmark CWRU (Case Western Reserve University) bearing dataset: 4 kondisi (healthy, inner race, outer race, ball fault), sekitar 10.000 sampel per kondisi, 21 fitur diekstrak dari gelombang mentah 50.000 titik. Random Forest 100 trees mencapai accuracy 98,5% dan F1=0,97 (macro-average), jauh mengungguli pendekatan rule-based (sekitar 75% akurasi).

LOGISTIC REGRESSION: LINEAR BASELINE CLASSIFIER UNTUK HEALTHY/FAULT DETECTION

$$P(y=1|x) = \text{sigma}(w^T x + b) \quad \text{sigma}(z) = 1/(1+e^{(-z)}) \quad \text{cross-entropy loss}$$

koefisien w = log-odds



Gambar 3. Fungsi sigmoid mengubah kombinasi linier $z=w^T x+b$ menjadi probabilitas 0-1 (kiri); decision boundary linier Logistic Regression memisahkan healthy dan fault pada fitur RMS-Peak (kanan).

Gambar 3 menunjukkan bagaimana fungsi sigmoid memetakan kombinasi linier fitur menjadi probabilitas, dan bagaimana decision boundary yang dihasilkan berbentuk garis lurus (linier) pada ruang fitur; koefisien w dapat diinterpretasikan sebagai log-odds, memberi Logistic Regression keunggulan interpretability dibanding model kotak-hitam.

Tabel 2. Perbandingan empat algoritma klasifikasi: kecepatan training/prediksi, interpretability, dan kecocokan data. Random Forest dan SVM dibahas mendalam di Pertemuan 15.

Algoritma	Train Speed	Predict Speed	Interpretability	Cocok Data
Logistic Regression	Sangat cepat	Sangat cepat	Tinggi (koefisien w)	Linear separable, <100K
Random Forest	Sedang	Cepat	Sedang (feature_imp)	Mixed, <1M, baseline terbaik
SVM (RBF)	Lambat $O(n^2)$	Sedang	Rendah	Small-medium, high-dim
XGBoost / LightGBM	Sedang	Cepat	Sedang (SHAP)	Large tabular, Kaggle-grade

Tabel 2 membandingkan Logistic Regression dengan tiga algoritma lain yang akan dibahas lebih dalam di Pertemuan 15.

Contoh Konkret: Bearing Fault Classification (CWRU). Pada benchmark CWRU 4-kelas (21 fitur, 8000 train + 2000 test): Logistic Regression mencapai accuracy 86% dan $F1=0,84$; Random Forest (default) accuracy 98,5% dan $F1=0,97$; SVM RBF ($C=10$, $\gamma=scale$) accuracy 97,2% dan $F1=0,96$; XGBoost (default) accuracy 98,8% dan $F1=0,98$. Logistic Regression tertinggal dari model non-linear, tetapi tetap menjadi baseline wajib karena kesederhanaan dan interpretabilitasnya.

EVALUASI DASAR: CONFUSION MATRIX, ACCURACY, PRECISION, RECALL & F1-SCORE

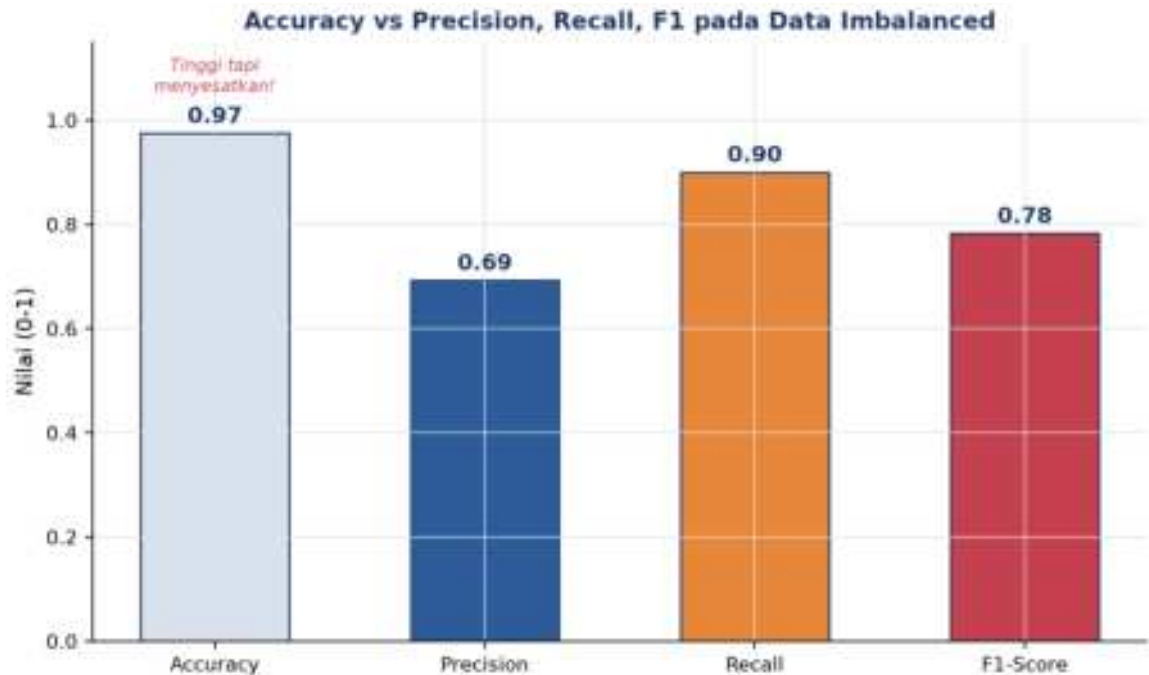
*Confusion matrix 2x2: $[[TN, FP], [FN, TP]]$ $Accuracy=(TP+TN)/N$
 $Precision=TP/(TP+FP)$ $Recall=TP/(TP+FN)$ $F1=2PR/(P+R)$*

Confusion Matrix: Bearing Fault (Imbalanced, 5% Positif)

Actual Healthy	TN 9300	FP 200
Actual Fault	FN 50	TP 450
	Predicted Healthy	Predicted Fault

Gambar 4. Confusion matrix studi kasus bearing fault imbalanced (5% positif): $TN=9300$, $FP=200$, $FN=50$, $TP=450$.

Gambar 4 menunjukkan confusion matrix untuk dataset dengan 9500 sampel healthy dan 500 sampel fault (rasio imbalance 19:1).



Gambar 5. Accuracy (0,975) tampak sangat tinggi tetapi menyesatkan pada data imbalanced; precision (0,69), recall (0,90), dan F1 (0,78) memberi gambaran lebih jujur tentang kinerja model.

Gambar 5 membuktikan mengapa accuracy saja tidak cukup untuk data imbalanced: naive predictor yang selalu memprediksi "healthy" sudah mencapai accuracy 95% padahal gagal total mendeteksi fault (recall=0%). Precision, recall, dan F1-score memberi gambaran yang jauh lebih jujur tentang kinerja model pada kelas minoritas yang justru paling penting.

Peringatan, Trap Evaluasi yang Sering Diabaikan: Tujuh jebakan evaluasi yang sering diabaikan: memakai accuracy untuk data imbalanced; hanya mengandalkan satu kali split (variance tinggi); data leakage (informasi test bocor ke proses training); menggunakan ulang test set berkali-kali untuk tuning; distribusi data training tidak sama dengan kondisi deployment (concept drift); memakai threshold default 0,5 tanpa pertimbangan biaya kesalahan; serta melaporkan metrik yang salah untuk konteks masalah.

ANALOGI MACHINE LEARNING DI SEHARI-HARI

- **Email Spam Filter (Binary Classification):** Gmail mencapai akurasi lebih dari 99,9% dengan feature engineering dari kata kunci, link, dan reputasi pengirim yang diumpungkan ke classifier biner.
- **Rekomendasi YouTube (Multi-Class + Regression):** YouTube menggabungkan collaborative filtering dan content-based filtering untuk memprediksi watch time dan merekomendasikan video.

- **Face Detection di Kamera (Computer Vision):** Viola-Jones (2001) memakai Haar cascade dan AdaBoost untuk deteksi wajah real-time; pendekatan modern beralih ke CNN yang dilatih pada jutaan foto.
- **Bank Credit Scoring (Logistic Regression Klasik):** skor kredit FICO memakai Logistic Regression yang diregulasi karena butuh interpretability tinggi (Fair Lending Act).
- **Diagnosis Medis ML (Decision Tree + Ensemble):** IBM Watson Health dan Google DeepMind Health memakai ensemble XGBoost dan deep learning untuk diagnosis medis, meski persetujuan FDA berjalan lambat karena kebutuhan explainability.
- **Self-Driving Car Object Detection (Real-Time ML):** Tesla Autopilot, Waymo, dan Mobileye memakai YOLO/SSD untuk deteksi objek real-time dengan trade-off latency edge vs cloud di bawah 100ms.

Pola yang Sama di Semua Analogi: Tiga elemen kunci yang sama di semua analogi: kualitas data berlabel, feature engineering yang relevan, dan kompleksitas model yang sesuai kebutuhan; kualitas data jauh lebih menentukan daripada pemilihan algoritma semata.

APLIKASI ML PREDICTIVE MAINTENANCE DI INDUSTRI 4.0 & SMART MANUFACTURING

- **Pratt & Whitney Engine Health Management:** 5000+ mesin PW1100G/A320neo dipantau via edge ETL dan ensemble RF/XGBoost/LSTM untuk prediksi Remaining Useful Life; 80% kegagalan terdeteksi 30+ hari lebih awal, menghemat \$5-10 juta per AOG yang dihindari.
- **Siemens Mobility Train Predictive Maintenance:** platform Railigent memakai Random Forest untuk klasifikasi kondisi bogie, motor traksi, gearbox, dan rem; degradasi bearing terdeteksi 2-4 minggu lebih awal, terbukti pada 600+ trainset dengan availability naik 15% dan MTBF naik 30%.
- **GE Predix APM:** 50.000+ aset terhubung dengan digital twin dan deteksi anomali ML untuk komponen hot gas path (umur maksimal 24.000 jam), menurunkan downtime 5-15%.
- **Tesla Battery Management ML:** 7104+ sel baterai per mobil dipantau real-time dengan gradient boosting dan neural network, update model mingguan via OTA, mendukung garansi 8 tahun/240.000 km dengan penurunan kapasitas di bawah 30%.

Peringatan, Jebakan Praktis ML Predictive Maintenance: Delapan jebakan praktis ML predictive maintenance di industri: data berlabel tidak cukup; mengabaikan class

imbalance; concept drift tidak dipantau; data leakage; deploy model kotak-hitam tanpa explainability; tidak melakukan A/B test sebelum full rollout; mengabaikan MLOps (versioning, monitoring, retraining); serta ML dianggap menggantikan rule-based padahal seharusnya saling melengkapi.

IMPLEMENTASI PYTHON: MACHINE LEARNING KLASIFIKASI DI JUPYTER NOTEBOOK

Empat cell berikut membangun classifier bearing fault dari nol. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib, scikit-learn; notebook p13_ml_classification.ipynb.

```
p13_ml_classification.ipynb – Cell 1
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

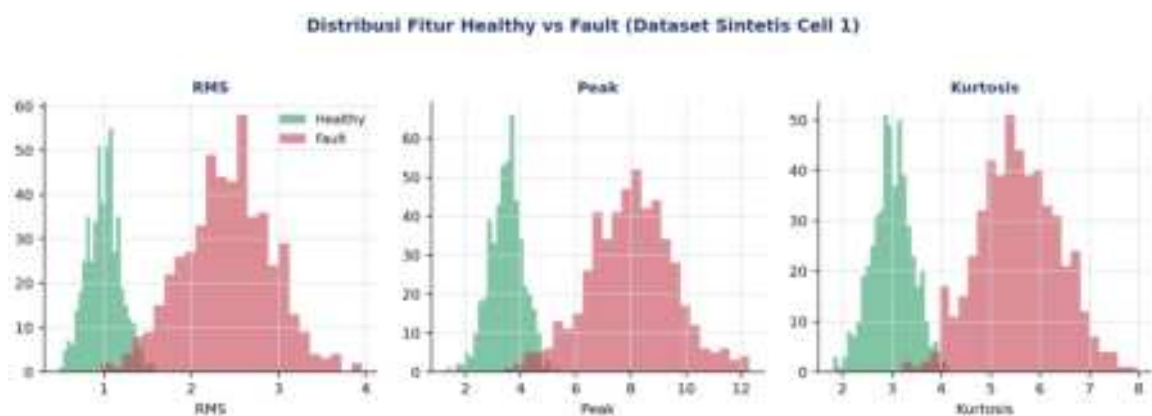
scaler = StandardScaler().fit(X_train)
X_train_s, X_test_s = scaler.transform(X_train), scaler.transform(X_test)

model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_s, y_train)

print('Accuracy: {model.score(X_test_s, y_test):.3f}'.format(model=model))
print('Coefficient (log-odds): {model.coef_[0]:.3f}'.format(model=model))
```

Gambar 6. Cuplikan kode train/test split stratified dan training Logistic Regression baseline.

Gambar 6 menunjukkan cuplikan kode inti Cell 1.



Gambar 7. Distribusi tiga fitur (RMS, Peak, Kurtosis) untuk kelas healthy vs fault pada dataset sintetis Cell 1; overlap minimal menunjukkan fitur cukup diskriminatif.

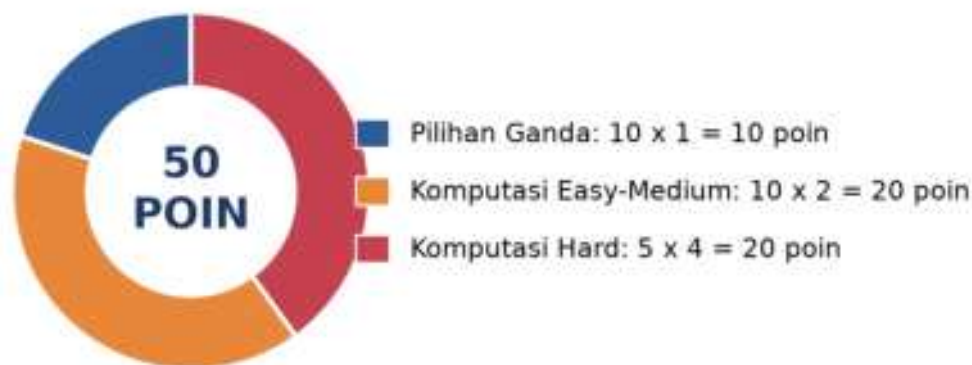
Gambar 7 menunjukkan distribusi fitur healthy vs fault pada dataset sintetis 1000 sampel (500/kelas) yang dipakai sepanjang Cell 1-4.

- **Cell 1:** generate dataset sintetis 1000 sampel (RMS, Peak, Kurtosis, seed=42);
`train\_test\_split(test\_size=0.2, random\_state=42, stratify=y)`; `StandardScaler` fit di train saja; `LogisticRegression(max\_iter=1000)`; cetak accuracy, F1, classification_report, dan koefisien.
- **Cell 2:** `RandomForestClassifier(n\_estimators=100, random\_state=42)` dilatih pada data mentah (tanpa scaling); cetak confusion matrix dan `feature\_importances\_` terurut; preview singkat materi Pertemuan 15.
- **Cell 3:** `Pipeline([StandardScaler, SVC(kernel="rbf", probability=True)])`; perbandingan ROC-AUC tiga model (LogReg, RF, SVM); preview materi Pertemuan 15.
- **Cell 4:** `cross\_val\_score(cv=5, scoring="f1\_macro")` dan `GridSearchCV` untuk tuning Random Forest; preview materi Pertemuan 15.

TUGAS PERTEMUAN 14

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 14



Gambar 8. Distribusi 50 poin Tugas Pertemuan 14 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Pendekatan supervised learning berbeda dari unsupervised learning karena...
 - A. Supervised butuh GPU, unsupervised tidak

- **B.** Supervised butuh data labeled (X, y), model belajar mapping input ke output dari contoh. Unsupervised hanya X (cari pattern intrinsik: clustering, dimensionality reduction)
- **C.** Supervised hanya untuk regresi, unsupervised untuk klasifikasi
- **D.** Tidak ada perbedaan signifikan

2. Tujuan train_test_split dengan stratify=y di scikit-learn adalah...

- **A.** Mempercepat training
- **B.** Menjaga proporsi tiap kelas sama di train dan test set, penting untuk imbalanced data (misal 1% fault). Tanpa stratify, test set bisa kebetulan tidak punya sampel positif sama sekali
- **C.** Mengubah label jadi numerik
- **D.** Menghapus duplicate sample

3. Logistic Regression memprediksi probabilitas kelas positif dengan cara...

- **A.** Menghitung jarak Euclidean ke centroid kelas
- **B.** Menerapkan fungsi sigmoid pada kombinasi linier fitur: $P(y=1|x)=\sigma(wTx+b)$, menghasilkan nilai 0-1
- **C.** Membangun banyak decision tree lalu voting
- **D.** Mengurutkan data dan mengambil median

4. Keunggulan utama Logistic Regression dibanding model black-box (misal deep neural network) adalah...

- **A.** Selalu lebih akurat pada semua jenis data
- **B.** Koefisien w dapat diinterpretasikan langsung sebagai log-odds tiap fitur, memberi interpretability tinggi yang penting untuk domain teregulasi
- **C.** Tidak butuh data untuk training
- **D.** Tidak pernah overfitting

5. Untuk dataset imbalanced 1% fault, 99% healthy, accuracy 99% dari naive predict-all-healthy adalah...

- **A.** Excellent, model sukses
- **B.** Menyesatkan! Recall=0% (semua fault missed). Untuk imbalanced data, accuracy tidak relevan, pakai F1-score, precision, recall, ROC-AUC
- **C.** Tidak bisa diinterpretasi tanpa data lebih
- **D.** Bukti overfit

6. Confusion matrix 2x2 (TN, FP, FN, TP): untuk safety-critical predictive maintenance, metric paling kritis adalah...

- **A.** FP (false alarm), yang utama dihindari

- **B.** FN (missed fault), paling berbahaya. Predict "healthy" padahal actual "fault" menyebabkan mesin tidak diperbaiki, catastrophic failure. Solusi: minimize FN via threshold rendah (recall tinggi)
- **C.** TN (correct normal), yang harus dimaksimalkan
- **D.** Semua sama pentingnya

7. F1-score = $2 \cdot P \cdot R / (P + R)$ berguna karena...

- **A.** Selalu lebih besar dari accuracy
- **B.** Harmonic mean precision dan recall, single metric yang menyeimbangkan keduanya. F1 tinggi hanya saat P dan R keduanya tinggi (penalize jika salah satu rendah). Cocok untuk imbalanced data
- **C.** F1 = accuracy untuk binary classification
- **D.** Hanya bekerja untuk 2 kelas

8. Parameter `random_state=42` pada `train_test_split` dan model scikit-learn berfungsi untuk...

- **A.** Mengatur jumlah fitur yang dipakai
- **B.** Menjamin reproducibility, hasil split/training yang sama setiap kali kode dijalankan ulang
- **C.** Menentukan jumlah kelas target
- **D.** Mempercepat training 42 kali lipat

9. StandardScaler perlu diterapkan sebelum Logistic Regression karena...

- **A.** Logistic Regression tidak bisa menerima angka negatif
- **B.** Gradient descent pada cross-entropy loss konvergen lebih stabil dan cepat saat fitur memiliki skala yang sebanding; scaler harus di-fit hanya di train set untuk mencegah data leakage
- **C.** StandardScaler mengubah masalah klasifikasi jadi regresi
- **D.** Tidak pernah diperlukan untuk model linear

10. Praktik terbaik arsitektur ML predictive maintenance production adalah...

- **A.** ML 100% menggantikan rule-based
- **B.** Hybrid rule-based + ML: rule-based ISO 10816 (Pertemuan 12-13) sebagai safety net dan compliance; ML (Pertemuan 14) sebagai early-warning dan diagnosis multi-fitur. Dashboard operator terpadu, model di-retrain berkala
- **C.** Hanya ML, abaikan rule-based
- **D.** Hanya rule-based, ML tidak diperlukan

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: confusion matrix $[[TN, FP], [FN, TP]]$; $accuracy = (TP + TN) / N$; $precision = TP / (TP + FP)$; $recall = TP / (TP + FN)$; $F1 = 2PR / (P + R)$.

C1. Hitung accuracy dari confusion matrix: TP=80, FP=10, TN=85, FN=5. Formula: $(TP+TN)/(TP+FP+TN+FN)$. Print persentase desimal (misal 91.67).

- 💡 **HINT:** Accuracy = (TP+TN) dibagi (TP+FP+TN+FN), lalu kalikan 100 untuk persen. Substitusikan nilai confusion matrix dari soal.

C2. Hitung precision dari confusion matrix: TP=80, FP=10. Formula: $TP/(TP+FP)$. Print desimal (3 digit).

- 💡 **HINT:** Precision = TP dibagi (TP+FP), proporsi prediksi positif yang benar. Substitusikan nilai dari soal.

C3. Hitung recall (sensitivity): TP=80, FN=5. Formula: $TP/(TP+FN)$. Print desimal (3 digit).

- 💡 **HINT:** Recall = TP dibagi (TP+FN), proporsi positif aktual yang terdeteksi. Substitusikan nilai dari soal.

C4. Hitung F1-score dari precision=0,750 dan recall=0,600. Formula: $2.P.R/(P+R)$. Print desimal (3 digit).

- 💡 **HINT:** F1 = 2.P.R dibagi (P+R), harmonic mean precision dan recall. Substitusikan P dan R dari soal.

C5. Untuk dataset N=1000 sampel dengan test_size=0.2, berapa sampel di training set? Print integer.

- 💡 **HINT:** Training set = $N \times (1 - \text{test_size})$; sisanya menjadi test set. Substitusikan N dan test_size dari soal.

C6. 5-fold Cross-Validation: untuk dataset 800 sampel, berapa sampel di tiap fold sebagai test? Print integer.

- 💡 **HINT:** Ukuran fold test = total sampel dibagi jumlah fold (k); sisanya jadi train tiap iterasi. Substitusikan nilai dari soal.

C7. Untuk Random Forest dengan n_estimators=100 dan max_features='sqrt' pada dataset dengan 16 fitur, berapa fitur dipertimbangkan per split? Print integer (round).

- 💡 **HINT:** Dengan max_features='sqrt', jumlah fitur per split = akar(total fitur), dibulatkan. Substitusikan jumlah fitur dari soal.

C8. StandardScaler: untuk fitur dengan mean=5,0, std=2,0, hitung scaled value dari raw=8,0. Formula: $z=(x-\text{mean})/\text{std}$. Print desimal.

- 💡 **HINT:** Scaled value $z = (x - \text{mean})$ dibagi std. Substitusikan raw, mean, dan std dari soal. Standardisasi wajib untuk SVM/logistic, tidak untuk tree-based.

C9. GridSearchCV: untuk param_grid dengan n_estimators: [50, 100, 200] dan max_depth: [None, 5, 10, 20], berapa total kombinasi hyperparameter? Print integer.

- 💡 **HINT:** Total kombinasi = perkalian jumlah nilai tiap hyperparameter di grid. Substitusikan jumlah nilai tiap parameter dari soal.

C10. Hitung FNR (False Negative Rate) = missed fault rate, dari TP=80, FN=5. Formula: $FN/(TP+FN)$. Print persentase desimal.

- 💡 **HINT:** FNR = FN dibagi (TP+FN), kalikan 100 untuk persen; ini komplemen recall (FNR=1-Recall). Substitusikan nilai dari soal.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

C11 (Hard). Logistic Regression Baseline, Synthetic Bearing Dataset: generate dataset 2-class healthy vs fault dengan separasi jelas. `np.random.seed(42); n=500; X_h=np.random.randn(n,3); X_f=np.random.randn(n,3)+2.0; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Split 80/20 stratified, train LogisticRegression, print test accuracy (3 desimal).

- 💡 **HINT:** Split data 80/20 stratified dengan `train_test_split`, latih LogisticRegression, lalu evaluasi dengan `.score()` pada test set.

C12 (Hard). Random Forest + 5-Fold Cross-Validation: generate dataset healthy vs fault. `np.random.seed(42); n=300; X_h=np.random.randn(n,4); X_f=np.random.randn(n,4)+2.0; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Train `RandomForestClassifier(n_estimators=100, random_state=42)` dengan 5-fold CV, print mean accuracy (3 desimal).

- 💡 **HINT:** Gunakan `cross_val_score` dengan `RandomForestClassifier`, `cv=5`, scoring accuracy, lalu ambil rata-ratanya dengan `.mean()`.

C13 (Hard). SVM RBF Pipeline + StandardScaler: dataset multi-skala (butuh scaling sebelum SVM). `np.random.seed(42); n=250; X_h=np.random.randn(n,5)*10; X_f=np.random.randn(n,5)*10+5; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Bangun `Pipeline([StandardScaler, SVC(kernel="rbf", C=1.0, gamma="scale", random_state=42)])`. Split 75/25 stratified, train, print test accuracy (3 desimal).

- 💡 **HINT:** Bangun Pipeline berisi StandardScaler lalu `SVC(kernel="rbf")` agar scaling otomatis dan bebas data leakage; split stratified, latih, lalu evaluasi `.score()` di test set.

C14 (Hard). GridSearchCV, Hyperparameter Tuning RandomForest: dataset 4 fitur. `np.random.seed(42); n=250; X_h=np.random.randn(n,4); X_f=np.random.randn(n,4)+2.0; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Tune `n_estimators=[50,100]`, `max_depth=[3,5,None]` dengan `GridSearchCV(cv=3, scoring="accuracy")`. Print `best_score_` (3 desimal).

-  **HINT:** Jalankan GridSearchCV dengan param_grid dan cv dari soal, panggil .fit(), lalu baca skor terbaik dari atribut best_score_.

C15 (Hard). Engineering Pipeline, End-to-End ML Comparison (3 model): dataset bearing fault. `np.random.seed(42); n=300; X_h=np.random.randn(n,5); X_f=np.random.randn(n,5)+1.8; X=np.vstack([X_h,X_f]); y=np.concatenate([np.zeros(n),np.ones(n)])`. Split 80/20 stratified. Train 3 model: LogReg, RandomForest(n=100), SVM-RBF Pipeline (Scaler+SVC probability=True). Print best ROC-AUC dari 3 model di test set (3 desimal).

-  **HINT:** Untuk tiap model ambil probabilitas kelas positif via `predict_proba(X_test)[:,-1]`, hitung `roc_auc_score(y_test, proba)`, lalu pilih AUC terbaik. SVM perlu `probability=True`.

DAFTAR PUSTAKA

- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>
- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Nguyen, T. T., Phi, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>

MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul

Otomatisasi Prediktif: Random Forest,

Abstrak

Pertemuan penutup mata kuliah ini memperluas Logistic Regression baseline (Pertemuan 14) ke algoritma

Sub - CPMK

Sub-CPMK 5.2 – Mampu menginterpretasikan data kinerja dan memberikan rekomendasi perbaikan

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-14.html>. Pada layar masuk, pilih tombol “ Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Logistic Regression (Pertemuan 14) adalah baseline yang cepat dan interpretable, tetapi memiliki tiga keterbatasan: hanya bisa membentuk decision boundary linier, threshold 0,5 tetap belum tentu optimal, dan hasil satu kali split rawan variance tinggi. Pertemuan kelima belas, penutup mata kuliah Optimalisasi & Otomasi, memperkenalkan dua algoritma yang mengatasi keterbatasan tersebut: Random Forest dan Support Vector Machine kernel RBF, dilengkapi evaluasi lanjutan (ROC-AUC, k-fold cross-validation) dan hyperparameter tuning sistematis (GridSearchCV).



Gambar 1. Posisi Pertemuan 15 (Random Forest, SVM & Hyperparameter Tuning) sebagai modul terakhir mata kuliah, sebelum capstone project.

Gambar 1 menunjukkan posisi Pertemuan 15 sebagai modul penutup mata kuliah.

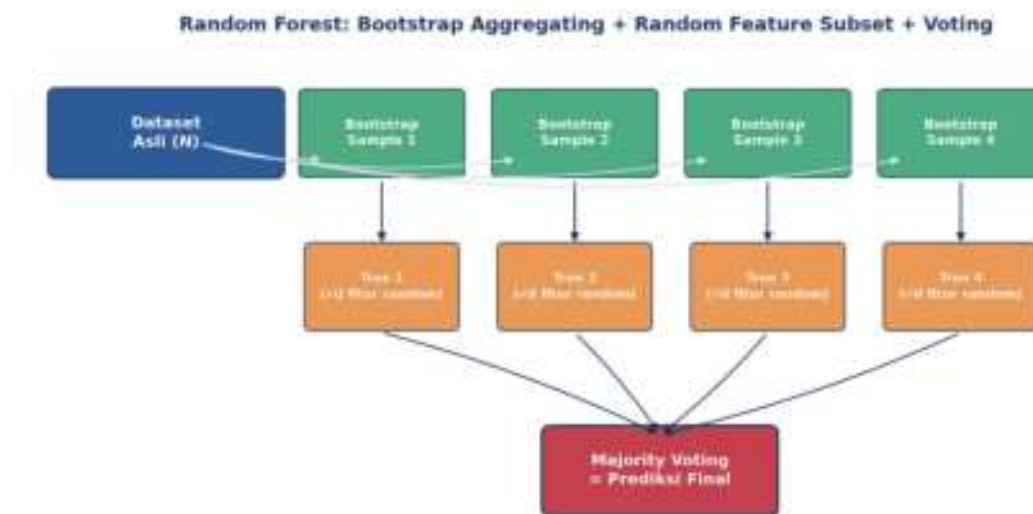
Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 5.2:** Mampu menginterpretasikan data kinerja dan memberikan rekomendasi perbaikan, yaitu menginterpretasikan metrik kinerja lanjutan (ROC-AUC, cross-validation score) untuk memberikan rekomendasi pemilihan model dan hyperparameter terbaik pada sistem klasifikasi prediktif (CPMK 5).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan cara kerja Random Forest (bagging, voting) dan SVM kernel RBF; membangun Pipeline anti-leakage; menginterpretasikan kurva ROC-AUC; serta melakukan hyperparameter tuning dengan k-fold cross-validation dan GridSearchCV.

RANDOM FOREST: ENSEMBLE DECISION TREE DENGAN BOOTSTRAP + FEATURE BAGGING

Bootstrap sampling *Random feature subset (akar(n))* *Majority voting*
n_estimators *feature_importances_* *Out-of-bag (OOB) error*

Bagging & Random Feature Subset: Diperkenalkan Leo Breiman (2001), Random Forest membangun banyak decision tree, masing-masing dilatih pada bootstrap sample (sampling dengan pengembalian) dan hanya mempertimbangkan subset fitur acak (biasanya akar(jumlah fitur)) di tiap split. Prediksi akhir diambil lewat majority voting dari seluruh tree.



Gambar 2. Diagram Random Forest: dataset asli menghasilkan beberapa bootstrap sample, tiap sample melatih satu tree dengan subset fitur acak, prediksi akhir via majority voting.

Gambar 2 mengilustrasikan pipeline lengkap Random Forest, dari bootstrap sampling hingga majority voting; diversifikasi antar-tree inilah yang menurunkan variance dan membuat Random Forest robust terhadap noise dibanding satu decision tree tunggal.

Out-of-Bag (OOB) Error sebagai Validasi Gratis: Out-of-bag (OOB) error menawarkan cara elegan untuk mengevaluasi performa Random Forest tanpa perlu menyisihkan data validasi terpisah: karena tiap tree hanya dilatih pada sekitar 63% data (akibat bootstrap sampling dengan pengembalian), sisa 37% data yang tidak terpakai (out-of-bag) pada tree tersebut dapat dipakai untuk mengevaluasi tree itu secara independen. Rata-rata error OOB di seluruh tree memberi estimasi generalisasi yang hampir setara dengan k-fold cross-validation, namun dihitung hampir gratis sebagai efek samping proses training itu sendiri (`RandomForestClassifier(oob_score=True)`).

Interpretasi Feature Importance: Feature importance yang dihasilkan Random Forest (`feature_importances_`) mengukur seberapa besar tiap fitur berkontribusi menurunkan impurity (Gini atau entropy) di seluruh split pada seluruh tree, dinormalisasi menjadi total

1.0. Metrik ini sangat berguna untuk interpretasi model pada konteks predictive maintenance: fitur dengan importance tertinggi biasanya mengarahkan engineer pada parameter fisik mana (RMS, kurtosis, frekuensi karakteristik bearing) yang paling diagnostik untuk kondisi kerusakan tertentu, memberi insight yang actionable, bukan sekadar prediksi black-box.

Tabel 1. Tujuh tahap workflow supervised learning (sama dengan Pertemuan 14), berlaku juga untuk Random Forest dan SVM.

Tahap Workflow	Aksi	scikit-learn API	Output
1. Load data	Read CSV/Parquet -> DataFrame	pd.read_csv()	X, y arrays
2. Preprocess	Standardize, encode, impute	StandardScaler, OneHotEncoder	X_scaled
3. Split	Train/test 80/20 stratified	train_test_split(stratify=y)	X_train, X_test, y_train, y_test
4. Train	Fit model di train	model.fit(X_train, y_train)	Trained model object
5. Evaluate	Predict + metrics di test	classification_report	Accuracy, F1, confusion matrix
6. Tune	Grid search hyperparams	GridSearchCV(cv=5)	Best model + params
7. Deploy	Save model, integrasi API	joblib.dump(model)	File .pkl untuk production

Tabel 1 mengingatkan kembali tujuh tahap workflow supervised learning yang berlaku untuk semua algoritma, termasuk Random Forest.

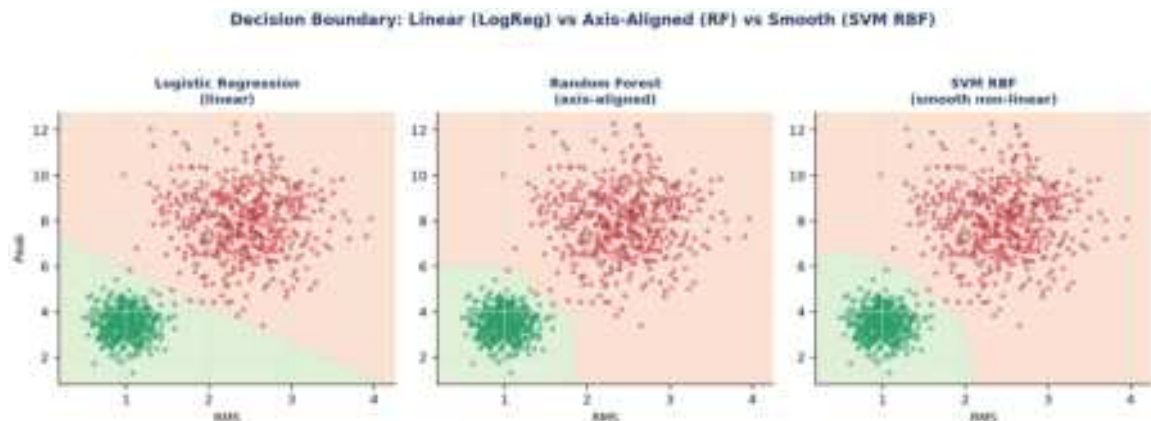
Contoh Konkret: Bearing Fault Detection (CWRU Dataset). Pada benchmark CWRU bearing dataset (4 kondisi, ~10.000 sampel/kondisi, 21 fitur), Random Forest 100 tree mencapai accuracy 98,5% dan F1=0,97 (macro-average), jauh mengungguli pendekatan rule-based murni (sekitar 75%).

SUPPORT VECTOR MACHINE + PIPELINE: KERNEL TRICK & ANTI-LEAKAGE WORKFLOW

*Margin maximization Kernel trick: $K(x,y)=\exp(-\gamma||x-y||^2)$ (RBF)
Hyperparameter C & gamma StandardScaler wajib Pipeline anti-leakage*

Margin, Kernel Trick & Kewajiban Scaling: Diperkenalkan Vladimir Vapnik (1995), SVM mencari hyperplane pemisah dengan margin maksimum antar kelas. Untuk data yang tidak linearly separable, kernel trick (RBF, polynomial) secara implisit memetakan data ke ruang berdimensi lebih tinggi tempat batas linier menjadi cukup. SVM SANGAT sensitif terhadap skala fitur, sehingga `StandardScaler` wajib diterapkan, dan harus dibungkus dalam

`Pipeline` agar scaler di-fit HANYA pada data training di setiap fold cross-validation (mencegah data leakage).



Gambar 3. Perbandingan decision boundary tiga algoritma pada data yang sama: linier (Logistic Regression), axis-aligned bertingkat (Random Forest), dan smooth non-linear (SVM RBF).

Gambar 3 menunjukkan karakteristik decision boundary tiap algoritma: Logistic Regression membentuk garis lurus, Random Forest membentuk pola bertingkat axis-aligned (karena struktur pohon keputusan), sedangkan SVM RBF membentuk kurva halus non-linear yang mengikuti bentuk data secara alami.

Tabel 2. Perbandingan empat algoritma klasifikasi (diulang dari Pertemuan 14, dengan fokus baris SVM dan XGBoost).

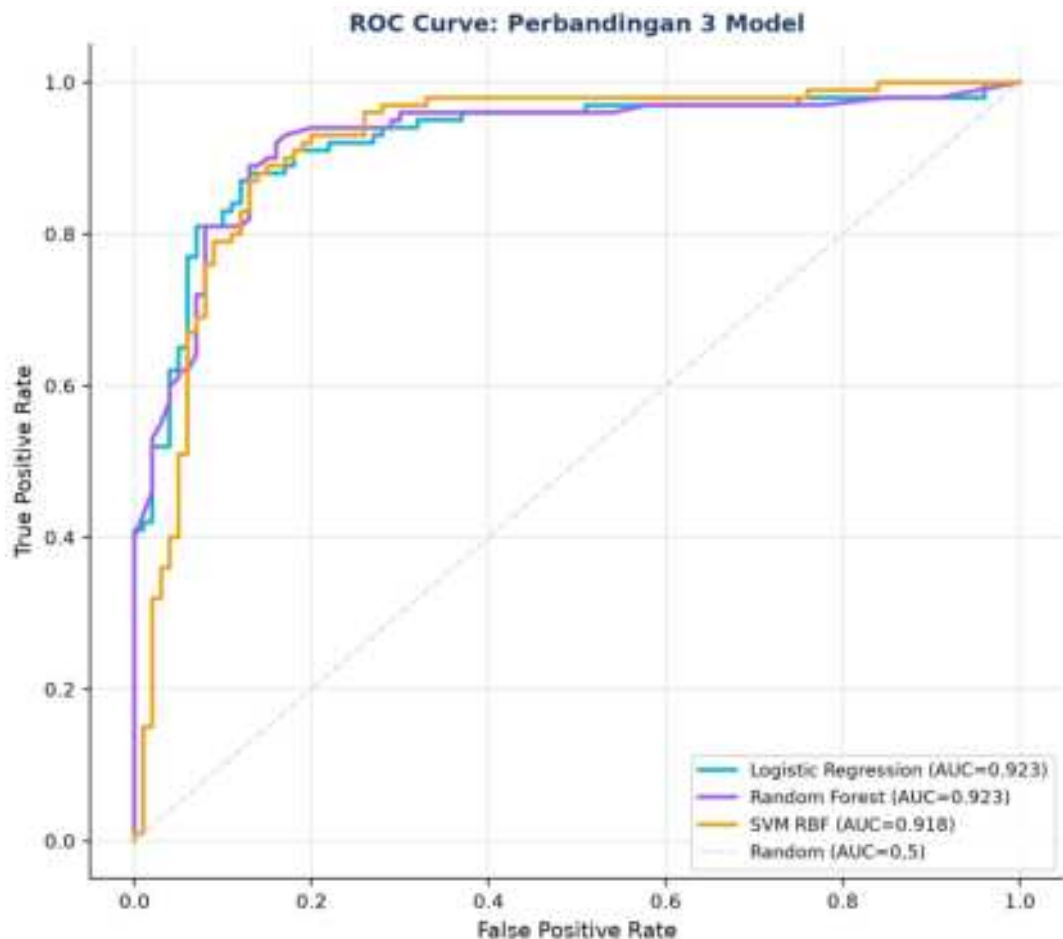
Algoritma	Train Speed	Predict Speed	Interpretability	Cocok Data
Logistic Regression	Sangat cepat	Sangat cepat	Tinggi (koefisien w)	Linear separable, <100K
Random Forest	Sedang	Cepat	Sedang (feature_imp)	Mixed, <1M, baseline terbaik
SVM (RBF)	Lambat $O(n^2)$	Sedang	Rendah	Small-medium, high-dim
XGBoost / LightGBM	Sedang	Cepat	Sedang (SHAP)	Large tabular, Kaggle-grade

Tabel 2 melengkapi perbandingan algoritma dari Pertemuan 14 dengan detail lebih pada SVM dan Gradient Boosting.

Contoh Konkret: Bearing Fault Classification (CWRU). Benchmark CWRU 4-kelas: SVM RBF ($C=10$, $\gamma=\text{scale}$) mencapai accuracy 97,2% dan $F1=0,96$, XGBoost (default) mencapai accuracy 98,8% dan $F1=0,98$, sedikit di atas Random Forest.

ROC-AUC, CROSS-VALIDATION & GRIDSEARCHCV: EVALUASI LANJUTAN & HYPERPARAMETER TUNING

*ROC curve & AUC (0,5-1,0) Stratified k-fold CV: mean +/- std GridSearchCV
exhaustive class_weight='balanced' Threshold tuning cost-sensitive*



Gambar 4. Kurva ROC tiga model (Logistic Regression, Random Forest, SVM RBF) pada dataset dengan overlap kelas lebih realistis; AUC mengukur kemampuan model me-ranking sampel positif di atas negatif, independen dari threshold.

Gambar 4 menunjukkan kurva ROC ketiga model pada dataset dengan overlap kelas yang lebih realistis (bukan dataset Cell 1 yang hampir sempurna terpisah); AUC=0,5 setara tebakan acak (garis diagonal), sedangkan AUC mendekati 1,0 menandakan model yang sangat baik dalam mengurutkan sampel positif di atas negatif, terlepas dari threshold keputusan yang dipilih.

Stratified k-Fold Cross-Validation: k-fold cross-validation (biasanya k=5) membagi data training menjadi k bagian; model dilatih k kali, tiap kali menggunakan 1 fold sebagai validasi dan sisanya sebagai training, lalu skor dirata-ratakan (mean +/- std). Ini jauh lebih robust dibanding satu kali train/test split yang bisa "beruntung" atau "sial".

Mengapa Stratified, Bukan k-Fold Biasa: Stratified k-fold (bukan k-fold biasa) menjadi pilihan wajib untuk data klasifikasi imbalanced: k-fold biasa membagi data secara acak murni sehingga ada risiko satu fold kebetulan berisi sangat sedikit atau bahkan nol sampel kelas minoritas (misal fault), menghasilkan skor evaluasi yang sangat bervariasi antar-fold dan tidak dapat diandalkan. Stratified k-fold menjaga proporsi kelas pada setiap fold tetap konsisten dengan proporsi kelas pada keseluruhan dataset, sehingga estimasi performa model jauh lebih stabil dan representatif terhadap kondisi deployment sesungguhnya.

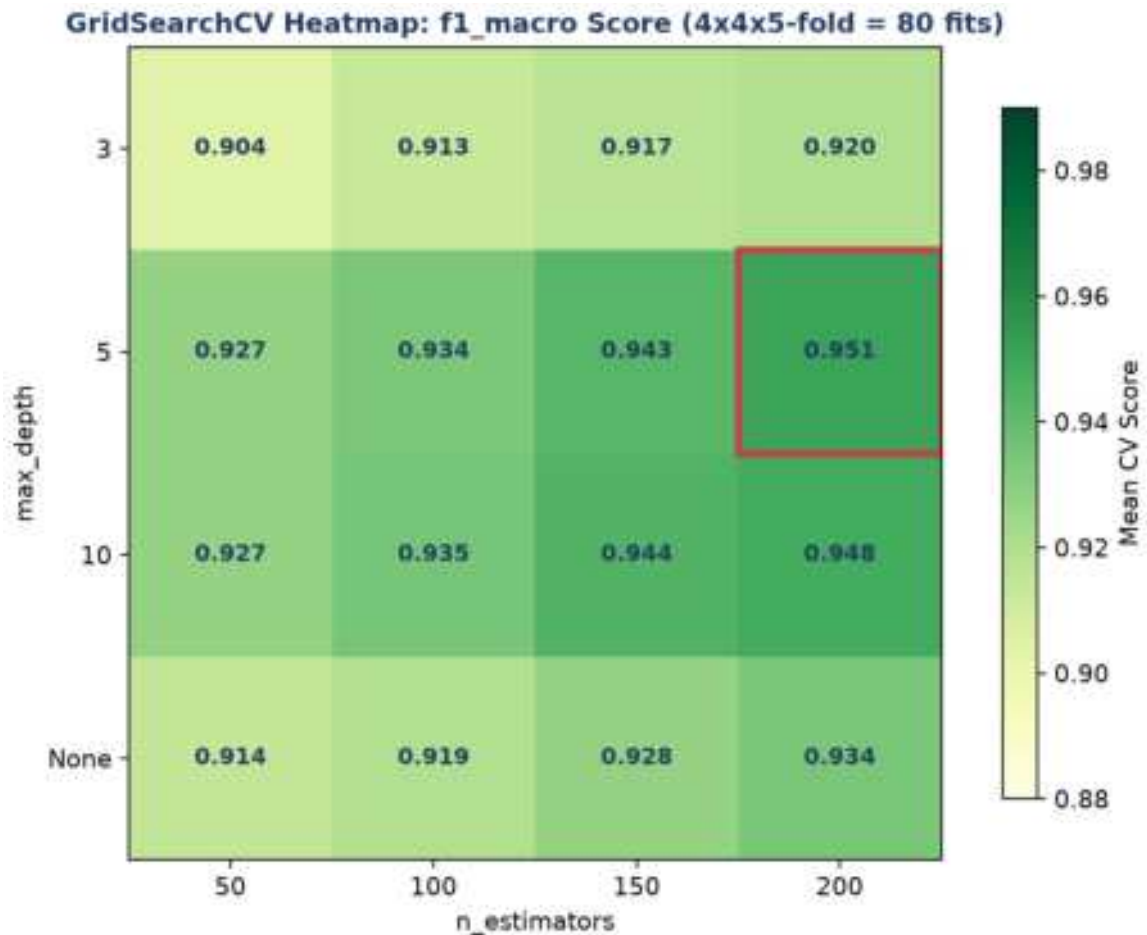


Gambar 5. 5-fold cross-validation: tiap fold bergiliran menjadi test set sedangkan 4 fold lainnya menjadi train set; skor akhir adalah rata-rata dari 5 iterasi.

Gambar 5 mengilustrasikan rotasi fold pada 5-fold cross-validation; setiap sampel data pada akhirnya pernah menjadi bagian dari test set tepat satu kali.

GridSearchCV: Exhaustive Hyperparameter Search. GridSearchCV menguji seluruh kombinasi hyperparameter dalam grid secara exhaustive, masing-masing dievaluasi dengan k-fold cross-validation. Untuk grid `n_estimators=[50,100,150,200]` x `max_depth=[3,5,10,None]` dengan `cv=5`, total dilakukan $4 \times 4 \times 5 = 80$ kali fitting model.

Skalabilitas Hyperparameter Search: Biaya komputasi GridSearchCV tumbuh secara kombinatorial terhadap jumlah hyperparameter yang di-grid-search sekaligus, sehingga pada praktiknya jarang lebih dari 2-3 hyperparameter digrid bersamaan untuk model kompleks seperti Random Forest atau SVM. Alternatif yang lebih efisien untuk ruang pencarian besar adalah RandomizedSearchCV (mengambil sampel acak dari kombinasi hyperparameter, bukan seluruhnya) atau pendekatan Bayesian optimization (Optuna, Hyperopt) yang secara adaptif mengarahkan pencarian ke region hyperparameter yang lebih menjanjikan berdasarkan hasil evaluasi sebelumnya, jauh lebih hemat komputasi dibanding exhaustive grid search pada ruang pencarian berdimensi tinggi.



Gambar 6. Heatmap GridSearchCV untuk Random Forest: kombinasi $n_estimators$ dan max_depth terbaik (kotak merah) menghasilkan skor $f1_macro$ tertinggi.

Gambar 6 menunjukkan heatmap hasil GridSearchCV; kombinasi $n_estimators=200$ dan $max_depth=5$ menghasilkan skor cross-validation tertinggi pada simulasi ini, ditandai kotak merah.

Imbalanced Data & Threshold Tuning: Untuk data yang sangat imbalanced (misal 5% fault), gunakan `class_weight='balanced'` agar model memberi bobot lebih besar pada kelas minoritas saat training, atau teknik SMOTE untuk oversampling sintetis. Threshold keputusan juga sebaiknya di-tuning (bukan default 0,5) berdasarkan biaya relatif false negative vs false positive.

Peringatan, Trap Evaluasi yang Sering Diabaikan: Tujuh jebakan evaluasi lanjutan yang sering diabaikan: memakai accuracy untuk data imbalanced; hanya mengandalkan satu kali split; data leakage (scaler di-fit sebelum split, bukan di dalam Pipeline); menggunakan ulang test set untuk tuning berulang kali; distribusi data training berbeda dari kondisi deployment (concept drift); threshold default 0,5 tanpa mempertimbangkan biaya kesalahan; serta melaporkan metrik yang tidak sesuai konteks masalah.

ANALOGI: MACHINE LEARNING DI SEHARI-HARI

- **Email Spam Filter (Binary Classification):** filter spam Gmail mencapai akurasi lebih dari 99,9% dengan menggabungkan banyak model dan aturan; klasifikasi biner spam/bukan-spam adalah contoh paling matang dari supervised learning.
- **Rekomendasi YouTube (Multi-Class + Regression):** sistem rekomendasi menggabungkan collaborative filtering dan content-based filtering, mirip bagaimana Random Forest menggabungkan banyak "opini" tree.
- **Face Detection di Kamera (Computer Vision):** Viola-Jones (2001) memakai Haar cascade dan AdaBoost (ensemble learning, sekeluarga dengan Random Forest), sedangkan sistem modern beralih ke CNN.
- **Bank Credit Scoring (Logistic Regression Klasik):** skor kredit FICO tetap memakai Logistic Regression karena regulasi menuntut interpretability tinggi, meski model yang lebih kompleks (Random Forest, gradient boosting) bisa lebih akurat.
- **Diagnosis Medis ML (Decision Tree + Ensemble):** IBM Watson Health dan Google DeepMind Health memakai ensemble XGBoost dan deep learning; hyperparameter tuning menjadi krusial karena taruhannya adalah keselamatan pasien.
- **Self-Driving Car Object Detection (Real-Time ML):** Tesla Autopilot, Waymo, dan Mobileye memakai YOLO/SSD hasil hyperparameter tuning ekstensif untuk mencapai latency di bawah 100ms tanpa mengorbankan akurasi deteksi.

Pola yang Sama di Semua Analogi: Pola yang sama di semua analogi: tidak ada satu algoritma yang selalu terbaik untuk semua kasus; pemilihan model dan hyperparameter yang tepat membutuhkan evaluasi sistematis (cross-validation) dan pertimbangan trade-off interpretability vs akurasi vs kecepatan.

APLIKASI ML PREDICTIVE MAINTENANCE DI INDUSTRI 4.0 & SMART MANUFACTURING

- **Pratt & Whitney Engine Health Management:** 5000+ mesin PW1100G/A320neo dipantau dengan ensemble RF+XGBoost dan LSTM untuk prediksi Remaining Useful Life; 80% kegagalan terdeteksi 30+ hari lebih awal, menghemat \$5-10 juta per AOG yang dihindari.
- **Siemens Mobility Train Predictive Maintenance:** platform Railigent memakai Random Forest untuk klasifikasi kondisi bogie, motor traksi, gearbox, dan rem; terbukti pada 600+ trainset dengan availability naik 15% dan MTBF naik 30%.

- **GE Predix APM:** 50.000+ aset terhubung dengan digital twin dan hybrid rule+ML untuk komponen hot gas path, menurunkan downtime 5-15%.
- **Tesla Battery Management ML:** 7104+ sel baterai per mobil dipantau dengan gradient boosting dan neural network, update model mingguan via OTA, mendukung garansi 8 tahun/240.000 km dengan penurunan kapasitas di bawah 30%.

Kaitan ke Capstone Project: Modul ini menjadi penutup mata kuliah Optimalisasi & Otomasi, menggabungkan optimasi matematis (LP/NLP/ANOVA, Pertemuan 9-11), monitoring rule-based (Pertemuan 12-13), dan machine learning (Pertemuan 14-15) menjadi satu kerangka pemikiran utuh untuk merancang sistem otomasi cerdas di industri manufaktur modern.

IMPLEMENTASI PYTHON: MACHINE LEARNING KLASIFIKASI DI JUPYTER NOTEBOOK

Empat cell berikut melanjutkan notebook Pertemuan 14, menambahkan Random Forest, SVM Pipeline, ROC-AUC, dan GridSearchCV. Lingkungan: conda env optoauto dengan paket numpy, pandas, matplotlib, scikit-learn; notebook p13_ml_classification.ipynb (lanjutan).

```
p13_ml_classification.ipynb – Cell 4
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier

param_grid = {'n_estimators': [50, 100, 200],
              'max_depth': [None, 5, 10, 20],
              'min_samples_leaf': [1, 2, 5]}

grid = GridSearchCV(RandomForestClassifier(random_state=42),
                    param_grid, cv=5, scoring='f1_macro', n_jobs=-1)
grid.fit(x_train, y_train)

print('Best params: {grid.best_params_}')
print('Best CV score: {grid.best_score_: .3f}')
```

Gambar 7. Cuplikan kode GridSearchCV untuk tuning hyperparameter Random Forest dengan 5-fold cross-validation.

Gambar 7 menunjukkan cuplikan kode inti Cell 4.

- **Cell 2 (lanjutan):** `RandomForestClassifier(n\_estimators=100, max\_depth=None, min\_samples\_leaf=2, random\_state=42)` dilatih pada data mentah (tanpa scaling, RF tidak butuh); cetak confusion matrix dan `feature\_importances\_` terurut.
- **Cell 3 (lanjutan):** `Pipeline([StandardScaler, SVC(C=1.0, kernel='rbf', gamma='scale', probability=True)])`; perbandingan ROC-AUC tiga model (Logistic Regression, Random Forest, SVM) via `roc\_auc\_score` dan `predict\_proba`.

- **Cell 4:** ``cross_val_score(cv=5, scoring='f1_macro')`` untuk validasi Random Forest; ``GridSearchCV`` dengan grid ``n_estimators=[50,100,200]``, ``max_depth=[None,5,10,20]``, ``min_samples_leaf=[1,2,5]`` (36 kombinasi x 5-fold=180 fit); evaluasi ``best_estimator_`` di test set; simpan model final dengan ``joblib.dump()``.

TUGAS PERTEMUAN 15

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin.

Distribusi Bobot Tugas Pertemuan 15



Gambar 8. Distribusi 50 poin Tugas Pertemuan 15 berdasarkan tipe soal.

Gambar 8 merangkum distribusi bobotnya.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Untuk dataset PdM yang sangat imbalanced (misal 1% fault, 99% healthy), mengapa metrik accuracy menyesatkan?

- A. Accuracy selalu bernilai nol pada data imbalanced
- B. Accuracy hanya bisa dihitung untuk regresi
- C. Model yang selalu memprediksi "healthy" sudah memberi accuracy 99% padahal gagal total mendeteksi fault; pakai recall/F1/AUC/balanced accuracy
- D. Accuracy membutuhkan GPU

2. ROC-AUC unggul sebagai metrik evaluasi karena...

- A. Bergantung pada threshold tetap 0,5
- B. Threshold-independent, mengukur kemampuan model me-ranking sampel positif di atas negatif (probabilitas fault acak lebih besar dari healthy acak). 0,5=acak, 1,0=sempurna

- C. Hanya berlaku untuk data seimbang sempurna
- D. Selalu sama dengan accuracy

3. Random Forest classifier menggabungkan banyak decision tree dengan strategi...

- A. Setiap tree dilatih di seluruh dataset yang sama, hasil dirata-rata
- B. Tiap tree dilatih di bootstrap sample dan random subset features; prediksi via majority voting. Diversifikasi mengurangi variance, robust terhadap noise
- C. Sequential boosting per tree dari error tree sebelumnya
- D. Single tree dengan max_depth tinggi

4. Support Vector Machine (SVM) dengan kernel RBF mampu menangani batas non-linear karena...

- A. Menggunakan banyak hidden layer seperti neural network
- B. Kernel trick: implicit map data ke ruang dimensi tinggi via kernel function (RBF, polynomial). Di ruang tersebut, batas linear cukup untuk memisahkan kelas yang non-linear di ruang asli
- C. Pakai gradient descent agresif
- D. Hanya bekerja untuk binary classification

5. Mengapa StandardScaler harus dibungkus dalam Pipeline bersama SVM, bukan di-fit terpisah sebelum train_test_split?

- A. Agar kode lebih pendek
- B. Mencegah data leakage: scaler harus di-fit HANYA pada data training (di tiap fold cross-validation), bukan pada seluruh dataset termasuk test set
- C. SVM tidak bisa menerima data yang sudah di-scale
- D. Pipeline mempercepat training 10x lipat

6. k-fold cross-validation (k=5) lebih reliable daripada single train/test split karena...

- A. Single split selalu underperform CV
- B. CV memberikan mean +/- std dari k metric estimates, single split bisa lucky/unlucky (variance tinggi). CV reduce variance estimate via averaging across k folds, lebih robust untuk model selection dan hyperparameter tuning
- C. CV gunakan algoritma berbeda
- D. CV lebih cepat 5 kali lipat

7. GridSearchCV dengan param_grid n_estimators:[50,100,200] dan max_depth:[None,5,10,20] serta cv=5 melakukan berapa kali fitting model?

- A. 5 kali
- B. 7 kali
- C. $3 \times 4 \times 5 = 60$ kali fitting
- D. 12 kali saja tanpa cv

8. Perbedaan utama Gradient Boosting dan Random Forest adalah...

- A. Keduanya identik
- B. RF melatih pohon secara sekuensial pada residual
- C. Gradient Boosting tidak memakai pohon keputusan
- D. RF = bagging pohon paralel (vote/average untuk mengurangi variance); Gradient Boosting = pohon sekuensial yang tiap pohon mengoreksi error pohon sebelumnya (mengurangi bias)

9. Permutation feature importance mengukur pentingnya fitur dengan cara...

- A. Menghapus fitur lalu melatih ulang model dari awal untuk tiap fitur
- B. Mengacak (shuffle) nilai satu fitur pada data test dan mengukur penurunan skor model; penurunan besar berarti fitur penting; bersifat model-agnostic
- C. Mengurutkan fitur secara abjad
- D. Hanya menghitung korelasi fitur dengan target

10. Setelah model PdM di-deploy, model drift/data drift ditangani dengan...

- A. Tidak perlu dipantau, model valid selamanya
- B. Mengganti model dengan rule-based setiap hari
- C. Memantau distribusi input dan metrik kinerja; bila terdeteksi drift (AUC turun, distribusi fitur bergeser), trigger retraining dengan data terbaru
- D. Menambah jumlah kelas target

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

Catatan: $\text{recall (sensitivity)} = \text{TP}/(\text{TP} + \text{FN})$; $\text{specificity} = \text{TN}/(\text{TN} + \text{FP})$;
 $\text{balanced_acc} = (\text{recall} + \text{specificity})/2$; $\text{Youden J} = \text{sensitivity} + \text{specificity} - 1$; $\text{G-mean} = \sqrt{\text{sensitivity} \times \text{specificity}}$; $\text{F-beta} = (1 + b^2) \cdot \text{P.R.} / (b^2 \cdot \text{P} + \text{R})$; $\text{class_weight balanced: } w_{\text{kelas}} = n_{\text{samples}} / (n_{\text{classes}} \times n_{\text{kelas}})$.

C1. Confusion matrix: TP=80, FP=10, TN=85, FN=5. Hitung $\text{specificity} = \text{TN}/(\text{TN} + \text{FP})$.

- 💡 **HINT:** Specificity = true negative rate.

C2. Dari confusion matrix yang sama (TP=80, FP=10, TN=85, FN=5), hitung $\text{balanced accuracy} = (\text{recall} + \text{specificity})/2$.

- 💡 **HINT:** Recall=TP/(TP+FN); specificity=TN/(TN+FP); rata-ratakan.

C3. Dataset PdM: 950 sampel healthy, 50 sampel fault. Hitung $\text{rasio imbalance} = \text{mayoritas}/\text{minoritas}$.

- 💡 **HINT:** Bagi jumlah mayoritas dengan minoritas.

C4. Untuk menyeimbangkan 50 sampel minoritas (fault) hingga sama dengan 950 mayoritas via oversampling, hitung jumlah sampel sintetis yang dibuat.

- 💡 **HINT:** Target jumlah minoritas = mayoritas; sintetis = selisihnya.

C5. Hitung Youden's $J = \text{sensitivity} + \text{specificity} - 1$ untuk $TP=80$, $FP=10$, $TN=85$, $FN=5$.

- 💡 **HINT:** $\text{sensitivity} = \text{recall}$; $J = \text{sensitivity} + \text{specificity} - 1$.

C6. Hitung F2-score (menekankan recall) untuk $\text{precision}=0,75$, $\text{recall}=0,60$. Formula: $F2 = 5 \cdot P \cdot R / (4 \cdot P + R)$.

- 💡 **HINT:** Substitusi P dan R ke formula F-beta dengan $\text{beta}=2$.

C7. Skor prediksi kelas positif= $[0.9, 0.4, 0.8]$ dan kelas negatif= $[0.5, 0.3, 0.6]$. Hitung ROC-AUC=fraksi pasangan (pos, neg) dengan skor pos>neg (Mann-Whitney).

- 💡 **HINT:** Bandingkan tiap skor positif dengan tiap skor negatif; hitung fraksi pos>neg.

C8. Untuk 1000 sampel, 2 kelas, minoritas 50 sampel, hitung bobot kelas minoritas ala $\text{class_weight} = \text{'balanced'} = n_samples / (n_classes \times n_minority)$.

- 💡 **HINT:** Bagi total sampel dengan (jumlah kelas x jumlah minoritas).

C9. Biaya $FP=1$ dan biaya $FN=10$ (FN 10 kali lebih mahal). Untuk $FP=10$, $FN=5$, hitung total biaya $= FP \times 1 + FN \times 10$.

- 💡 **HINT:** Jumlahkan $FP \times \text{biaya_FP} + FN \times \text{biaya_FN}$.

C10. Hitung G-mean= $\sqrt{\text{sensitivity} \times \text{specificity}}$ untuk $TP=80$, $FP=10$, $TN=85$, $FN=5$.

- 💡 **HINT:** Akar dari ($\text{recall} \times \text{specificity}$).

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan 20-50+ baris kode dengan algoritma lengkap dari awal. Hint minimal. Jawaban benar: +4 poin. Jawaban salah (kode ada tapi hasil tidak sesuai/error): +1 poin partial. Kode kosong = 0 poin (bisa retry).

C11 (Hard). ROC-AUC via Mann-Whitney: `np.random.seed(42); skor_positif=np.random.normal(0.7,0.15,100); skor_negatif=np.random.normal(0.4,0.15,100)`. Hitung AUC=rata-rata atas seluruh pasangan (pos,neg): +1 bila pos>neg, +0,5 bila sama.

- 💡 **HINT:** Bandingkan tiap skor positif dengan tiap negatif (100x100 pasangan); rata-ratakan.

C12 (Hard). Threshold tuning (Youden): dengan skor pos/neg dari soal sebelumnya (seed 42), sapu semua threshold unik; di tiap threshold hitung TPR dan FPR; print nilai Youden's J maksimum= $\max(\text{TPR}-\text{FPR})$.

- 💡 **HINT:** Untuk tiap threshold t, prediksi positif bila skor $\geq t$; hitung TPR-FPR; ambil maksimum.

C13 (Hard). SMOTE 50%: 920 sampel mayoritas, 80 minoritas. Oversample minoritas hingga mencapai 50% jumlah mayoritas. Hitung jumlah sampel sintetis yang ditambahkan.

- 💡 **HINT:** Target minoritas=0,5 x mayoritas; sintetis=target-minoritas awal.

C14 (Hard). Precision pada recall tinggi: dengan skor dan label dari soal ROC-AUC (positif=label 1, negatif=label 0, seed 42), urutkan skor menurun, lalu hitung precision pada titik pertama dimana recall $\geq$ 0,9.

- 💡 **HINT:** Urut skor menurun, akumulasi TP/FP, cari recall pertama $\geq$ 0,9, ambil precision di titik itu.

C15 (Hard). Optimasi threshold untuk F1: dengan data seed 42 dari soal ROC-AUC, sapu semua threshold; hitung F1 di tiap threshold; print F1 maksimum.

- 💡 **HINT:** Untuk tiap threshold hitung precision, recall, lalu F1; ambil maksimum.

DAFTAR PUSTAKA

- Altaf, M., Akram, T., Khan, M. A., Iqbal, M., Ch, M. M. I., & Hsu, C.-H. (2022). A new statistical features based approach for bearing fault diagnosis using vibration signals. *Sensors*, 22(5), 2012. <https://doi.org/10.3390/s22052012>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Tiboni, M., Remino, C., Bussola, R., & Amici, C. (2022). A review on vibration-based condition monitoring of rotating machinery. *Applied Sciences*, 12(3), 972. <https://doi.org/10.3390/app12030972>
- Nguyen, T. T., Phi, B. P., Tran, V. T., Nguyen, V.-T., & Nguyen, V. T. T. (2025). Three-stage optimization of surface finish in WEDM of D2 tool steel via Taguchi design and ANOVA analysis. *Metals*, 15(6), 682. <https://doi.org/10.3390/met15060682>